

Embedded Systems und Feldbusse, Teil: Embedded Systems

(2 SWS)

Inhaltsverzeichnis

0.	EMBEDDED RESSOURCEN.....	5
1.	ANWENDUNGSFELDER VON EMBEDDED SYSTEMEN	7
1.1.	Automatisierung von Maschinen.....	7
1.2.	Automatisierung von technischen Anlagen in der Prozessautomatisierung	7
1.3.	Einsatz im Automobil	10
1.4.	Einsatz in der Medizintechnik (Bsp.).....	12
1.5.	Einsatz in der Gebäudetechnik	12
1.6.	Einsatz im Bereich drahtloser Systeme	13
1.7.	Topologien vernetzter drahtloser Systeme	15
1.8.	Fragen zum Kapitel.....	18
2.	BEGRIFFSDEFINITIONEN UND ANFORDERUNGEN	19
2.1.	Begriffsdefinition Embedded System	19
2.2.	Eigenschaften und Anforderungen an ein Embedded System	21
2.3.	Begriffsdefinition Echtzeitsystem (Real-time system)	22
2.4.	Anforderungen an ein Echtzeit-Betriebssystem.....	23
2.4.1.	Rechtzeitigkeit.....	23
2.4.1.1	Arten der Rechtzeitigkeit (harte u. weiche Echtzeit)	23
2.4.1.2	Vier Zeitbedingungen	25
2.4.2.	Übung: Beispielanwendungen zu gegebenen Zeitbedingungen	26
2.4.3.	Gleichzeitigkeit (Multitasking).....	31
2.4.4.	Vorhersehbarkeit (Determinismus)	33
2.4.5.	Zusammenfassung Anforderungen an ein Echtzeitsystem:.	38
3.	GLEICHZEITIGE UND PARALLELE AUFGABENBEARBEITUNG	38

3.1.	Lösungsvariante 1: Einsatz eines Mehrprozessorsystems	40
3.2.	Lösungsvariante 2: Ereignisgesteuerte Aufgabenbearbeitung	42
3.2.1.	Definition Ereignisse	43
3.2.2.	Bewertung der Lösungsvariante 2: Ereignisgesteuerte Aufgabenbearbeitung	44
3.2.3.	Programmtechnische Realisierung einer ereignisgesteuerten Aufgabenbearbeitung (Bsp.)	45
3.2.4.	Signaltypen in Embedded Systemen	46
3.2.5.	Problemlösung: Periodischer Hardware-Timer muss mittelbar Periodizität herstellen.....	47
4.	AUFBAUSTRUKUR VON EMBEDDED SYSTEMEN.....	49
4.1.	Vergleich Mikrocontroller versus Mikroprozessor	50
4.2.	Aufbau und Funktionsweise einer Central Processing Unit.....	51
4.2.1.	Bestandteile einer CPU.....	51
4.2.2.	Funktionsweise einer CPU (Von-Neumann-Zyklus).....	55
4.2.3.	Übung: Beschleunigungsverfahren für Von-Neumann Rechner	60
4.2.4.	CPU-Zugriff auf den Schreib-Lese Speicher.....	61
4.3.	Hauptfunktionskomponenten eines Embedded Systems	63
4.3.1.	Mikrocontrollerfamilien (Auswahl)	66
4.3.2.	Auswahl geeigneter Mikrocontroller - Bsp. TI MSP430.....	70
4.4.	Der Adressraum der MSP430-Mikrocontrollerfamilie.....	75
4.4.1.	MSP430 Memory Map im Detail	77
4.4.2.	Das Device Description File msp430F449.ddf	79
4.5.	Mikrocontrollerspeicher.....	81
4.6.	Die CPU.....	82
4.6.1.	Addressing Modes	84
4.6.2.	The Instruction Set.....	84
4.7.	Beurteilungskriterien für die Auswahl eines Mikrocomputersystems	89
4.8.	Speichertypen in der Programmiersprache C/C++	90
	Stack und den Heap - Wertetypen und Verweistypen.....	91
4.10.	Testfragen	92

5.	ENTWICKLUNGSSCHRITTE FÜR EIN MIKROCOMPUTERSYSTEM	95
6.	DER SOFTWARE BUILD PROZESS BEI EMBEDDED SYSTEMEN.....	95
7.	SCHNITTSTELLE ZUM PERIPHERIEMODUL (I/O-BAUSTEIN).....	100
8.	DIGITAL I/O: PORTS UND BITMANIPULATIONEN	101
8.1.	Begriffsdefinitionen	101
8.2.	Einsatz von symbolischen Konstanten zur Erhöhung der Lesbarkeit von Programmen	102
8.3.	Definition von Bitgruppen.....	103
8.4.	Bitmanipulationen	103
8.4.1.	Selektives Eins-Setzen von Bits.....	104
8.4.2.	Selektives Rücksetzen von Bits	104
8.4.3.	Bitmaskierung	105
8.4.4.	Bitweises Invertieren	106
8.4.5.	Das 1er und 2er-Komplement.....	107
8.4.6.	Übersicht der Bitoperatoren in ISO Standard C	109
8.4.7.	Übungen zu Bitoperationen.....	110
8.5.	Digital Input und Digital Output Unit.....	115
8.5.1.	MSP430x44x Terminal Functions	115
8.5.2.	Non-Interruptible I/O.....	117
1.1.1.	Interruptible I/O	119
8.5.3.	Digital I/O-Registers	121
8.5.4.	Electrical characteristics of Inputs/Outputs	126
8.5.5.	Programmbeispiel 1	128
8.5.6.	Programmbeispiel 2	128
8.5.7.	Programmbeispiel 3	129
8.5.8.	Übung 1.....	130
8.5.9.	Übung 2.....	131
8.5.10.	Übung 3.....	132
8.5.11.	Übung 4.....	132
9.	INTERRUPTS UND INTERRUPT SERVICE ROUTINEN (ISR)	134

9.1. Arten von Interrupts	134
9.1.1. Interne Unterbrechungsursachen.....	135
9.1.2. Externe Unterbrechungsursache	136
9.1.3. Hardware Interrupt Request-Leitungen (IRQs)	136
9.2. Übertragung des Interruptvektors von der Peripherie an die CPU .	140
9.2.1. Maskierbare und nicht maskierbare Interrupts.....	141
9.2.2. Software-Interrupts (BIOS-Interrupt)	142
9.2.3. Interrupt Service Routinen (ISR).....	143
9.3. Der Programmable Interrupt-Controller 8259 (PIC).....	144
9.3.1. Zusammenspiel zwischen CPU und 8259-Controller.....	146
9.4. CPU interne Bearbeitung von Interrupts und ISRs	154
9.4.1. Der normale Programmfluss	154
9.4.2. Priorisierung bei mehreren Interrupt-Quellen.....	156
9.5. Das MSP430 Interruptsystem	156
9.5.1. Behandlung eines Interrupts bei der TI MSP430-Familie ..	159
9.5.2. Binden eines Interrupts an eine Interrupt Service Routine.	162
9.5.3. Übersicht der Interrupt Vektoren (aus msp430x44x.h)	163
9.6. Beispiele und Übungen zum Interruptsystem MSP430I	164
9.6.1. Anwendungsbeispiel I	164
9.6.2. Anwendungsbeispiel II	168
9.6.3. Übung I.....	168
10. WEITERE FRAGEN:	169
10.1.1. Übung II.....	171
10.1.2. Einsatz von Low Power Modes bei der MSP430F4xx-Familie	173

Document History

Version, Date	Author(s) email address	Changes and other notes
16.4.2019	ludwig.eckert@fhws.de	Kap. 2.4.1 u. Kap. 2.4.2 überarbeitet
19.5.2019	ludwig.eckert@fhws.de	Kap. 3 überarbeitet
4.6.2019	ludwig.eckert@fhws.de	Kap. 3.2 überarbeitet
24.6.2019	ludwig.eckert@fhws.de	Kap. 4 überarbeitet
2.7.2019	ludwig.eckert@fhws.de	Kap. 7 eingefügt, Kap. 8 überarbeitet

0. EMBEDDED RESSOURCEN

Embedded Systems Hardware Design und Aufbaustrukturen

- [1] Embedded Microprocessor Systems: Real World Design
Stuart Ball
Verlag: Elsevier LTD, Oxford; Auflage: 3 Sub (4. Dezember 2002)
- [2] Eingebettete Systeme
Peter Marwedel und Lars Wehmeyer
Springer; Auflage: 1. Aufl. 2007. Korr. Nachdruck 2008 (8. September 2008)
- [3] Designing Embedded Hardware
John Catsoulis von O'Reilly Media (24. Mai 2005)
O'Reilly Media; Auflage: 2nd ed. (24. Mai 2005)
- [4] Mikroprozessortechnik: Grundlagen, Architekturen, Schaltungstechnik und Betrieb von Mikroprozessoren und Mikrocontrollern
Klaus Wüst
Vieweg+Teubner Verlag; Auflage: 4. Aufl. 2011 (25. November 2010)

Embedded Systems Software Design und Programmierung, Echtzeitsysteme

- [5] An Embedded Software Primer
David E. Simon
Addison Wesley; Auflage: Pap/Cdr (5. August 1999)
- [6] Barr, M.
Programming Embedded Systems
Programmierung, Entwicklungswerkzeuge, Speicher und Peripherie, O'Reilly, 1999
- [7] Ganssle, Jack G.
The Art of Programming Embedded Systems
San Diego, California, Academic Press, Inc., 1992
- [8] Zöbel, D.; Albrecht, W.
Echtzeitsysteme - Grundlagen und Techniken
Echtzeitanalyse, Echtzeitprogrammierung
Informatik Lehrbuch Reihe, International Thomson Publishing, 1995
- [9] Wood, Mike and Tom Barrett
A Real-Time Primer
Embedded Systems Programming, February 1990 p20-28

EDN Magazine (Electronic Design Network)

At the [EDN Magazine](http://www.edn.com) website, you can read articles and write letters to the editors.
<http://www.edn.com>

EDN provides engineers and engineering managers with in-depth design features and deep technical information on microprocessors and other digital ICs, analog ICs, ASICs, computers and software, computer peripherals, test and measurement, components, power sources, interconnections and packaging, and much more.

C51 Primer

Der [C51 Primer](#) (englisch) ist ein einführender Text der beschreibt, wie C Programme für den 8051 zu schreiben sind. Die verschiedenen Speicher Modelle, special function register, die Architektur des 8051 werden im Detail beschrieben.

C166 Primer

Der [C166 Primer](#) (englisch) liefert Informationen über die Siemens 166 (16Bit) Mikrocontroller Familie. Eine detaillierte Referenz liefert Details der Interrupts, der I/O Ports, der Bus Modi und mehr.

Chip Directory

Sehr umfangreiches Verzeichnis von Halbleiter ICs: [Chip Directory](#). Dokus zu den auf dem Markt verfügbaren ICs mit Links zu den jeweiligen Herstellern.

Embedded Systems Programming

[Embedded Systems Programming](#): Englischsprachige Zeitschrift speziell über Mikrocontrollersysteme. Die Webseite des Verlags beinhaltet eine Menge aktuelle Informationen sowie code listings von erschienenen Artikeln.

<http://www.elektroniknet.de/>

<http://www.mikrocontroller.net/>

<http://www.techonline.com/>

DEPATISnet

Recherchen zum Stand der Technik in Patentdokumenten aus aller Welt
Mit DEPATISnet können Sie in einer weltweiten Sammlung von Patentveröffentlichungen recherchieren. Sie nutzen dabei den Datenbestand des amtsinternen deutschen Patentinformationssystems DEPATIS.

http://www.deutsches-patentamt.de/service/e_dienstleistungen/depatismet/index.html

Google Books

Search and preview millions of books from libraries and publishers worldwide using Google Book Search. Discover a new favorite or unearth an old classic
books.google.com / books.google.de

<https://code.google.com>

<https://developers.google.com/>

1. ANWENDUNGSFELDER VON EMBEDDED SYSTEMEN

1.1. Automatisierung von Maschinen

- Werkzeugmaschinen
- Industrieroboter
- Sensorsysteme
(Bildverarbeitung)
- Kraftfahrzeuge
(Motorsteuerung,
Bremsystem,
Getriebesteuerung,
Fahrplanung,
Abstandswarnsystem, usw.)
- Prüfmaschinen
- Messmaschinen
- Küchengeräte
- Waschmaschinen
- Nähmaschinen
- Alarmanlagen
- Navigationssysteme
- Heizungssysteme
- Spielzeug
- Filmkameras
- Telefonanrufbeantworter

1.2. Automatisierung von technischen Anlagen in der Prozessautomatisierung

- Kraftwerksanlagen
(Dampferzeuger, Turbinen,
Generatoren)
- Energieversorgungsanlagen
(Lastverteiler, Netzleitsysteme)
- Fertigungssysteme
- Stahlwerksanlagen
- Walzwerksanlagen
- Schienenverkehrssysteme
(Fernbahnen, Stadtbahnen,
U-Bahnen)
- Gasversorgungssysteme
- Prüfstände, Prüffelder
- Verfahrenstechnische Anlagen
- Laborautomatisierung
- Gebäude- und haustechnische
Anlagen
- Aufzüge
- Intensivstationen
- Hochregallager
- Verkehrstechnik
- Papiermaschinen
- Textilmaschinen

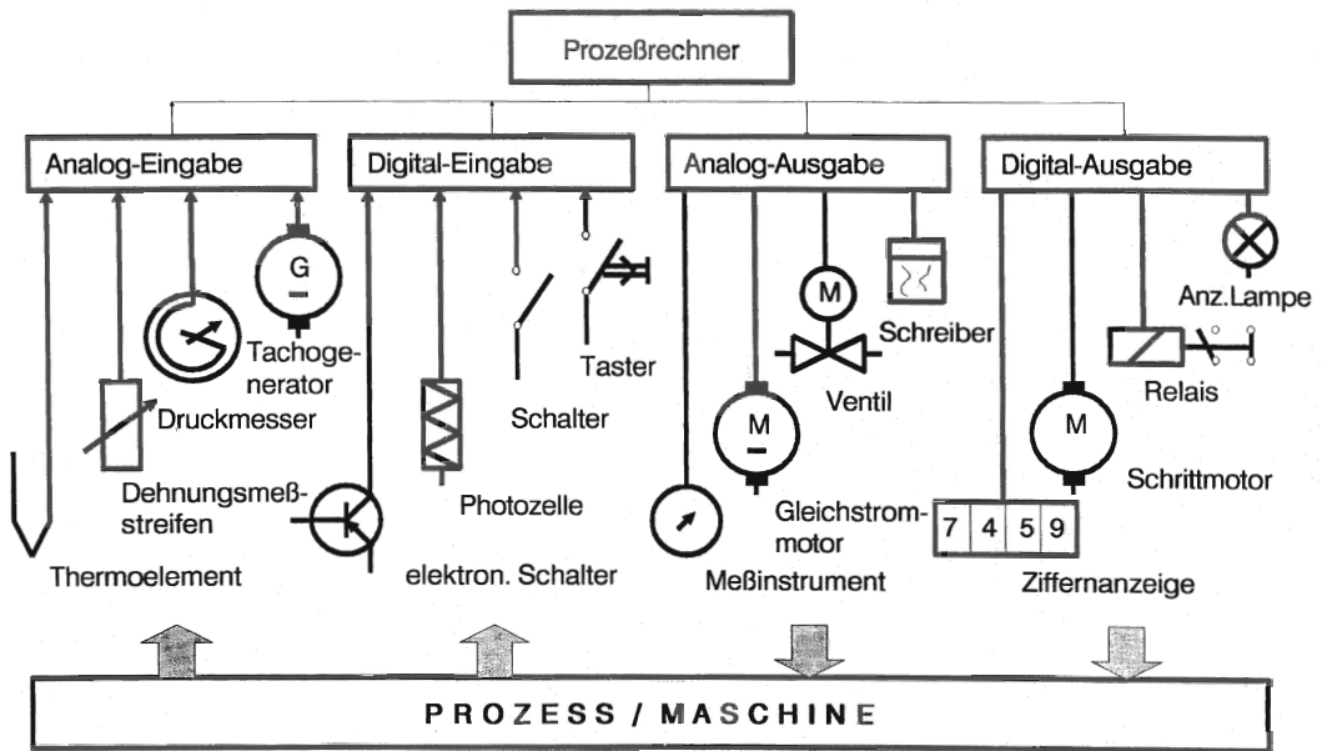


Bild: Prozessperipherie

Einsatz in der Handhabungstechnik in der Prozessautomatisierung

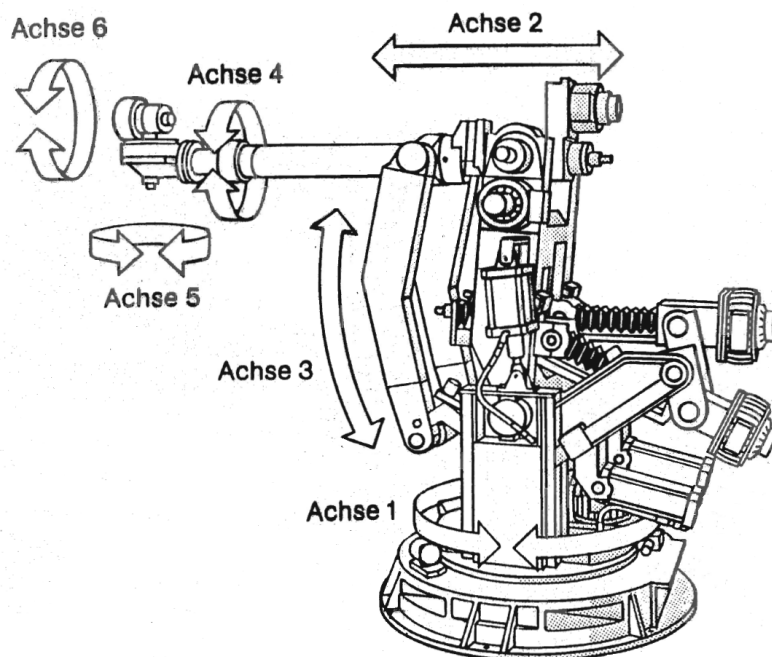
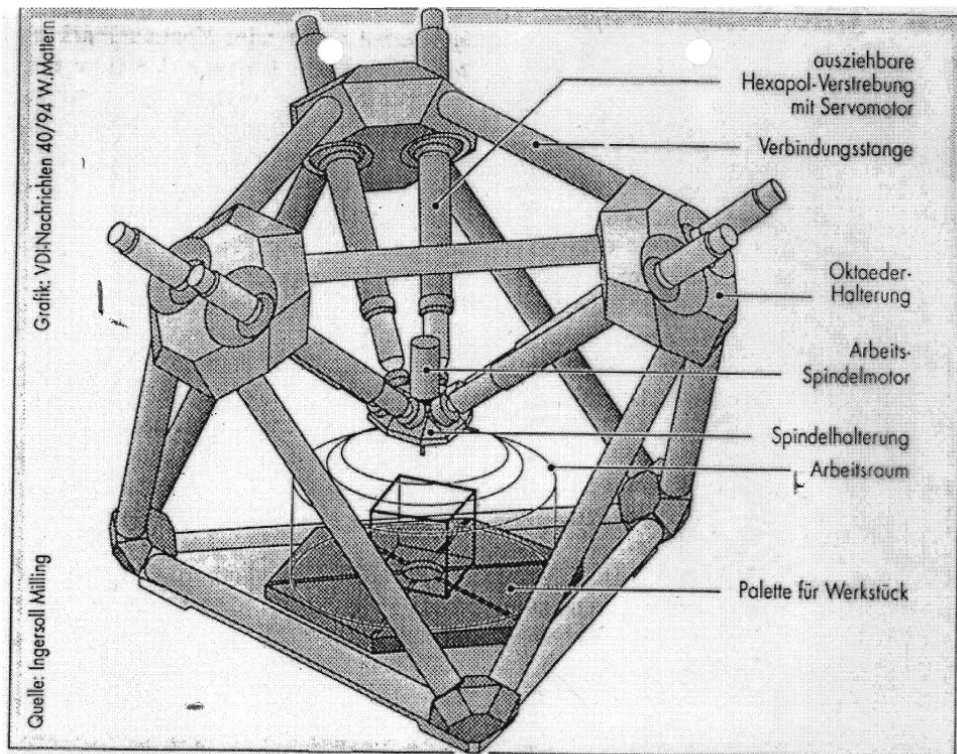


Bild: Industrieroboter mit 6 Achsen



Der oktaederförmige Maschinenkörper basiert auf zwölf Streben, die acht Dreieckselemente bilden. Über der Werkstück-Spannfläche im unteren Teil der 3 m hohen „Octahedral Hexapod“-Maschine befindet sich die Trägerplattform für Bearbeitungseinheiten, die von sechs Teleskopstangen frei im Raum bewegt werden kann. Alle Kräfte werden an den Ecken der acht Flächenelemente eingeleitet, so daß keine Biegemomente auftreten.

Bild: Ingersoll

Bild: Hexapod

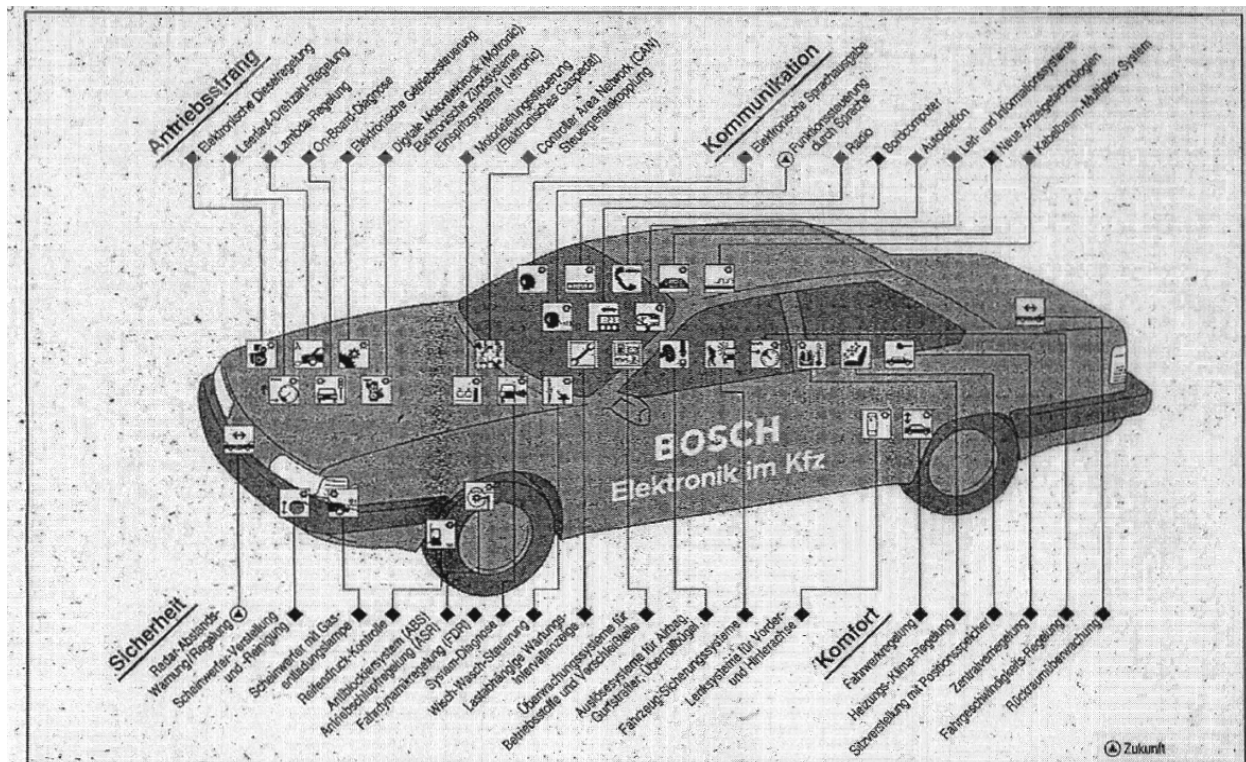
Anm.: Ein Hexapod (Hexa griech.: Sechs, pod griech.: Fuß) ist eine räumliche Bewegungs-Maschine mit sechs Antriebselementen.



Bild: Hexapod

http://www.automationline.de/wp-content/uploads/2016/07/fau172_Bild6.jpg - 18.10.2016

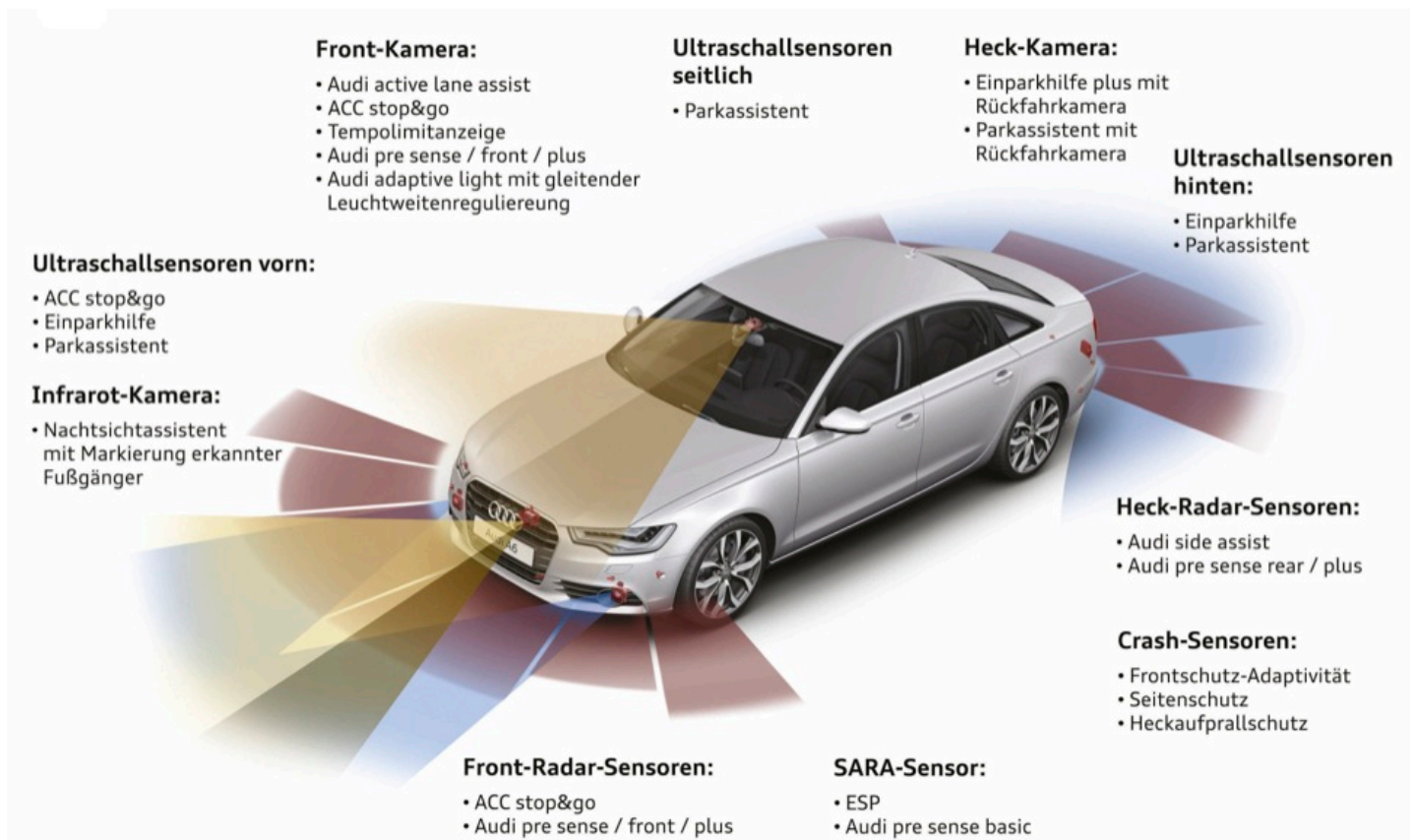
1.3. Einsatz im Automobil



Elektronik gehört zur Steuerung von Sicherheit und Komfort im Auto längst zur unverzichtbaren Ausstattung. Foto: Bosch

Foto: Bosch

http://www.linzing.de/wp-content/gallery/elektronik-trendes/rl11elektronik02_02_1024.jpg; 18.10.2016



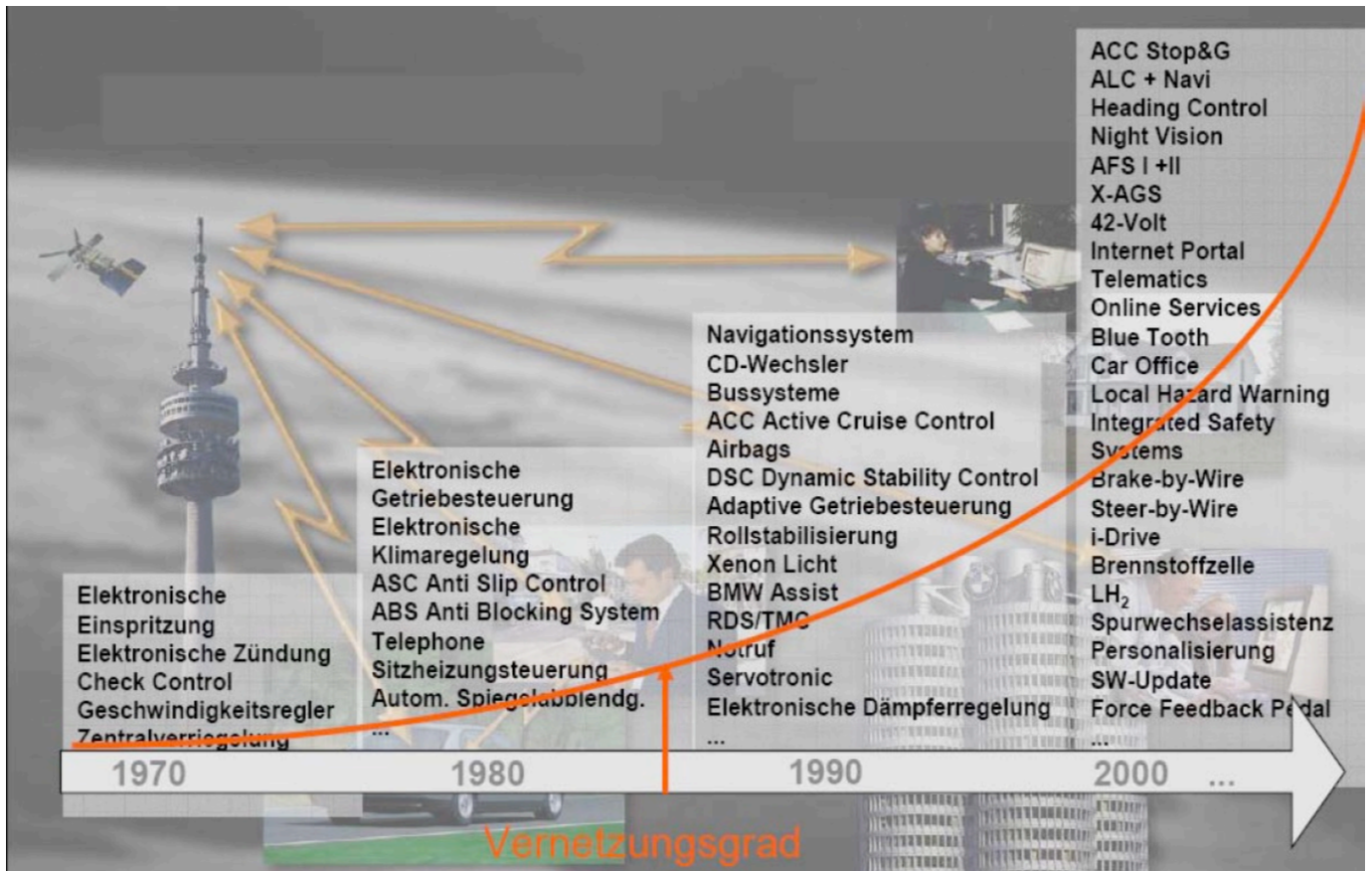


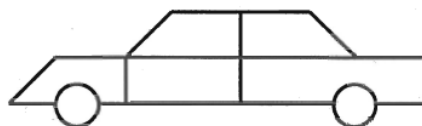
Bild: Wachsender Vernetzungsgrad

Mechatronik-System: "intelligentes" Produkt

bestehend aus:

- mechanisches System
- Sensoren, Aktoren
- Mikroprozessor für Steuerung, Regelung, Optimierung

Beispiel Automobil



Motor:	Kennlinienabhängige Zündung
Getriebe:	Automatisches Getriebe, ASR
Bremse:	Antiblockiersystem ABS
Diagnose:	Fahrzeugdiagnose

1.4. Einsatz in der Medizintechnik (Bsp.)

SANITAS ... gesund von Kopf bis Fuß.



NEU BEI METRO

Mückenstichheiler SIS 40¹⁾

- Das Heizplättchen des Mückenstichheilers erwärmt lokal die Einstichstelle auf über 50 °C – die beim Mückenstich übertragenen Giftstoffe werden somit unschädlich
- Inklusive 9-V-Batterie
- Medizinprodukt nach 93/42/EEC

29,99

16,79*

19,98



MEDISANA[®] made for Life

Anti-Allergiegerät Medinose¹⁾

- Behandlung von Allergien der Nasenschleimhaut durch Phototherapie: Heuschnupfen, Hausstaub- und Tierhaarallergie • Auch zur prophylaktischen Anwendung • Einfach und sicher

4½ min 3x täglich

Keine Nebenwirkungen

99,95

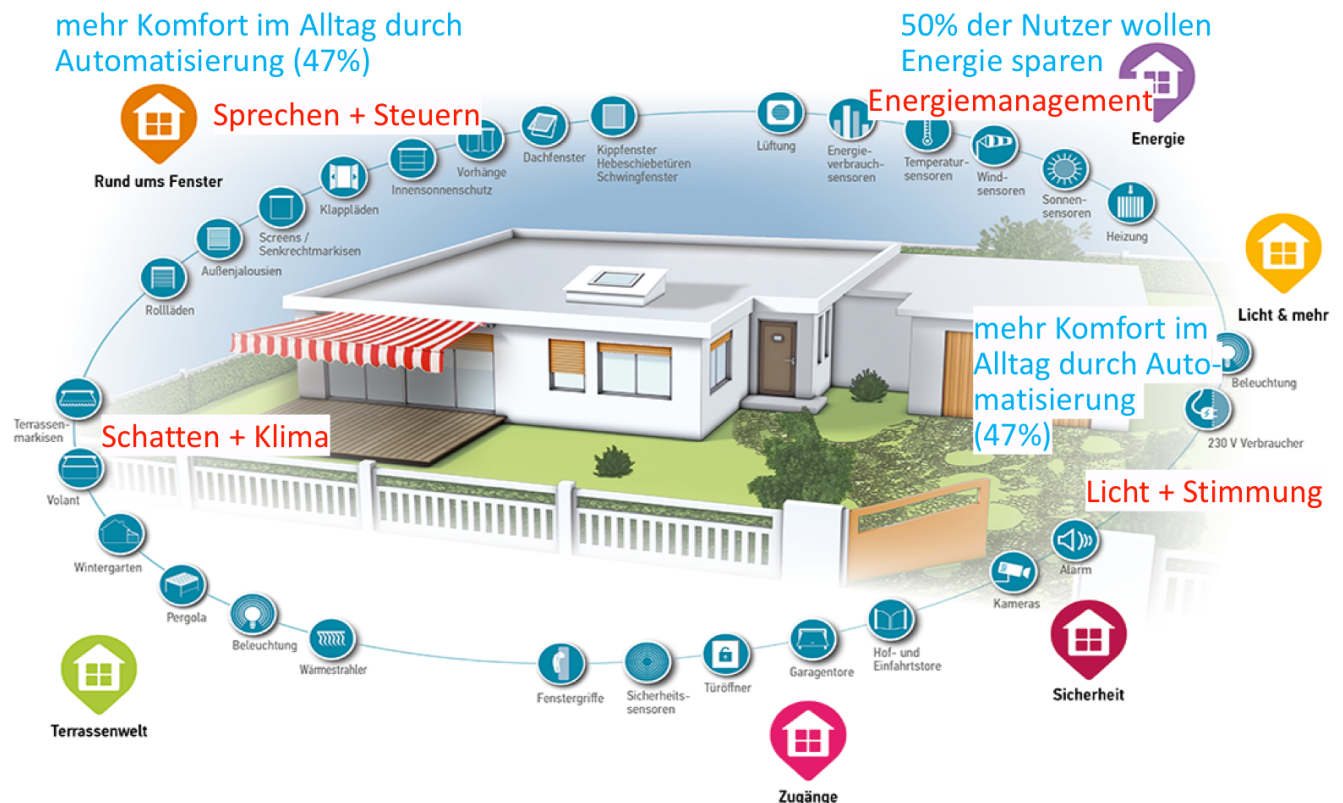
39,99*

47,59

Bild: Embedded Systems in der Medizintechnik

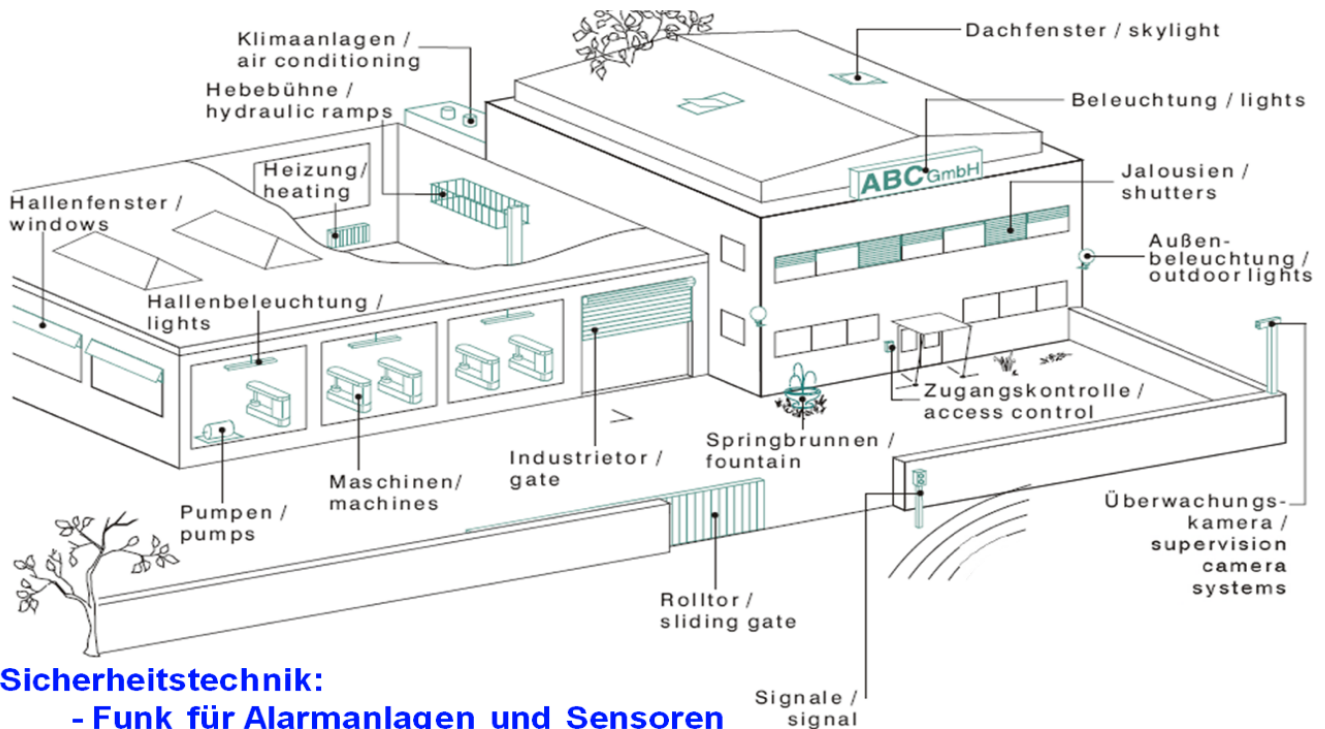
1.5. Einsatz in der Gebäudetechnik

Beispiele für den Einsatz von Embedded Systemen im Bereich „Smart Home“



<https://www.somfy.de/smart-home/was-ist-tahoma/tahoma> 17.3.2019 – 13:13 Uhr

Beispiele für den Einsatz von Embedded Systemen im Zweckbau/Industrie



Sicherheitstechnik:

- Funk für Alarmanlagen und Sensoren
- Zutrittssysteme
- Funk für Notrufsysteme

1.6. Einsatz im Bereich drahtloser Systeme

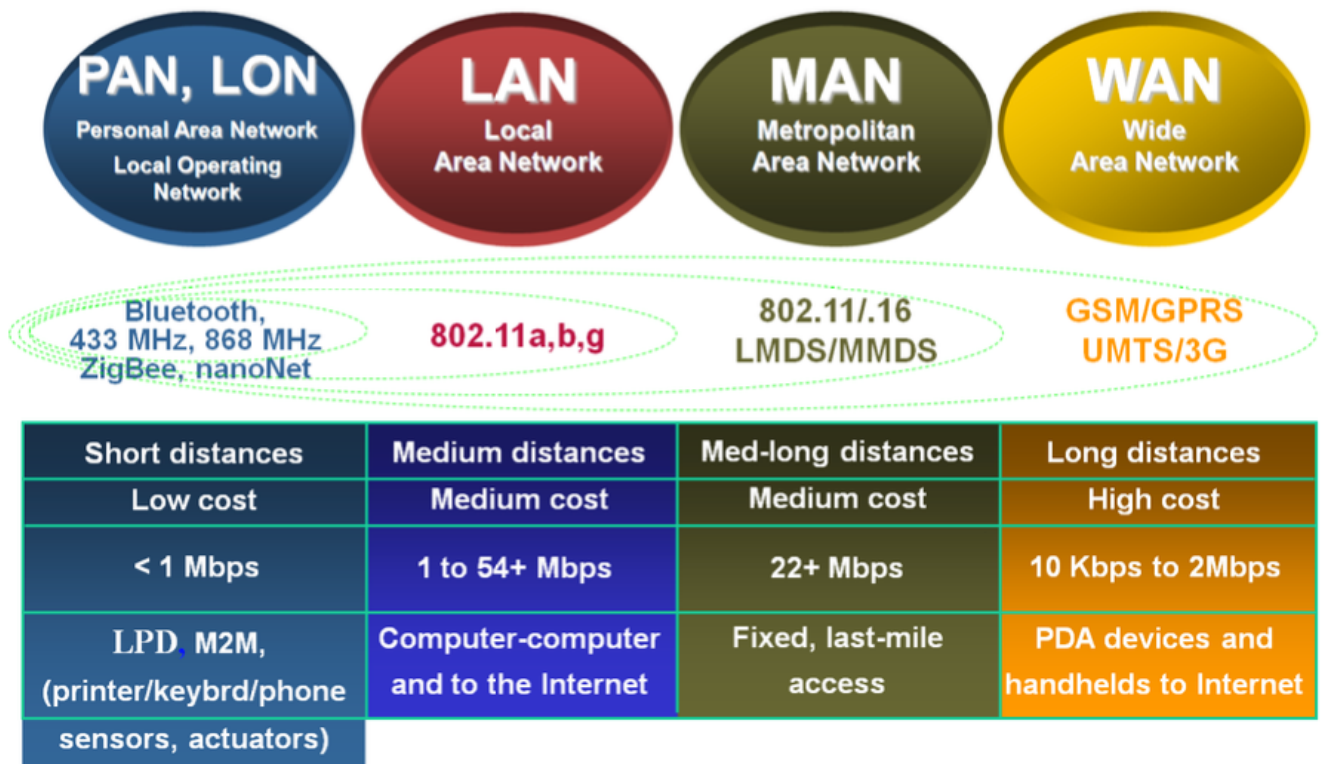


Bild: Wireless Network Distances

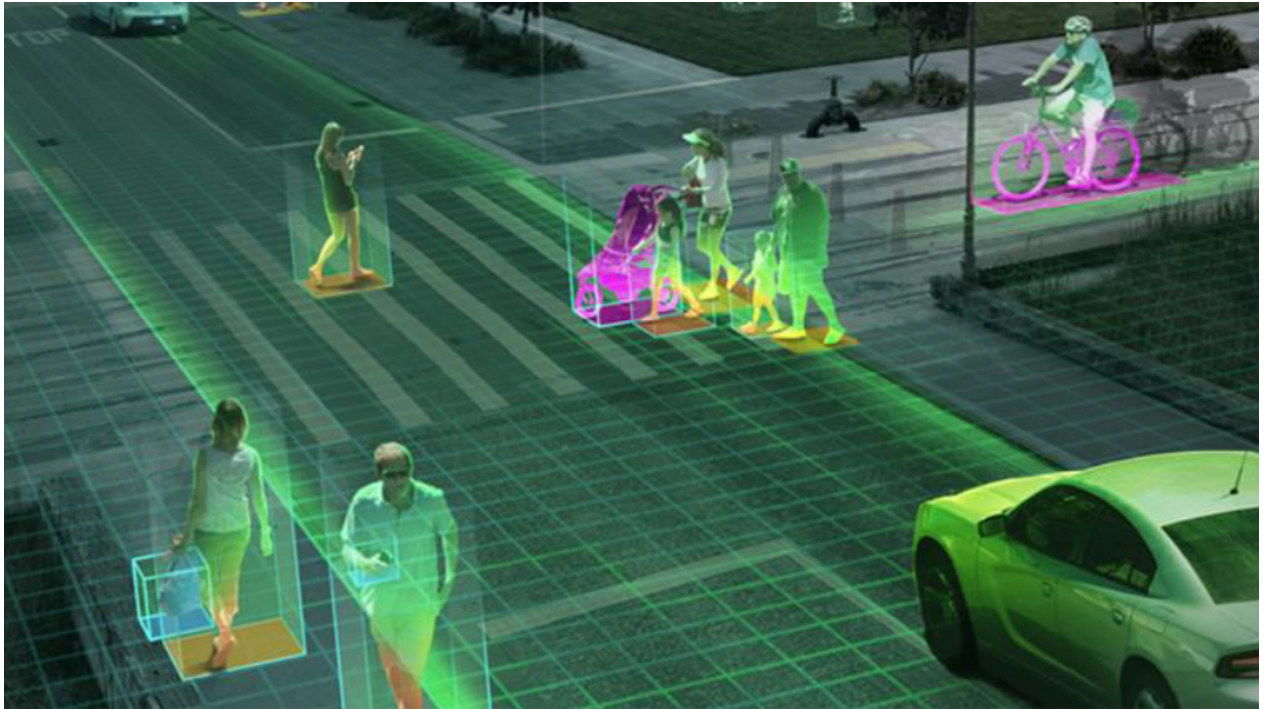
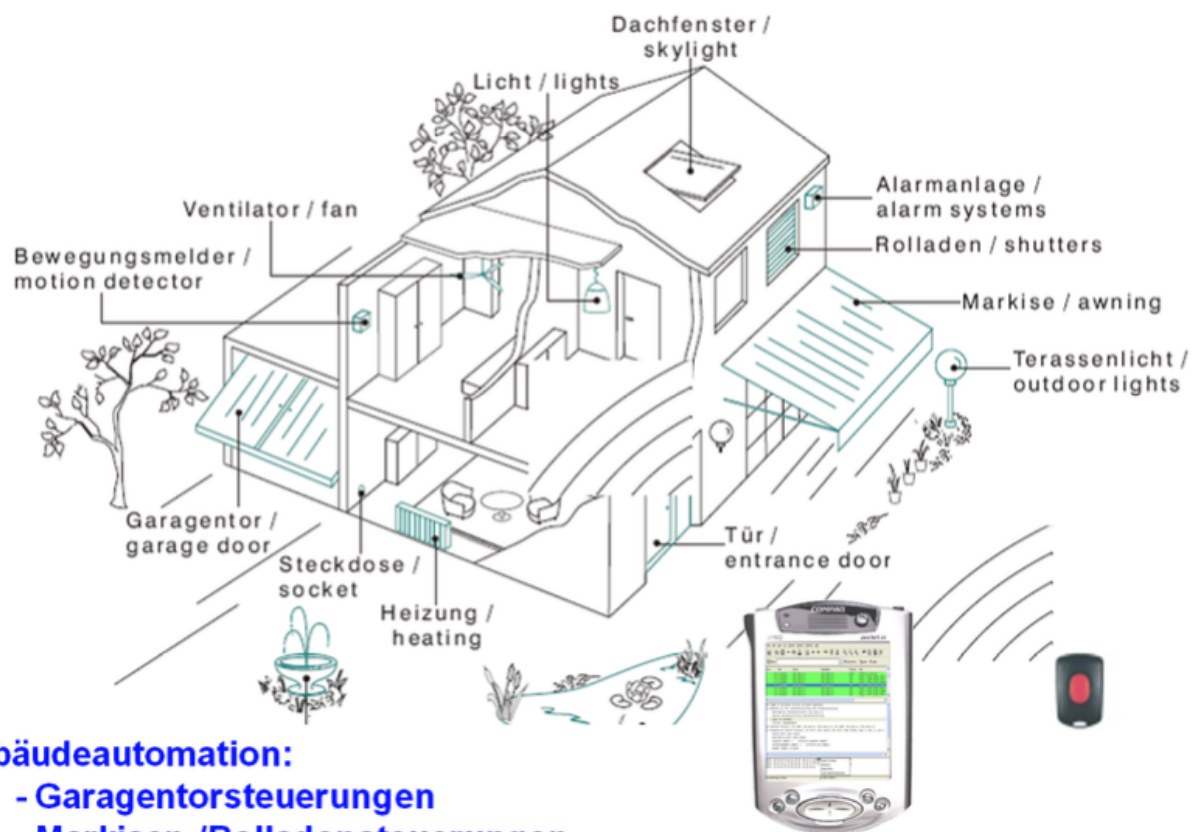


Bild: Lokale Überwachung / Monitoring der Vitalfunktionen

Anm.: Als Vitalfunktionen (lateinisch vita ‚Leben‘ und lateinisch functio ‚Verrichtung‘) werden in der Medizin die lebenswichtigen Vorgänge im Wachzustand, der Atmung und dem Kreislauf bezeichnet.



Gebäudeautomation:

- Garagentorsteuerungen
- Markisen-/Rolladensteuerungen
- Beleuchtungssteuerungen etc.

Bild: Multilink wireless networks in Home Automation (Bsp.1)

Automation Systems

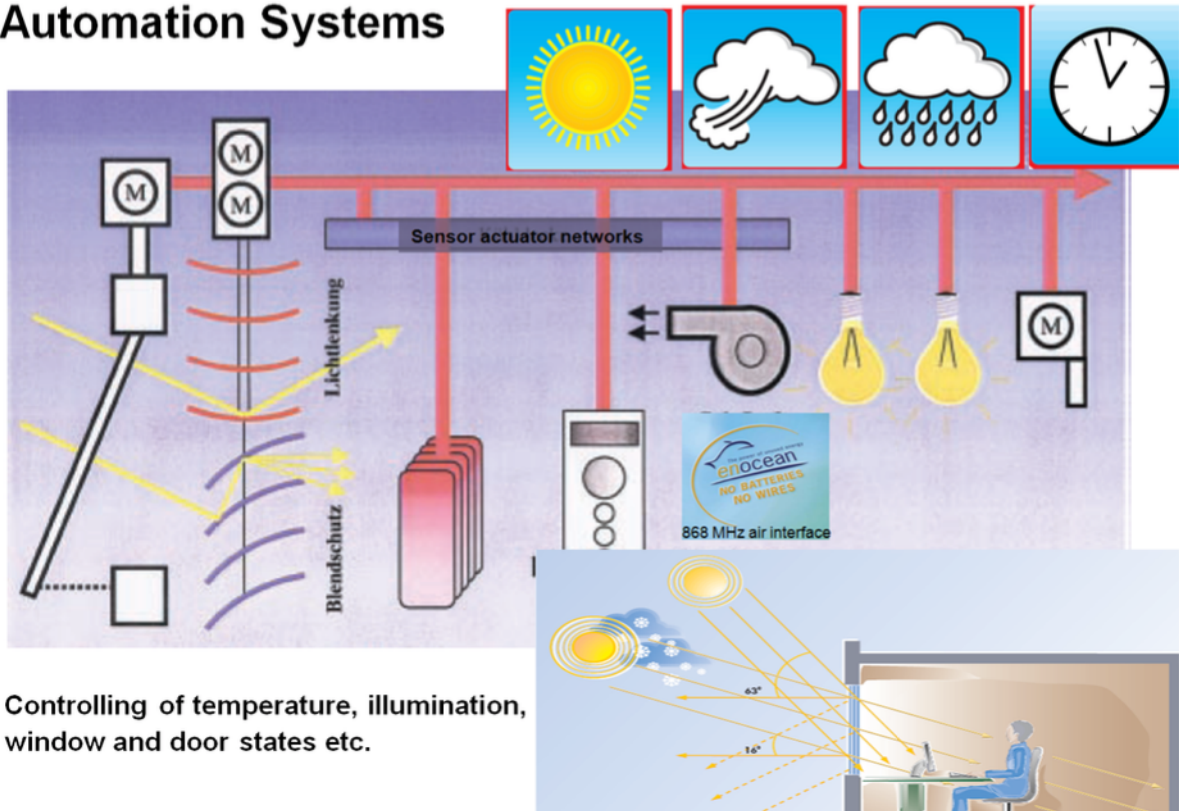


Bild: Multilink wireless networks in Home Automation (Bsp.2)

1.7. Topologien vernetzter drahtloser Systeme

Types of ZigBee Networks

ZigBee networks can be configured to operate in a variety of different ways to suit the application and environment. Supported topologies include:

Peer to Peer Network

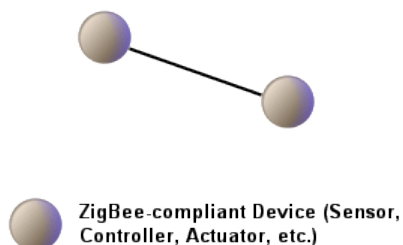
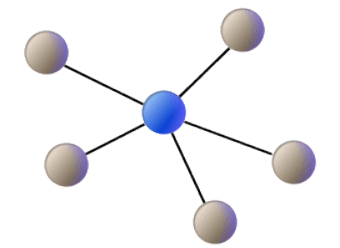


Figure: Peer to Peer (Ad-hoc)

ZigBee nodes connect directly to each other for peer to peer communication

Star Topology Network



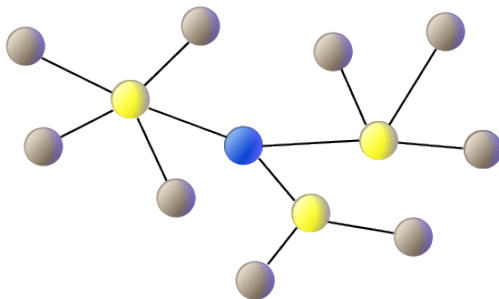
Reduced Function Device (Sensor, Controller, Actuator, etc.)

PAN Coordinator

Figure: Star Configuration

Using a single PAN coordinator, each node connects directly to the central coordinator – all inter-node communications are passed through the coordinator.

Cluster Network



Reduced Function Device (Sensor, Controller, Actuator, etc.)

PAN Coordinator

Full Function Device (Performs network routing functions)

Figure: Cluster Tree

A cluster tree network consists of a number of star networks connected whose central nodes are also in direct communications with the single PAN Coordinator.

Mesh Network

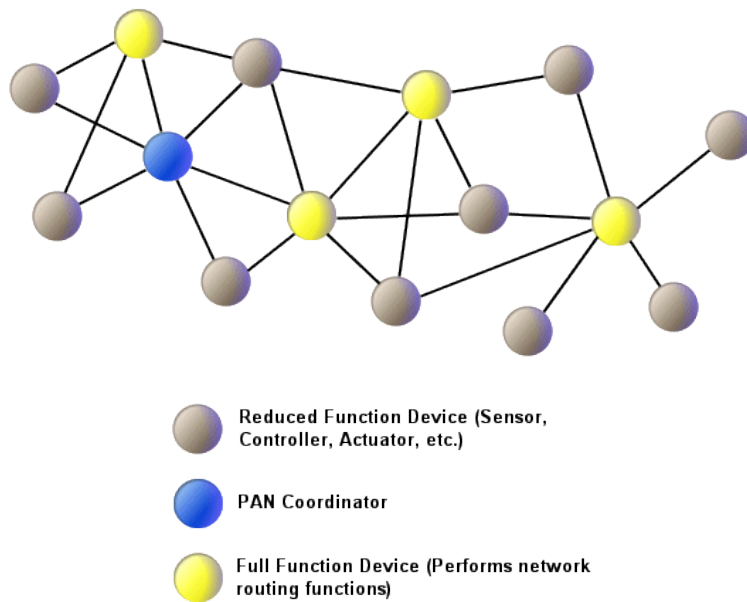


Figure: Mesh Network

Using a set of routers and a single PAN coordinator, the network is formed into an interconnected mesh of routers and end nodes which pass information from node to node using the most cost effective path. Should any individual router become inaccessible, alternate routes can be discovered and used providing a robust and reliable network topography.

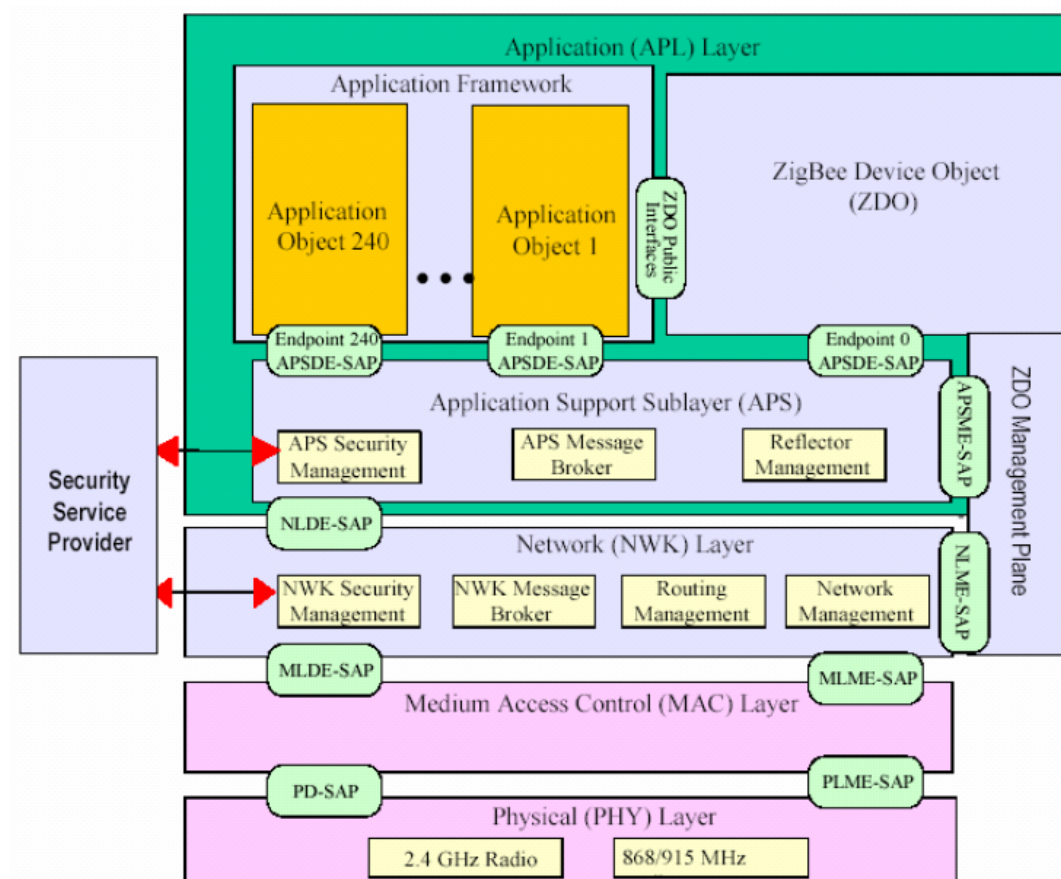
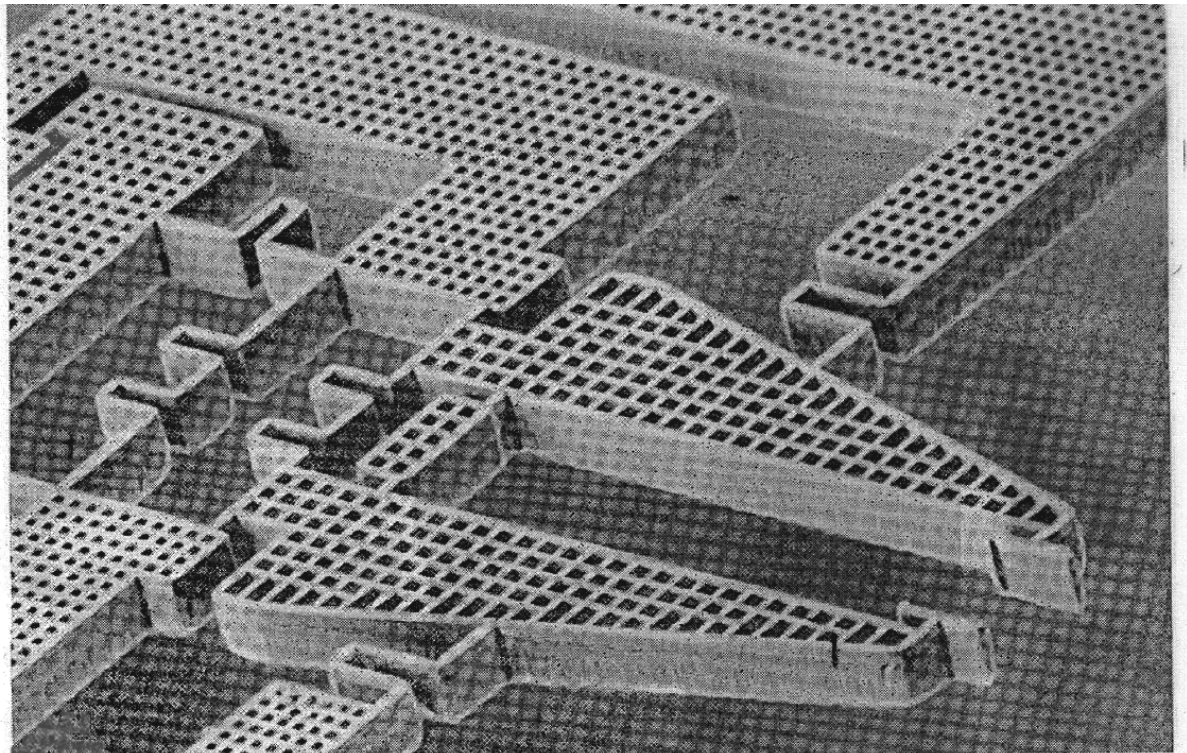


Bild: Architektur des ZigBee Protokollstack



Einen, winzigen Greifer haben kalifornische Wissenschaftler für die Montage von Mikrostrukturen entwickelt. Die Greifzangen auf der Abbildung sind etwa hundert Mikrometer weit geöffnet. Mit dem sogenannten Hexsil-Verfahren können solche Hohlstrukturen aus Polysilizium hergestellt und in einem weiteren Schritt mit dotiertem Polysilizium oder

mit Metall aufgefüllt werden. Allerdings besteht nicht die gesamte Konstruktion des Greifers aus gefüllten Hohlstrukturen, sondern nur der thermische Antrieb und die elektrischen Zuleitungen. Für die Finger und alle übrigen Teile ist dieser Aufwand nicht nötig. Das Füllmaterial für den thermischen Antrieb ist dotiertes Polysilizium, das sich im Gegen-

satz zum undotierten Polysilizium ausdehnt, sobald ein elektrischer Strom zu fließen beginnt. Dieser Effekt lässt sich dazu verwenden, über entsprechende Hebel die Finger des Greifers in Bewegung zu setzen. Die Hohlstrukturen für die elektrischen Zuleitungen des thermischen Antriebs werden dagegen mit Nickel aufgefüllt. Foto Universität von Kalifornien, Berkeley

Bild: Embedded Systeme in der Mikrosystemtechnik

1.8. Fragen zum Kapitel

1. Welche eingebetteten Systeme gibt es hier im Raum?
2. Mit welchen Embedded Systemen sind Sie heute bereits in Berührung gekommen?
3. Wo und in welchen Anwendungen werden zukünftig Embedded Systeme eingesetzt werden?

2. BEGRIFFSDEFINITIONEN UND ANFORDERUNGEN

2.1. Begriffsdefinition Embedded System

Anwendungsbeispiel Düsenstrahltriebwerk

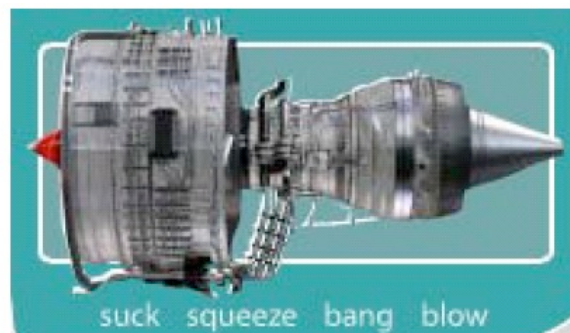


Bild: Aufbau eines Düsenstrahltriebwerkes

Funktionsweise siehe <https://www.youtube.com/watch?v=JxkJ-FwFeVI> 00:30 – 01:00

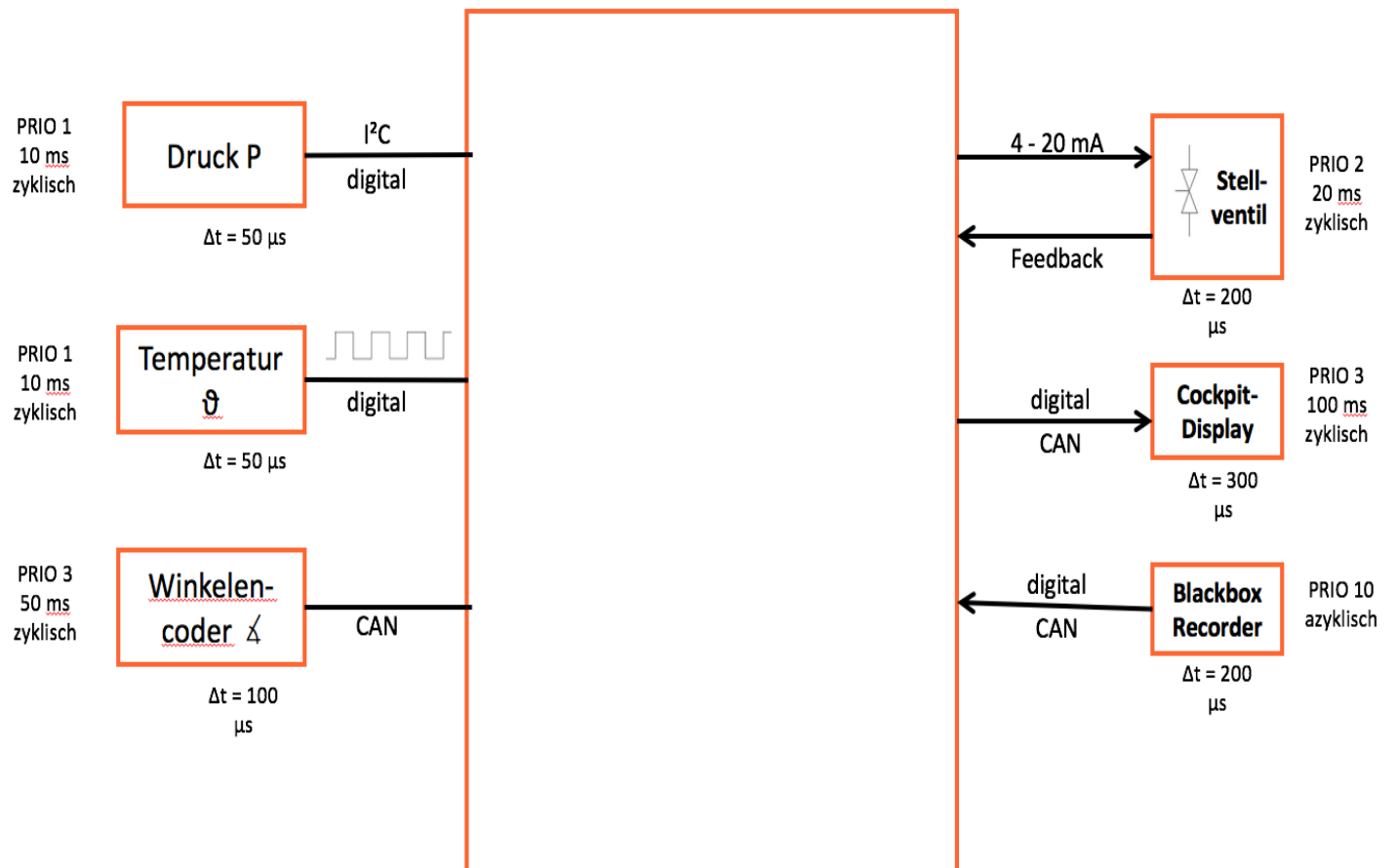


Bild: Aufbaustruktur eines Embedded Systems

Merkmale eines Embedded Systems?

Antwort:

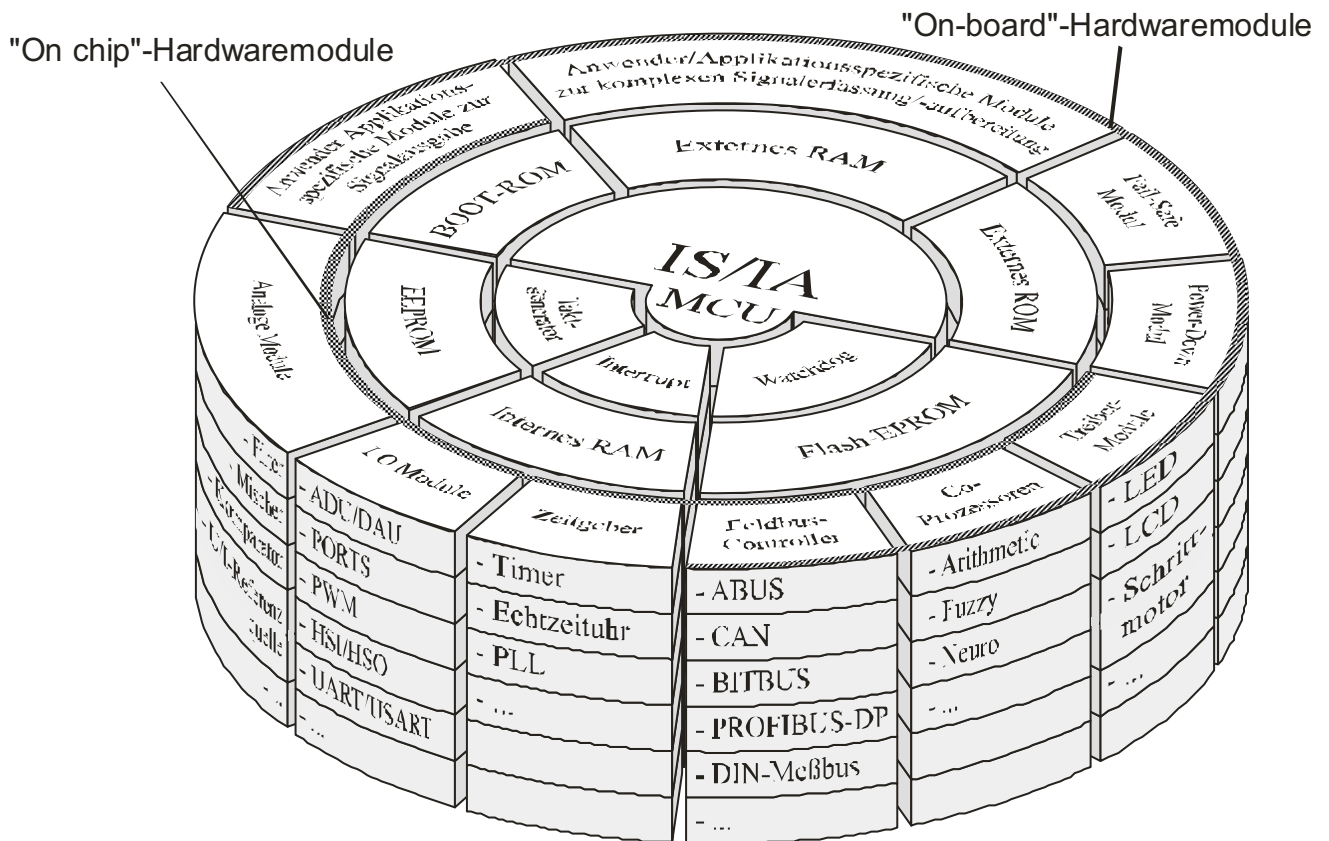


Bild: Komponentenmodell eines Embedded Systems

2.2. Eigenschaften und Anforderungen an ein Embedded System

2.3. Begriffsdefinition Echtzeitsystem (Real-time system)

DIN 44300 (Begriffe der Informationsverarbeitung, 1985):

Echtzeitbetrieb ist ein Betrieb eines Rechensystems, bei dem **Programme zur Verarbeitung anfallender Daten ständig betriebsbereit** sind, und zwar derart, dass die **Verarbeitungsergebnisse innerhalb einer vorgegebenen Zeitspanne verfügbar sind**. Die Daten können je nach Anwendungsfall nach einer zeitlich zufälligen Verteilung oder zu vorbestimmten Zeitpunkten anfallen.

John A. Stankovic, K. Ramamritham (University of Virginia):

What is predictability for Real-Time Systems? (1990): **Real-time systems** are those systems in which *the correctness of the system depends not only on the logical results of computations*, but also on the *time at which the results are produced*.

- ⇒ Ein Echtzeitsystem ist ein Rechensystem, dessen Korrektheit
 - nicht nur von der logischen Korrektheit der Ergebnisse selbst, sondern auch
 - von der Einhaltung vorgegebener Zeitbedingungen abhängt.
- ⇒ Die Nichteinhaltung der vorgegebenen Zeitforderungen ist gleichbedeutend mit dem Versagen des Systems. Bsp.: Herzschrittmacher

2.4. Anforderungen an ein Echtzeit-Betriebssystem

2.4.1. Rechtzeitigkeit

Rechtzeitigkeit, d. h. innerhalb garantierter Zeiten auf ein Signal von außen reagieren

- Rechnersystem muss mit den im technischen Prozess ablaufenden Vorgängen Schritt halten, um Zeitbedingungen erfüllen zu können.
- D. h. auf jedes Ereignis des Prozesses muss innerhalb einer vorgegebenen Zeitspanne ein entsprechendes Ergebnis an den Prozess zurück geliefert werden.
 - ⇒ das Ereignis muss in dieser Zeit erfasst, die Berechnungen ausgeführt und die Reaktion zum Prozess ausgegeben worden sein.
 - ⇒ Die Reaktionszeiten werden von dem technischen Prozess bestimmt und müssen für jeden auftretenden Ereignistyp spezifiziert werden.

Die Forderung nach Rechtzeitigkeit des Rechensystems bedeutet, dass

- die Eingabedaten **rechtzeitig** abgerufen werden müssen und
- dass die Ausgabedaten **rechtzeitig in Bezug auf die Anforderungen** des betreffenden technischen Prozesses verfügbar sein müssen.

Man unterscheidet zwei Kategorien von Zeitbedingungen:

- **Absolutzeit-Bedingungen**, z. B. um 13.30 Uhr soll ein Signal zur Abfahrt eines Zuges ausgegeben werden
- **Relativ-Zeitbedingungen**, z. B. nach dem Auftreten des Streckensignals soll das Stellsignal für die Weichenstellung innerhalb von 10 sec. ausgegeben werden

2.4.1.1 Arten der Rechtzeitigkeit (harte u. weiche Echtzeit)

Harte Echtzeit (Hard Real-Time): Nichteinhaltung der Zeitbedingung wird als Fehler gewertet und führt zu einem Versagen des Systems.

Hard Real-Time: Provides, without fail, a response to some kind of event within a specified time window (timeliness of result), **The response must be predictable and independent** of other operating System or application activities.

Stellt auf jeden Fall eine Antwort auf ein Ereignis innerhalb eines bestimmten Zeitfensters (Zeitnähe des Ergebnisses) bereit. Die Antwort muss vorhersehbar und unabhängig von anderen Aktivitäten des Betriebssystems oder anderen Anwendung sein.

Definition: Real-Time conditions, newsgroup.comp.realtime

Weiche Echtzeit (Soft Real-Time): Ergebnis kann nach Ablauf der Zeitbedingung noch verwendet werden, ist allerdings mit Nachteilen verbunden, z. B. Verspätung, höhere Kosten etc.. Zeitbedingung sollte nur selten überschritten werden.

Soft Real-Time: *It has reduces constraints on "Lateness", but still must operate quickly within fairly consistent time constraints. The response to the serviced events should be satisfactory, on average.*

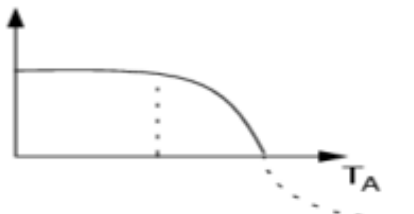
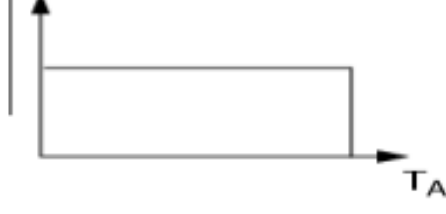
Die Anforderungen bzgl. "Verspätungen" sind reduziert, das System muss aber dennoch schnell und innerhalb eines einheitlichen Zeitrahmens arbeiten. Die Reaktion auf die eintretenden Ereignisse sollte im Durchschnitt zufriedenstellend sein.

Definition: Real-Time conditions, newsgroup.comp.realtime

Voraussetzungen:

Laufzeitverzögerungen (Latenzzeiten) durch Betriebssystemaufrufe müssen von dem Betriebssysteminnern her bekannt sein.

Maximale Verzögerungszeiten müssen durch den Betriebssystem-Hersteller garantiert und im laufenden Betrieb eingehalten werden.

Harte Echtzeit (hard real time)	Weiche Echtzeit (soft real time)	Nicht-Echtzeit (Best Effort)
- Definierte Zeitanforderungen: $P(T_A \leq T_{A,max}) = 1$ - Verletzung der Zeitanforderung führt zu einem unbrauchbaren Ergebnis oder sogar zu einem Schaden am System - deterministisch	- Definierte Zeitanforderungen, aber $P(T_A \leq T_{A,max}) \leq 1$ - Verletzung der Zeitanforderung ist für einen Teil der Ergebnisse tolerierbar - Das Ergebnis hat nach der Zeitüberschreitung noch einen gewissen Wert	- Kein $T_{A,max}$ vorgegeben, nur T_A minimiert - System trifft keine Vorkehrungen zur Einhaltung von Zeitschranken
		

Weitere allgemein gültige Aussagen für harte und weiche Echtzeit-Systeme

Gegenüber weichen Echtzeit-Systemen müssen **harte Echtzeit-Systeme eine höhere „Zeit-**

Qualität“ erfüllen. Sie sind charakterisiert durch:

- ⇒ Sie besitzen **mindestens eine** spezifizierte harte Echtzeitbedingung.
- ⇒ Harte Echtzeitbedingungen müssen unter allen Systembelastungen (auch bei Systemfehlern) eingehalten werden, ansonsten liegt ein Systemversagen vor.
- ⇒ Das Einhalten der Echtzeitbedingungen lässt sich trotz deterministischem Verhalten der Hardware, des Betriebssystems und der Applikationen **nicht vollständig testen**.
- ⇒ Harte Echtzeitbedingungen haben keine Korrelation mit schneller Verarbeitung oder hoher Prozessor-Leistung.
- ⇒ Harte Echtzeitbedingungen sind selten und treten im Allgemeinen verkoppelt mit weichen Echtzeitbedingungen auf.

Bei dem Design und der Bewertung von harten Echtzeitsystemen müssen nichtdeterministische Komponenten wie z. B. Hardware-Controller, Suchprozesse in Datenbanken etc. berücksichtigt werden.

Aufgaben mit hohen Echtzeitanforderungen und hoher Priorität

-
-
-
-

2.4.1.2 Vier Zeitbedingungen

Fall 1: Das Erfassen eines Messwertes oder die Ausgabe eines Stellsignals hat genau zu bestimmten Zeitpunkten zu erfolgen.

Fall 2: Eine Funktion hat innerhalb eines Zeitfensters zu erfolgen ($t_1 + t_{\text{Delta}}$) (Toleranzbereich).

Fall 3: Eine Funktion hat spätestens zu einem bestimmten Zeitpunkt zu erfolgen.

Fall 4: Eine Aktion darf frühestens zu einem bestimmten Zeitpunkt erfolgen.



2.4.2. Übung: Beispielanwendungen zu gegebenen Zeitbedingungen

Finden Sie Beispielanwendungen zu den nachstehenden Zeitbedingungen.

Fall 1: Das Erfassen eines Messwertes oder die Ausgabe eines Stellsignals hat genau zu bestimmten Zeitpunkten zu erfolgen

Fall 2: Eine Funktion hat innerhalb eines Zeitfensters zu erfolgen ($t_1 + t_{\text{Delta}}$) (Toleranzbereich)

Fall 3: Eine Funktion hat spätestens zu einem bestimmten Zeitpunkt zu erfolgen

Fall 4: Eine Aktion darf frühestens zu einem bestimmten Zeitpunkt erfolgen

Weitere Beispiele:

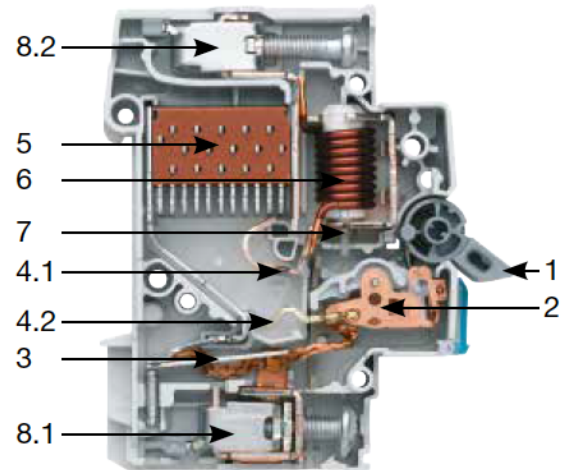
Leitungsschutzschalter

In DIN VDE 0100-430 sind die Anforderungen für den „Schutz von Kabeln und Leitungen bei Überstrom“ festgelegt.

Sicherungsautomaten dienen dem Kabel- und Leitungsschutz in der Installation. Sie sollen selbsttätig abschalten, sobald der Strom **durch Anstieg** und **Zeitdauer** eine für die Leitung oder ein Betriebsmittel **zu hohe Erwärmung** erzeugt.

Wichtige Elemente eines Leitungsschutzschalters:

- 1 Kipphebel mit Schaltschloss
- 2 Freilauf – für den Fall, dass der Kipphebel in der Ein-Position blockiert ist
- 3 Bimetall – Thermisches Auslösesystem
- 4.1 Schaltkontakt (fest)
- 4.2 Schaltkontakt (beweglich)
- 5 Funken-Löschkammer
- 6 Elektromagnet – Elektromagnetisches Auslösesystem
- 7 Schlaganker
- 8.1 Anschlussklemme (Bi-Connect)
- 8.2 Anschlussklemme



Die **Auslösecharakteristik** definiert den Strombereich, bei dem der **elektromagnetische Auslöser** den Stromkreis unterbricht.

Die **thermische Auslösung** ist bei allen Charakteristiken identisch. Aufgrund von Temperaturveränderungen und der Mechanik handelt es sich bei der ganzen Auslösekurve um einen Bereich und nicht um einen festen Wert.

Bei einem großen Überstrom beträgt die Auslösezeit ca. 10 Millisekunden. Schnellere Auslösezeiten sind nicht möglich, da der Mechanik physikalische Grenzen gesetzt sind.

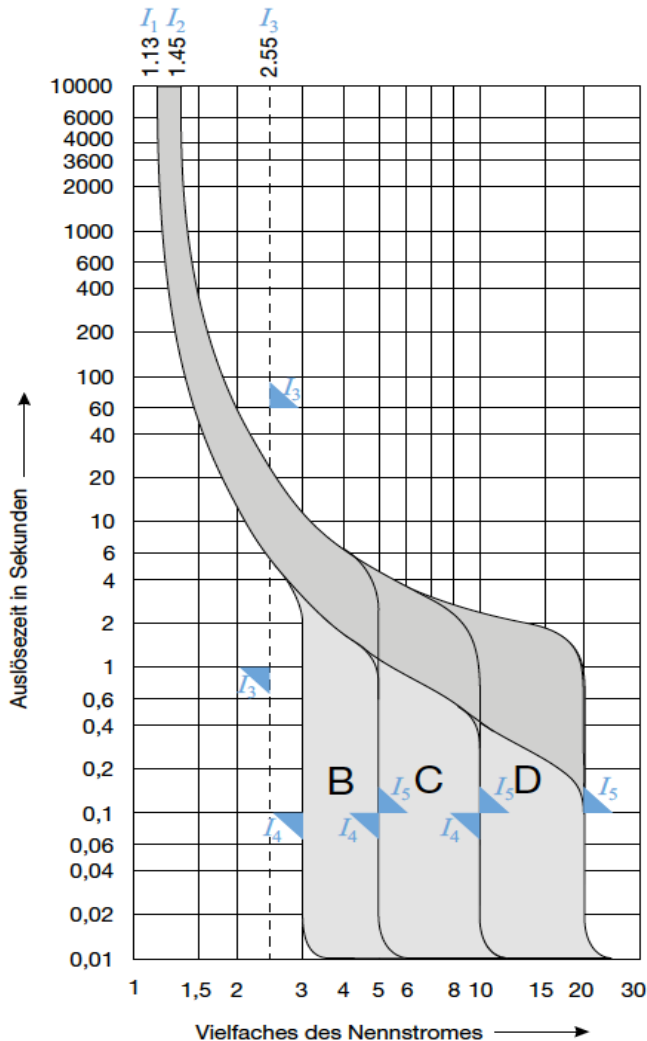


Bild: Auslösecharakteristik LS B, C, D

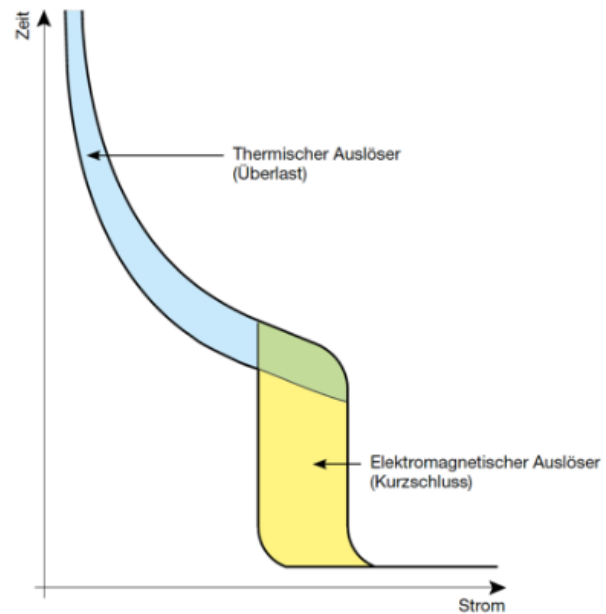


Bild: Zeit / Strom Auslösecharakteristik eines LS

I_1 Kleiner Prüfstrom

Der bis zu 1,13 fache Nennstrom darf nicht vor einer Stunde ausgeschaltet werden.

I_2 Grosser Prüfstrom

Der ab oder grösser gleich 1,45 fache Nennstrom muss spätestens nach einer Stunde ausgeschaltet werden.

I_3 Prüfstrom Toleranzeingrenzung (auch undefinierter Auslösebereich genannt)

Die Auslösekurve des thermischen Auslösers muss innerhalb der Toleranzeingrenzung sein.

Dieser Prüfstrom ist für die Dimensionierung und Planung nicht relevant.

I_4 Haltestrom

Der elektromagnetische Auslöser darf bis zu diesem Strom nicht ansprechen, z. B. den Einschaltstrom.

I_5 Auslösestrom

Der elektromagnetische Auslöser muss bei diesem Strom innert 0.1 s ansprechen, z. B. einen Kurzschlussstrom.

Auslösebedingungen für Sicherungsautomaten für AC-Anwendungen

Auslösecharakteristik	B	C	D		K	Z	
Norm	DIN EN 60898-1, -2 (VDE 0641-11, -12)				in Anlehnung an DIN EN 60947 (VDE 0660)		
Zeit-Strom-Kennlinie*	$x I_n$ (30 °C)			Zeiten	$x I_n$ (20 °C)		Zeiten
Nicht-Auslösen	1,13			1 h ($I_n \leq 63$ A)	1,05		1 h
Auslösen	1,45			1 ... 60 s ($I_n \leq 63$ A)	1,2		1 h
Auslösen	2,55			1 ... 60 s ($I_n \leq 32$ A)	1,5		< 120 s
					6		> 2 s
Sofort-Auslösung (magn.)	$x I_n$			Zeiten	$x I_n$		Zeiten
Nicht-Auslösen	3	5	10	0,1 s	10	2	0,2 s
Auslösen	5	10	20	0,1 s	14	3	0,2 s

* Für abweichende Umgebungstemperaturen gelten Reduktionsfaktoren!

Der elektromagnetische Auslöser beim Typ B löst bis zum 3-fachen Nennstrom nicht aus und ab dem 5-fachen Nennstrom sicher innerhalb von 0,1 Sekunden.

Auslösecharakteristik	Halten I_4	Auslösen I_5	Auslösezeit
B	$3 \times I_n$		> 0.1 s
		$5 \times I_n$	< 0.1 s
C	$5 \times I_n$		> 0.1 s
		$10 \times I_n$	< 0.1 s
D	$10 \times I_n$		> 0.1 s
		$20 \times I_n$	< 0.1 s
CS (SLS)	$6.5 \times I_n$		> 0.1 s
		$10 \times I_n$	< 0.3 s
E (SLS)	$5 \times I_n$		> 0.1 s
		$6.25 \times I_n$	< 0.3 s

Bild: Elektromagnetischer Auslöser – mag. Sofort-Auslösung

Aufgabe: Geben Sie die Echtzeitbedingungen für die obige Anwendung an.

Fehlerstrom-Schutzeinrichtungen



Das Auslöseverhalten zeigt auf, dass der kurzzeitverzögerte RCD Typ A HI unempfindlicher ist gegenüber Fehlerströmen als der RCD Typ A (unverzögert). Bei beiden Versionen wird der Personenschutz eingehalten.

Die Auslösung der Version selektiv 300 mA S erfolgt klar später und ist somit selektiv zu einem RCD mit 30 mA.

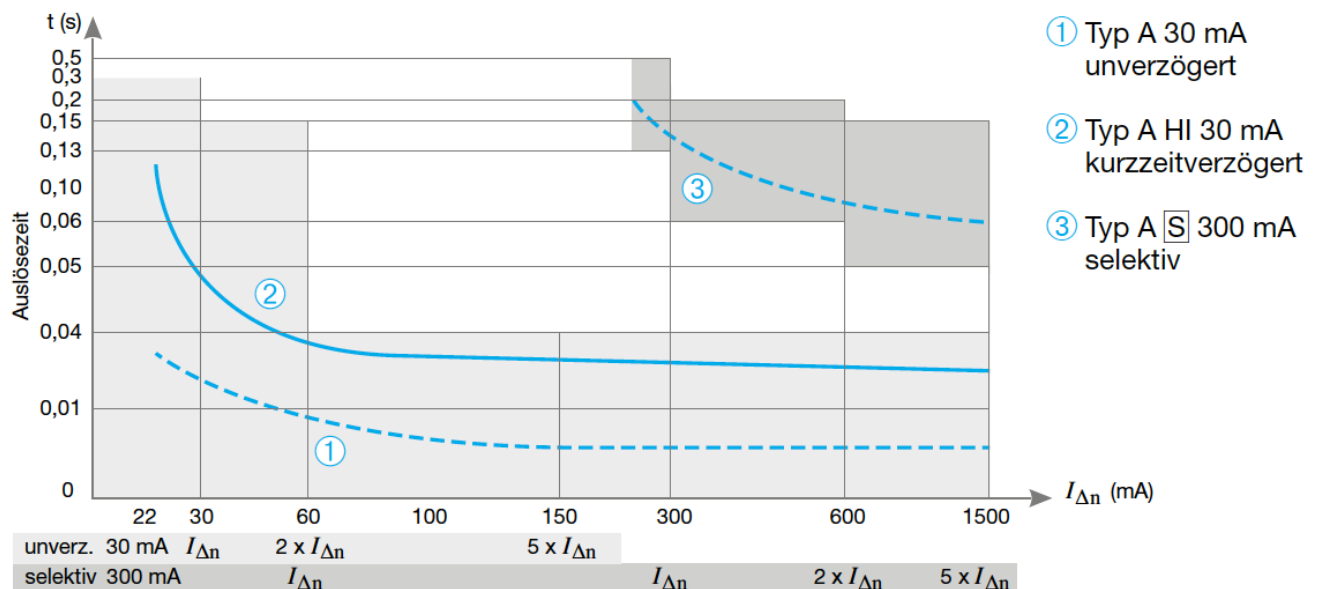


Bild: Auslöseverhalten von Fehlerstrom-Schutzeinrichtungen (RCD)

Aufgabe: Geben Sie die Echtzeitbedingungen für die obige Anwendung an.

2.4.3. Gleichzeitigkeit (Multitasking)

Gleichzeitigkeit gewährleistet, dass die Rechenprozesse alle (quasi-) gleichzeitig vom Rechner (Prozessor) abgearbeitet werden.

Die **Forderung nach Gleichzeitigkeit (Multitaskingfähigkeit)** ergibt sich aus der Tatsache, dass Echtzeit-Rechensysteme auf Vorgänge in ihrer Umwelt reagieren müssen, die gleichzeitig ablaufen.

Beispiele:

- ⇒ **Schienenverkehrssystem:** Ein Prozessrechensystem muss auf die gleichzeitige Fahrt mehrerer Züge reagieren und entsprechend die Weichenstellung vornehmen, d. h. gleichzeitig anfallende Messwerte müssen erfasst und ausgewertet werden und ggf. müssen auch gleichzeitig mehrere Stellsignale ausgegeben werden.
- ⇒ **Rechnerbetrieb:** Hier möchte man ein Programm editieren, d. h. am Bildschirm eingeben, ändern etc. und gleichzeitig ein Peripheriegerät, hier Drucker betätigen. Der Prozessor muss so zwischen dem Editier- und dem Druckprozess ständig hin und her schalten.

Ein Rechnersystem muss die Fähigkeit besitzen, mehrere gleichzeitig ablaufende Vorgänge eines Systems zu bearbeiten.

Probleme:

1. **Computerprogramme** werden typischerweise als eine Folge sequentieller Anweisungen ausgeführt.
2. **Computersprachen** unterstützen überwiegend den sequentiellen Ablauf von Instruktionen, vgl. Assembler, C etc.
3. Eher **sequentielle Prozessorlogik statt parallele Logik** in den CPUs vorherrschend, ungeachtet dessen, ob John von Neumann- oder Harvard-Architektur vorliegt
4. Meist nur **ein Prozessor** in Embedded Systemen wirksam

Wie lässt sich Gleichzeitigkeit herstellen? Möglichkeiten der Realisierung:

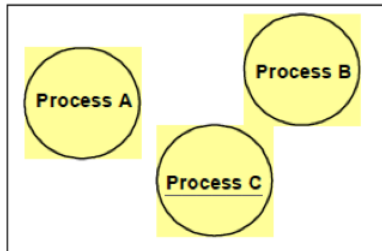
1. **Parallel arbeitende hardwarebasierende Einheiten** (mehrere Prozessoren, Co-Prozessoren, Eingabe/Ausgabe-Einheiten etc.)
 - z. B. PC mit Hardware-Druckerbox (Spooler). In diesem Fall arbeiten dann die Programme des so entstehenden Mehrrechner-Systems echt parallel und gleichzeitig. (Nachteil: sehr teuer)
2. **Nebenläufige Abarbeitung von Rechenprozessen auf einem Prozessor**

Die Forderung nach Gleichzeitigkeit kann auch mit einem einzigen Computer näherungsweise erfüllt werden. Voraussetzung: Programmabarbeitungszeiten müssen im Vergleich zu den Vorgängen in der Prozess-Umwelt sehr klein sein.

Durch **schnelles und intelligentes Umschalten** in kurzen Zeitabständen gelingt es so, gleichzeitig ablaufende Vorgänge in der „Umwelt“ gleichzeitig zu bedienen.

Multitasking

Processor



Process Swapping

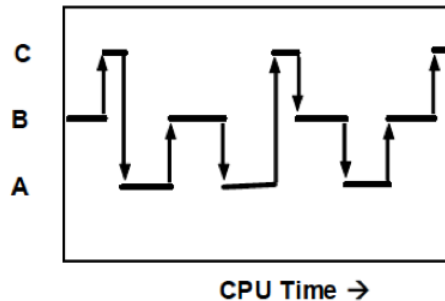


Bild: Multitasking mittels Process Swapping

3. **Zeitscheibenverfahren (Time-Slicing)**, d. h. die Prozessorleistung wird einem Task nur für die Zeit einer kurzen Zeitscheibe (Time Slice) zur Verfügung gestellt. Nach Ablauf der Zeitscheibe geht die Rechenzeit an den nächsten Prozess oder Thread über, usw.

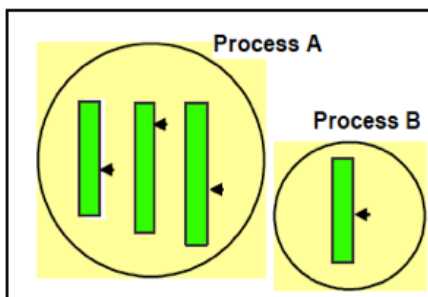
=> Einsatz eines Prozess-Schedulers, der die zeitliche Ausführung mehrerer Prozesse in Betriebssystemen regelt.

Lt. Definition <http://de.wikipedia.org/wiki/Prozess-Scheduler>

Ein Prozess-Scheduler ist eine Arbitrationslogik, die die zeitliche Ausführung mehrerer Prozesse in Betriebssystemen regelt. Prozess-Scheduler können grob in unterbrechende (präemptiv) und nicht unterbrechende (non-preemptive, auch als kooperativ Scheduler bezeichnet) aufteilen.

Multitasking und Multithreading

Processor



Process Swapping u. Time-Slicing

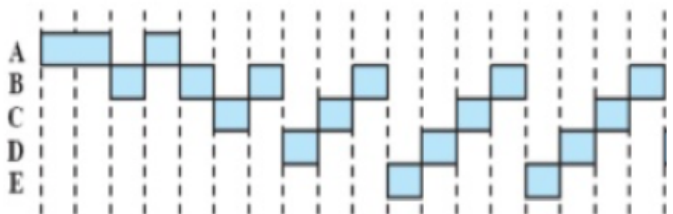


Bild: Multitasking u. Multithreading mittels Process Swapping u. Time-Slicing

2.4.4. Vorhersehbarkeit (Determinismus)

Vorhersehbarkeit: Das Systemverhalten muss auch bei zufällig anfallenden Ereignissen **deterministisch und vorhersagbar** sein.

Bei einem **deterministischen Betrieb** treten nur definierte und reproduzierbare Zustände auf. Auf die gleichen Eingaben folgen auch immer die gleichen Ausgaben, zusätzlich wird die gleiche Folge an Zwischenzuständen durchlaufen. Zu jedem Zeitpunkt ist der nachfolgende Abarbeitungsschritt des eindeutig festgelegt.

Auch **bei gleichzeitigem Auftreten mehrerer Ereignisse**, die zu einer Konkurrenzsituation bezüglich der Verarbeitung führen, muss das Verhalten des Systems bekannt und **vorhersagbar** sein.

Die Bearbeitung der einzelnen Ereignisse darf nur in einer vorhersagbaren Form verzögert werden. Die Reaktionszeiten müssen weiterhin in den Reaktionszeitgrenzen liegen.

Umsetzung:

- ⇒ Nicht formal nachweisbar, wenn periodische Anforderungen (diese sind einplanbar) und unbekannte, spontane Anforderungen (Signale, Alarme, Interrupts) gemischt auftreten.
- ⇒ Logische und zeitliche Korrektheit einzelner Routinen reichen nicht aus, da Nebenläufigkeit und zufällige Verteilung von Ereignissen die Verifikationsmöglichkeit beschränken.
- ⇒ Analyse für den schlechtesten Fall d. h. für maximal zu erwartenden Signalraten, ist notwendig („worst case“ Betrachtung). Hierzu müssen die Programmlaufzeiten bekannt sein.

2.4.4.1 Alternative: Empirische¹ Untersuchung des Lastverhaltens

Auf RTOS basierende Embedded Systeme werden z. B. mit dem Stress Workload Generator mit verschiedenen Lastarten beansprucht.

(The University of Oklahoma, Stress Workload Generator,
<http://weather.ou.edu/apw/projects/stress/>)

¹ Empirisch - Eine wissenschaftliche Vorgehensweise betreffend, die nicht auf theoretischen Begründungen, sondern auf nachvollziehbaren Erfahrungen und beschreib- bzw. messbaren Beobachtungen beruht. (z. B. Experiment).

Hier wird zwischen Rechenlast, Harddisk Transfer, Buslast (Netzwerk-/ Feldbus-kommunikation) und den Zugriffen auf die Subsysteme (Input/Output-Aktivität) unterschieden.

- Die Rechenlast, auch als Algorithmische CPU Last bezeichnet, wird durch das fortgesetzte Berechnen der Wurzel aus Zufallszahlen über den Aufruf *sqrt (rand())* erzeugt.
- Harddisk Transfers werden durch Festplattenzugriffe erzeugt.
- Das Memory Subsystem wird durch das stetige Allokieren *malloc()* und Freigeben *free()* von Speicherbereichen stimuliert.

Die RTOS wurden durch einzelne Lasten oder eine Kombination mehrerer Lasten beansprucht und daraus sog. Lastprofilen definiert.

Anwendungsbeispiel Lastprofil 1: Kommunikations-Gateway

10 % Algorithmische CPU Last
30 % Harddisk Transfer
55 % Netzwerk-/ Feldbuskommunikation
5 % I/O-Aktivität

100 % Auslastung

Bild: Diagramm zum Lastprofil 1

2.4.4.2 Übung: Lastprofil Software Interpolator

Eine numerische Bahnsteuerung soll jede Millisekunde eine Koordinate $f(x_j)$ ausgeben. Als Steuerdaten erhält sie alle 10 ms eine Koordinate x_j als Stützstelle für ein Polynom zweiten Grades mit $f(x) = ax^2 + bx + c$.

Die Bestimmung eines Zwischenwertes (jede 1 ms) benötigt 0,2 ms.

Die Bestimmung neuer Polynomkoeffizienten (alle 10 ms) benötigt 2,0 ms.

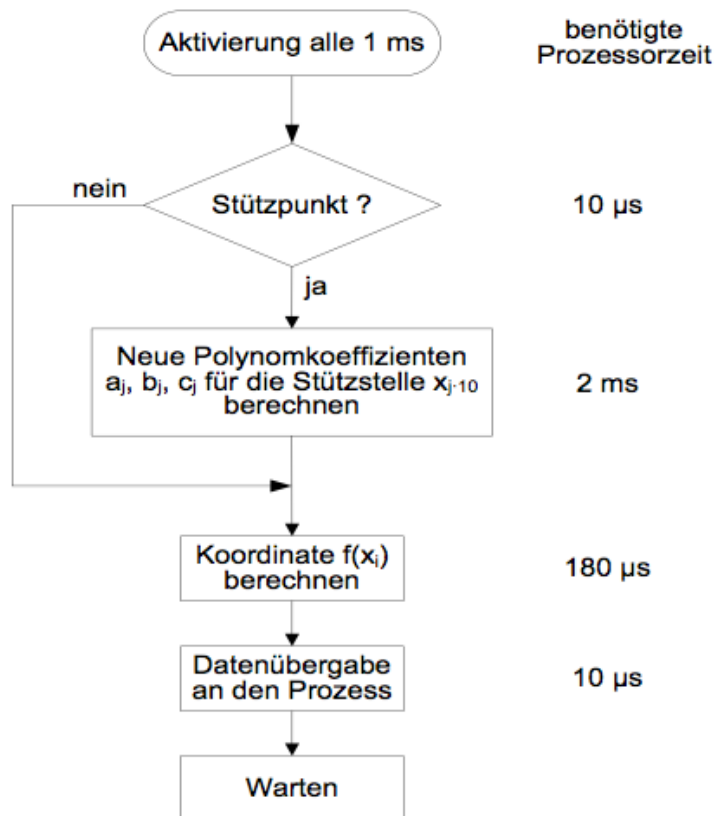


Bild: Ablaufdiagramm Software Interpolator

Bild: Prinzip Interpolation / Extrapolation

Die Kommunikation mit den Achsen x, y, z in der Prozessperipherie erfolgt nach dem Prinzip eines „verteilten Schieberegisters“. Das nachfolgende Bild zeigt dieses Schema:

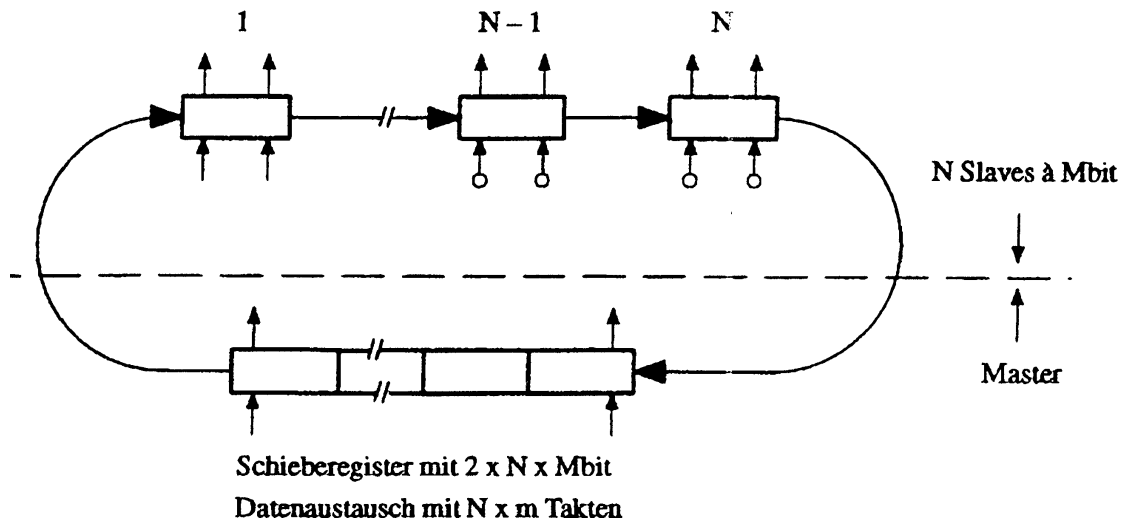


Bild: Ringleitungssystem nach dem Prinzip eines verteiltes Schieberegisters

In jedem Teilnehmer sind Schieberegister mit M bit, die Teilnehmer werden zu einem Gesamt-Schieberegister von $N \times M$ bit verbunden. auf Seiten des Masters gibt es ebenfalls ein Schieberegister derselben Länge. Zu Beginn des Zeitrahmens wird die Eingabe-Information parallel in die Schieberegister übernommen, es folgt ein Schiebevorgang mit $n \times M$ Takten, danach steht die Information an den entsprechenden Ausgängen der Teilnehmer und des Master-Schieberegisters zur Verfügung.

Aufgabe:

Definieren Sie ein Lastprofil zum Anwendungsfall *Software Interpolator*.

- % Algorithmische CPU Last
- % Harddisk Transfer
- % Netzwerk-/ Feldbuskommunikation
- % I/O-Aktivität

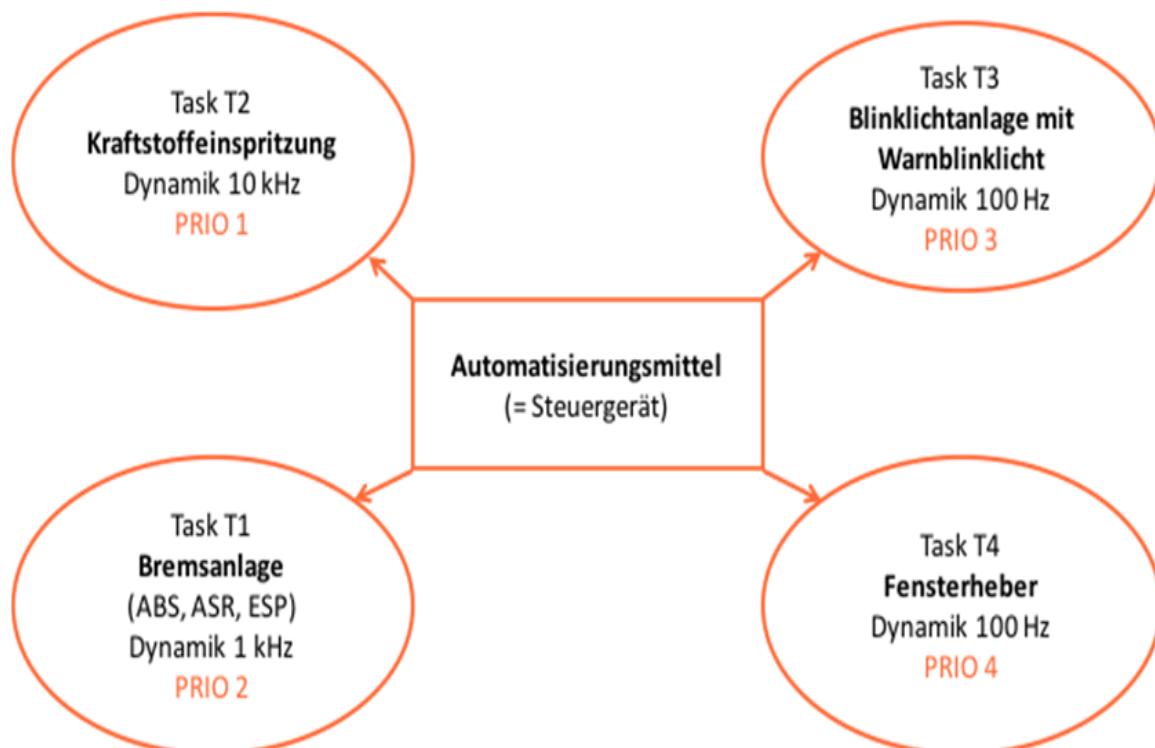
100 % Auslastung

2.4.5. Zusammenfassung Anforderungen an ein Echtzeitsystem:

- **Gleichzeitigkeit: Bereitstellung paralleler Prozesse**
Alle Rechenprozesse werden (quasi-)gleichzeitig vom Prozessor abgearbeitet.
- **Betriebsmitteleinplanung mit Echtzeitverfahren (Kernproblem: Prozessor,...)**
- **Interaktion der Prozesse (Kommunikation, Synchronisation)**
- **Rechtzeitigkeit: Reaktion innerhalb garantierter Zeiten auf Signale von außen**
z.B. kurze Reaktionszeiten auf einen Interrupt.
- **Prozesse müssen zugunsten wichtigerer Prozesse unterbrechbar sein**
- **Ein-/Ausgabeabwicklung mit determiniertem Zeitverhalten**
- **Zeitfunktionen, Uhrensynchronisation**

3. GLEICHZEITIGE UND PARALLELE AUFGABENBEARBEITUNG

Anwendungsszenario: Automatisierung im PKW



Σ Automatisierungsmittel = Automatisierungssystem (Steuerungssystem)

Anforderungen an das Steuergerät
/Embedded System

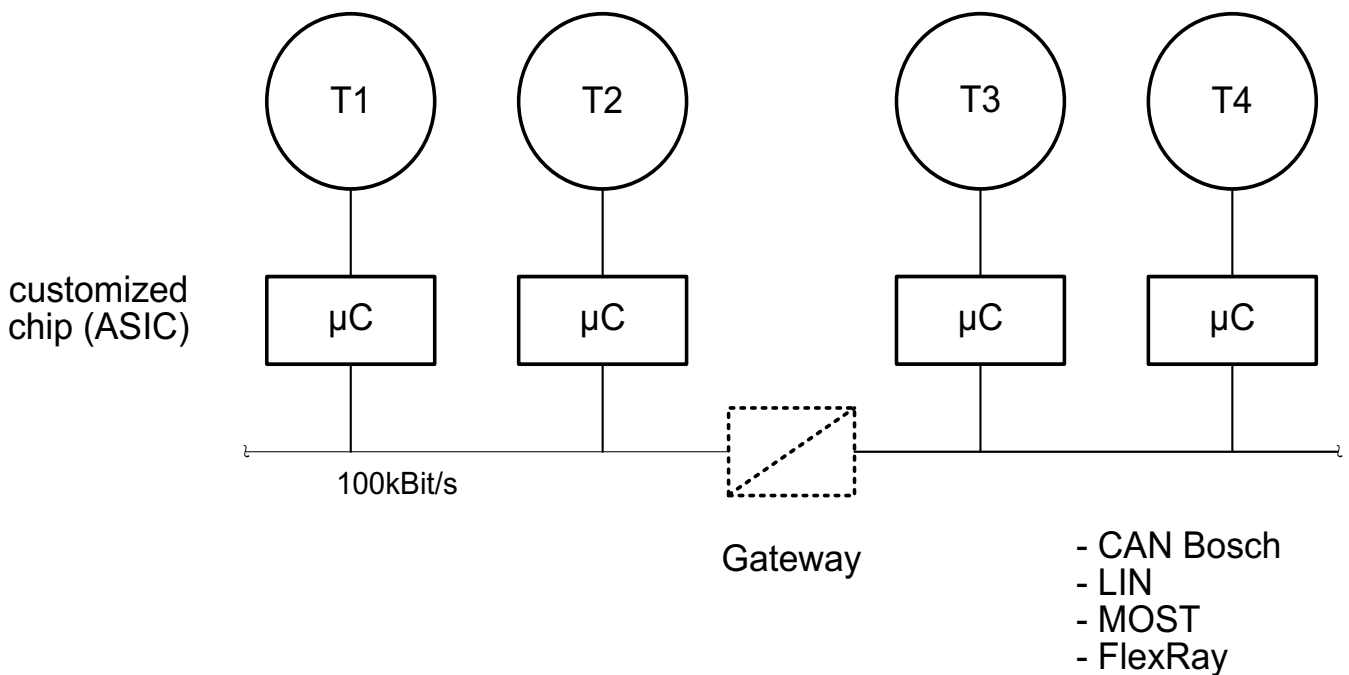
Gegebenheiten

Geben Sie im Folgenden mögliche Systemarchitekturen an, die möglichst viele dieser Anforderungen an das Gesamtsystem erfüllen.

Gesucht werden Lösungsarchitekturen zur parallelen und gleichzeitigen Aufgabebearbeitung.

3.1. Lösungsvariante 1: Einsatz eines Mehrprozessorsystems

Prinzip: Für jede Aufgabe wird ein separates Steuergerät verwendet, also dedizierte Systeme, alternativ Multi-Core Systeme)



Vorteile

- Transparente Aufbaustruktur
- Einfaches Handling im Reparaturfall, z. B. Austausch des kompletten Steuergerätes
- Parallele Entwicklung von Steuereinheiten möglich
=> Concurrent Engineering (=verteilte gleichzeitige Aufgabenbearbeitung -> time-to market
=> Möglichkeit für Zukaufteile -> time-to-market

Anm.: Simultaneous Engineering (im Amerikanischen: Concurrent Engineering, auf Deutsch: „verteilte gleichzeitige Entwicklung“) bezeichnet eine Vorgehensweise in der Produktentwicklung, Damit wird die Entwicklungszeit eines neuen Produktes verkürzt

Nachteile

- Teure Lösung wegen verteilter Multiprozessorarchitektur
- Problem der externen Kommunikation wie Konformität, Interoperabilität, verschiedene Bussysteme und Netzkoppler (Hub, Bridge, Switch, Gateway)

Einsatz von Netzkopplern

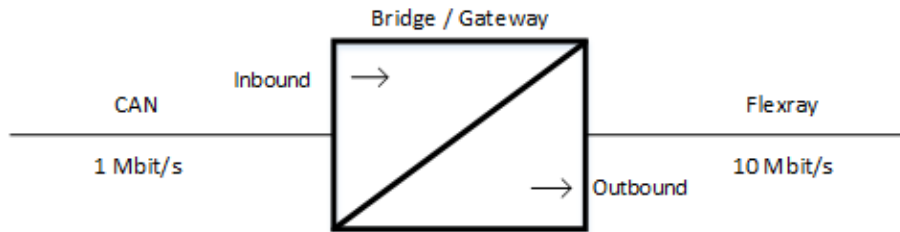


Bild: Gateway im Automotive Bereich

Vorzüge des Einsatzes von Netzkopplern

- Netzkoppler separiert Kommunikationsflüsse -> Bandbreitenerhöhung, z. B. statt 1 Mbit/s -> 2 Mbit/s
- Fehler bleiben auf das jeweilige Segment begrenzt wie z. B. Kollisionen, Leitungskurzschlüsse, Reflexionen am Leitungsende oder bei Unterbrechungen etc. -> erleichtert die Fehlersuche
- Filterung von Datenströmen, d. h.
 - defekte Telegramme werden vom Bus entfernt,
 - Access Listen auf Inbound / Outbound Interfaces erlauben einen gerichteten und gefilterten Datenverkehr
- Anpassung von unterschiedlichen Übertragungsraten, z. B. Ethernet 10 MBit/s <-> CAN 32,5 kBit/s
- Ankopplung unterschiedlicher Bussysteme mit unterschiedlichen Protokollen, unterschiedlichem Telegrammaufbau und spez. Verhalten, z. B. LIN <-> CAN <-> FlexRay

3.2. Lösungsvariante 2: Ereignisgesteuerte Aufgabenbearbeitung

Prinzip:

Signaländerung am Eingangsport führt zu einer Ressourcenzuweisung (Rechenzeit, Arbeitsspeicher, SSD, Timer, CPU Core u. weitere Betriebsmittel) und zum Aufruf der Anwendertask (IST).

Behandlung der Ereignisse erfolgt zunächst durch eine Interrupt Service Routine (ISR) und schließlich mittels einer nachgeschalteten Anwendertask (Interrupt Service Task = IST).

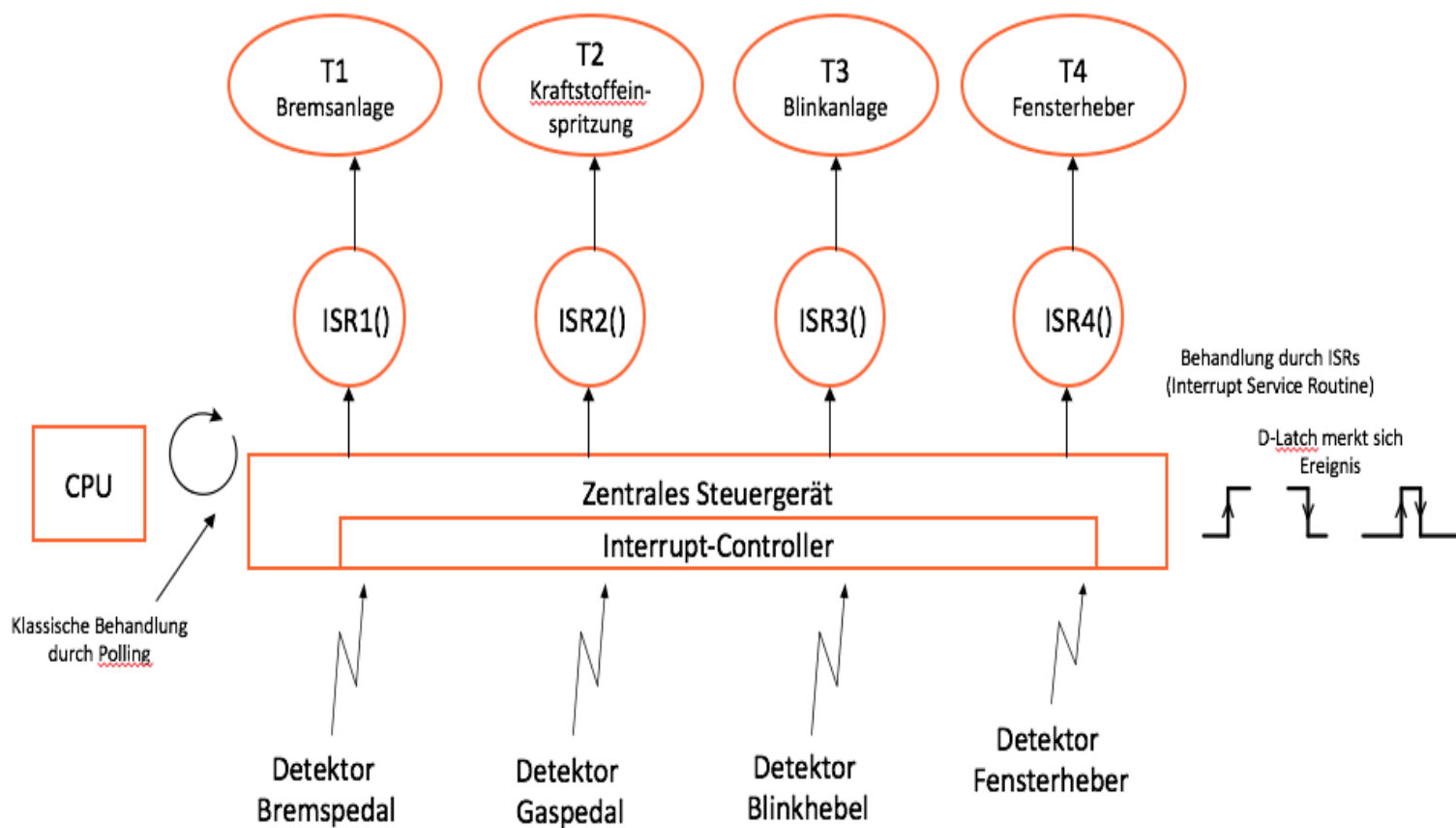
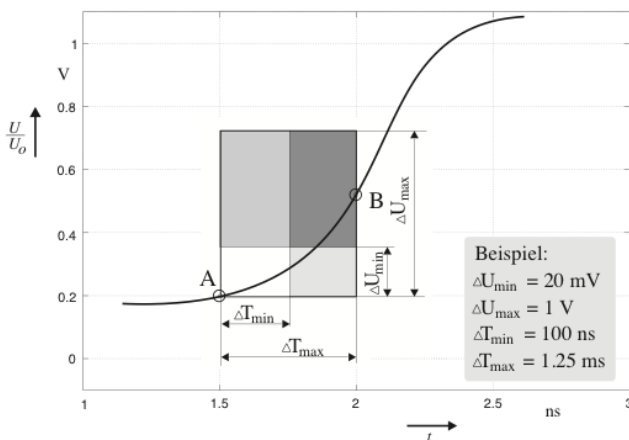


Bild: Prinzipbild einer ereignisgesteuerten Aufgabenbearbeitung

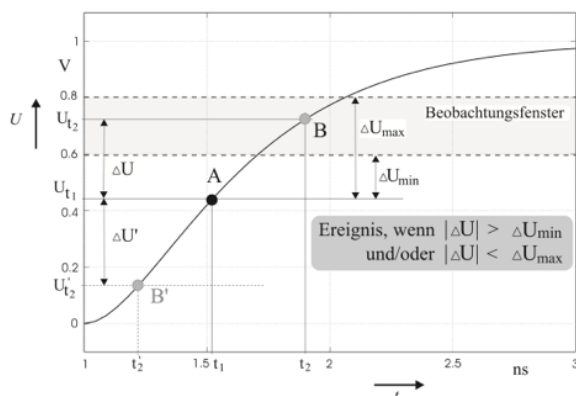
3.2.1. Definition Ereignisse

- Signaländerungen (Level)
- Grenzwertüber- bzw. unterschreitungen LOW_LOW_LIM, LOW_LIM, HIGH_LIM, HIGH_HIGH_LIM
- Signalfrequenzänderung
- Signal-Gradienten-Änderung in positiver oder negativer Richtung
- Signalzustand in Kombination mit einem Zeitfenster (vgl. Trendbeobachtung)

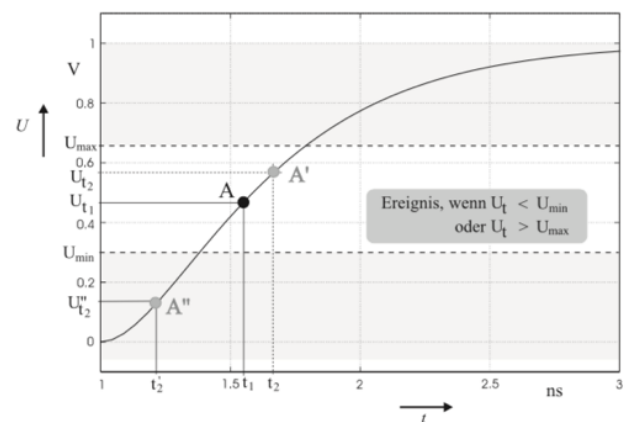
Ereigniskriterien:



a) Zeitvarianzkriterium



b) Ortsvarianzkriterium



c) Gradientenkriterium

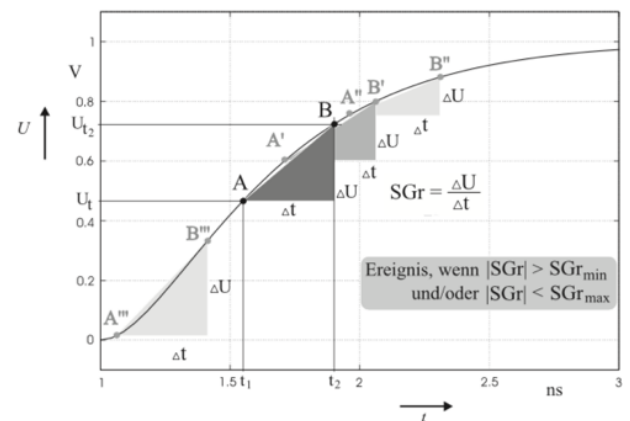


Bild: Ereignisgesteuerte Aufgabenbearbeitung

3.2.2. Bewertung der Lösungsvariante 2: Ereignisgesteuerte Aufgabenbearbeitung

Vorteile

- Gute Priorisierung der Aufgaben möglich durch Konfiguration der Prioritäten des PICs (Programmable Interrupt Controller)
- Preiswerte Ein-Prozessor- bzw. Single Chip-Lösung (SoC)
- Freigewordene Rechenzeit kann für die Diagnose verwendet werden, wie ROM-Check, Ping auf Localhost, Funktionstest der Peripherie wie LoopTest von Kommunikationscontrollern, z. B. für SPI- und I²C-Verbindungen oder Verwendung von Low Power Modes.

Nachteile

- Nicht alle Peripherieeinheiten sind interruptfähig.
- Ungünstig für passive Sensoren und für periodische Vorgänge (Regelung, Abtastung, Signalmustererzeugung, Filterung), da zentrale Zeitbasis fehlt.

Abhilfe: Einsatz von Hardware Timer: Timer löst periodisch Interrupts aus, die mit den zyklisch auszuführenden Aufgaben verknüpft werden (ISR, IST).

Beispielprogrammierung eines USART-Bausteins

```
Funktion void main(void)
{
    char zeichen;           // Speicher fuer empfangenes Zeichen
    WDTCTL = WDTPW + WDTHOLD; // Stop watchdog timer
    init_uart();

    for ( ;; )
    {
        while ( (IFG2 & URXIFG1)==0 )
        {
            ; // Warten auf empfangenes Zeichen Interrupt Flag auslesen!
        }
        zeichen=RXBUF1;      // empfangenes Zeichen im Speicher lesen
        while ( (IFG2 & UTXIFG1)==0 )
        {
            ; // Ist Transmitter frei fuer neue Daten?
        }
        TXBUF1=zeichen;     // Zeichen zurueck senden
    }
}
```

Bild: Negativbeispiel für blockierendes Empfangen und Senden von Daten über die V24-Schnittstelle

```
#pragma vector=UART1RX_VECTOR __interrupt void usart1_rx (void)
// Interruptroutine Serieller Empfang

/* <blockkommentar> */
IE2 = IE2 ^ URXIE1; // Interrupt fuer seriellen Empfang abschalten

((char*)buffer)[buffer_counter++] = RXBUF1; // Zeichen in Puffer lesen und Pufferzaehler erhoehen

if ( RXBUF1==0x0d // Abfrage ob Return gesendet wurde )
then
else
    if ( (buffer_counter+1) % 10 == 0 // Ist Puffer voll? )
    then
        // weitere 10 Byte am Anschluss von Buffer reservieren
        if ( (buffer_tmp=realloc(buffer,buffer_counter+1+10))==0 )
        then
            flag=1; // Wenn kein Speicherplatz mehr frei, dann Speicher auslesen aktivieren
        else
    else
        if ( flag==0 )
        then
            IE2 = IE2 | URXIE1; // Interrupt fuer Empfang wieder freigeben
        else
```

Bild: Alternative Lösung des Datenempfangs mit Hilfe einer ISR

3.2.3. Programmtechnische Realisierung einer ereignisgesteuerten Aufgabenbearbeitung (Bsp.)

```
unsigned int branch=0;    // Zeitbasis = 25µs
```

```
main()
{
    while (1)  // Endlosschleife
    {
        switch (branch)
        {
            case 10: Kraftstoffeinspritzung();
                    break;
            case 40: Bremsanlage();
                    break;
            case 2000: Fensterheber();
                    break;
            case 40000: Blinkanlage();
                    break;
            default: ;
        }
        branch=0;
    } // end while
} // end main
```

```
// Belegung der Interrupt-Vektortabelle
```

```
#pragma vector = HardwareINT1 _interrupt = KraftstoffeinspritzungISR();
{
    branch = 10;
}
```

```
#pragma vector = HardwareINT2 _interrupt = BremsanlageISR()
{
    branch = 40;
}
```

```
#pragma vector = HardwareINT1 _interrupt = Fensterheber()
{
    branch = 2000;
}
```

```
#pragma vector = HardwareINT4 _interrupt = Blinkanlage ()
{
    branch = 40000;
}
```

3.2.4. Signaltypen in Embedded Systemen

3.2.5. Problemlösung: Periodischer Hardware-Timer muss mittelbar Periodizität herstellen

Die Periodizität, Turnus oder Wiederkehr bezeichnet allgemein die Eigenschaft einer Sache oder eines Vorgangs, die bezüglich des Auftretens eines bestimmten Ereignisses eine Regelmäßigkeit aufweist. Die regelmäßige Reihe der Ereignisse ist ein Zyklus.

<https://de.wikipedia.org/wiki/Periodizität>

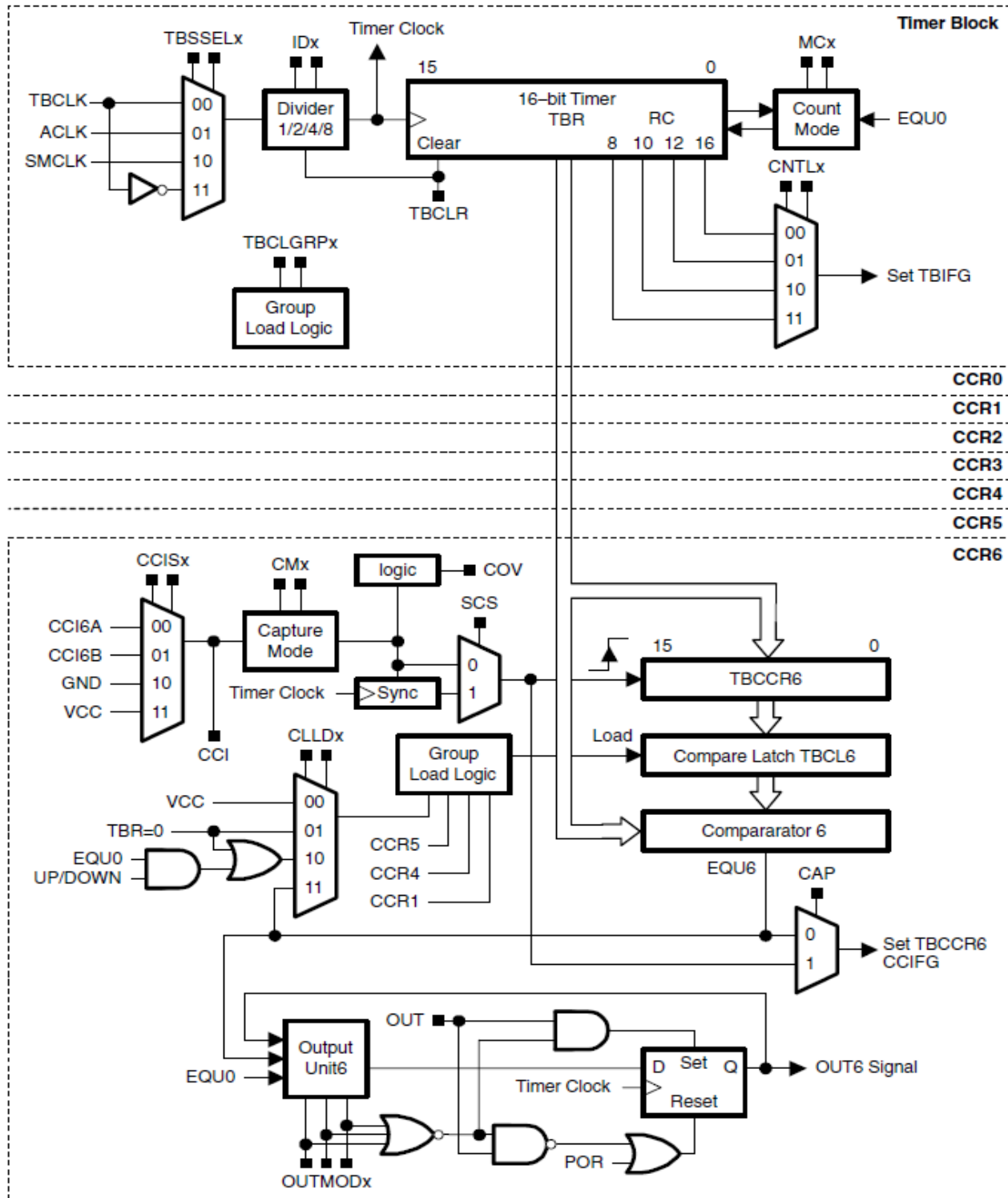
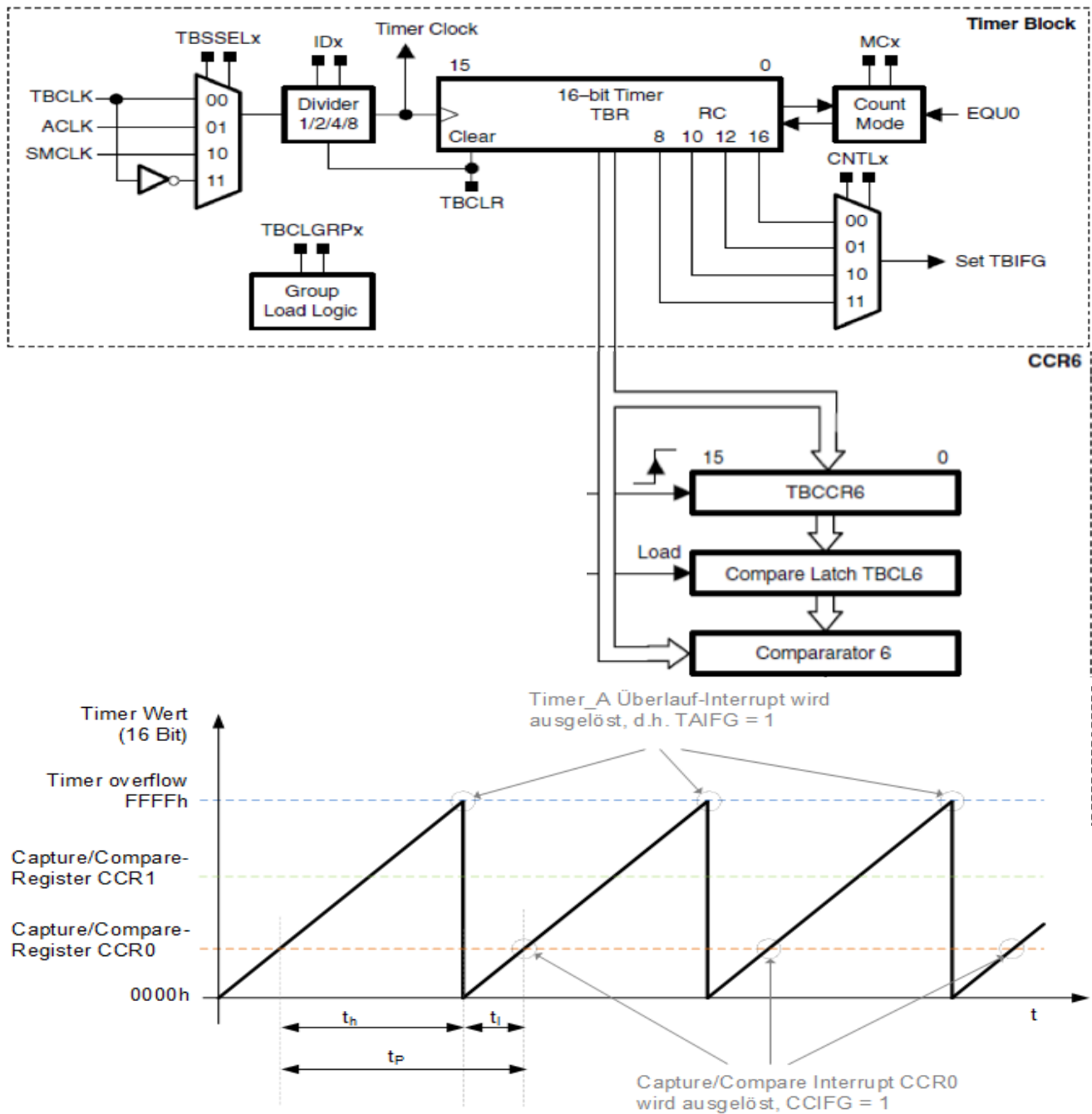


Bild: Hardware-Timer, Capture Unit und Compare Unit, Output-Unit

⇒ Realisierung mittels eines Hardware-Timers (TBR) und der Verwendung der Capture Compare Unit (CCU)



4. AUFBAUSTRUKTUR VON EMBEDDED SYSTEMEN

Hardware-Aufbaustruktur von Embedded Systemboards

On Chip-Peripherie, Komponentenaufbaustruktur

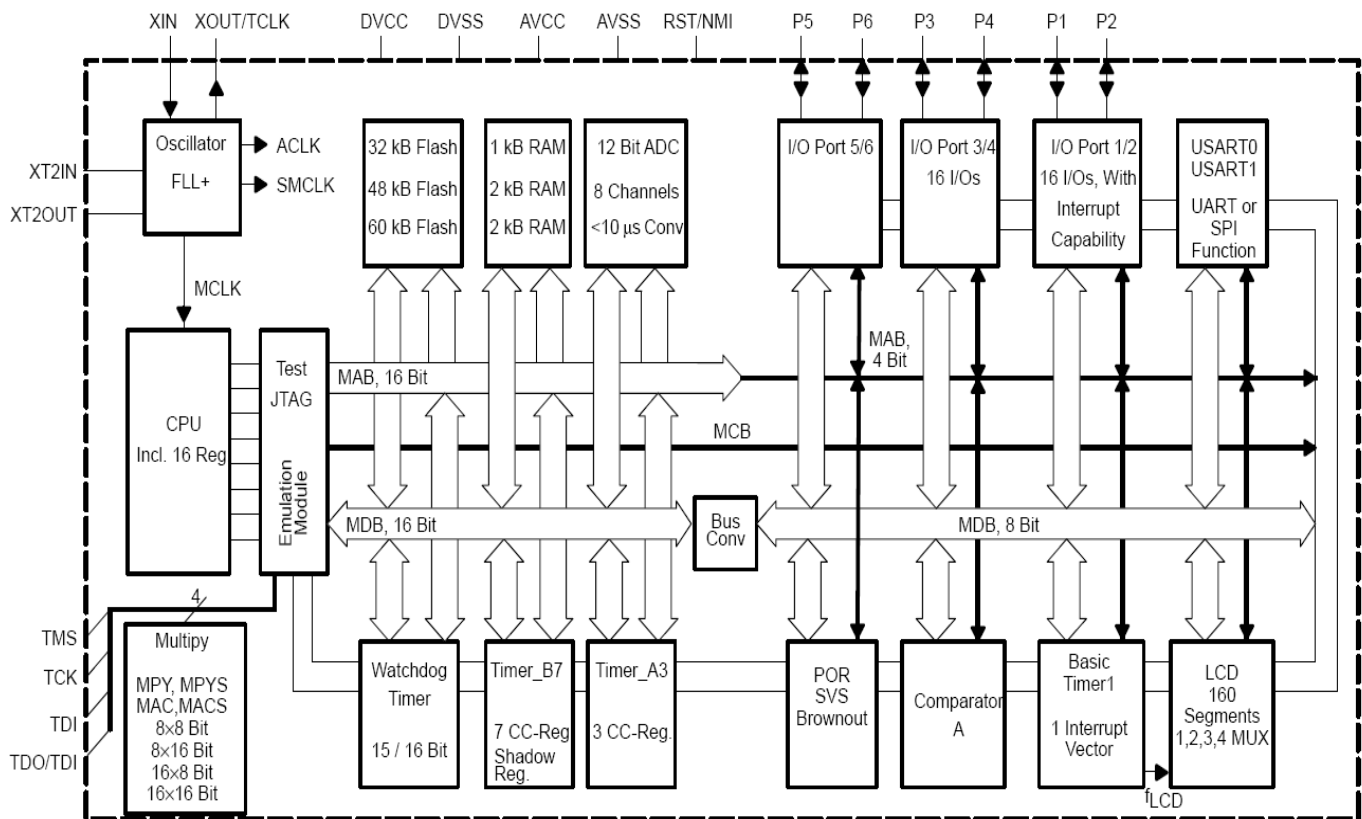


Bild: MSP430x44x functional block diagrams

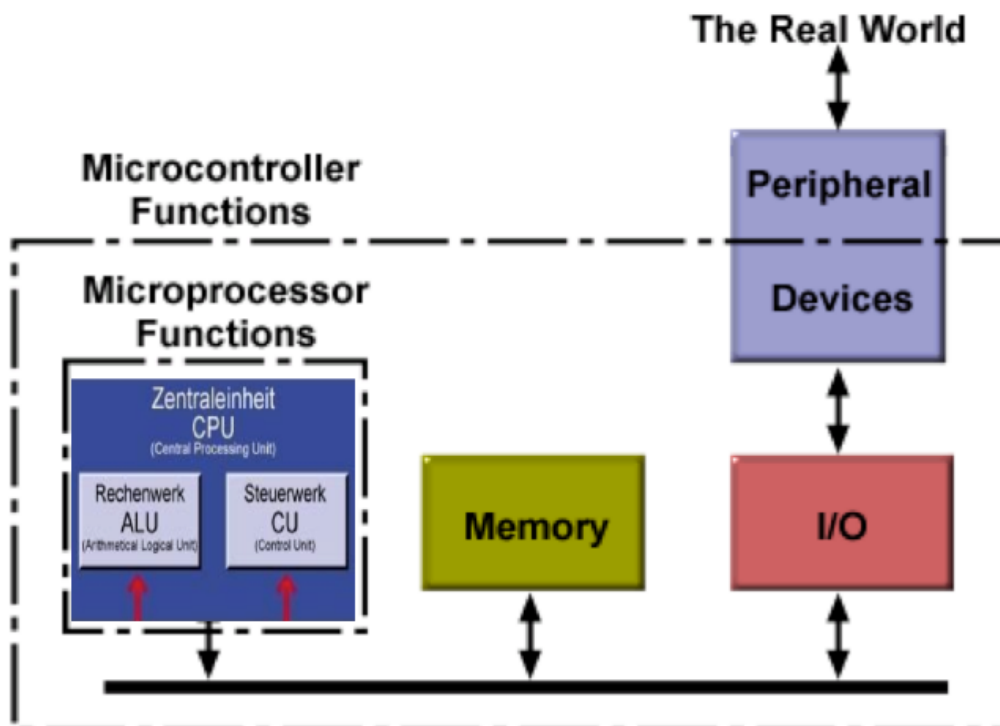


Bild: µController versus µProzessor

4.1. Vergleich Mikrocontroller versus Mikroprozessor

- | | |
|--|---|
| <ul style="list-style-type: none">▪ µC Chip Includes:<ul style="list-style-type: none">• Central Processor• Program Memory• Data Memory• I/O▪ Highly Integrated▪ Low Cost▪ Specialized Architectures | <ul style="list-style-type: none">▪ µP Chip Includes:<ul style="list-style-type: none">• Central Processor▪ Separate Chips for:<ul style="list-style-type: none">• Central Processor• Program Memory• Data Memory• I/O▪ Highest Performance▪ Highest Cost |
|--|---|

Vor- und Nachteile von Single Chip Mikrocontrollern gegenüber CPUs

4.2. Aufbau und Funktionsweise einer Central Processing Unit

4.2.1. Bestandteile einer CPU

STEUERWERK:

RECHENWERK:

CPU-interner Registersatz:

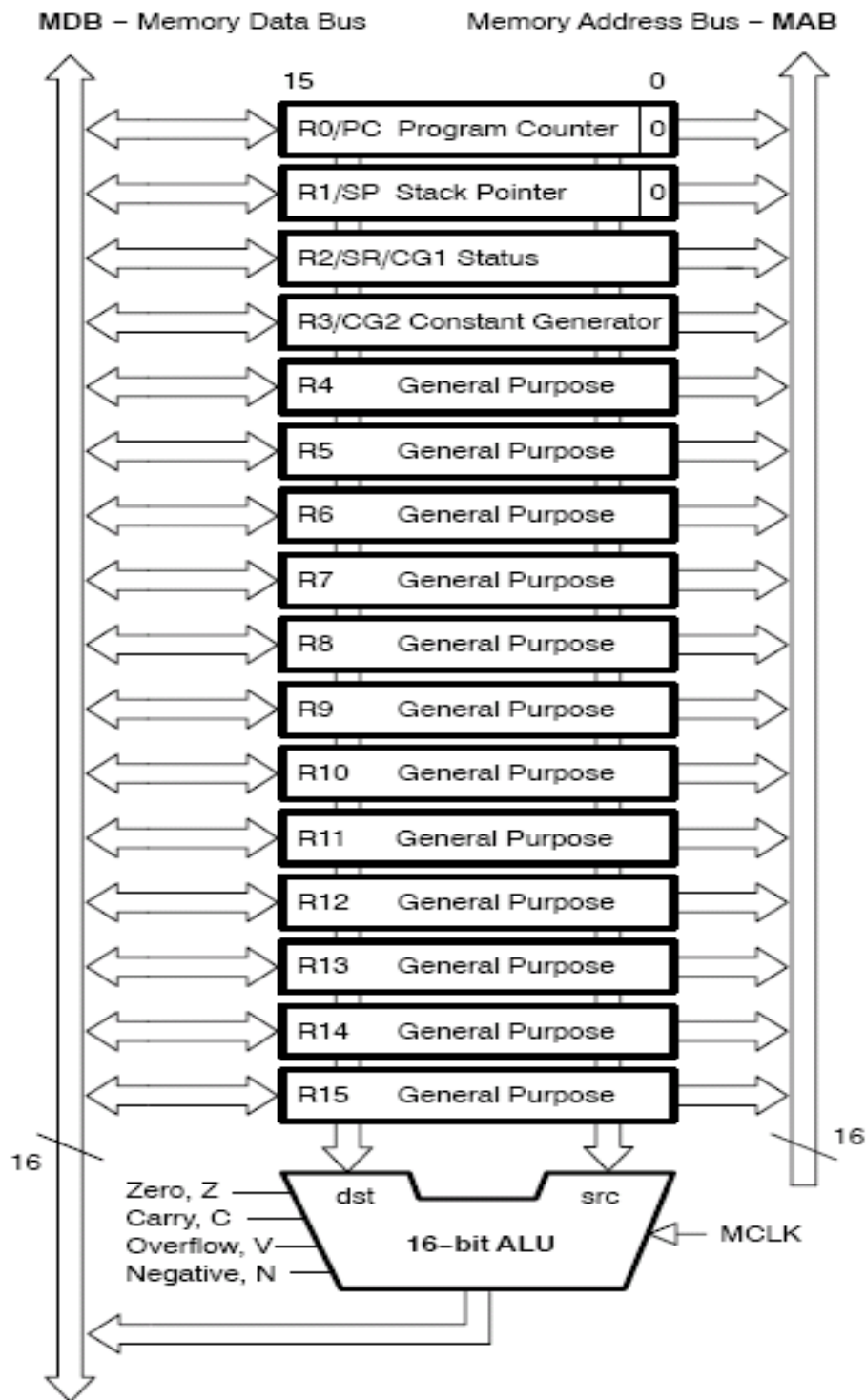


Bild: Der CPU interne Registersatz beim TI MSP430-Mikrocontroller

Status Register

The status register (SR/R2), used as a source or destination register, can be used in the register mode only addressed with word instructions. The remaining combinations of addressing modes are used to support the constant generator. The figure shows the SR bits.

Bit	Description
V	Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range. ADD (.B) , ADDC (.B) Set when: Positive + Positive = Negative Negative + Negative = Positive, otherwise reset SUB (.B) , SUBC (.B) , CMP (.B) Set when: Positive - Negative = Negative Negative - Positive = Positive, otherwise reset
SCG1	System clock generator 1. This bit, when set, turns off the DCO dc generator, if DCOCLK is not used for MCLK or SMCLK.
SCG0	System clock generator 0. This bit, when set, turns off the FLL+ loop control
OSCOFF	Oscillator Off. This bit, when set, turns off the LFXT1 crystal oscillator, when LFXT1CLK is not used for MCLK or SMCLK
CPUOFF	CPU off. This bit, when set, turns off the CPU.
GIE	General interrupt enable. This bit, when set, enables maskable interrupts. When reset, all maskable interrupts are disabled.
N	Negative bit. This bit is set when the result of a byte or word operation is negative and cleared when the result is not negative. Word operation: N is set to the value of bit 15 of the result Byte operation: N is set to the value of bit 7 of the result
Z	Zero bit. This bit is set when the result of a byte or word operation is 0 and cleared when the result is not 0.
C	Carry bit. This bit is set when the result of a byte or word operation produced a carry and cleared when no carry occurred.

Bild: Description of Status Register Bits (SFR)

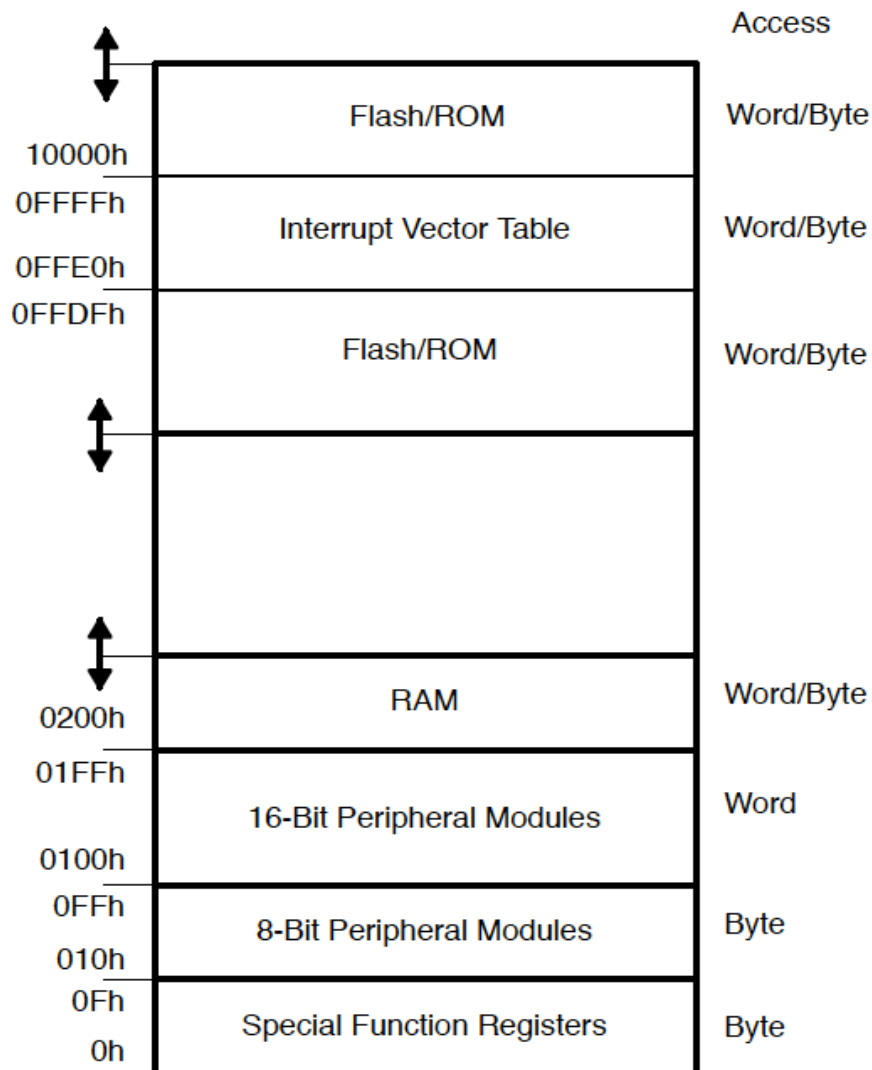


Bild: Memory Map der Texas Instruments MCU-Familie MSP430F44x

4.2.2. Funktionsweise einer CPU (Von-Neumann-Zyklus)

Der Von-Neumann Rechner ist eine sequentiell arbeitende Maschine.

Der Prozessor (=CPU) verarbeitet Schritt für Schritt Befehle und Daten, die aus dem gleichen Speicher kommen (nur eine Busverbindung!).

Jede Speicherstelle besitzt eine fest zugeordnete physikalische Adresse, die allgemein in hexadezimaler Schreibweise angegeben wird.

Zentraleinheit

Die Zentraleinheit (Central Processing Unit CPU) stellt den Rechnerkern dar. Sie realisiert

- die Ablaufsteuerung des Rechners selbst,
- die Durchführung arithmetisch-logischer Operationen,
- den Transfer von Daten und
- den Ablauf des Programms.

Der Von-Neumann-Zyklus ist wie folgt:

1. FETCH (Holphase)

Der Befehlszählregister (Program Counter) zeigt auf den nächsten auszuführenden Maschinenbefehl.

Die CPU holt sich diesen Befehl aus dem Arbeitsspeicher (RAM) und speichert den Maschinenbefehl (Opcode) in den Befehlsdecoder.

2. DECODE (Dekodierphase)

Das Steuerwerk untersucht den Maschinenbefehl im Befehlsdecoder hinsichtlich der vorzunehmenden Arbeitsschritte und sorgt dafür, dass der Befehl ausgeführt wird.

Liegt ein fest implementierter Befehl vor (z. B. Inc, Dec etc.), werden sofort die entsprechenden Schritte in der Ablaufsteuerung eingeleitet.

Bei komplexeren Maschinenbefehlen ermittelt der Befehlsdekoder aus den Befehlsbits die Einsprungsadresse für das Ablaufsteuerprogramm im Microcode-ROM.

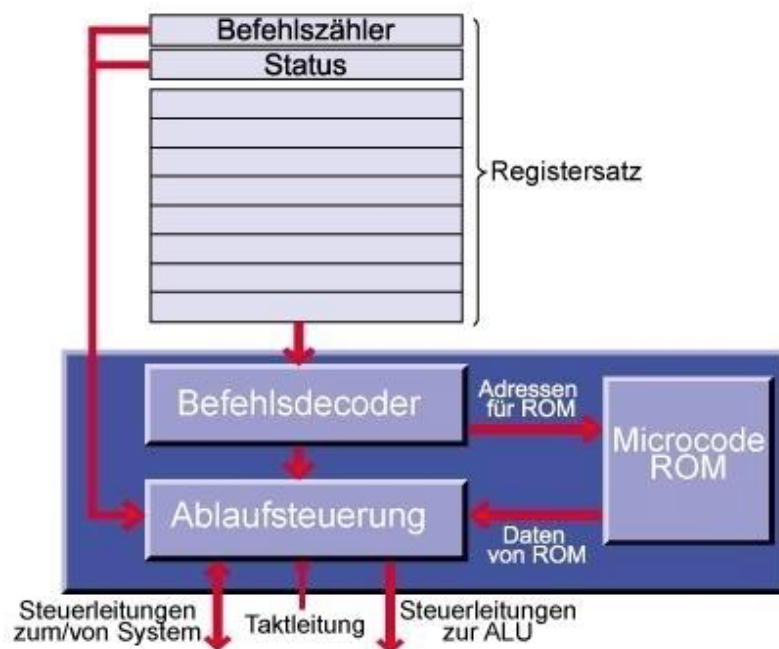


Bild: Das Steuerwerk (Control Unit CU) ist die Kommandozentrale der CPU. Sie steuert alle Abläufe im Innern des Prozessors

3. FETCH OPERANDS (Operanden nachladen)

Bei der Dekodierung des Maschinenbefehls wird ggf. festgestellt, dass für die Ausführung des Befehls noch weitere Daten (Operanden) aus dem Speicher geholt werden müssen.

Das Steuerwerk veranlasst das Adresswerk aus dem CPU-internen Cache, dem Arbeitsspeicher oder von der Festplatte (virtuelles Memory) weitere Operanden, d. h. Werte und Parameter, in die Register nachzuladen, die Gegenstand des Befehls (2-Operanden bzw. 3-Operanden Befehl) sind und zur vollständigen Ausführung des Befehls benötigt werden, z. B. *MOV R1, (R2)*.

4. EXECUTE (Ausführungsphase)

Die ALU wird vom Steuerwerk angewiesen, die gewünschte Operation mit den zuvor geladenen Daten auszuführen.

In Abhängigkeit des auszuführenden Befehls werden

- Peripheriemodule angesteuert,
- Operationen im Rechenwerk (ALU) durchgeführt (ADD, INC, etc.) oder
- ein Sprung im Programm bewerkstelligt (JMP, JNZ etc.).

Ein Sprung im Programm bedeutet, dass das Befehlszählerregister verändert wird.

WRITE BACK (Schreibphase)

Das Ergebnis der Operation in der ALU wird in Abhängigkeit des ausgeführten Maschinenbefehls in

- ein CPU internes Register R1, R2 etc. oder
- im Arbeitsspeicher (RAM) abgelegt.

Um langwierige Zugriffe auf das externe RAM zu vermeiden, sammelt die CPU die Daten eventuell in einem schnellen CPU internen Zwischenspeicher (Cache), damit sie später in einem schnellen Blockschreibvorgang in das RAM übertragen werden können.

Mikroprogramm

Das Mikroprogramm ist der Algorithmus eines Mikroprogrammsteuerwerks in einem Prozessor zum Steuern seines Rechenwerks (ALU).

Mikroprogramme werden mittels Mikroprogrammierung erstellt. Dazu werden Mikrobefehle eines Mikroprogramms in einer adressierbaren Reihenfolge angeordnet und in einem Festwertspeicher, dem sog. Mikroprogrammspeicher, bereitgestellt.

Die Mikrobefehle enthalten in binärer Form codierte Anweisungen mit fester Bit-Anzahl **zur Aktivierung und Deaktivierung logischer Funktionseinheiten**. Auf diese Weise bestimmt das Mikroprogrammsteuerwerk mit Hilfe des Mikroprogramms, welche logischen Schaltvorgänge in welcher Reihenfolge zur Umsetzung von Maschinenbefehlen für ein Rechenwerk (ALU) benötigt werden.

Die Gesamtheit aller im Mikroprogrammspeicher binär codierten Mikrobefehle bilden den Mikrocode.

Mikrocodesimulator <http://www.mikrocodesimulator.de/>
https://sourceforge.net/projects/j8085sim/?source=typ_redirect

Programmablaufplan (PAP) zur Befehlsausführung

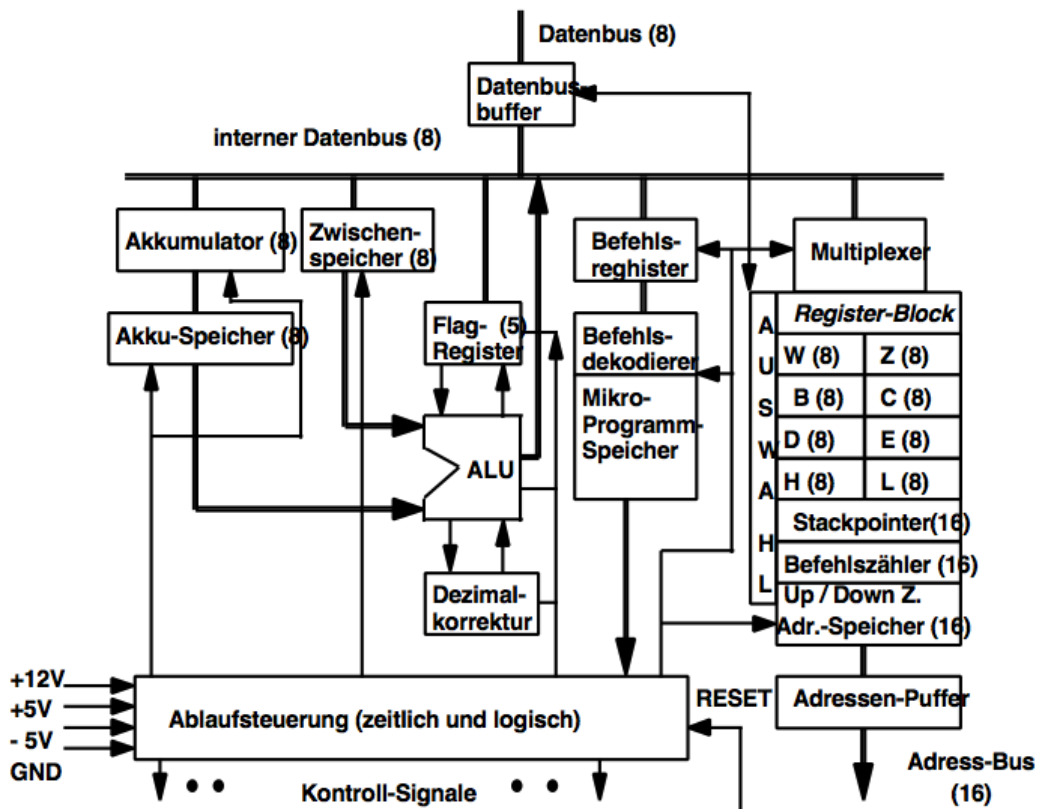


Bild: Innere Struktur des 8080-Mikroprozessors

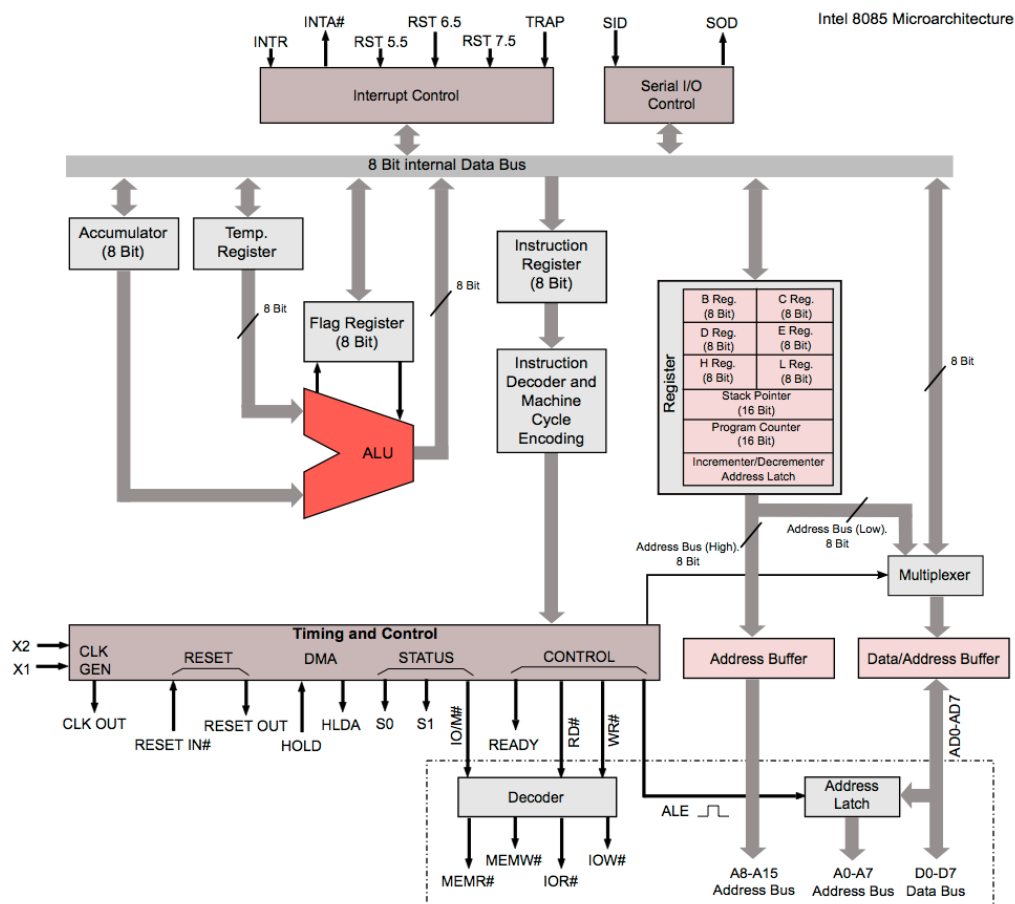


Bild: Innere Struktur des 8085-Mikroprozessors

4.2.3. Übung: Beschleunigungsverfahren für Von-Neumann Rechner

4.2.4. CPU-Zugriff auf den Schreib-Lese Speicher

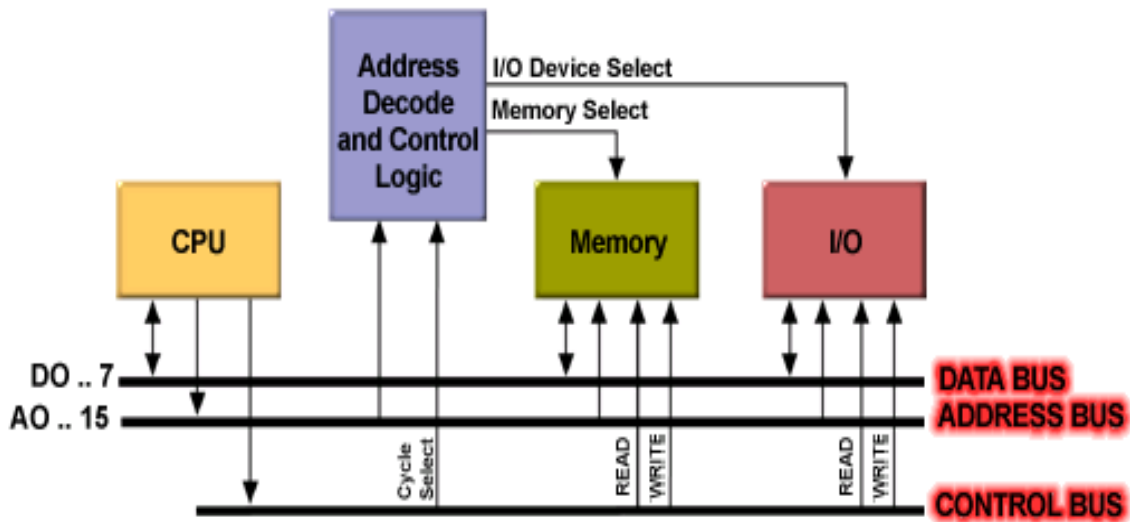


Bild: Busorientierter Mikrocontroller, typischer Aufbau eines μ Controllers

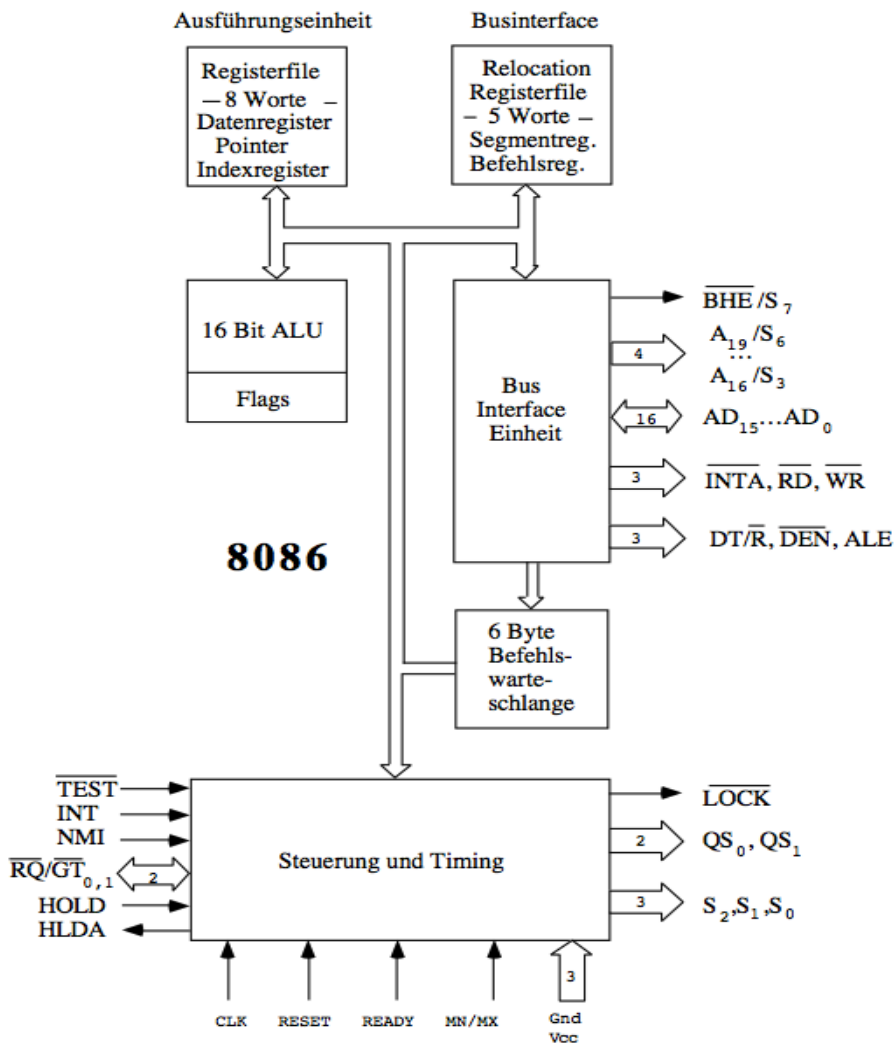


Bild: Innere Struktur des 16-Bit Intel 8086-Mikroprozessors, Vorläufer der Intel 80386/80486 und Pentium-Prozessoren

Schreibe-Lese Maschinenzyklus einer CPU

Speicher 4-Bit-Speicher

- 4 Daten-Signale
- 16 Adress-Signale
- 2 Steuer-Signale -CE, RD/-WR

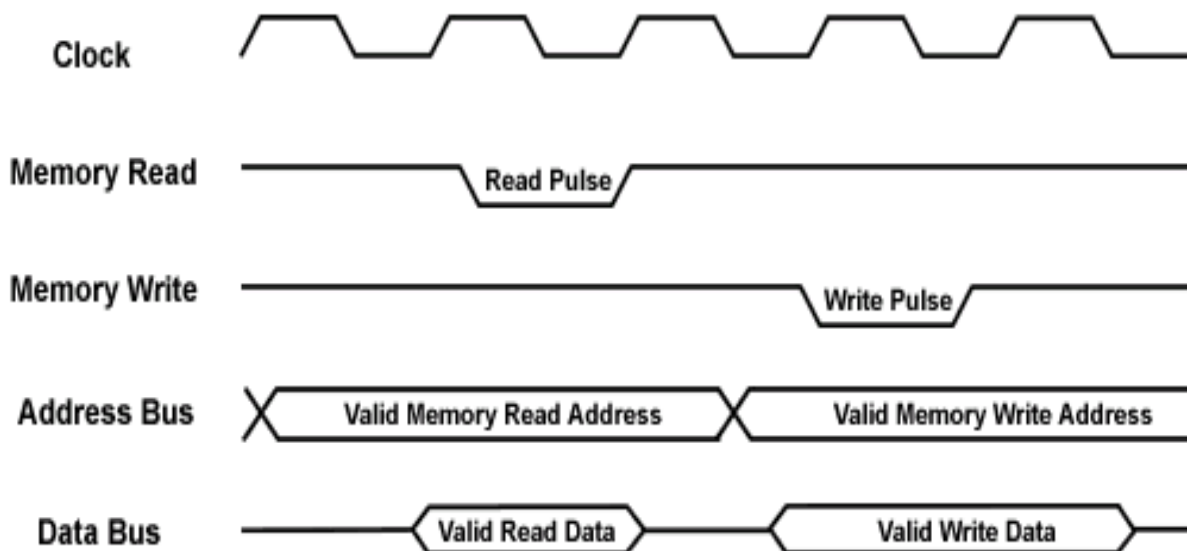
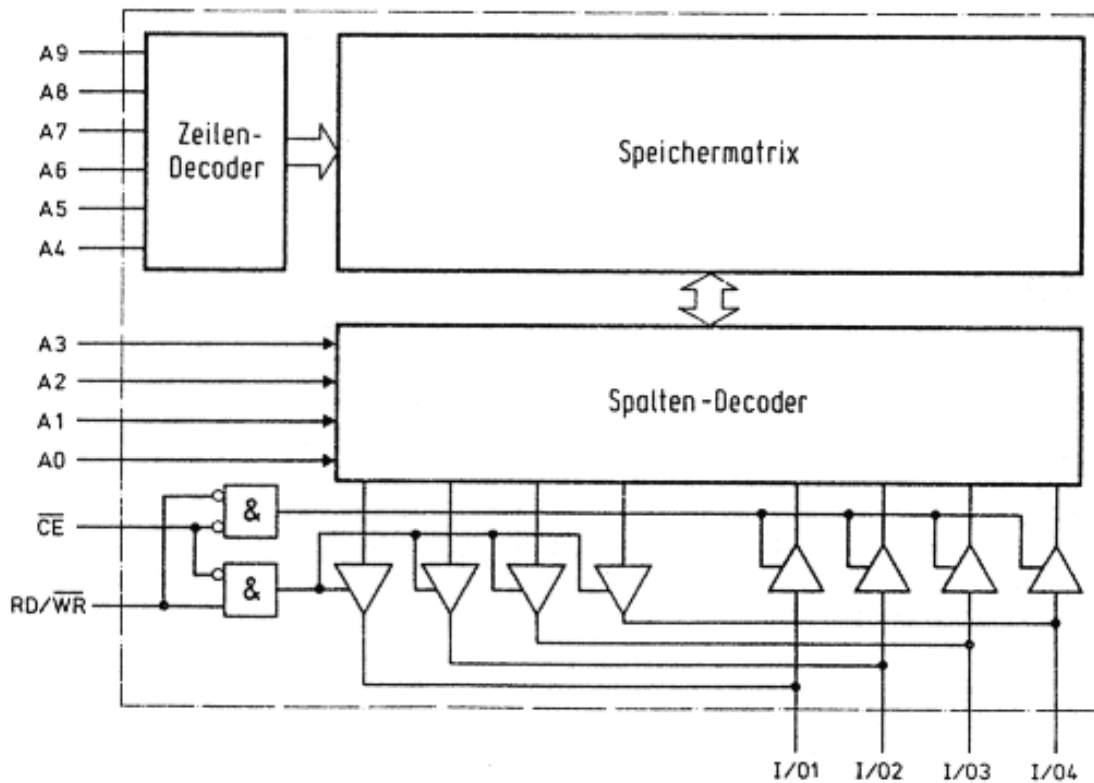


Bild: Memory Read/Write Cycle

4.3. Hauptfunktionskomponenten eines Embedded Systems

Der Hardware-Core: Mikrocontroller

Mikrocontroller-OnChip Funktionseinheiten

- Digitale I/O
- Standard I/O
- Capture- and Compare-Einheiten
- PWM (Pulsweitenmodulierte Ausgänge)
- Analoge I/O
- AD/DA-Wandler
- RS232-asynchrone und synchrone Einheiten

Das Interrupt-System (siehe Unterlagen Dr. Jacob)

- Interruptcontroller
- Daisy-Chain
- Mechanismus Interruptleitung, Interruptvektor und Interruptserviceroutine (ISR)

Das Speichersystem

- Speicherarten -> siehe Berufungsvortrag an der FH Augsburg
- Flash-Speicher, z.B. für Images und Disklessssysteme
- EPROM
- DRAMs, SRAMs etc.

Kommunikationsadapter

- Digitale Chip-to-Chip-Bussysteme (I2C, SPI etc.)
- Ethernet-Controller (Dallas D800C40 etc.)
- Feldbus-Controller (Aufbau und Schnittstelle zum Mikrocontroller)
- Ggf. USB, Wireless etc.

Signalkonditionierung

- Bsp. Signal-Conditioning--AD7710.pdf

Bsp. Board

- 68000 Board aus dem Praktikum
 - Universal-Boards (Tiny, x-Boards, Board im Automobilbereich für den Embeddedbereich etc.)
 - Router-Board
 - Settop-Box Board
-
- CPU
 - MCU (MCU = CPU mit On-Chip Peripherie)
 - On-Board Peripherie
 - Schnittstellen
 - Watchdog
 - Diagnosemöglichkeiten mittels JTAG
 - Memory Map eines Embedded Systems

Zentraleinheit

Die Zentraleinheit (Central Processing Unit CPU) stellt den Rechnerkern dar, um den sich alles dreht. Sie realisiert die Ablaufsteuerung des Rechners selbst, die Durchführung arithmetisch-logischer Operationen, den Transfer von Daten und den Ablauf des Programms.

Bei der Familie MSP430 handelt es sich um eine 16-Bit-RISC-CPU mit verschiedenen universellen Registern, einem breiten Spektrum an Adressierarten, einem kleinen, aber flexiblen Befehlssatz, alles ausgelegt auf hohe Leistung bei besonders kleinem Stromverbrauch.

Zum Test des gesamten Systems und zur Programmierung ist die CPU von einer JTAG-Schnittstelle umgeben, die es erlaubt, auf den gesamten Mikrocontroller mit nur vier Leitungen effektiv zuzugreifen.

Peripherieeinheiten

Die Familie MSP430 zeichnet sich durch eine breite Palette intelligenter und voneinander und von der CPU weitestgehend unabhängiger Peripherie aus, die die Realisierung komplexer Systeme mit nur geringer externer Beschaltung erlaubt.

Die Peripherie ist in Modulen realisiert, die mit der CPU durch den Speicheradressbus MAB, den Speicherdatenbus MDB und Unterbrechungsdienst- und -anforderungsleitungen verbunden sind.

Die Befehlsausführung bei 16-Bit-Modulen arbeitet ohne Einschränkungen (Watchdog, ADC, TIMER A, Hardware-Multiplier). Die 8 bit-Peripheriemodule sind byteorientiert. Ihre SFR's werden ausnahmslos mit einem Datenbereich von 8 Bit betrieben.

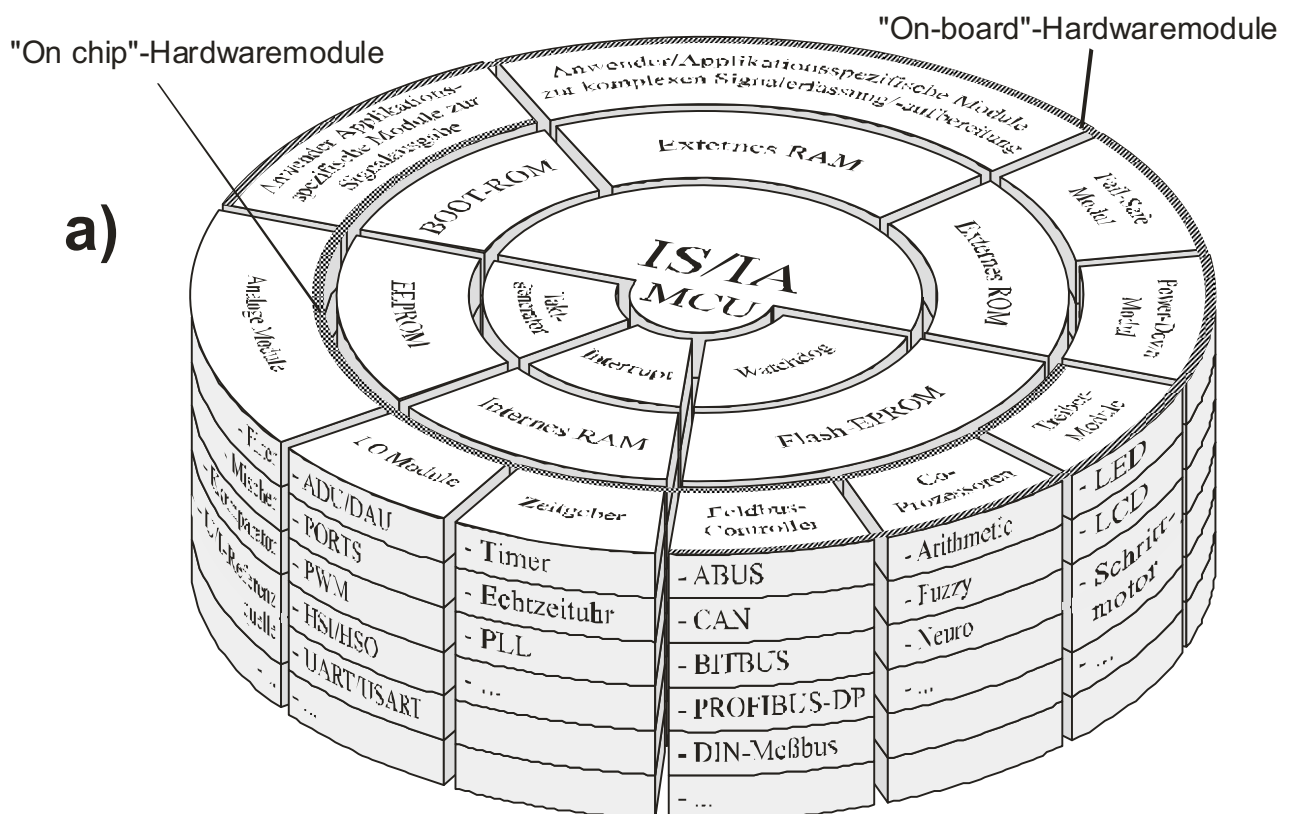


Bild: Komponentenmodell eines mikrocontrollergestützten Embedded Systems

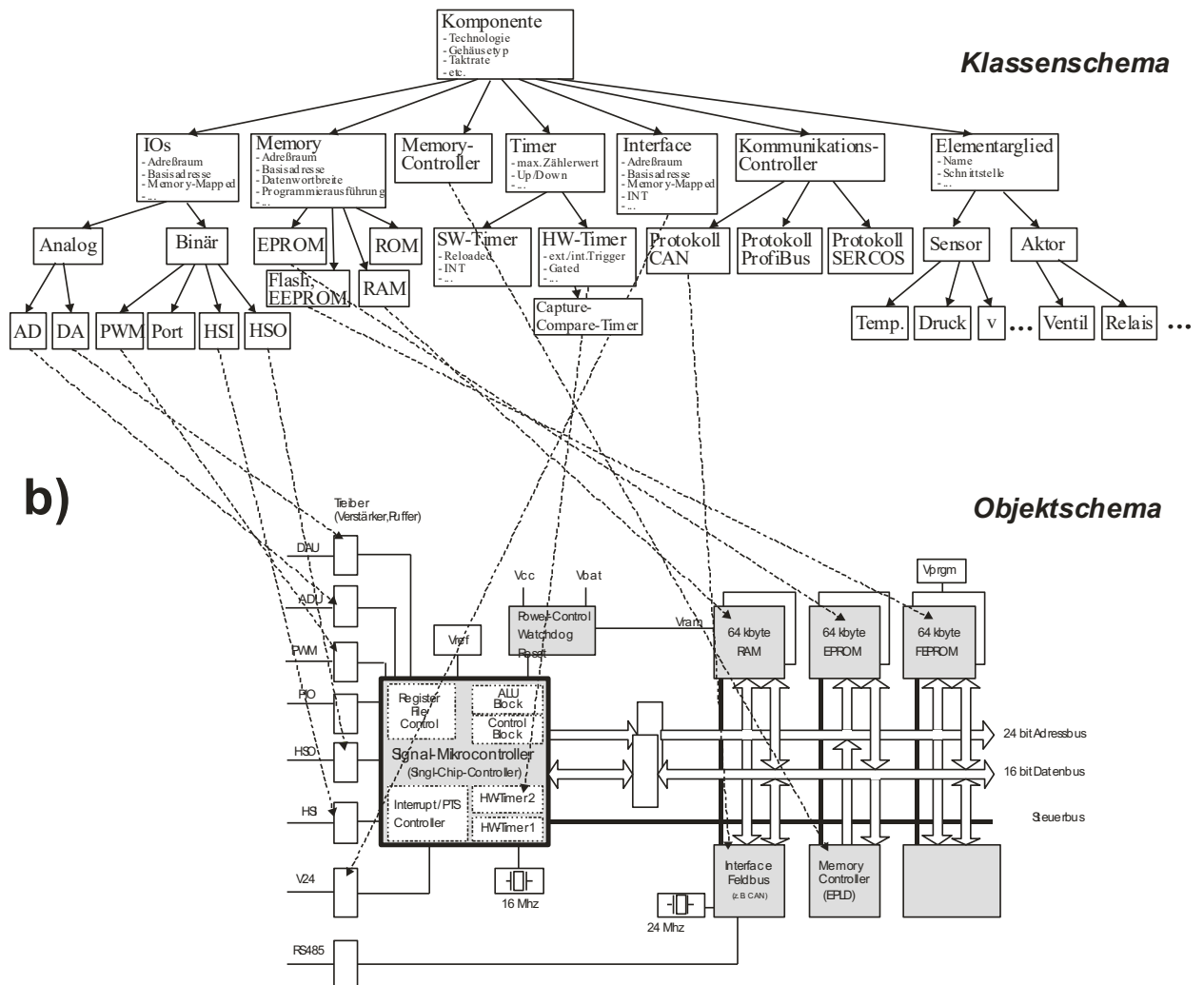


Bild: Klassenschema und Objektschema zur topologischen Modellierung eines Embedded Systems

4.3.1. Mikrocontrollerfamilien (Auswahl)

- Atmel **AVR**
- Atmel, NXP (Philips), OKI, ST, TI **ARM7**
- Atmel **ARM9**
- Axis Communications **ETRAX**
- Microchip **PICmicro**
- Parallax (Ubicom/Scenix) **SX**
- Parallax **Propeller Chip**
- Intel, Infineon, NXP (Philips), Dallas, Silicon Labs... **8051**
- Infineon **C16x**
- Intel, AMD **x86**
- Freescale **ColdFire**
- Freescale **HC08 / HCS08**
- Freescale **HC11**
- Freescale **HC12 / HCS12 / S12X**
- Texas Instruments **MSP430**
- Zilog, Toshiba **Z80**
- Parallax **BASIC Stamp** und **Javelin Stamp**
- weitere **µController, Software und Peripherie** (Interfacekonverter, IF-Module, PIF-Bus, ST6, SH-7032...)

Bild: Mikrocontrollerfamilien

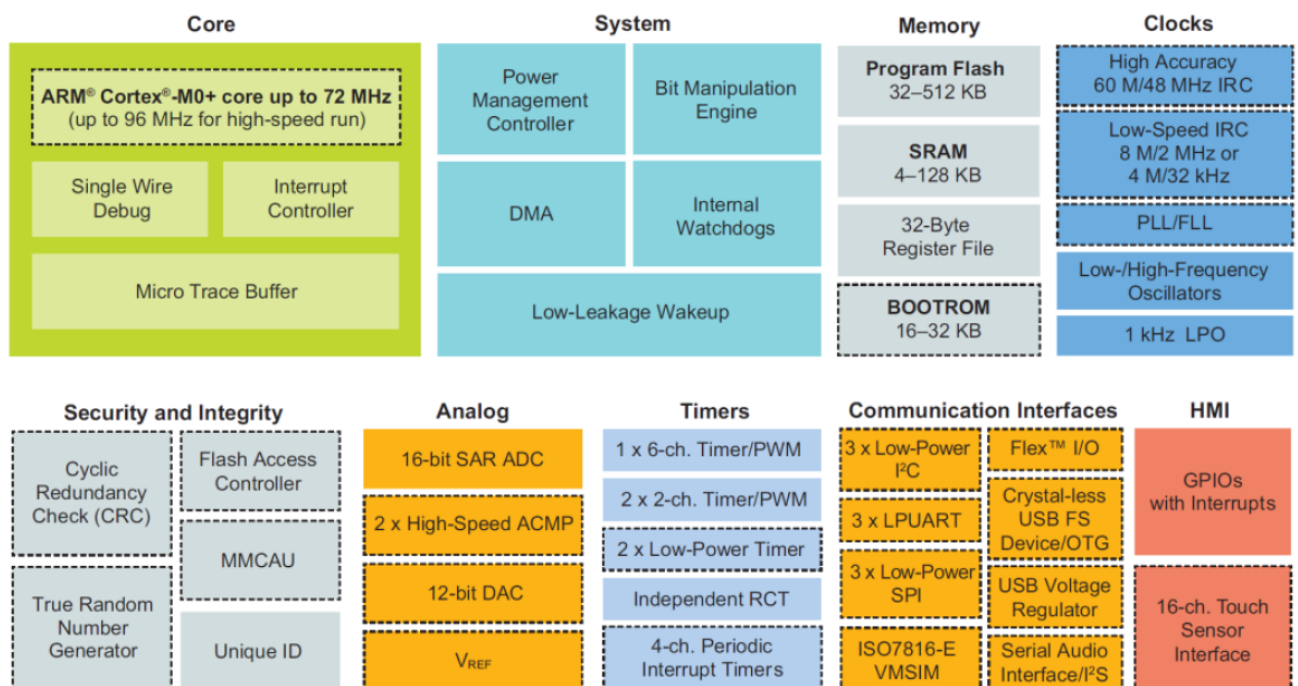


Bild: NXP Kinetis KL2x MCU Family Block Diagram

KL2x: Kinetis® KL2x-48 MHz, USB Ultra-Low-Power Microcontrollers (MCUs) based on

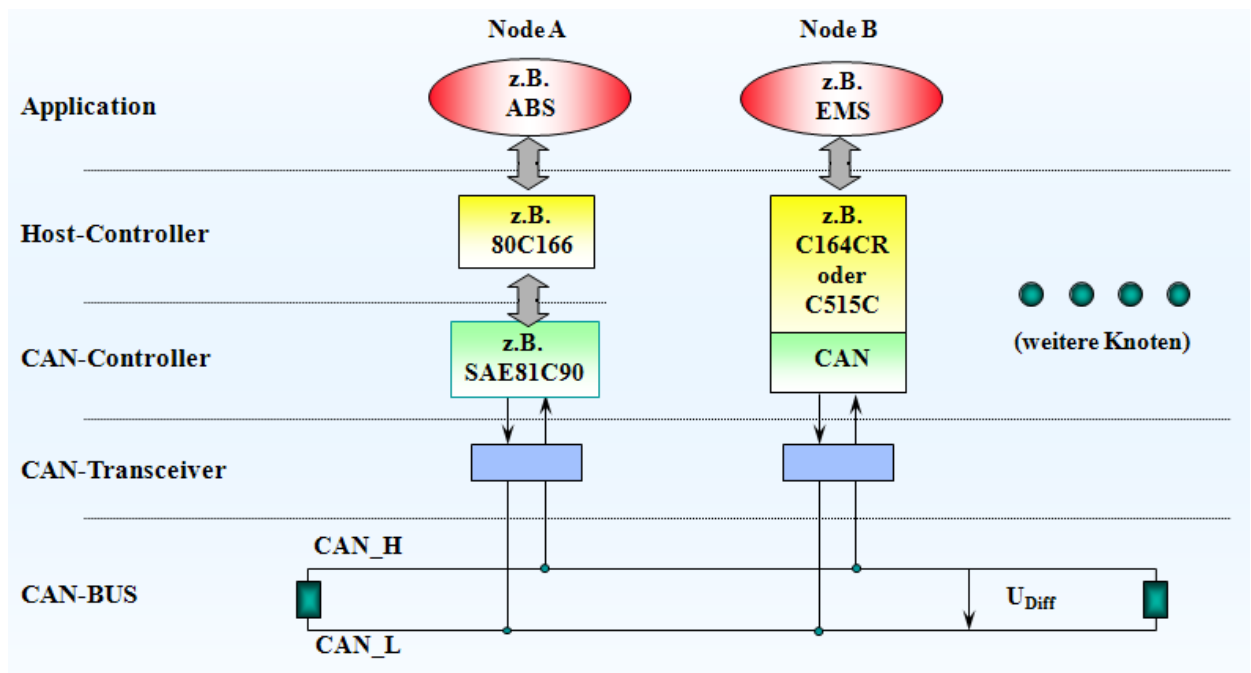


Bild: Mikrocontroller mit CAN-Interface (typische Struktur von CAN-Knoten)

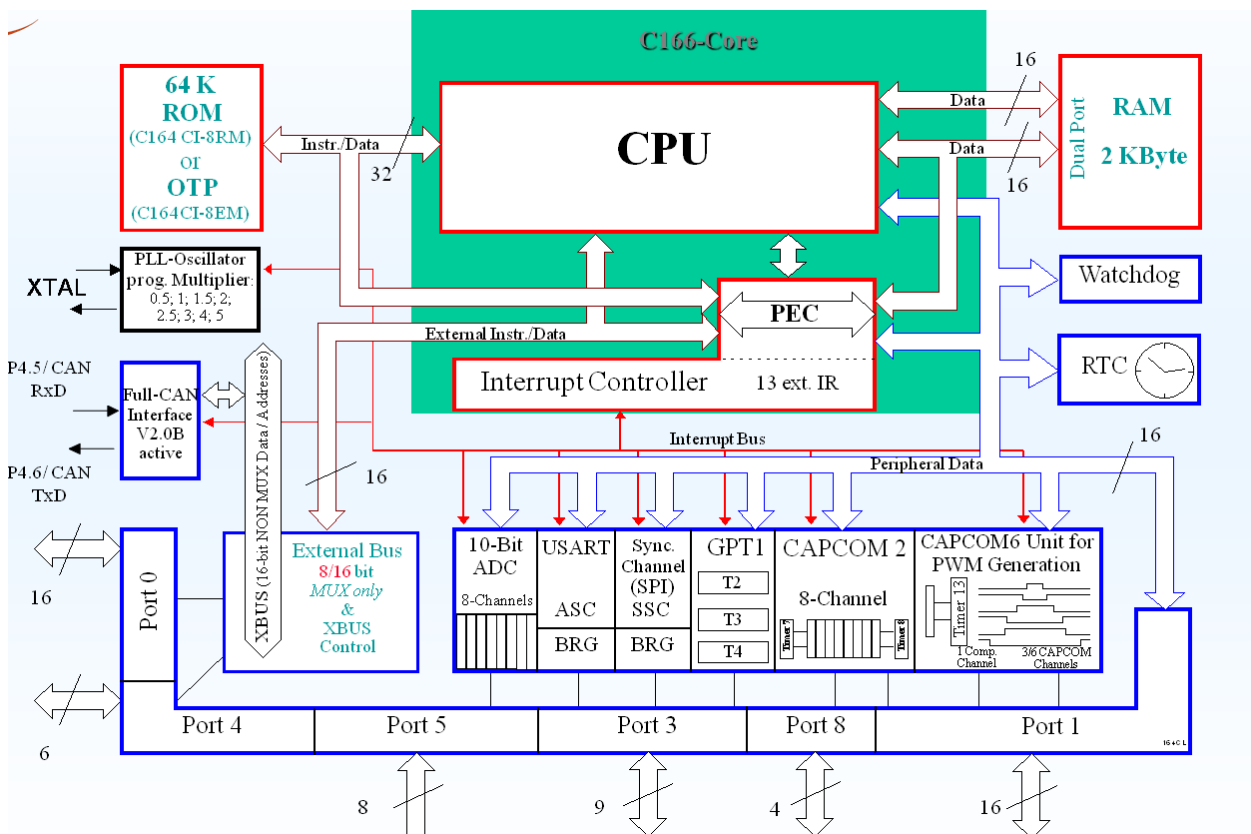


Bild: C164CI - CAN-Schnittstelle

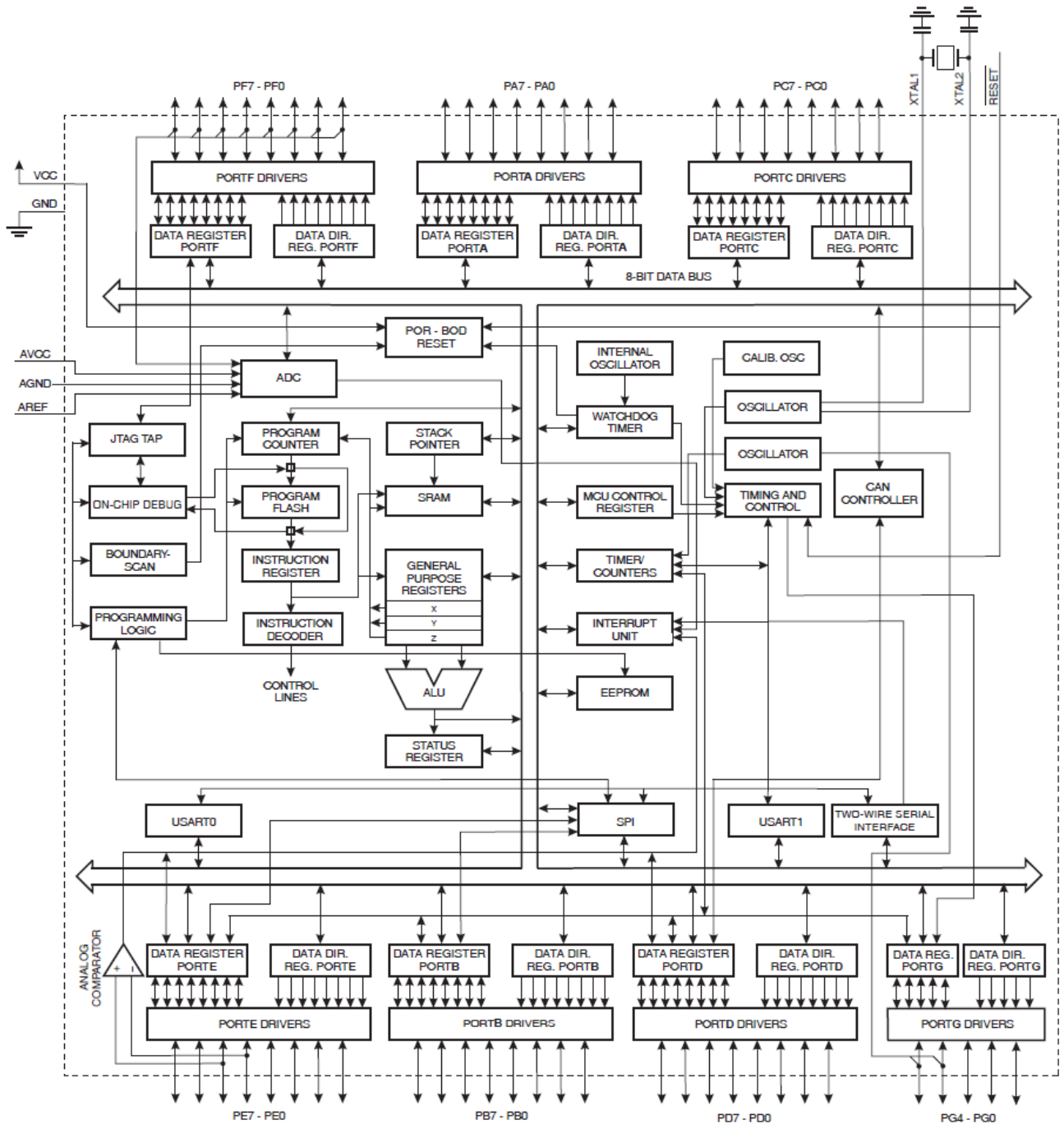


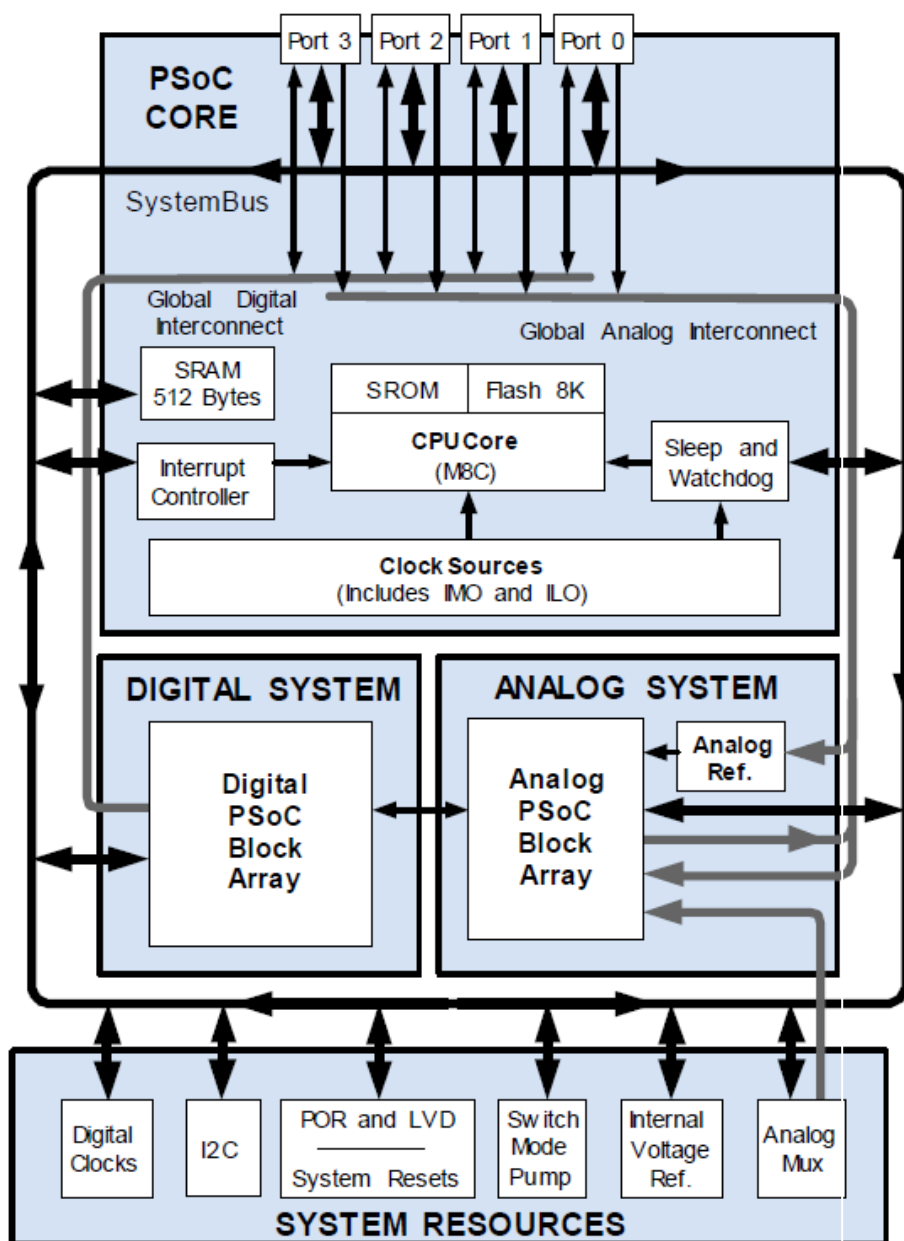
Bild: Block Diagram AT90CAN128

PSoC™ Mixed-Signal Array

The PSoC™ family consists of many Mixed-Signal Array with On-Chip Controller devices. These devices are designed to replace multiple traditional MCU-based system components with one, low cost single-chip programmable component.

A PSoC device includes configurable blocks of analog and digital logic, as well as programmable interconnect. This architecture allows the user to create customized peripheral configurations, to match the requirements of each individual application.

Additionally, a fast CPU, Flash program memory, SRAM data memory, and configurable IO are included in a range of convenient pinouts.



Features

- **Powerful Harvard Architecture Process**
 - M8C Processor Speeds to 24 MHz
 - Low Power at High Speed
 - 2.4V to 5.25V Operating Voltage
 - Operating Voltages Down to 1.0V Using On-Chip Switch Mode Pump (SMP)
 - Industrial Temperature Range: -40°C to +85°C
- **Advanced Peripherals (PSoC Blocks)**
 - 4 Analog Type "E" PSoC Blocks Provide:
 - 2 Comparators with DAC Refs
 - Two 8-Bit 28 Channel ADCs
 - 4 Digital PSoC Blocks Provide:
 - 8- to 32-Bit Timers, Counters, and
 - CRC and PRS Modules
 - Full-Duplex UART, SPI™ Master or Slave
 - Connectable to All GPIO Pins
 - Complex Peripherals by Combining Blocks
- **Flexible On-Chip Memory**
 - 8K Flash Program Storage 50,000 Erase Cycles
 - 512 Bytes SRAM Data Storage
 - In-System Serial Programming (ISSP)
 - Partial Flash Updates
 - Flexible Protection Modes
 - EEPROM Emulation in Flash
- **Complete Development Tools**
 - Free Development Software (PSoC™ Designer)
 - Full-Featured, In-Circuit Emulator and Programmer
 - Full Speed Emulation
 - Complex Breakpoint Structure
 - 128K Bytes Trace Memory
- **Precision, Programmable Clocking**
 - Internal $\pm 2.5\%$ 24/48 MHz Oscillator
 - Internal Oscillator for Watchdog and Sleep
- **Programmable Pin Configurations**
 - 25 mA Drive on All GPIO
 - Pull Up, Pull Down, High Z, Strong, or Drain Drive Modes on All GPIO
 - Up to 8 Analog Inputs on GPIO
 - Configurable Interrupt on All GPIO
- **Versatile Analog Mux**
 - Common Internal Analog Bus
 - Simultaneous Connection of IO Components
 - Capacitive Sensing Application Capable
- **Additional System Resources**
 - I2C™ Master, Slave and Multi-Master 400 kHz
 - Watchdog and Sleep Timers
 - User-Configurable Low Voltage Detector
 - Integrated Supervisory Circuit
 - On-Chip Precision Voltage Reference

Bild: Block Diagram Cypress PSoC™ CY8C21434

4.3.2. Auswahl geeigneter Mikrocontroller - Bsp. TI MSP430

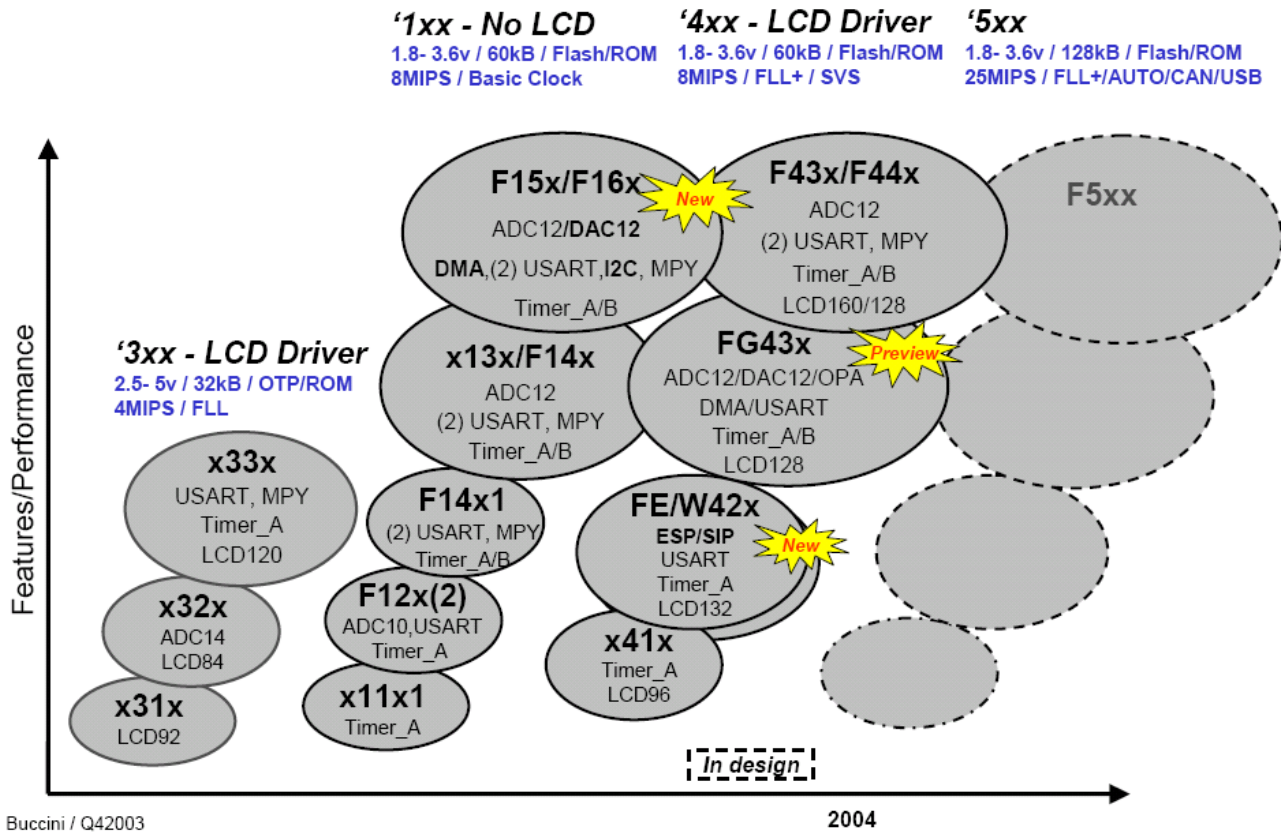


Bild: Mikrocontroller- Derivate am Beispiel der MSP430 Roadmap

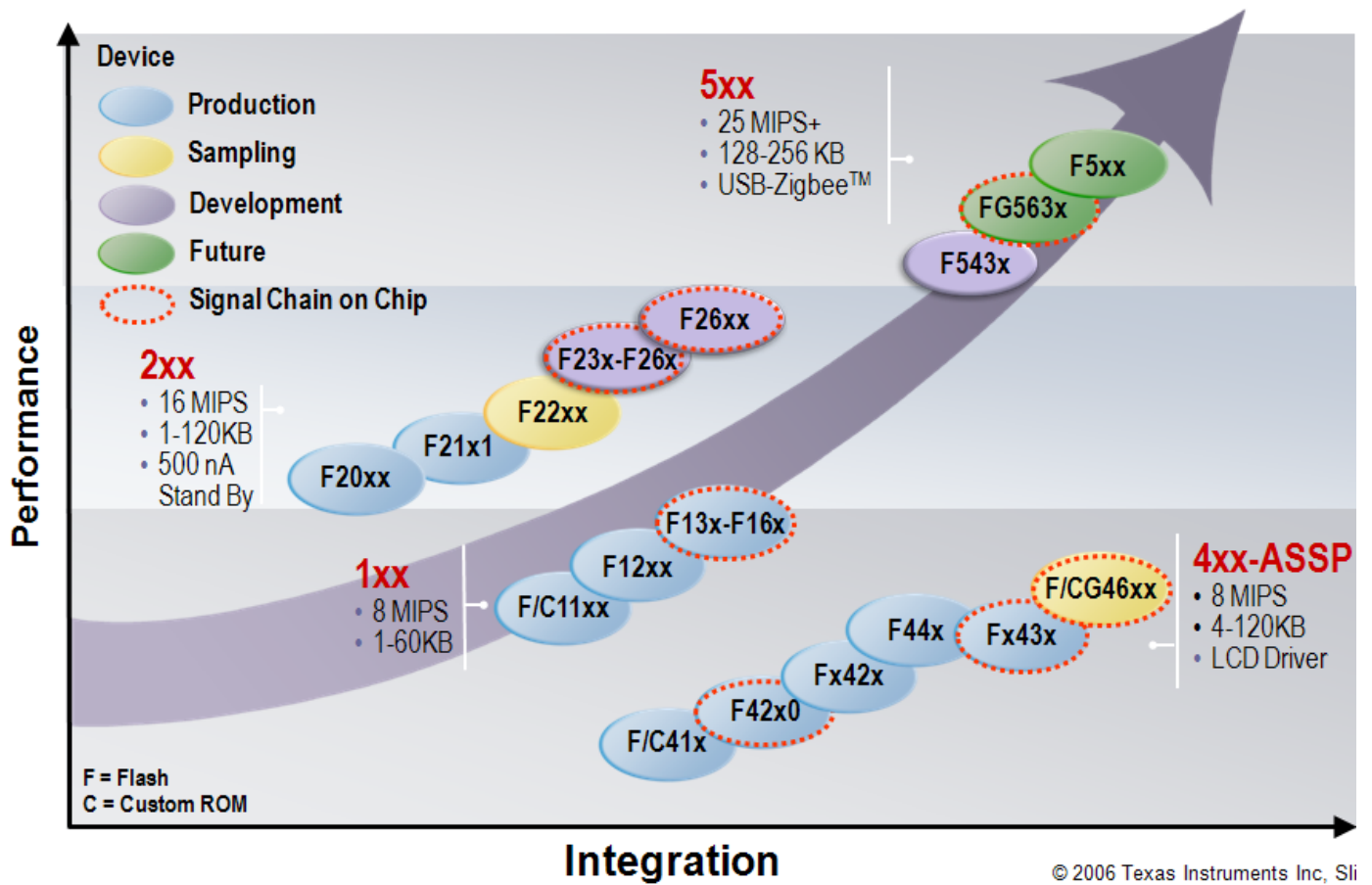


Bild: MSP430 Roadmap (Bsp)

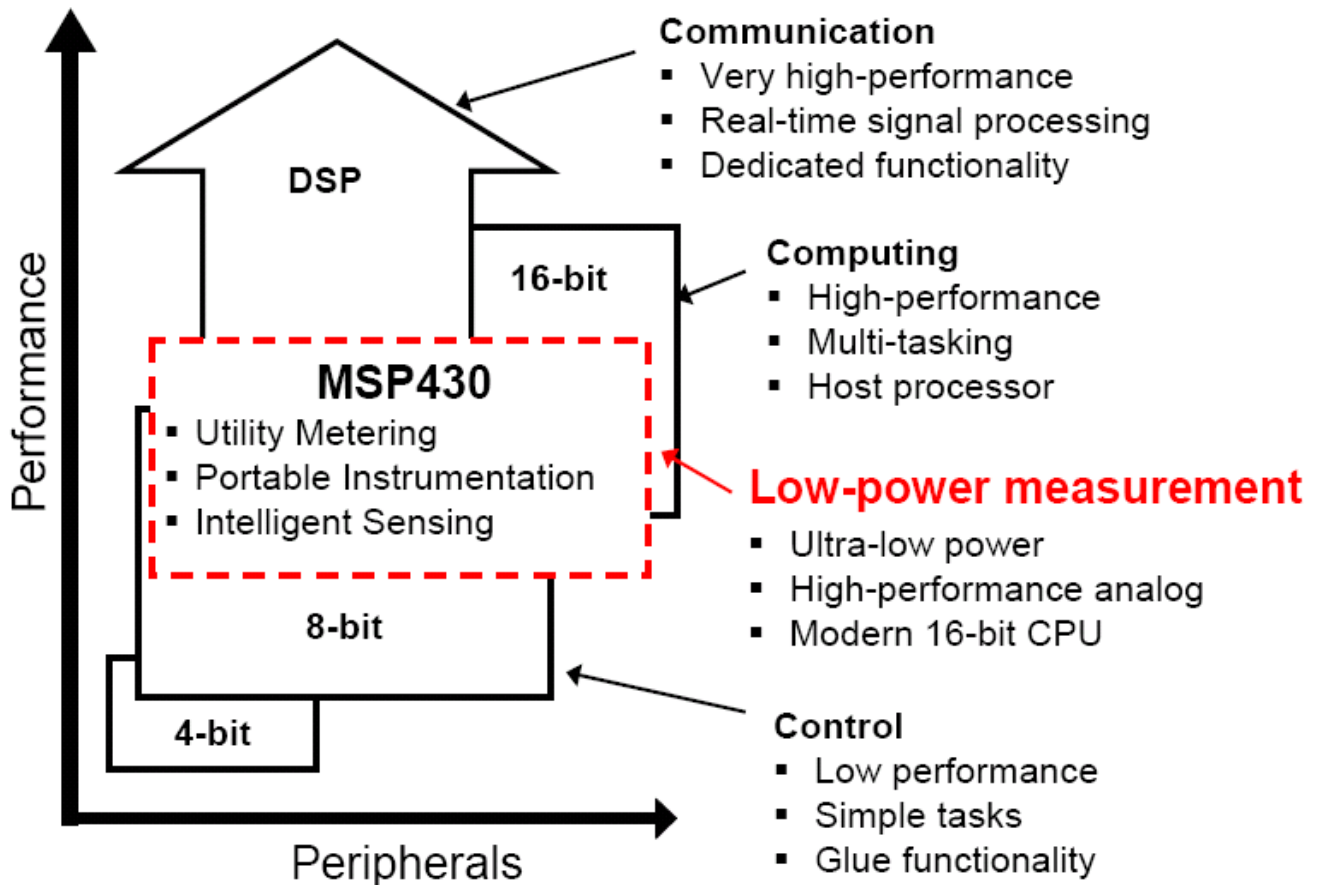


Bild: Positionierung der Mikrokontrollerfamilie MSP430 (2)

16 MIPS

Flash-Based F2xx Family with 16-Bit Watchdog (V_{CC} 1.8-3.6V)

(C) ROM (F) Flash	Program (KB)	SRAM (B)	I/O	16-Bit Timers		USCI ²	USI ²	DMA	SVS	Brown-Out Reset	MPY	Comp_A+	Temp Sensor	ADC Ch/Res	Additional Analog	Package(s)	Price ¹
				A	B												
MSP430F2001	1	128	10	2	—	—	—	—	—	✓	—	✓	—	Slope	—	14 PW, N, 16 RSA	\$0.55
MSP430F2011	2	128	10	2	—	—	—	—	—	✓	—	✓	—	Slope	—	14 PW, N, 16 RSA	\$0.70
MSP430F2002	1	128	10	2	—	—	✓	—	—	✓	—	—	✓	1/10	—	14 PW, N, 16 RSA	\$0.99
MSP430F2012	2	128	10	2	—	—	✓	—	—	✓	—	—	✓	1/10	—	14 PW, N, 16 RSA	\$1.15
MSP430F2003	1	128	10	2	—	—	✓	—	—	✓	—	—	✓	1/16	—	14 PW, N, 16 RSA	\$1.50
MSP430F2013	2	128	10	2	—	—	✓	—	—	✓	—	—	✓	1/16	—	14 PW, N, 16 RSA	\$1.65
MSP430F2101	1	128	16	3	—	—	—	—	—	✓	—	✓	—	slope	—	20 DGV, DW, PW, 24 RGE	\$0.90
MSP430F2111	2	128	16	3	—	—	—	—	—	✓	—	✓	—	slope	—	20 DGV, DW, PW, 24 RGE	\$0.99
MSP430F2121	4	256	16	3	—	—	—	—	—	✓	—	✓	—	slope	—	20 DGV, DW, PW, 24 RGE	\$1.35
MSP430F2131	8	256	16	3	—	—	—	—	—	✓	—	✓	—	slope	—	20 DGV, DW, PW, 24 RGE	\$1.70
MSP430F2234	8	512	32	3	3	✓	—	—	—	✓	—	—	—	1/10	(2) OPAMP	38 DA, 40 RHA	\$2.75
MSP430F2254	16	512	32	3	3	✓	—	—	—	✓	—	—	—	1/10	(2) OPAMP	38 DA, 40 RHA	\$3.10
MSP430F2274	32	1024	32	3	3	✓	—	—	—	✓	—	—	—	1/10	(2) OPAMP	38 DA, 40 RHA	\$3.55

¹ Suggested resale price in U.S. dollars in quantities of 1,000.

² USCI includes UART, IrDA, SPI and I²C. USI includes I²C and SPI.

New devices are listed in bold red.

Bild: MSP430 Mikrokontrollerfamilie (16 MHz), Quelle: MSP430-slab034j.pdf

Products for MSP430 Ultra-Low Power 16-bit Microcontrollers

<http://www.ti.com/lit/sg/slab034x/slab034x.pdf>

-> TI Product Selection Webseite

Products for MSP430 Ultra-Low Power 16-bit MCUs

http://www.ti.com/lstds/ti/microcontroller/16-bit_msp430/products.page

Frequency (MHz)	Flash	ROM	RAM	GPIO	ADC	Other Integrated Peripherals	LCD Segments	Timers
8 (81) 16 (19)	1 KB (5) 2 KB (5) 4 KB (6) 8 KB (12) 16 KB (11) 24 KB (3) 32 KB (16) 48 KB (6)	1 KB (1) 2 KB (2) 4 KB (3) 8 KB (4) 12 KB (1) 16 KB (3) 24 KB (1) 32 KB (1)	1 KB (21) 2 KB (12) 4 KB (2) 5 KB (2) 8 KB (2) 10 KB (1) 128 B (13) 256 B (27)	10 (6) 14 (28) 16 (4) 22 (4) 32 (12) 40 (5) 48 (37) 80 (4)	Slope (45) 10-Bit SAR (6) 10-bit SAR (6) 12-bit SAR (27) 14-bit SAR (5) 16-bit Sigma Delta (11)	Analog Comparator (58) DAC12 (3) DMA (16) Hardware Multiplier (29) SVS (37) 2 DAC 12 (16) 2 Op Amp (3) 3 Op Amp (7)	56 (3) 84 (5) 92 (7) 96 (9) 120 (5) 128 (12) 160 (10)	1 16-bit (2CCR) (6) 1 16-bit (3CCR) (55) 1 16-bit (5CCR) (10) 1 16-bit (7CCR) (19) 1 Watchdog/Interval (100) 2 16-bit (3CCR) (22) 2 8-bit (43)

00 Results - Show All Results < Previous | 71 - 80 of 100 Results | Next > < Previous 5 | Page 6 | 7 | 8 | 9 | 10 Customize Columns Download Spreadsheet

Compare Uncheck All	Part Number	Status	Sub Family	Frequency (MHz)	Flash	RAM	GPIO	ADC	Other Integrated Peripherals	Timers	Interface	Temp Sensor	Brown Out Reset
☐	MSP430F4270	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	32 KB	256 B	32	16-bit Sigma Delta	DAC12	1 16-bit (3CCR) 1 Watchdog/Interval	Timer UART	Yes	Yes
☐	MSP430F435	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	16 KB	512 B	48	12-bit SAR	Analog Comparator SVS	1 16-bit (3CCR) 2 16-bit (3CCR) 2 8-bit	1 USART (SPI or UART)	Yes	Yes
☐	MSP430F436	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	24 KB	1 KB	48	12-bit SAR	Analog Comparator SVS	1 16-bit (3CCR) 2 16-bit (3CCR) 2 8-bit	1 USART (SPI or UART)	Yes	Yes
☐	MSP430F437	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	32 KB	1 KB	48	12-bit SAR	Analog Comparator SVS	1 16-bit (3CCR) 2 16-bit (3CCR) 2 8-bit	1 USART (SPI or UART)	Yes	Yes
☐	MSP430F447	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	32 KB	1 KB	48	12-bit SAR	Analog Comparator Hardware Multiplier SVS	1 16-bit (3CCR) 1 16-bit (7CCR) 1 Watchdog/Interval 2 8-bit	2 USART (SPI or UART)	Yes	Yes
☐	MSP430F448	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	48 KB	2 KB	48	12-bit SAR	Analog Comparator Hardware Multiplier SVS	1 16-bit (3CCR) 1 16-bit (7CCR) 1 Watchdog/Interval 2 8-bit	2 USART (SPI or UART)	Yes	Yes
☐	MSP430F449	ACTIVE	MSP430x4xx - Flash/ROM LCD	8	60 KB	2 KB	48	12-bit SAR	Analog Comparator Hardware Multiplier SVS	1 16-bit (3CCR) 1 16-bit (7CCR) 1 Watchdog/Interval 2 8-bit	2 USART (SPI or UART)	Yes	Yes

-> Auswahl mittels Product Selection Tool (10.5.2014)

http://focus.ti.com/en/multimedia/flash/selection_tools/mcu/mcu.html

[TI Home](#) > [Microcontrollers \(MCU\)](#) > [MCU Selection Tool](#)

Microcontrollers (MCU)



Design Support ■ **Getting Started** ■ **Selection Tool** ■ **Training & Events** ■ **Developer Network** ■ **University**

MCU Selection Tool

All Microcontrollers (419) [Reset All Criteria](#) [Hide Criteria](#)

MCU Core Type:

Program Memory Type:

Program Memory (KB):

RAM (KB):

Max Speed (MHz):

CPU Features:

Standby Current (µA):

Active Current (µA/MHz):

Pins:

Package Type:

Package area (mm²):

Price (US\$) 1ku:

Timers:

PWM Channels:

Capture Pins:

Quad Encoder:

ADC Channels:

ADC Resolution (bits):

ADC Sample Rate:

Other Analog:

Temp. Range:

SPI:

CAN:

Buffered Serial Port UART:

I2C:

Ethernet: ☐

IEEE 1588: ☐

I2S: ☐

Hibernation Module: ☐

EPI/EMIF: ☐

Memory Protection Unit: ☐

Flexray: ☐

USB:

419 Results found [To sort/re-order/resize columns, drag-&-drop or click column headers.](#)

[Compare Selected](#) [Export to Spreadsheet](#)

Part Number	Flash	ROM	RAM	Max Speed	MPV	DMA	Standby Current	Active Current	MAX GPIO	Pins/Package	Package area	Timers	PWM	Capture Pins	QEP	LCD Segments	ADC Channels	ADC Resolution	Op Amp	CAN	Buffered Serial Port	UART	SPI	I2C	Price	Temp Range	ADC Sample Rate	Ethernet	IEEE 1588	EP/E MIF	USB	
LM3S101	8	0	2	20	0	0	0	0	18	28SOIC,48L	49,134.6,13	4	0	1	0	0	0	0	0	0	0	1	1	0	2.05	T,I	0	0	0	0	0	0
LM3S102	8	0	2	20	0	0	0	0	18	48LQFP,48V	49	4	0	2	0	0	0	0	0	0	1	1	1	2.05	T,I	0	0	0	0	0	0	
LM3S1110	64	0	16	25	0	0	0	0	41	100LQFP,100	100,196	5	0	2	0	0	0	0	0	0	2	1	0	2.75	T,I	0	0	0	0	0	0	

-> siehe auch Product Selection Guide TI MSP430.xls

Microsoft Excel - MSP430 Ultra-Low Power Microcontrollers.xls

Frage hier eingeben

75%

Arial 11

Family Name: MSP430 Ultra-Low Power Microcontrollers

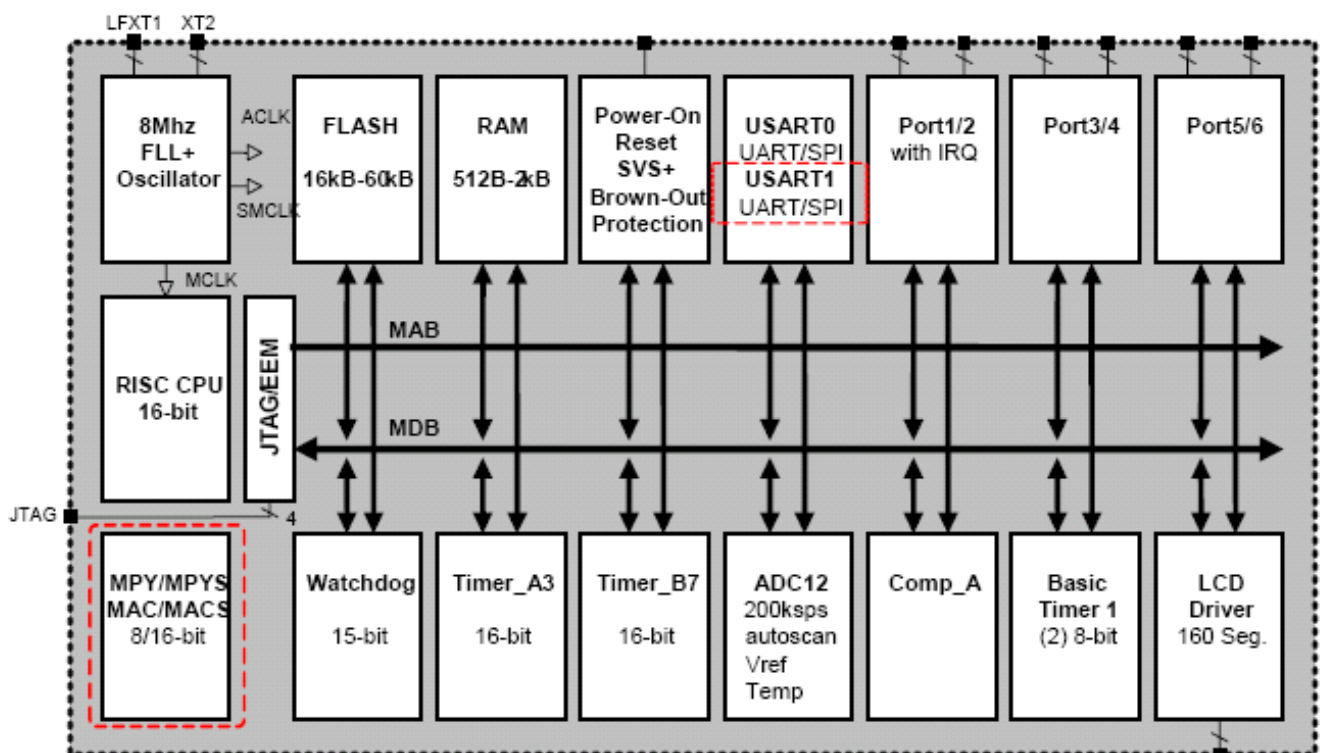
Part Number	Status	MSP430 Family	Type of Program Memory	Program Memory (kB)	RAM (Bytes)	I/O Pins	LCD Segments	ADC	Other Integrated Peripherals
MSP430C101	ACTIVE	FLASH/ROM NO LCD (x1xx FAMILY)	ROM	1	128	14		Slope	Analog Comparator
MSP430C101H	ACTIVE	FLASH/ROM NO LCD (x1xx FAMILY)	ROM	2	128	14		Slope	Analog Comparator
MSP430C102	ACTIVE	FLASH/ROM NO LCD (x1xx FAMILY)	ROM	4	256	14		Slope	Analog Comparator
MSP430C102H	ACTIVE	FLASH/ROM NO LCD (x1xx FAMILY)	ROM	8	256	48		Slope	Analog Comparator
MSP430C105	ACTIVE	FLASH/ROM NO LCD (x1xx FAMILY)	ROM	16	512	48		Slope	Analog Comparator
MSP430C311	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	2	128	14	92	Slope	
MSP430C312	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	4	256	14	92	Slope	
MSP430C313	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	8	256	14	92	Slope	
MSP430C314	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	12	512	14	92	Slope	
MSP430C315	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	16	512	14	92	Slope	
MSP430C323	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	8	256	14	84	14-bit SAR	
MSP430C325	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	16	512	14	84	14-bit SAR	
MSP430C336	ACTIVE	ROM/OTP LCD (x3xx FAMILY)	ROM	24	1024	40	120	Slope	Hardware Multiplier

4.4. Der Adressraum der MSP430-Mikrocontrollerfamilie

Die MSP430-Familie arbeitet nur über einen Adressraum, der auf eine Adressbreite von 16 Bit festgelegt ist. Dieser Adressraum umfasst

- alle Spezialregister der CPU und der weiteren im Chip integrierten peripheren Module,
- den Schreib-Lese-Speicher, der zur Aufnahme des Stapelspeichers (Stack), variabler Daten und Programme verwendet wird,
- den Programmspeicher
- leere Bereiche ohne Speicher
- Interruptvektoren

in einem 16-Bit-Bereich, also den Adressen zwischen 0 und 0FFFFH bzw. 0 bis 65535 dezimal.



100 TQFP
Buccini / Q42003

‘44x - Only

Bild: Blockschaltbild MSP430

- Low Supply-Voltage Range, 1.8 V to 3.6 V
 - Ultralow-Power Consumption:
 - Active Mode: 280 μ A at 1 MHz, 2.2 V
 - Standby Mode: 1.1 μ A
 - Off Mode (RAM Retention): 0.1 μ A
 - Five Power Saving Modes
 - Wake-Up From Standby Mode in less than 6 μ s
 - 16-Bit RISC Architecture, 125-ns Instruction Cycle Time
 - 12-Bit A/D Converter With Internal Reference, Sample-and-Hold and Autoscan Feature
 - 16-Bit Timer_B With Three[†] or Seven[‡] Capture/Compare-With-Shadow Registers
 - 16-Bit Timer_A With Three Capture/Compare Registers
 - On-Chip Comparator
 - Serial Communication Interface (USART), Select Asynchronous UART or Synchronous SPI by Software:
 - Two USARTs (USART0, USART1) — MSP430x44x Devices
 - One USART (USART0) — MSP430x43x Devices
 - Brownout Detector
 - Supply Voltage Supervisor/Monitor With Programmable Level Detection
 - Serial Onboard Programming, No External Programming Voltage Needed
 - Programmable Code Protection by Security Fuse
 - Integrated LCD Driver for Up to 160 Segments
 - Family Members Include:
 - MSP430F435: 16KB+256B Flash Memory, 512B RAM
 - MSP430F436: 24KB+256B Flash Memory, 1KB RAM
 - MSP430F437: 32KB+256B Flash Memory, 1KB RAM
 - MSP430F447: 32KB+256B Flash Memory, 1KB RAM
 - MSP430F448: 48KB+256B Flash Memory, 2KB RAM
 - MSP430F449: 60KB+256B Flash Memory, 2KB RAM
 - For Complete Module Descriptions, See The *MSP430x4xx Family User's Guide*, Literature Number SLAU056
- [†] 'F435, 'F436, and 'F437 devices
[‡] 'F447, 'F448, and 'F449 devices

Bild: Feature List MSP430x43x und MSP430x44x

		Funktion	Zugriff	Inhalt
0FFFFH	Unterbrechungs- vektorentabelle	ROM	W/B	Vektoren
0FFE0H				
0FFDFH	Festwertspeicher	ROM	W/B	Programm, Tabellen, Konstanten
	nichts	keine		
0200H	Schreib-Lese- Speicher	RAM	W/B	Daten, Stapelspeicher, Programm
01FFH	16-Bit-Peripherie- Module	Timer, ADC,..	Wort	Peripherie- daten/steuerung
0100H				
0FFH	8-Bit-Peripherie- Module	I/O,LCD, Timer,..	Byte	Peripherie- daten/steuerung
010H				
0FH	Spezialregister	SFR	Byte	Systemdaten/ -steuerung
0				

Bild: MSP430x44x Speicherorganisation (Memory Map)

Damit sind dieselben Befehle zur Speicherbearbeitung, für Peripherieaktivitäten und für Ein-/Ausgabeoperationen zuständig, und das mit allen Adressierungsarten.

In Zukunft soll der Speicher durch segmentierte Speicherblöcke erweitert werden. Dazu werden im Statusregister die führenden Adressbits von Codesegmenten (Programmsegmenten) als Codesegmentzeiger (Code Segment Pointer CSP) und getrennt davon ein Teil der Datenadresse als Datenzeiger (Data Pointer DPP) gespeichert.

Separate Memory Address Spaces für

Programme - Non-volatile (nichtflüchtig, permanent)

- Internal ROM
- External EPROM

Daten - Volatile

- External SRAM
- Internal RAM
 - General Purpose Registers
 - Bit Addressable Registers
 - Special Function Registers

4.4.1. MSP430 Memory Map im Detail

Memory Address	Description
0FFE0h–0FFFFh	Interrupt Vectors
0FFDFh	End of code space - All device
0F800h	Start of code space - 2K device
0F000h	Start of code space - 4K device
0E000h	Start of code space - 8K device
0D000h	Start of code space - 12K device
0C000h	Start of code space - 16K device
0A000h	Start of code space - 24K device
08000h	Start of code space - 32K device
04000h	Start of code space - 48K device

01100h	Start of code space - 60K device
010FFh	End of Information Memory: Flash devices except 'F110 and 'F1101
0107Fh	End of Information Memory: 'F110 and 'F1101
01000h	Start of Information Memory: Flash devices only
0FFFh	End of Boot Memory: Flash devices only
0C00h	Start of Boot Memory: Flash devices only
09FFh	End of RAM-2k devices
05FFh	End of RAM-1k devices
03FFh	End of RAM-512 byte devices
02FFh	End of RAM-256 byte devices
027Fh	End of RAM-128 byte devices
0200h	Start of RAM-All devices
01B0h – 01FFh	Unused (All devices)
01A0h – 01AFh	ADC Control ('1xx and '4xx devices) / Unused ('3xx devices)
0160h – 017Fh	Timer A (All devices)
0140h – 015Fh	ADC Conversion ('1xx and '4xx devices) / Unused ('3xx devices)
0130h – 013Fh	Multiplier (All devices)
0110h – 011Fh	ADC ('3xx devices) / Unused ('1xx and '4xx devices)
0100h – 010Fh	Unused (All devices)
00B0h – 00FFh	Unused (All devices)
0090h – 00AFh	LCD (Byte addressed, '4xx devices) / Unused ('1xx and '3xx devices)
0080h – 008Fh	ADC memory control (Byte addressed, '1xx and '4xx devices) / Unused ('3xx devices)
0070h – 007Fh	USART (Byte addressed, All devices)
0060h – 006Fh	Unused (All devices)
0050h – 005Fh	System Clock (Byte addressable, All devices) / Comparator ('1xx and '4xx devices) / Brownout ('4xx devices) / EPROM and crystal buffer ('3xx devices)
0040h – 004Fh	Basic Timer and 8-bit Counter (Byte addressable, '3xx and '4xx devices) / Unused ('1xx devices)

0030h – 003Fh	I/O ports 5 and 6 control (Byte addressable, '1xx and '4xx devices) / LCD (Byte addressable, '3xx devices)
0020h – 002Fh	I/O ports 1 and 2 control (Byte addressable, All devices)
0010h – 001Fh	I/O ports 3 and 4 control (Byte addressable, All devices), I/O port 0 (Byte addressable, '3xx devices)
0006h – 000Fh	Unused (All devices)
0005h	Module Enables 2 (Byte Addressable, all devices)
0004h	Module Enables 1 (Byte Addressable, all devices)
0003h	Interrupt Flags 2 (Byte Addressable, all devices)
0002h	Interrupt Flags 1 (Byte Addressable, all devices)
0001h	Interrupt Enables 2 (Byte Addressable, all devices)
0000h	Interrupt Enables 1 (Byte Addressable, all devices)

Bild: Memory Map (detailliert)

All die physikalisch getrennten Speicherbereiche, sowohl die internen Bereiche mit Festwertspeicher (ROM,...), Schreib-/Lesespeicher (RAM,...), Spezialregistern (SFR) und Peripheriemodulen, als auch -wenn vorhanden- die externen Bereiche sind in einem einzigen Adressraum positioniert (mapped).

Für alle Prozessoren mit einer maximalen Größe des Adressraumes von 64 kByte (65.536 Bytes), gilt das "kleine Speichermodell" (small memory model).

Dieses Speichermodell benutzt einen eindimensionalen linearen Adressraum mit Adressen von Null (00000h) bis 65.535 (0FFFFh).

4.4.2. Das Device Description File msp430F449.ddf

```
;;/*****
;; * Definitions of I/O register description file,
;; * interrupt/exception vectors, interrupt control registers,
;; * I/O register reset values.
;; *
;; * Copyright 2001-2003 IAR Systems. All rights reserved.
;; * $Revision: 1.9 $
;; *****/

;; Memory information ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;
;; Used to define address zones within the MSP430 address space (Memory).
;;
;; Name      may be almost anything
;; AddrSpace must be Memory
;; StartAdr  start of memory block
;; EndAdr    end of memory block
;; AccType   type of access, read-only (R) or read-write (RW)
```

[Memory]

```

;;          Name          AdrSpace   StartAdr   EndAdr     AccType
Memory0    = SFR          Memory     0x0000     0x01FF     RW
Memory1    = RAM          Memory     0x0200     0x09FF     RW
Memory2    = INFO         Memory     0x1000     0x10FF     R
Memory3    = FLASH        Memory     0x1100     0xFFDF     R

;; I/O Register description file ::::::::::::::::::::::::::::::::::::::

[SfrInclude]
File = MSP430F449.sfr

;; Interrupt definitions ::::::::::::::::::::::::::::::::::::::::::::::
[InterruptList]
;          Id          Table Adr     Prio     Enable          Pending
Interrupt0 = BASICTIMER_VECTOR 0x00     2      IE2.BTIE        IFG2.BTIFG
Interrupt1 = PORT2_VECTOR      0x02     2      P2IE.P2IE_0     P2IFG.P2IFG_0
Interrupt2 = PORT2_VECTOR      0x02     2      P2IE.P2IE_1     P2IFG.P2IFG_1
Interrupt3 = PORT2_VECTOR      0x02     2      P2IE.P2IE_2     P2IFG.P2IFG_2
Interrupt4 = PORT2_VECTOR      0x02     2      P2IE.P2IE_3     P2IFG.P2IFG_3
Interrupt5 = PORT2_VECTOR      0x02     2      P2IE.P2IE_4     P2IFG.P2IFG_4
Interrupt6 = PORT2_VECTOR      0x02     2      P2IE.P2IE_5     P2IFG.P2IFG_5
Interrupt7 = PORT2_VECTOR      0x02     2      P2IE.P2IE_6     P2IFG.P2IFG_6
Interrupt8 = PORT2_VECTOR      0x02     2      P2IE.P2IE_7     P2IFG.P2IFG_7
Interrupt9 = USART1TX_VECTOR   0x04     2      IE2.UTXIE1      IFG2.UTXIFG1
Interrupt10 = USART1RX_VECTOR  0x06     2      IE2.URXIE1      IFG2.URXIFG1
Interrupt11 = PORT1_VECTOR     0x08     2      P1IE.P1IE_0     P1IFG.P1IFG_0
Interrupt12 = PORT1_VECTOR     0x08     2      P1IE.P1IE_1     P1IFG.P1IFG_1
Interrupt13 = PORT1_VECTOR     0x08     2      P1IE.P1IE_2     P1IFG.P1IFG_2
Interrupt14 = PORT1_VECTOR     0x08     2      P1IE.P1IE_3     P1IFG.P1IFG_3
Interrupt15 = PORT1_VECTOR     0x08     2      P1IE.P1IE_4     P1IFG.P1IFG_4
Interrupt16 = PORT1_VECTOR     0x08     2      P1IE.P1IE_5     P1IFG.P1IFG_5
Interrupt17 = PORT1_VECTOR     0x08     2      P1IE.P1IE_6     P1IFG.P1IFG_6
Interrupt18 = PORT1_VECTOR     0x08     2      P1IE.P1IE_7     P1IFG.P1IFG_7
Interrupt19 = TIMERA1_VECTOR   0x0A     2      TACCTL1.CCIE    TACCTL1.CCIFG
Interrupt20 = TIMERA1_VECTOR   0x0A     2      TACCTL2.CCIE    TACCTL2.CCIFG
Interrupt21 = TIMERA1_VECTOR   0x0A     2      TACTL.TAIE      TACTL.TAIFG
Interrupt22 = TIMERA0_VECTOR   0x0C     2      TACCTL0.CCIE    TACCTL0.CCIFG
Interrupt23 = ADC_VECTOR       0x0E     2      ADC12IE         ADC12IFG
Interrupt24 = USART0TX_VECTOR  0x10     2      IE1.UTXIE0      IFG1.UTXIFG0
Interrupt25 = USART0RX_VECTOR  0x12     2      IE1.URXIE0      IFG1.URXIFG0
Interrupt26 = WDT_VECTOR       0x14     2      IE1.WDTIE       IFG1.WDTIFG
Interrupt27 = COMPARATORA_VECTOR 0x16     2      CACTL1.CAIE     CACTL1.CAIFG
Interrupt28 = TIMERB1_VECTOR   0x18     2      TBCCTL1.CCIE    TBCCTL1.CCIFG
Interrupt29 = TIMERB1_VECTOR   0x18     2      TBCCTL2.CCIE    TBCCTL2.CCIFG
Interrupt30 = TIMERB1_VECTOR   0x18     2      TBCCTL3.CCIE    TBCCTL3.CCIFG
Interrupt31 = TIMERB1_VECTOR   0x18     2      TBCCTL4.CCIE    TBCCTL4.CCIFG
Interrupt32 = TIMERB1_VECTOR   0x18     2      TBCCTL5.CCIE    TBCCTL5.CCIFG
Interrupt33 = TIMERB1_VECTOR   0x18     2      TBCCTL6.CCIE    TBCCTL6.CCIFG
Interrupt34 = TIMERB1_VECTOR   0x18     2      TBCTL.TBIE      TBCTL.TBIFG
Interrupt35 = TIMERB0_VECTOR   0x1A     2      TBCCTL0.CCIE    TBCCTL0.CCIFG
Interrupt36 = NMI_VECTOR       0x1C     2      IE1.OFIE        IFG1.NMIIFG
Interrupt37 = RESET_VECTOR     0x1E     1
;; End of file

```

4.5. Mikrocontrollerspeicher

Speichertyp		Löschen	Programmieren	Speicherinhalt ist bei
RAM	Random Access Memory "Speicher mit wahlfreiem Zugriff" Schreib-Lese-Speicher Read-Write-Memory (R/W-Speicher)	elektrisch	elektrisch	Stromabschaltung flüchtig
ROM	Read-Only-Memory Nur-Lese-Speicher Festwertspeicher	nicht möglich	durch Masken beim Herstellungsprozeß	Stromabschaltung nicht flüchtig
PROM	Programmable ROM Programmierbarer Festwert speicher			
EPROM	Erasable PROM Löschbarer Festwertspeicher	durch UV-Licht	elektrisch	
RPROM	Reprogrammable ROM Neuprogrammierbarer Festwertspeicher			
EEPROM	Electrically Erasable ROM Elektrisch löschbarer Festwertspeicher	elektrisch		
EAROM	Electrically Alterable ROM Elektrisch umprogrammierbarer Festwertspeicher			

Bild: Merkmale von Speichertypen auf Halbleiterbasis

Der Speicher der MSP430-Familie ist vollständig auf dem Silizium-Chip enthalten.

Auf den gesamten Speicher wird über zwei 16-Bit-Busse zugegriffen:

- **Speicher-Adress-Bus (Memory Address Bus MAB)** zur Bereitstellung der Speicheradresse durch die CPU
- **Speicher-Daten-Bus (Memory Data Bus MDB)** zum Lesen (Speicher => CPU) und Schreiben (CPU => Speicher)

Der Speicher ist technologisch einzuteilen in die beiden Arten

- **Festwertspeicher:** ein Speicher, dessen Inhalt durch die CPU nicht beliebig geändert werden kann (ROM, OTPROM, EPROM, EEPROM,...)
- **Schreib-Lese-Speicher:** ein Speicher, dessen Inhalt jederzeit geändert werden kann und der bei Abschaltung der Versorgungsspannung seinen Wert verliert (Random Access Memory RAM)

Festwertspeicher kann bei der Definition eines Familienmitgliedes entweder benutzerpro-

grammierbar (One Time Programmable Read-Only Memory OTPROM, Erasable Programmable Read-Only Memory EPROM) oder bei der Fertigung festgelegt (Read-Only Memory ROM) werden. Der Zugriff auf gewöhnlichen externen Speicher ist für spätere Familienmitglieder vorgesehen. Programmseitig ist der Speicher jedoch einzuteilen in Programmspeicher und Datenspeicher.

Programmspeicher

Der Zugriff auf den Programmspeicher ist beim Zugriff auf Anweisungen immer Word-orientiert, d. h. die CPU greift nur auf gerade Adressen wortweise mit jeweils 16 Bit zu. Daten können wortweise (16 Bit an gerader Adresse) oder byteweise (8 Bit an beliebiger Adresse) gelesen oder geschrieben werden.

Jeder Zugriff auf den Speicher benutzt den 16-Bit-Datenbus und so viele der niederwertigen Adressleitungen wie für die Speicherunterscheidung gerade benötigt werden.

Speicherblöcke werden erst bei Zugriff automatisch selektiert und durch Modulaktivierungssignale (Module Enable Signals) aktiviert, um den Stromverbrauch des MSP430 möglichst klein zu halten.

Datenspeicher

Der gesamte Befehlssatz arbeitet wort- oder byteorientiert. Alle Zugriffe auf den Stapelspeicher und den Programmzähler haben wortorientiert mit geraden Adressen zu erfolgen.

Spezialregister

Der Betrieb der verschiedenen Module der MSP430-Familie wird im wesentlichen durch Informationen gesteuert, die in Spezialregistern (SFR) gespeichert sind. Die verschiedenen Bits in den SFR erlauben Unterbrechungen (Interrupts), liefern dem Programm Informationen über die Ursache einer Unterbrechung und steuern den Betrieb peripherer Module.

Angehaltene Peripheriegeräte stellen ihren Betrieb ein, um den Stromverbrauch zu minimieren. Dabei bleiben alle in den Registern des Moduls enthaltenen Daten erhalten. Die SFR's sind in zwei Adressbereichen untergebracht, einem für Byte-Register und einem für Word-Register.

4.6. Die CPU

Die CPU Register

The 16-bit program counter (PC/R0) points to the next instruction to be executed. Each instruction uses an even number of bytes (two, four, or six), and the PC is incremented accordingly. Instruction accesses in the 64-KB address space are performed on word boundaries, and the PC is aligned to even addresses.

The PC can be addressed with all instructions and addressing modes.

Examples

MOV #LABEL, PC	; Branch to address LABEL
MOV LABEL, PC	; Branch to address contained in LABEL
MOV @R14, PC	; Branch indirect to address in R14

Stack Pointer

The stack pointer (SP/R1) is used by the CPU to store the return addresses of subroutine calls and interrupts. It uses a predecrement, postincrement scheme. In addition, the SP can be used by software with all instructions and addressing modes.

Examples

```
MOV 2(SP), R6      ; Item I2 -> R6
MOV R7, 0(SP)      ; Overwrite TOS with R7
PUSH #0123h        ; Put 0123h onto TOS
POPR 8             ; R8 = 0123h
```

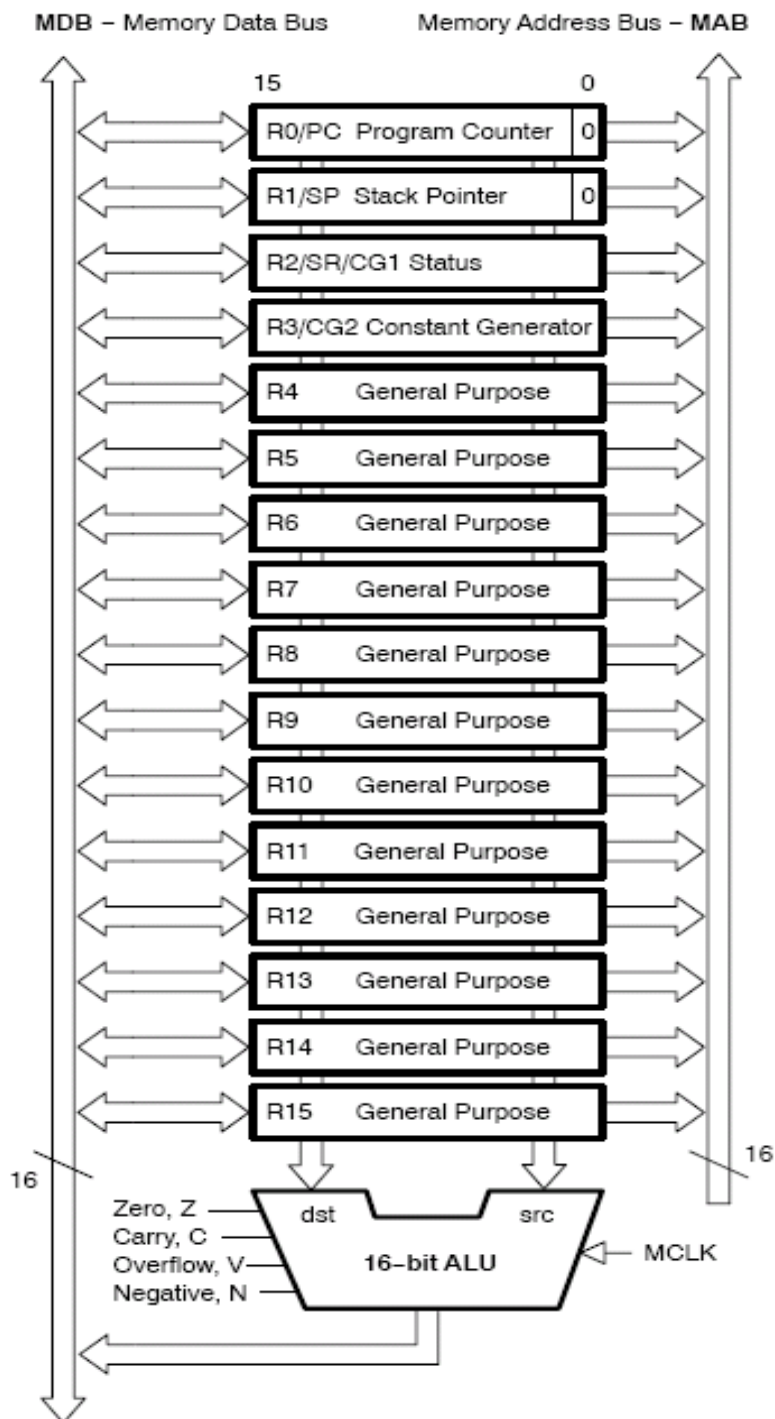


Bild: CPU Block Diagramm

4.6.1. Addressing Modes

Seven addressing modes for the source operand and four addressing modes for the destination operand can address the complete address space with no exceptions. The bit numbers in Table 3–3 describe the contents of the As(source) and Ad (destination) mode bits.

As/Ad	Addressing Mode	Syntax	Description
00/0	Register mode	Rn	Register contents are operand
01/1	Indexed mode	X(Rn)	(Rn + X) points to the operand. X is stored in the next word.
01/1	Symbolic mode	ADDR	(PC + X) points to the operand. X is stored in the next word. Indexed mode X(PC) is used.
01/1	Absolute mode	&ADDR	The word following the instruction contains the absolute address. X is stored in the next word. Indexed mode X(SR) is used.
10/–	Indirect register mode	@Rn	Rn is used as a pointer to the operand.
11/–	Indirect autoincrement	@Rn+	Rn is used as a pointer to the operand. Rn is incremented afterwards by 1 for .B instructions and by 2 for .W instructions.
11/–	Immediate mode	#N	The word following the instruction contains the immediate constant N. Indirect autoincrement mode @PC+ is used.

Bild: Seven Addressing Modes

4.6.2. The Instruction Set

Format I Source, Destination	Format II Single Operand	Format III +/- 9bit Offset	Support
add(.b)	br	jmp	clrc
addc(.b)	call	jc	setc
and(.b)	swpb	jnc	clrz
bic(.b)	sxt	jeq	setz
bis(.b)	push(.b)	jne	clrn
bit(.b)	pop(.b)	jge	setn
cmp(.b)	rra(.b)	j1	dint
dadd(.b)	rrc(.b)	jn	eint
mov(.b)	inv(.b)		nop
sub(.b)	inc(.b)		ret
subc(.b)	incd(.b)		reti
xor(.b)	dec(.b)		
	decdd(.b)		
	adc(.b)		
	sbc(.b)		
	clr(.b)		
	dadc(.b)		
	rla(.b)		
	rlc(.b)		
	tst(.b)		

Bild: Befehlsübersicht

Mnemonic		Description		V	N	Z	C
ADC(.B) [†]	dst	Add C to destination	dst + C → dst	*	*	*	*
ADD(.B)	src, dst	Add source to destination	src + dst → dst	*	*	*	*
ADDC(.B)	src, dst	Add source and C to destination	src + dst + C → dst	*	*	*	*
AND(.B)	src, dst	AND source and destination	src .and. dst → dst	0	*	*	*
BIC(.B)	src, dst	Clear bits in destination	.not.src .and. dst → dst	-	-	-	-
BIS(.B)	src, dst	Set bits in destination	src .or. dst → dst	-	-	-	-
BIT(.B)	src, dst	Test bits in destination	src .and. dst	0	*	*	*
BR [†]	dst	Branch to destination	dst → PC	-	-	-	-
CALL	dst	Call destination	PC+2 → stack, dst → PC	-	-	-	-
CLR(.B) [†]	dst	Clear destination	0 → dst	-	-	-	-
CLRC [†]		Clear C	0 → C	-	-	-	0
CLR [†]		Clear N	0 → N	-	0	-	-
CLRZ [†]		Clear Z	0 → Z	-	-	0	-
CMP(.B)	src, dst	Compare source and destination	dst - src	*	*	*	*
DADC(.B) [†]	dst	Add C decimally to destination	dst + C → dst (decimally)	*	*	*	*
DADD(.B)	src, dst	Add source and C decimally to dst.	src + dst + C → dst (decimally)	*	*	*	*
DEC(.B) [†]	dst	Decrement destination	dst - 1 → dst	*	*	*	*
DECD(.B) [†]	dst	Double-decrement destination	dst - 2 → dst	*	*	*	*
DINT [†]		Disable interrupts	0 → GIE	-	-	-	-
EINT [†]		Enable interrupts	1 → GIE	-	-	-	-
INC(.B) [†]	dst	Increment destination	dst + 1 → dst	*	*	*	*
INCD(.B) [†]	dst	Double-increment destination	dst+2 → dst	*	*	*	*
INV(.B) [†]	dst	Invert destination	.not.dst → dst	*	*	*	*
JC/JHS	label	Jump if C set/Jump if higher or same		-	-	-	-
JEQ/JZ	label	Jump if equal/Jump if Z set		-	-	-	-
JGE	label	Jump if greater or equal		-	-	-	-
JL	label	Jump if less		-	-	-	-
JMP	label	Jump	PC + 2 x offset → PC	-	-	-	-
JN	label	Jump if N set		-	-	-	-
JNC/JLO	label	Jump if C not set/Jump if lower		-	-	-	-
JNE/JNZ	label	Jump if not equal/Jump if Z not set		-	-	-	-
MOV(.B)	src, dst	Move source to destination	src → dst	-	-	-	-
NO [†]		No operation		-	-	-	-
POP(.B) [†]	dst	Pop item from stack to destination	@SP → dst, SP+2 → SP	-	-	-	-
PUSH(.B)	src	Push source onto stack	SP - 2 → SP, src → @SP	-	-	-	-
RET [†]		Return from subroutine	@SP → PC, SP + 2 → SP	-	-	-	-
RETI		Return from interrupt		*	*	*	*
RLA(.B) [†]	dst	Rotate left arithmetically		*	*	*	*
RLC(.B) [†]	dst	Rotate left through C		*	*	*	*
RRA(.B)	dst	Rotate right arithmetically		0	*	*	*
RRC(.B)	dst	Rotate right through C		*	*	*	*
SBC(.B) [†]	dst	Subtract not(C) from destination	dst + 0FFFFh + C → dst	*	*	*	*
SETC [†]		Set C	1 → C	-	-	-	1
SETN [†]		Set N	1 → N	-	1	-	-
SETZ [†]		Set Z	1 → C	-	-	1	-
SUB(.B)	src, dst	Subtract source from destination	dst + .not.src + 1 → dst	*	*	*	*
SUBC(.B)	src, dst	Subtract source and not(C) from dst.	dst + .not.src + C → dst	*	*	*	*
SWPB	dst	Swap bytes		-	-	-	-
SXT	dst	Extend sign		0	*	*	*
TST(.B) [†]	dst	Test destination	dst + 0FFFFh + 1	0	*	*	1
XOR(.B)	src, dst	Exclusive OR source and destination	src .xor. dst → dst	*	*	*	*

[†] Emulated Instruction

Bild: MSP430 Instruction Set

	000	040	080	0C0	100	140	180	1C0	200	240	280	2C0	300	340	380	3C0
0xxx																
4xxx																
8xxx																
Cxxx																
1xxx	RRC	RRC.B	SWPB		RRA	RRA.B	SXT		PUSH	PUSH.B	CALL		RETI			
14xx																
18xx																
1Cxx																
20xx	JNE/JNZ															
24xx	JEQ/JZ															
28xx	JNC															
2Cxx	JC															
30xx	JN															
34xx	JGE															
38xx	JL															
3Cxx	JMP															
4xxx	MOV, MOV.B															
5xxx	ADD, ADD.B															
6xxx	ADDC, ADDC.B															
7xxx	SUBC, SUBC.B															
8xxx	SUB, SUB.B															
9xxx	CMP, CMP.B															
Axxx	DADD, DADD.B															
Bxxx	BIT, BIT.B															
Cxxx	BIC, BIC.B															
Dxxx	BIS, BIS.B															
Exxx	XOR, XOR.B															
Fxxx	AND, AND.B															

Bild: Core Instruction Map

Double Operand Instructions

Mnemonic	Source, Destination	
	Register	Register
	Indexed	Indexed
	Absolute	Absolute
	Symbolic	Symbolic
	Immediate	
	Register indirect	
	Register indirect autoincrement	

Single Operand Instructions

Mnemonic	Destination
	Register
	Indexed
	Absolute
	Symbolic
	Immediate
	Register indirect
	Register indirect autoincrement

12 Instructions 28 Combinations

7 Instructions 7 Addressing Modes

The source and destination of an instruction are defined by the following fields:

src	The source operand defined by As and S-reg
dst	The destination operand defined by Ad and D-reg
As	The addressing bits responsible for the addressing mode used for the source (src)
S-reg	The working register used for the source (src)
Ad	The addressing bits responsible for the addressing mode used for the destination (dst)
D-reg	The working register used for the destination (dst)
B/W	Byte or word operation: 0: word operation 1: byte operation

Double Operand (Format I) Instructions

Double Operand Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-Reg				Ad	B/W	As	D-Reg				

Table 3–11 lists and describes the double operand instructions.

Double Operand Instructions

Mnemonic	S-Reg, D-Reg	Operation	Status Bits			
			V	N	Z	C
MOV(.B)	src,dst	src → dst	–	–	–	–
ADD(.B)	src,dst	src + dst → dst	*	*	*	*
ADDC(.B)	src,dst	src + dst + C → dst	*	*	*	*
SUB(.B)	src,dst	dst + .not.src + 1 → dst	*	*	*	*
SUBC(.B)	src,dst	dst + .not.src + C → dst	*	*	*	*
CMP(.B)	src,dst	dst – src	*	*	*	*
DADD(.B)	src,dst	src + dst + C → dst (decimally)	*	*	*	*
BIT(.B)	src,dst	src .and. dst	0	*	*	*
BIC(.B)	src,dst	.not.src .and. dst → dst	–	–	–	–
BIS(.B)	src,dst	src .or. dst → dst	–	–	–	–
XOR(.B)	src,dst	src .xor. dst → dst	*	*	*	*
AND(.B)	src,dst	src .and. dst → dst	0	*	*	*

- * The status bit is affected
- The status bit is not affected
- 0 The status bit is cleared
- 1 The status bit is set

Single Operand (Format II) Instructions

Single Operand Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code									B/W	Ad	D/S-Reg				

Single Operand Instructions

Mnemonic	S-Reg, D-Reg	Operation	Status Bits			
			V	N	Z	C
RRC (.B)	dst	C → MSB →LSB → C	*	*	*	*
RRA (.B)	dst	MSB → MSB →LSB → C	0	*	*	*
PUSH (.B)	src	SP - 2 → SP, src → @SP	-	-	-	-
SWPB	dst	Swap bytes	-	-	-	-
CALL	dst	SP - 2 → SP, PC+2 → @SP dst → PC	-	-	-	-
RETI		TOS → SR, SP + 2 → SP TOS → PC, SP + 2 → SP	*	*	*	*
SXT	dst	Bit 7 → Bit 8.....Bit 15	0	*	*	*

- * The status bit is affected
- The status bit is not affected
- 0 The status bit is cleared
- 1 The status bit is set

Jumps

Jump Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code					C		10-Bit PC Offset								

The next Table lists and describes the jump instructions.

Jump Instructions

Mnemonic	S-Reg, D-Reg	Operation
JEQ/JZ	Label	Jump to label if zero bit is set
JNE/JNZ	Label	Jump to label if zero bit is reset
JC	Label	Jump to label if carry bit is set
JNC	Label	Jump to label if carry bit is reset
JN	Label	Jump to label if negative bit is set
JGE	Label	Jump to label if (N .XOR. V) = 0
JL	Label	Jump to label if (N .XOR. V) = 1
JMP	Label	Jump to label unconditionally

Conditional jumps support program branching relative to the PC and do not affect the status bits. The possible jump range is from -511 to +512 words relative to the PC value at the jump instruction. The 10-bit program-counter offset is treated as a signed 10-bit value that is doubled and added to the program counter:

$$PC_{\text{new}} = PC_{\text{old}} + 2 + PC_{\text{offset}} \times 2$$

4.7. Beurteilungskriterien für die Auswahl eines Mikrocomputersystems

- Datenformat (4, 8, 16, 32 bit)
- Befehlsvorrat, Adressierungsmöglichkeiten
- CPU-Struktur
- Mikroprogrammierbarkeit
- Geschwindigkeit
- Interrupt - Möglichkeiten
- Schnittstelleneigenschaften
- Speicherkonzept
- Versorgungsspannung
- Verfügbarkeit von Bausteinen
- Sicherung der Lieferfähigkeit (Zweitlieferant), Zukunftssicherheit des Systems
- Zusatz-Hard- und Software für Systementwicklung, Tool-Chain
- Unterstützung durch den Hersteller
- Dokumentations-und Ausbildungsangebot
- Kosten

4.8. Speichertypen in der Programmiersprache C/C++

Der verfügbare adressierbare Speicher wird in vier Bereiche eingeteilt:

Programmspeicher	-> ROM	
Statischer Speicher	-> RAM	-> globaler Speicher für globale Variablen
Haldenspeicher, Kellerspeicher (Heap)	-> RAM	-> für die dynamische Speicherverwaltung
Stapelspeicher (Stack)	-> RAM	

Programmspeicher

Der Programmspeicher beinhaltet, wie der Name schon verrät, das Programm. In zahlreichen Sprachen ist er strikt vom Datenspeicher getrennt. Manche Sprachen erlauben aus programmiertechnischen Gründen keine globalen Variablen. Bei C/C++ sind diese zwar erlaubt, es wird aber zwischen Programm- und Datenspeicher grundsätzlich unterschieden.

Neben dem Speicher für globale Variable bleiben noch zwei Bereiche für die Daten. Einer dieser Bereiche wird als Stapel (Stack) bezeichnet. Den zweiten Speicherbereich bezeichnet man als Halde (Heap). Er dient der dynamischen Speicherverwaltung.

Stapelspeicher (Stack)

Der Stack ist im Gegensatz zum Heap sehr klein. Er hat nur eine Größe von mehreren Kilobytes bis wenigen Megabytes. Der Stack arbeitet nach dem LIFO (Last In First Out) Prinzip, d. h. was zuletzt angefordert wurde, muss auch als erstes wieder freigegeben werden.

Wenn eine Funktion aufgerufen wird, wird die aktuelle Programmadresse (=Rücksprungadresse) auf den Stapel gelegt. Bei jedem Aufruf werden ebenso Informationen über den aktuellen Prozessorzustand (Prozessor Statuswort) und ggf. CPU interne Hilfsregister R1, R2 etc. auf dem Stack gesichert werden, soweit sie im Unterprogramm überschrieben werden. Innerhalb der Funktionen werden möglicherweise Variablen angelegt, die ebenso auf dem Stapel landen. Dies wird vom Compiler zur Übersetzungszeit festgelegt und stellt somit eine statische Speicherverwaltung dar.

Am Ende der Funktion werden die Speicherbereiche der Variablen wieder freigegeben und das Programm springt zur Rücksprungadresse, die jetzt oben auf dem Stapel liegt.

Die aktuelle Position im Stack markiert der „Stackpointer“. Wenn eine Funktion von einer anderen Funktion aufgerufen wird, wird dieser Pointer so im Stack verschoben, dass genügend Speicherplatz für die lokalen Variablen und den Zustandsdaten zur Verfügung steht. Wenn die Funktion durchlaufen ist, wird der Stack-Pointer auf die Rücksprungadresse gesetzt und der Speicher wieder freigegeben.

Haldenspeicher (Heap)

Der Haldenspeicher wird zur Laufzeit des Programmes, also dynamisch zu einem beliebigen Zeitpunkt angefordert und auch wieder freigegeben. Somit kann innerhalb einer Funktion

Halde Speicher angefordert, und nach Beendigung der Funktion ein Objekt, das auf der Halde liegt, weiterhin genutzt werden. Es wird also nicht mit Beendigung der Funktion ungültig. Der Speicher wird im Rahmen der sog. dynamischen Speicherverwaltung zur Laufzeit festgelegt.

Programmiersprachen unterstützen die dynamische Speichieranforderung auf unterschiedliche Weisen. In Standard ISO-C gibt es dafür beispielsweise die Funktionen malloc(), calloc() und realloc(). Mit der Funktion free() wird der Speicher wieder freigegeben.

Zur Speicherung von Daten werden in Programmiersprachen Variablen benutzt. Jede Variable besitzt dabei einen Typ (Datentyp). Dieser Datentyp gibt die Art der Daten vor, die in der Variable abgelegt werden können.

Stack und den Heap - Wertetypen und Verweistypen

Der Stack und der Heap werden auch zum Abspeichern von Variablen verwendet. Dabei unterscheidet man zwischen Wertetypen und Verweistypen.

Die Daten von Wertetypen werden auf dem Stack gespeichert:

- Alle numerischen Datentypen,
- Boolean,
- Char,
- alle Strukturen, auch wenn ihre Member Verweistypen sind, Enumerationen

Verweistypen hingegen schreiben ihren Wert auf dem Heap. Auf dem Stack wird lediglich ein Zeiger bzw. Verweis geschrieben, wo im Heap die Daten zu finden sind:

- Strings,
- alle Arrays, auch wenn ihre Elemente Wertetypen sind,
- Klassentypen (C++-Klasse)

Beispiel:

```
void *malloc(size_t size);           // Prototyp- Deklaration, siehe Library:  stdlib.h
char *String;                        // -> Stack
String = (char *) malloc(1000);      // -> Heap
```

int main()

```
{
    A *pObjekt (0);                  // Erzeuge Zeiger auf ein A-Objekt, mit Konstruktoraufruf -> Stack
    pObjekt = new A;                 // Instanzieren des Objektes pObjekt auf dem Heap
    delete pObjekt;                  // Objekt „pObjekt“ wird gelöscht -> im Heap (Garbage Collection)

    char *pszMemory = new char[1000]; // 1000 Bytes auf dem Heap allokalieren
    delete [ ] pszMemory;             // Speicherblock wieder freigeben
}
```

4.10. Testfragen

1. Skizzieren Sie die prinzipiellen Unterschiede zwischen einem Mikroprozessor (CPU) und einem Mikrocontroller (MCU)?
2. Aus welchen Bestandteilen besteht eine CPU prinzipiell?
3. Nennen Sie typische Komponenten eines Embedded-Microcontrollers (MCU)
4. Was versteht man unter einer sog. Memory Map und einem Memory Map I/O?
5. Was versteht man unter den Begriffen Mikrocontrollerfamilie, Derivate
6. Ordnen Sie die Begriffe Stack, Statischer Speicher, Kellerspeicher (Heap). Welche Variablentypen werden üblicherweise in diesen Speicherbereichen abgelegt?
7. In dem Stack legt der Prozessor nach dem First-in-Last-out-Prinzip Daten und Adressen ab. Diese Technik nutzt z.B. ein C-Compiler beim Aufruf einer Unterprogrammfunktion und dem anschließenden Rücksprung ins aufrufende Programm.

Beispiel:

```
int func1(int para1, int para2)
{
    return (para1+para2)
}
```

```
void main(void)
{
    int result;
    result = func1(2,4);
}
```

Wie verändert sich der Stackpointer während des Programmaufrufes von func1 in der Hauptfunktion main() und welche Daten werden in welcher Reihenfolge im Stack abgelegt bzw. „entnommen“.

8. Wozu benötigt ein Mikroprozessor einen Datenbus, Adressbus und einen Steuerbus?
9. Skizzieren Sie das prinzipiellen zeitlichen Ablauf eines Speicherzugriffes, einmal für Memory Read und einmal für einen Memory Write Cycle.

10. What functions are in a microcontroller chip but NOT in a microprocessor?

- Interrupts, Instruction Decoder, Instruction Register
- Program memory, Data memory, I/O
- Program Counter, Registers, Data Bus
- Peripheral Devices, Address Bus, Data Bus

11. What buses are present in a microcontroller system?

- Address, Data, and Control
- Input and Output
- Address and Instruction
- Address, Data and Bluebird

12. From what point of reference are the terms "read" and "write" defined?

- The data bus
- The I/O Device
- The CPU
- The Memory

13. What is the purpose of the address decoder?

- To translate logical addresses to physical addresses.
- It selects the appropriate CPU register.

14. Geben Sie an, welche Speicherarten jeweils vorliegen. Wo werden die Variablen jeweils abgespeichert? Sind die Funktionen a priori reentrant?

a)

```
void strcpy(char *dest, char *src)
{
    while (*dest++ = *src++)
        { ;
        }
    *dest = NUL;
}
```

b)

```
int temp;
void swap (int *x, int *y)
{
```

```
temp = *x;  
*x = *y;  
*y = temp;  
}
```

c)

```
void swap (int *x, int *y)  
{  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

d)

```
void swap (int *x, int *y)  
{  
    static int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

5. ENTWICKLUNGSSCHRITTE FÜR EIN MIKROCOMPUTERSYSTEM

Problemdefinition:	Leistungsumfang des Systems definieren, Datenflussplan erstellen (SADT); Requirementsanalyse (Functional, Non-Functional, System, Software, Hardware-, GUI- Requirements)
Wirtschaftlichkeitsprüfung (Realisierungsbeschluss):	Abschätzen von Geschwindigkeit, Speicherbedarf, Bauelementeaufwand, Prüfung auf technische und wirtschaftliche Realisierbarkeit
Systemanalyse:	Problemstruktur analysieren, Use Cases definieren, Klassendesign u. Transaktionsdiagramme erstellen (UML) Programmablaufplan erstellen, Blockschaltbild der Schaltung
Hardwareentwicklung:	Kauf/Eigenentwicklung
Softwareentwicklung:	Detaillierter Programmablaufplan, Codieren, Programmeingabe, Übersetzen, Simulation, Debugging
Modultest, Integrationstest, Systemtest, Fehlerkorrektur:	Anbindung des Prozesses, Test des Zusammenspiels
Dokumentation:	Hardware, Software

6. DER SOFTWARE BUILD PROZESS BEI EMBEDDED SYSTEMEN

Compiler

Die Aufgabe des Compilers liegt darin, den in einer Hochsprache abgefassten Quellcode, in Assembler-Quellcode zu übersetzen. Die vielen problemorientierten, für Menschen lesbaren Hochsprachen, müssen von Compilern in eine prozessorspezifische Maschinensprache übersetzt werden, die von dem Computer direkt verarbeitet werden kann.

Assembler

Der Assembler ist das Werkzeug, das den Assembler-Sourcecode (meistens erzeugt vom Compiler) in Maschinensprache übersetzt, in einen so genannten Objectcode. Dies ist die Sprache des Prozessors, die für Menschen nur sehr schwer lesbar ist.

Der Zwischenschritt von Compiler und Assembler über den Assembler-Sourcecode ist auf den ersten Blick unnötig. Tatsächlich gibt es ältere Compiler, die diesen Schritt auslassen. Es hat

sich jedoch gezeigt, dass dieser Zwischenschritt sinnvoll ist, da sehr viel Flexibilität erreicht werden kann. Zum Beispiel ist es möglich, Teile von Programmen in verschiedenen Hochsprachen zu schreiben, diese mit entsprechenden Compilern in Assembler-Sourcecode zu übersetzen und dann mit einem Assembler den Objektcode für den Prozessor zu erzeugen. Es wird so auch möglich Optimierungen oder Fehlerbehebungen durchzuführen, zu denen der Compiler nicht in der Lage ist.

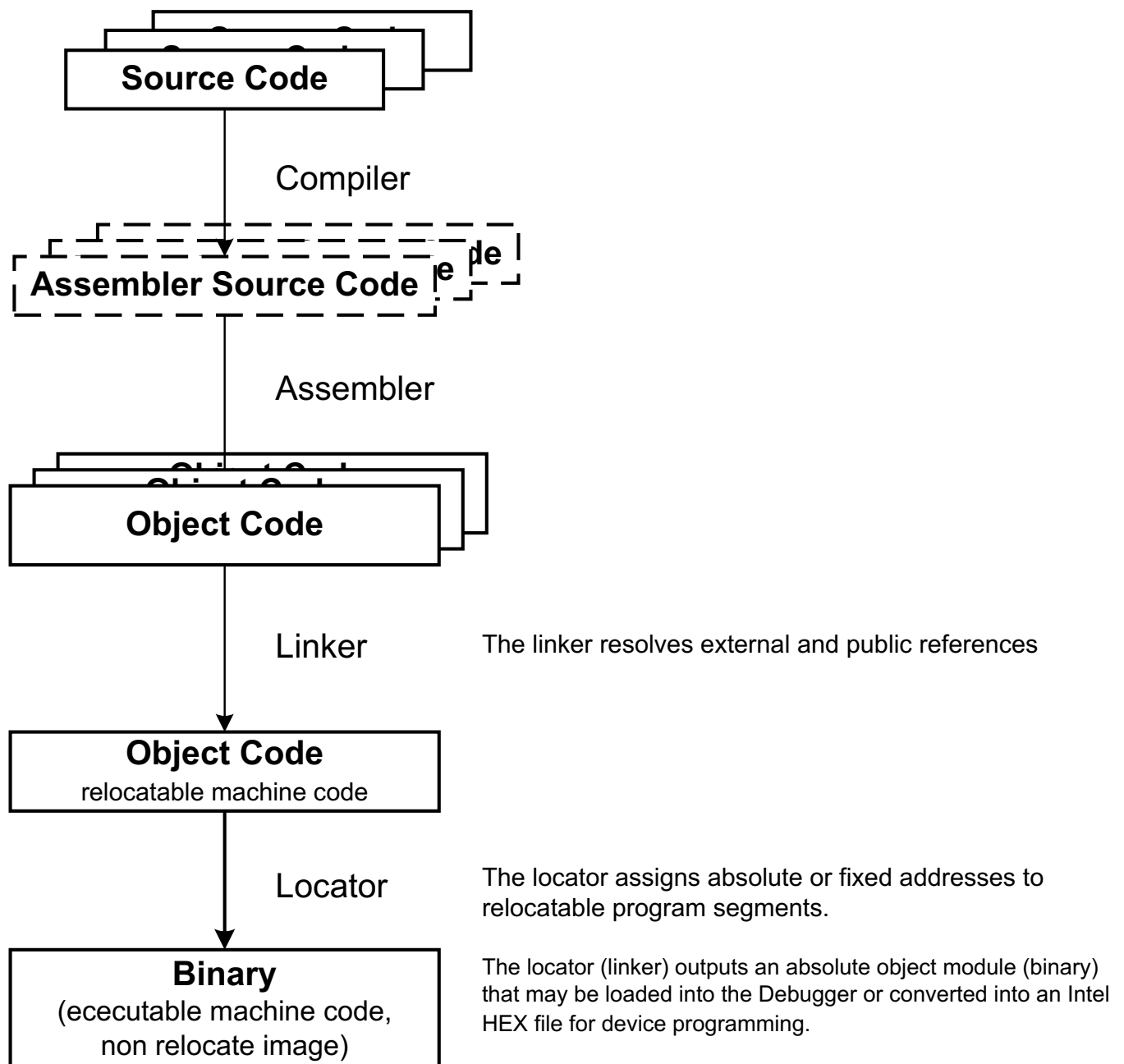


Bild: Ablaufdiagramm: Vom Quellcode zur Programmdatei und Toolchain

Linker

Der vom Assembler generierte Objektcode liegt in Bruchstücken vor. Die Aufgabe des Linkers besteht nun darin, alle benötigten Stücke zu sammeln und zusammenzufügen (engl. to link = verbinden, koppeln). Dabei können die einzelnen Object-Codedateien aus verschiedenen

Source-Code Dateien stammen und von verschiedenen Compilern (Pascal, C/C++) erzeugt worden sein.

Softwareroutinen, die nicht benötigt werden, z. B. unbenutzte Teile einer Softwarebibliothek (Library), können so weggelassen werden, um das ausführbare Programm möglichst klein zu halten.

Das Resultat des Linkers sind Gruppierungen von Bruchstücken, die Gemeinsamkeiten aufweisen wie z. B. gemeinsam genutzter Speicher etc.. Dies kann die Vorstufe zu einem ausführbaren Programm oder aber eine Softwarebibliothek (*library*) sein.

Locator

Als letzte Station tritt der Locator in Aktion. Er ist es, der die vom Linker erzeugten Gruppen an den richtigen Stellen platziert (engl. *to locate* = fixieren, siedeln). Das so erzeugte Programm steht bereit zur Ausführung.

Debugger

Der Begriff *debugging* bezeichnet den Prozess des Auffindens von Softwarefehlern, d. h. *Bugs* zu finden und zu entfernen. Mittels spezieller Tools kann der Entwickler das Programm Schritt für Schritt ausführen und alle logischen Abläufe überprüfen.

Debugging mit dem JTAG Interface

Das JTAG Interface oder auch als BDM (Background Debug Mode) Interface bezeichnet, macht sich die Boundary Scan Methode zu Nutzen.

In der Anfangszeit hatte man für das Debugging nur wenige Tools wie z. B. Oszilloskope zur Verfügung. Die später eingesetzten Emulatoren, spezielle Prozessoren für die Entwicklung mit herausgeführten Kontakten, waren für größere Prozessoren nicht mehr einsetzbar.

Das BDM/JTAG Interface ist auch bei komplexen CPUs verwendbar. Dabei hat das JTAG Entwicklungswerkzeug den Prozessor vollkommen im Griff und die darauf aufsetzende Debugoberfläche steuert wiederum das JTAG Interface. Idealerweise kann man so direkt aus der integrierten Entwicklungsoberfläche heraus Software debuggen.

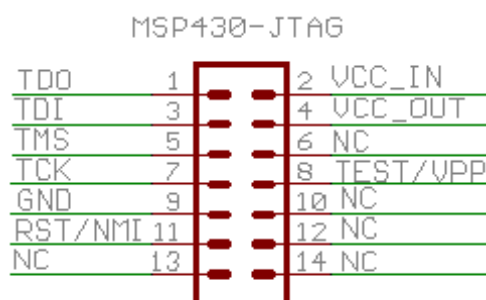


Bild: Pinbelegung JTAG Interface

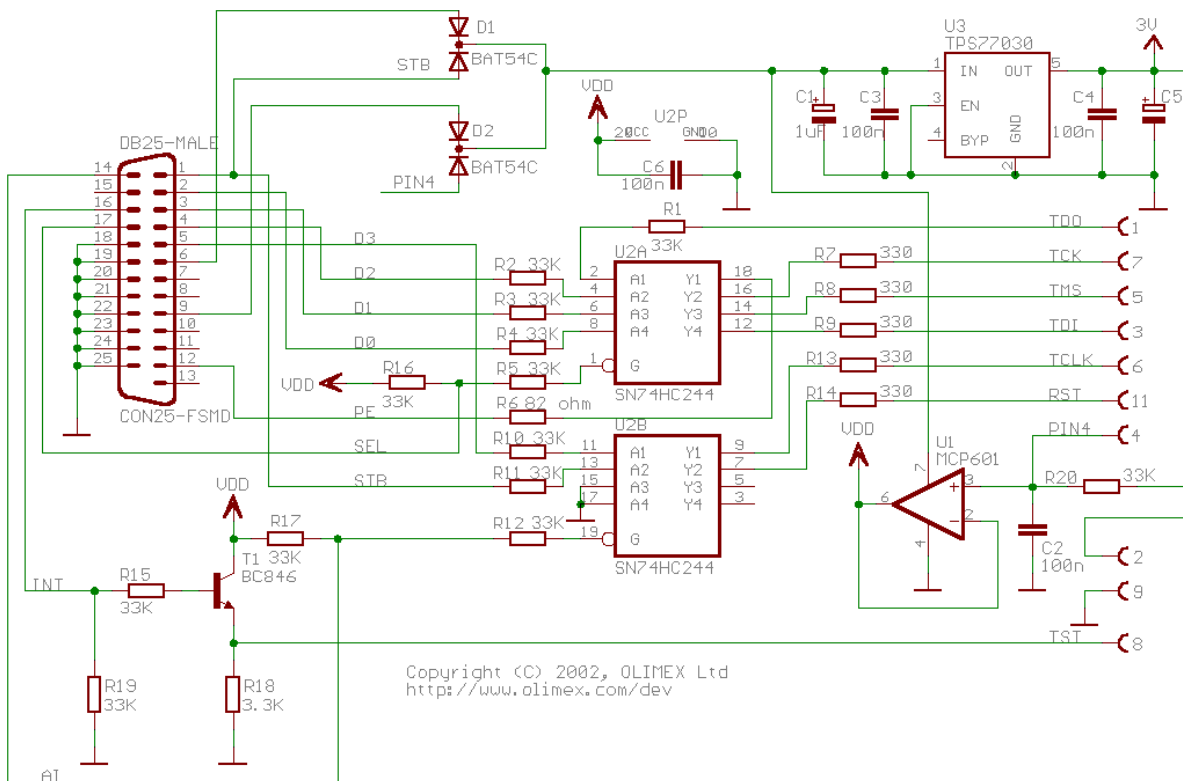


Bild: Schaltplan JTAG Interface

Der Programmcode kann editiert, compiliert, auf die Zielhardware übertragen und anschließend direkt im Prozessor debugged werden. Über ein paralleles oder serielles Interface kann man direkt auf den Speicher und auf die Register zugreifen. So ist es möglich, die Reaktionen der Hardware auszulesen, oder irgendwelche Zustände extern herbeizuführen um beispielsweise eine spezielle Situation auszutesten.

Über die Debuggeroberfläche kann Schritt für Schritt der Sourcecode ausgeführt werden und man hat dabei volle Kontrolle, ob man in Unterfunktionen verzweigen möchte, oder zu bestimmten Stellen springen möchte.

Mit so genannten Breakpoints ist es möglich, Haltepunkte innerhalb des Sourcecodes zu setzen, um während des Programmablaufs an dieser Stelle eine Pause zu machen, um sich in aller Ruhe den aktuellen Zustand anschauen zu können und kann bei Bedarf auch Register abzuändern, um z. B. einen konditionalen Sprung auszuführen, der durch eine falsche Rechenoperation doch nicht erreicht wurde.



7. SCHNITTSTELLE ZUM PERIPHERIEMODUL (I/O-BAUSTEIN)

Steuerregister

Datenregister

Statusregister

8. DIGITAL I/O: PORTS UND BITMANIPULATIONEN

Ziele:

Erfassung binärer Signalzustände

Schalten von externen Einheiten

8.1. Begriffsdefinitionen

8.2. Einsatz von symbolischen Konstanten zur Erhöhung der Lesbarkeit von Programmen

8.3. Definition von Bitgruppen

8.4. Bitmanipulationen

8.4.1. Selektives Eins-Setzen von Bits

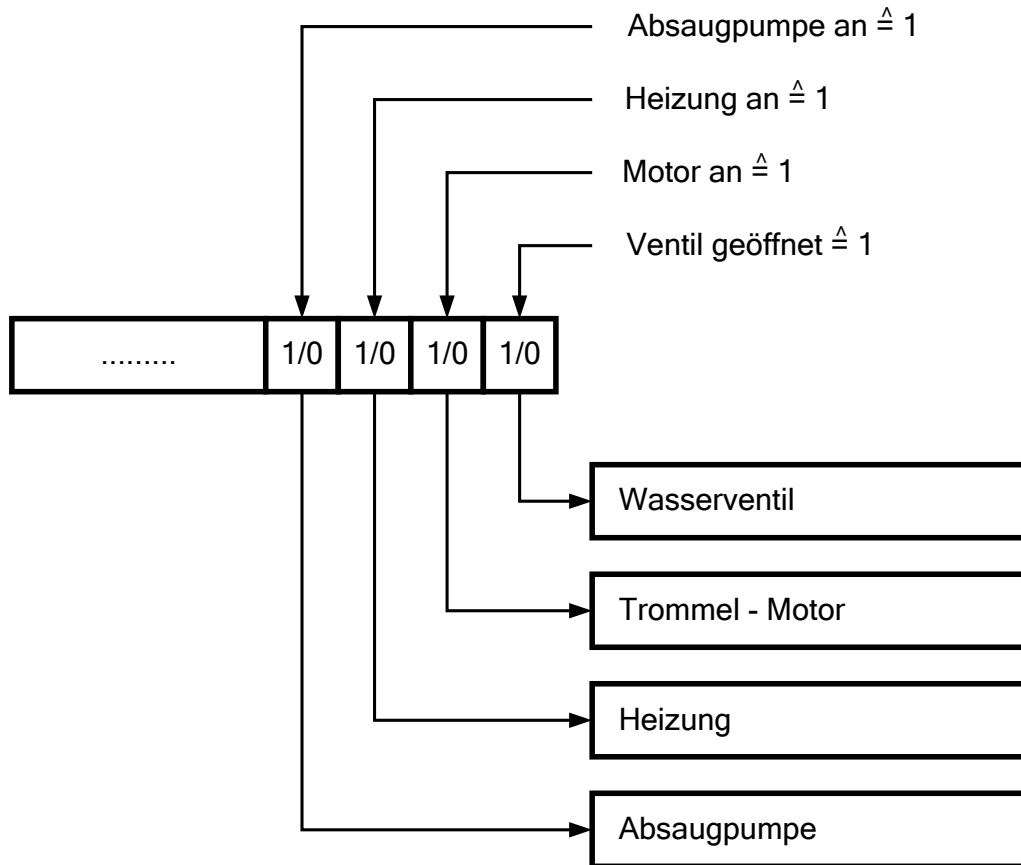


Bild: Zusammenfassung mehrerer möglicher Steuergrößen einer Waschmaschine in einem Steuerwort

Zum **selektiven Eins-Setzen** eines Bits wird das Steuerwort mit Hilfe einer **ODER-Verknüpfung** mit einer vorgegebenen konstanten Dualzahl **bitweise** verknüpft:

	X X X X	X X X X	Steuerwort-alt
V bzw.	0 0 0 0	0 1 0 1	Konstante

	X X X X	X 1 X 1	Ergebnis = Steuerwort-neu

wobei

X steht für 0/1, also wahlweise 0- oder 1-Pegel

1 Byte enthält 8 Bitpositionen mit Bit7, Bit6, Bit5, Bit4, Bit3, Bit2, Bit1, Bit0

Da $X \vee 1 = 1$ und $0 \vee 1 = 1$, wird das niederwertigste Bit (Bit 0), unabhängig von seinem vorherigen Wert, definiert auf 1 gesetzt. Alle anderen Bits (= Steuersignale) behalten ihren alten Wert bei.

Soll Bitposition 2 auf 1 gesetzt werden, dann muss hierfür die Konstante 0000 0010 verwendet werden.

8.4.2. Selektives Rücksetzen von Bits

Zum selektiven Rücksetzen (Null-Setzen) eines einzelnen Bits wird das Steuerwort mit Hilfe einer **UND-Verknüpfung** mit einer vorgegebenen konstanten Dualzahl bitweise verknüpft:

	X X X X	X X X X	Steuerwort-alt
$\wedge$ bzw. &	1 1 1 1	1 1 1 0	Konstante

	X X X X	X X X 0	Ergebnis = Steuerwort-neu

Anm.:

X steht für 0/1, also wahlweise 0- oder 1-Pegel

1 Byte enthält 8 Bitpositionen mit Bit7, Bit6, Bit5, Bit4, Bit3, Bit2, Bit1, Bit0

Da $X \wedge 1 = X$ und $1 \wedge 0 = 0$, wird das niederwertigste Bit, unabhängig von seinem vorherigen Wert, definiert auf 0 gesetzt. Alle anderen Bits behalten ihren alten Wert. Soll Bit 1 auf Null gesetzt werden, dann muss als Konstante die Dualzahl 1111 1101 gewählt werden.

8.4.3. Bitmaskierung

Da bei dem eingelesenen Zustandswort ein Bit nach dem andern selektiv abgefragt werden muss, müssen alle anderen Bits „unsichtbar“ gemacht werden. Dieser Vorgang wird als „Maskierung“ bezeichnet.

Vom originalen Statuswort soll nur das gewünschte Bit sichtbar sein, alle anderen Bits sollen auf einen vorgegebenen Wert, meist 0, gesetzt werden.

Das geschieht, indem mit einem UND-Befehl das Zustandswort bitweise mit einer konstanten Dualzahl (Maske) verknüpft wird:

	X X X X	X X X X	Statuswort
$\wedge$ bzw. &	0 0 0 0	0 0 0 1	Bit-Maske

	0 0 0 0	0 0 0 X	Ergebnis = Statuswort-neu

Anm.:

X steht für wahlweise 0- oder 1-Pegel

1 Byte enthält 8 Bitpositionen mit Bit7, Bit6, Bit5, Bit4, Bit3, Bit2, Bit1, Bit0

Es gilt hier: $X \wedge 1 = X$ und $X \wedge 0 = 0$

Da in diesem Beispiel nur das Bit0 seinen Wert behält, ist die Überprüfung einfach: Hat das Ergebnis der UND-Verknüpfung (Maskierung) einen Wert verschieden von Null, war das Bit gesetzt, andernfalls war es nicht gesetzt.

Soll anschließend das Bit1 abgefragt werden, muss die Maske lauten: 0000 0010 = 02h verwendet werden.

Auf diese Weise lassen sich auch Bitgruppen abfragen.

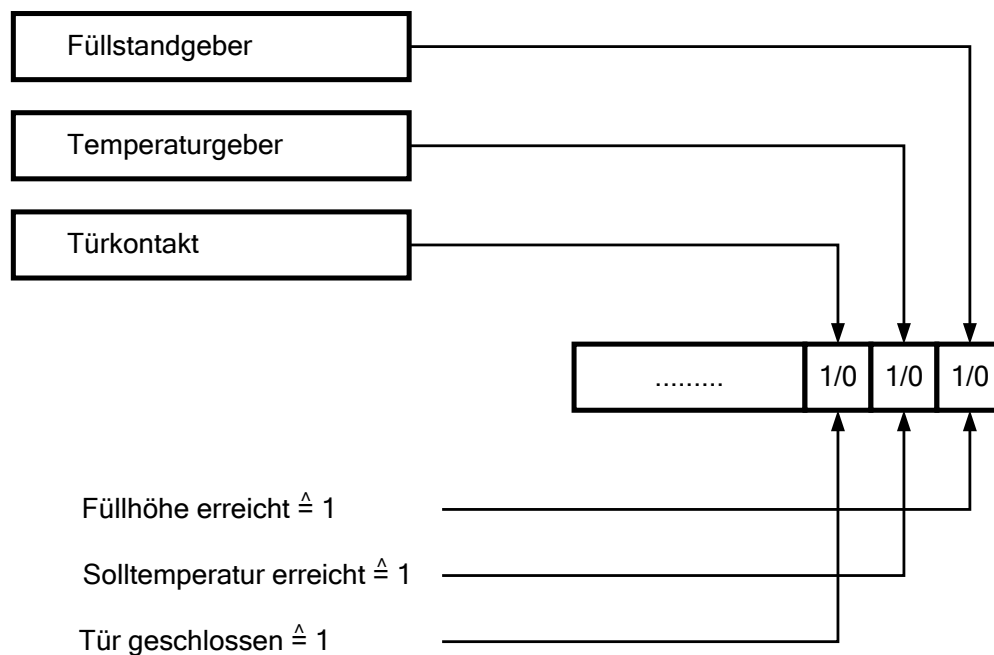


Bild: Zusammenfassung mehrerer Zustandsinformationen einer Waschmaschine in einem Statuswort

Waschprogramm: Es seien dies die Bits Bit3 und Bit4 (= 4. und 5. Bitposition von rechts). Die Abfrage dieser Bitgruppe sähe dann wie folgt aus:

	X X X X	X X X X	Statuswort
$\wedge$ bzw. $\&$	0 0 0 1	1 0 0 0	Bit-Maske

	0 0 0 X	X 0 0 0	Ergebnis

wobei

X steht für 0/1, also wahlweise 0- oder 1-Pegel

1 Byte enthält 8 Bitpositionen mit Bit7, Bit6, Bit5, Bit4, Bit3, Bit2, Bit1, Bit0

Nach Division des Ergebnisses durch acht bzw. dreimaligem Schieben des Wortes nach rechts (Rechts-Shift) ergibt sich die korrekte Programmnummer.

8.4.4. Bitweises Invertieren

Oft werden einzelne Signale des Steuer- und Statusregisters vor dem Einlesen bzw. der Ausgabe invertiert. Bei vielen Anwendungsfällen - vor allem wenn man die Informationen von Steuerregister und Statusregister als eine zusammenhängende Information verwendet - ist es lästig, bei der Programmierung zu berücksichtigen, dass einzelne Bits invertiert sind. Eine

Möglichkeit, die hardwaremäßige Negation einzelner Bits durch erneutes Invertieren wieder rückgängig zu machen, bietet die Verwendung der bitweisen Exklusiv-ODER-Verknüpfung:

```

          1 1 0 X   X X 0 1   eingelesenes Datenwort
^ oder XOR 0 0 0 0   0 0 1 1   Toggle-Bitmaske
-----
          1 1 0 X   X X 1 0   korrigiertes Datenwort

```

Da $X \text{ XOR } 0 = X$ und $X \text{ XOR } 1 = \bar{X}$ gilt, werden bei diesem Beispiel die beiden niederwertigen Bits Bit0 und Bit1 invertiert, während alle anderen Bits ihre alte Information behalten.

8.4.5. Das 1er und 2er-Komplement

Im Fall des Dualsystem sind noch zwei spezielle Zahlendarstellungen von besonderer Wichtigkeit:

- das Stellenkomplement $\sim z$, auch als 1er Komplement bezeichnet. Das 1er Komplement ist stellt die Invertierung aller Bits dar.
- 2er-Komplement $2c(z)$ einer Dualzahl z . Das 2er-Komplement ergibt sich aus dem 1er-Komplement plus 1.

Beispiel:

Zahl z	Stellenkomplement $\sim z$	2er-Komplement $2c(z)$
0100_2	1011_2	1100_2

Das 2er-Komplement wird benutzt, um negative Zahlen darzustellen.

$4 = 0100_2$ und

$-4 = 1100_2$.

	Dual	Dezimal	
	0000	0	
	0001	1	
	0010	2	
	0011	3	
	0100	4	
	0101	5	
	0110	6	
	0111	7	
2er Komplement (= 1er Kompl. + 1)	1000	-8	2er Komplement
	1001	-7	
	1010	-6	
	1011	-5	
	1100	-4	
	1101	-3	
	1110	-2	
	1111	-1	

Bild: Negative Zahlen im Dualsystem

Benutzt man positive und negative ganze Zahlen, dann wird der Bereich der dualen Zahlen wie folgt aufgeteilt:

Negative Zahlen	Positive Zahlen
$-2^{n-1} \leq p \leq -1$	$0 \leq q \leq 2^{n-1}-1$
$-2^{32-1} \leq p \leq -1$	$0 \leq q \leq 2^{32-1}-1$
$-2.147.483.648 \leq p \leq -1$	$0 \leq q \leq 2.147.483.647$

8.4.6. Übersicht der Bitoperatoren in ISO Standard C

Folgende Bitoperatoren stehen gemäß ISO Standard C zur Verfügung:

OPERATOR	ANWENDUNG	BEDEUTUNG
& AND	$n3 = n2 \& n1;$ $n2 = n2 \& n1;$ oder $n2 \&= n1;$	$n3$ erhält das Ergebnis von $n1$ AND $n2$; $n2$ erhält das Ergebnis von $n1$ AND $n2$;
 OR	$n3 = n2 n1;$ $n2 = n2 n1;$ oder $n2 = n1;$	$n3$ erhält das Ergebnis von $n1$ OR $n2$; $n2$ erhält das Ergebnis von $n1$ OR $n2$;
^ EXCLUSIV-OR (XOR)	$n3 = n2 ^ n1;$ $n2 ^= n1;$	$n3$ erhält das Ergebnis von $n1$ XOR $n2$; $n2$ erhält das Ergebnis von $n1$ XOR $n2$;
!		Logisch NICHT; logischer Verneinungsoperator Der Wert eines damit gebildeten logischen Verneinungsausdrucks <code>! <cast-expression></code> ist wahr, wenn der gewandelte Wert des Ausdrucks <code><cast-expression></code> "false" ist, und falsch, wenn der gewandelte Wert des Ausdrucks <code><cast-expression></code> den Wert "true" ist.
~ COMPLEMENT	$n3 = \sim n1;$ $n1 \sim= n1;$	$n3$ erhält das Ergebnis des KOMPLEMENTs von $n1$; $n1$ erhält das Ergebnis des KOMPLEMENTs von $n1$;
<< SHIFT LEFT shl	$n3 = n1 << x;$ $n1 <<= x;$	$n3$ erhält das Ergebnis von $n1$ mit x-mal SHIFT LINKS $n1$ erhält das Ergebnis von $n1$ mit x-mal SHIFT LINKS
>> SHIFT RIGHT shr	$n3 = n1 >> x;$ $n1 >>= x;$	$n3$ erhält das Ergebnis von $n1$ mit x-mal SHIFT RECHTS $n1$ erhält das Ergebnis von $n1$ mit x-mal SHIFT RECHTS

8.4.7. Übungen zu Bitoperationen

1) Was versteht man unter dem Begriff der Bitmaskierung?

2) Bitmanipulation -> Bitmaskierung (Bitselektion): Geben Sie in binärem 8 Bit-Format die Maske an zur Maskierung der 1. und der 6. Bitposition einer Bytezahl.

Bit _____ Bit _____

3) Bitmanipulation -> Bit setzen: Durch welche Verknüpfungsform erreicht man ein selektives Eins-Setzen eines Bits?

Setzen Sie jeweils das 2. und das 6. Bit der folgenden Dualzahlen auf 1. Geben Sie den vollständigen Befehl hierzu an (and, or, xor, shl, shr).

Befehl: _____ Befehl: _____

0 0 0 1 0 0 0 0 = Steuerwort 1 1 0 1 1 0 0 1 = Steuerwort

_____ = Konstante _____ = Konstante

_____ = Ergebnis _____ = Ergebnis

4) Bitmanipulation -> Bit rücksetzen (löschen): Durch welche Verknüpfungsform erreicht man ein selektives Rücksetzen eines Bits?

Setzen Sie jeweils das 1. und das 7. Bit der folgenden Dualzahlen auf 0. Geben Sie hierzu den vollständigen Befehl an (and, or, xor, shl, shr).

Befehl: _____ Befehl: _____

1 1 1 1 1 1 1 1 = Steuerwort 1 1 0 1 1 0 0 1 = Steuerwort

_____ = Konstante _____ = Konstante

_____ = Ergebnis _____ = Ergebnis

5) Bitmanipulation -> Bit invertieren: Durch welche Verknüpfungsform erreicht man ein selektives Invertieren einzelner Bits?

Invertieren Sie jeweils das 3. und das 6. Bit der folgenden. Geben Sie hierzu den vollständigen Befehl an (and, or, xor, shl, shr).

Befehl: _____

Befehl: _____

0 0 1 0 X X 1 0 = Steuerwort

0 X X 0 0 0 0 0 = Steuerwort

_____ = Konstante

_____ = Konstante

_____ = Ergebnis

_____ = Ergebnis

- 6) Bitmanipulation -> Bitselektion: In einem Statuswort ist eine Programmnummer (3. und 4. Bit von rechts) codiert. Diese soll zunächst maskiert (Bitgruppe) und anschließend durch mehrmaliges Schieben nach rechts in die korrekte Programmnummer überführt werden.

- 6.1) Geben Sie hierzu die benötigten Befehle in der richtigen Reihenfolge an.
Verwenden Sie hierzu die Befehle: and, or, xor, shl und shr.

- 6.2) Weisen Sie die Funktion der Befehlssequenz an einem Beispiel nach. Das Statuswort lautet 0011 0101 sein.

- 7) Gegeben ist der Ausgabewert A, der in den Ausgabewert B überführt werden soll. Geben Sie drei mögliche Verfahren an.

0 0 0 0 1 0 0 0₂ = A

0 0 0 0 0 1 0 0₂ = B

Geben Sie fünf mögliche Verfahren an:

- 8) Shift-Operatoren bewegen ein Bitmuster n in einem Bitfeld x Felder nach *links* ($<<$) oder nach *rechts* ($>>$). Mit dieser Technik kann man gezielt z. B. eine 1 über alle Positionen im Bitfeld bewegen und auf diese Weise jede Position mittels eines AND-Operators abfragen (s.u.).

Ergänzen Sie in der rechten Spalte die fehlende Anweisung:

2^4	2^3	2^2	2^1	2^0	Bezogen auf erste Zeile
0	0	0	0	1	
0	0	0	1	0	
0	0	1	0	0	
0	0	0	1	0	
1	0	0	0	0	
					...

- 9) Logische Operatoren. Ergänzen Sie die nachstehende Tabelle:

A	B	AND &	OR 	XOR ^	COMPLEMENT ~ (bezogen auf A)
0	0				
0	1				
1	0				
1	1				

10) Übung zur Bitmanipulation und Interrupts

Löschen eines Rücksetzten eines Interrupt Flags nach Bearbeiten der ISR.

Port	Register	Short Form	Address	Register Type	Initial State
P1	Input	P1IN	020h	Read only	–
	Output	P1OUT	021h	Read/write	Unchanged
	Direction	P1DIR	022h	Read/write	Reset with PUC
	Interrupt Flag	P1IFG	023h	Read/write	Reset with PUC
	Interrupt Edge Select	P1IES	024h	Read/write	Unchanged
	Interrupt Enable	P1IE	025h	Read/write	Reset with PUC
	Port Select	P1SEL	026h	Read/write	Reset with PUC
	Resistor Enable	P1REN	027h	Read/write	Reset with PUC
P2	Input	P2IN	028h	Read only	–
	Output	P2OUT	029h	Read/write	Unchanged
	Direction	P2DIR	02Ah	Read/write	Reset with PUC
	Interrupt Flag	P2IFG	02Bh	Read/write	Reset with PUC
	Interrupt Edge Select	P2IES	02Ch	Read/write	Unchanged
	Interrupt Enable	P2IE	02Dh	Read/write	Reset with PUC
	Port Select	P2SEL	02Eh	Read/write	0C0h with PUC
	Resistor Enable	P2REN	02Fh	Read/write	Reset with PUC

Bild: Digital I/O Registers P1-P2

Direction Registers PxDIR

Each bit in each PxDIR register selects the direction of the corresponding I/O pin, regardless of the selected function for the pin. PxDIR bits for I/O pins that are selected for other module functions must be set as required by the other function.

Bit = 0: The port pin is switched to input direction

Bit = 1: The port pin is switched to output direction

Function Select Registers PxSEL

Port pins are often multiplexed with other peripheral module functions. See the device-specific data sheet to determine pin functions. Each PxSEL bit is used to select the pin function — I/O port or peripheral module function.

Bit = 0: I/O function is selected for the pin

Bit = 1: Peripheral module function is selected for the pin

Setting PxSELx = 1 does not automatically set the pin direction. Other peripheral module functions may require the PxDIRx bits to be configured according to the direction needed for the module function. See the pin schematics in the device-specific data sheet.

Interrupt Edge Select Registers P1IES, P2IES

Each PxIES bit selects the interrupt edge for the corresponding I/O pin.

Bit = 0: The PxIFGx flag is set with a low-to-high transition

Bit = 1: The PxIFGx flag is set with a high-to-low transition

Interrupt Edge Select Registers P1IES, P2IES

Each PxIE bit enables the associated PxIFG interrupt flag.

Bit = 0: The interrupt is disabled

Bit = 1: The interrupt is enabled

Ergänzen Sie die entsprechenden Anweisungen in dem folgenden Programm:

```
void main(void)
```

```
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop WDT
    FLL_CTL0 = FLL_CTL0 | XCAP14PF;     // Configure load caps
    P2DIR = .....                       // Set P2.1 to output direction
    P1IES = .....                       // H-L transition
    P1IE = .....                       // Enable interrupt
    _EINT();                             // Enable interrupts
    while ( 1 );
} // end while
```

```
#pragma vector=PORT1_VECTOR             // P1 interrupt service routine
__interrupt void P1ISR (void)
{
    .....                               // toggle LED Pin
    P1IFG = P1IFG & ~.....              // Clear P1IFG
}
```

8.5. Digital Input und Digital Output Unit

Every system you will ever design has one thing in common: use of input and/or output. All I/Os on MSP430 devices are eight bits wide and controlled with **memory-mapped registers**. These ports, which are fundamentally the same across all device families, can be divided into two categories:

- interruptible ports (meaning that interrupts can be generated via these ports)
- and non-interruptible ports.

In all devices, ports 1 and 2 are interruptible, and the higher numbered ports are not. '3xx devices have a port 0, which is also interruptible (there is no port 0 in '1xx and '4xx devices).

Along with basic I/O functions, the port pins can be individually configured as special function I/Os, such as USARTs, Comparator signals, and ADCs. The number of ports and available functions vary by part, so consult your datasheet.

8.5.1. MSP430x44x Terminal Functions

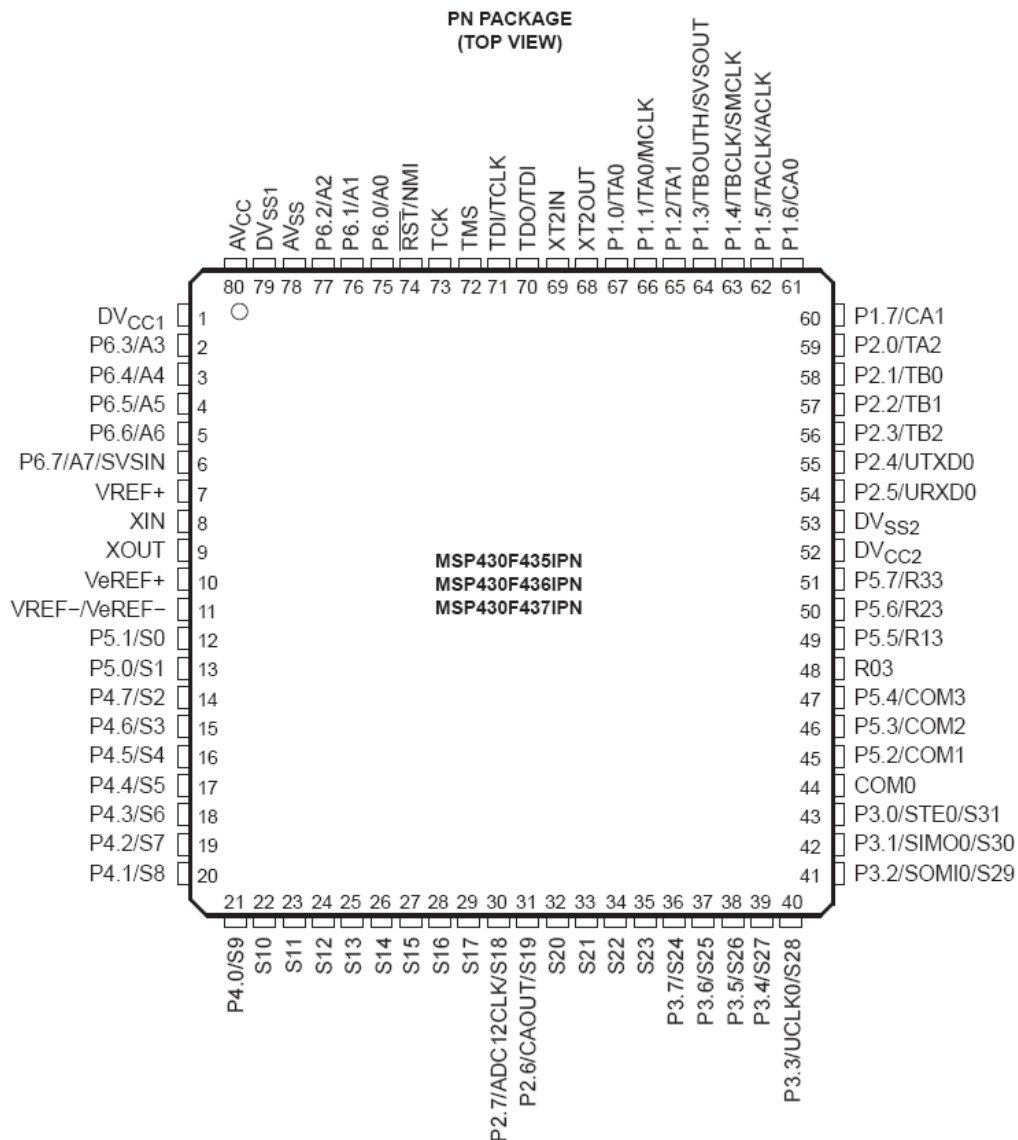


Bild: MSP430-Familie Pin-Layout

TERMINAL		I/O	DESCRIPTION
PN NAME	NO.		
P1.6/CA0	81	I/O	General-purpose digital I/O / Comparator_A input
P1.5/TACLK/ ACLK	82	I/O	General-purpose digital I/O / Timer_A, clock signal TACLK input / ACLK output (divided by 1, 2, 4, or 8)
P1.4/TBCLK/ SMCLK	83	I/O	General-purpose digital I/O / input clock TBCLK—Timer_B7 / submain system clock SMCLK output
P1.3/TBOutH/ SVSOut	84	I/O	General-purpose digital I/O / switch all PWM digital output ports to high impedance—Timer_B7 TB0 to TB6 / SVS: output of SVS comparator
P1.2/TA1	85	I/O	General-purpose digital I/O / Timer_A, Capture: CCI1A, compare: Out1 output
P1.1/TA0/MCLK	86	I/O	General-purpose digital I/O / Timer_A, Capture: CCI0B / MCLK output. Note: TA0 is only an input on this pin.
P1.0/TA0	87	I/O	General-purpose digital I/O / Timer_A, Capture: CCI0A input, compare: Out0 output
XT2OUT	88	O	Output terminal of crystal oscillator XT2
XT2IN	89	I	Input port for crystal oscillator XT2. Only standard crystals can be connected.
TDO/TDI	90	I/O	Test data output port. TDO/TDI data output or programming data input terminal
TDI	91	I	Test data input. TDI is used as a data input port. The device protection fuse is connected to TDI.
TMS	92	I	Test mode select. TMS is used as an input port for device programming and test.
TCK	93	I	Test clock. TCK is the clock input port for device programming and test.
RST/NMI	94	I	Reset input or nonmaskable interrupt input port
P6.0/A0	95	I/O	General-purpose digital I/O, analog input a0—12-bit ADC
P6.1/A1	96	I/O	General-purpose digital I/O, analog input a1—12-bit ADC
P6.2/A2	97	I/O	General-purpose digital I/O, analog input a2—12-bit ADC
AVSS	98		Negative terminal that supplies SVS, brownout, oscillator, FLL+, comparator_A, port 1, and LCD resistive divider circuitry.
DVSS1	99		Digital supply voltage, negative terminal. Supplies all digital parts, except those which are supplied via AVCC/AVSS.
AVCC	100		Positive terminal that supplies SVS, brownout, oscillator, FLL+, comparator_A, port 1, and LCD resistive divider circuitry; must not power up prior to DVCC1/DVCC2.

Figure: Terminal functions

TERMINAL		I/O	DESCRIPTION
PN NAME	NO.		
S29	41	O	LCD segment output 29
S30	42	O	LCD segment output 30
S31	43	O	LCD segment output 31
S32	44	O	LCD segment output 32
S33	45	O	LCD segment output 33
P4.7/S34	46	I/O	General-purpose digital I/O / LCD segment output 34
P4.6/S35	47	I/O	General-purpose digital I/O / LCD segment output 35
P4.5/UCLK1/S36	48	I/O	General-purpose digital I/O / external clock input—USART1/UART or SPI mode, clock output—USART1/SPI MODE / LCD segment output 36
P4.4/SOMI1/S37	49	I/O	General-purpose digital I/O / slave out/master in of USART1/SPI mode / LCD segment output 37
P4.3/SIMO1/S38	50	I/O	General-purpose digital I/O / slave in/master out of USART1/SPI mode / LCD segment output 38
P4.2/STE1/S39	51	I/O	General-purpose digital I/O / slave transmit enable—USART1/SPI mode / LCD segment output 39
COM0	52	O	COM0–3 are used for LCD backplanes.
P5.2/COM1	53	I/O	General-purpose digital I/O / common output, COM0–3 are used for LCD backplanes.
P5.3/COM2	54	I/O	General-purpose digital I/O / common output, COM0–3 are used for LCD backplanes.
P5.4/COM3	55	I/O	General-purpose digital I/O / common output, COM0–3 are used for LCD backplanes.
R03	56	I	Input port of fourth positive (lowest) analog LCD level (V5)
P5.5/R13	57	I/O	General-purpose digital I/O / Input port of third most positive analog LCD level (V4 or V3)
P5.6/R23	58	I/O	General-purpose digital I/O / Input port of second most positive analog LCD level (V2)
P5.7/R33	59	I/O	General-purpose digital I/O / Output port of most positive analog LCD level (V1)
DVCC2	60		
DVSS2	61		
P4.1/URXD1	62	I/O	General-purpose digital I/O / receive data in—USART1/UART mode
P4.0/UTXD1	63	I/O	General-purpose digital I/O / transmit data out—USART1/UART mode
P3.7/TB6	64	I/O	General-purpose digital I/O / Timer_B7 CCR6 / Capture: CCI6A/CCI6B input, compare: Out6 output
P3.6/TB5	65	I/O	General-purpose digital I/O / Timer_B7 CCR5 / Capture: CCI5A/CCI5B input, compare: Out5 output
P3.5/TB4	66	I/O	General-purpose digital I/O / Timer_B7 CCR4 / Capture: CCI4A/CCI4B input, compare: Out4 output
P3.4/TB3	67	I/O	General-purpose digital I/O / Timer_B7 CCR3 / Capture: CCI3A/CCI3B input, compare: Out3 output
P3.3/UCLK0	68	I/O	General-purpose digital I/O / external clock input—USART0/UART or SPI mode, clock output—USART0/SPI mode
P3.2/SOMI0	69	I/O	General-purpose digital I/O / slave out/master in of USART0/SPI mode
P3.1/SIMO0	70	I/O	General-purpose digital I/O / slave in/master out of USART0/SPI mode
P3.0/STE0	71	I/O	General-purpose digital I/O / slave transmit enable—USART0/SPI mode
P2.7/ADC12CLK	72	I/O	General-purpose digital I/O / conversion clock—12-bit ADC
P2.6/CAOUT	73	I/O	General-purpose digital I/O / Comparator_A output
P2.5/URXD0	74	I/O	General-purpose digital I/O / receive data in—USART0/UART mode
P2.4/UTXD0	75	I/O	General-purpose digital I/O / transmit data out—USART0/UART mode
P2.3/TB2	76	I/O	General-purpose digital I/O / Timer_B7 CCR2. Capture: CCI2A/CCI2B input, compare: Out2 output
P2.2/TB1	77	I/O	General-purpose digital I/O / Timer_B7 CCR1. Capture: CCI1A/CCI1B input, compare: Out1 output
P2.1/TB0	78	I/O	General-purpose digital I/O / Timer_B7 CCR0. Capture: CCI0A/CCI0B input, compare: Out0 output
P2.0/TA2	79	I/O	General-purpose digital I/O / Timer_A Capture: CCI2A input, compare: Out2 output
P1.7/CA1	80	I/O	General-purpose digital I/O / Comparator_A input

Figure: Terminal functions

8.5.2. Non-Interruptible I/O

I/O ports 3, 4, 5 and 6 are non-interruptible data ports. These ports are not implemented on all devices.

Use of non-interruptible I/O is simple and straight forward. Each bit is individually controllable, so inputs, outputs, and dedicated function I/O can be mixed in a single port.

Port pins are controlled by four byte-addressable registers:

- direction,
- input, output, and
- function select.

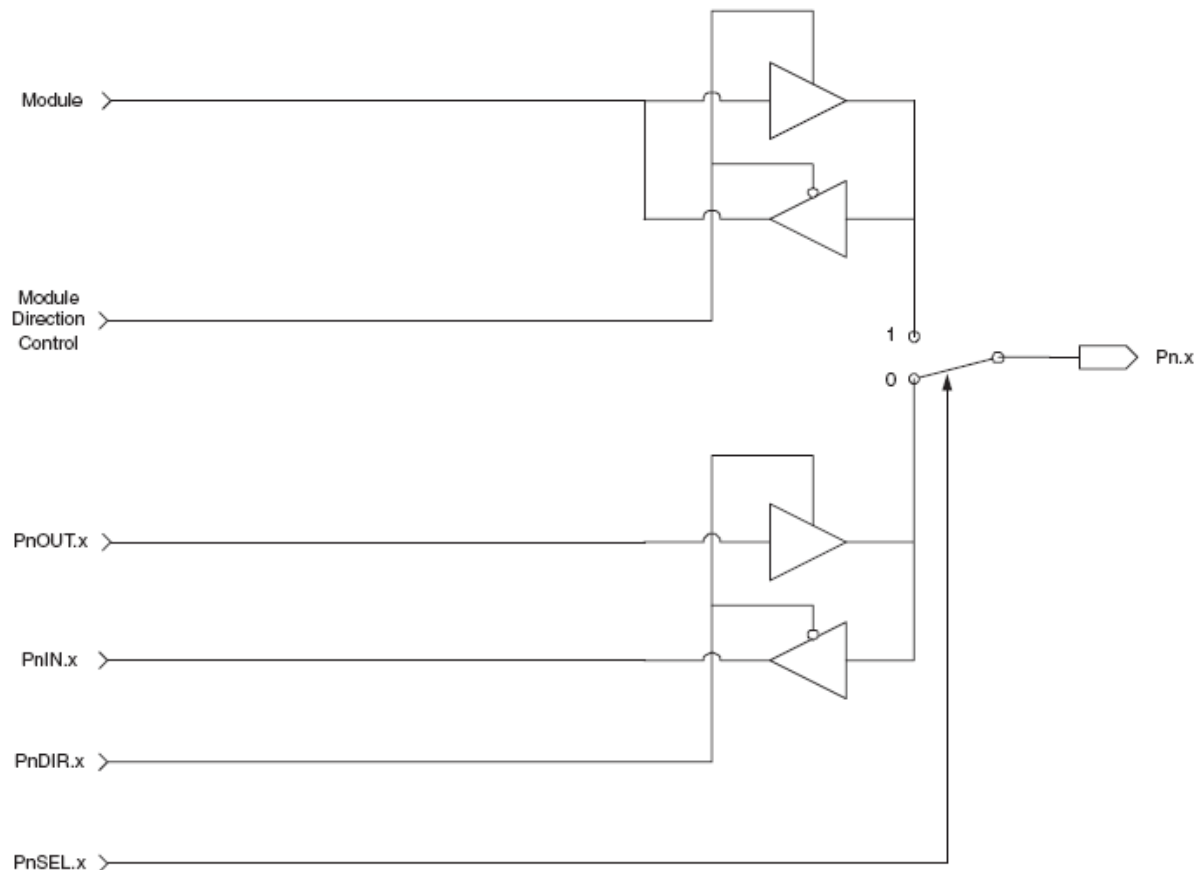


Figure: Non-Interruptible I/O Pin Logic

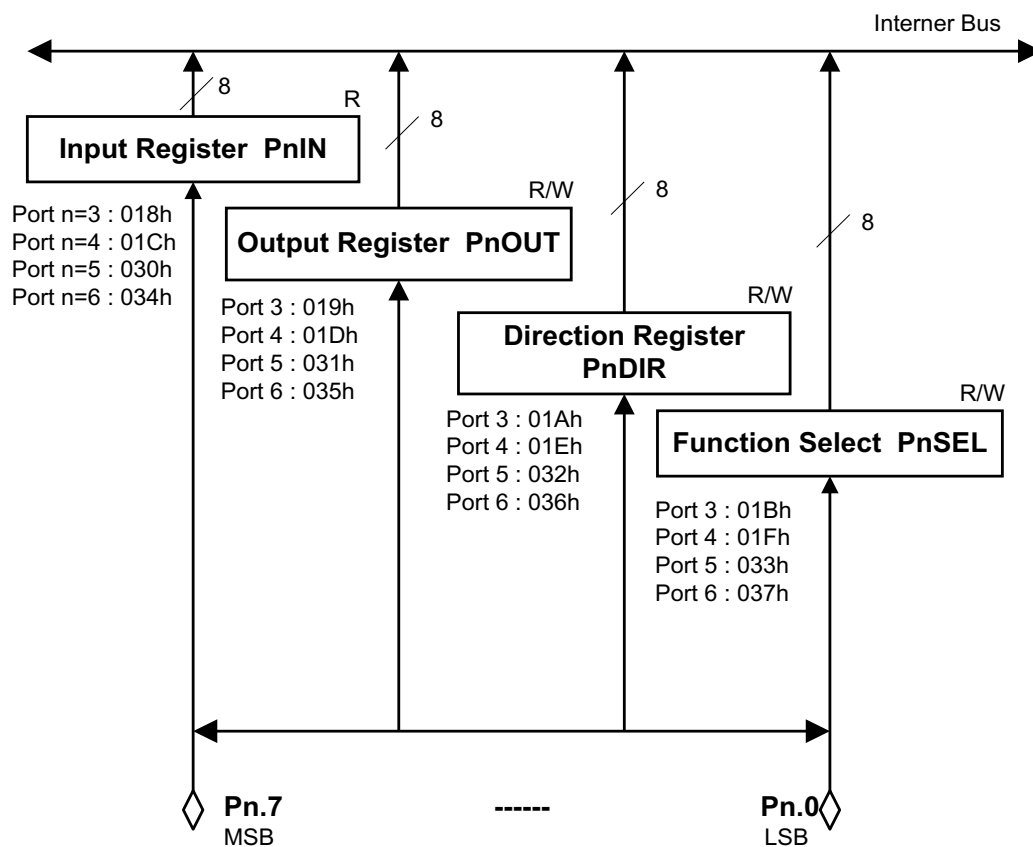


Bild: Digital I/O-Ports ohne Interruptfähigkeit (Ports 3, 4, 5 u. 6)

Direction Registers. These read/write registers control the signal direction for port pins. When a bit in the direction register is set, the corresponding port pin is set as an output, and when the bit in the direction register is cleared, the port pin is set as an input.

The direction registers need to be configured properly if the port pin is selected as a general purpose I/O or as a special function I/O. Direction registers are cleared on reset.

Input Registers. These are read-only registers, which reflect the input value on the port.

Output Registers. These registers are used to write to output ports, and can be read as well. When reading these registers, they will reflect the last value written to them. However, if the port is configured as an input, the output register will be in an indeterminate state. It will not necessarily reflect the input value on the associated pin.

Function Select Registers. These read/write registers determine the use of the individual pins on the I/O port. When the bit in the select register is set, the port pin is set as a function I/O, and when the bit in the direction register is cleared, the port pin is set as general purpose I/O.

1.1.1. Interruptible I/O

Ports 1 and 2 (and port 0 on '3xx devices) are interruptible ports. They contain all of the same control registers as non-interruptible ports (described in the previous section), along with three other byte-addressable registers: interrupt enable, interrupt edge select, and interrupt flags.

Interrupt Enable. This read-write register enables interrupts on individual pins. Interrupts on the pins are enabled when their corresponding bits in this register are set. This register is cleared on reset.

Interrupt Edge Select. This read-write register selects the transition on which an interrupt occurs. If set, an interrupt occurs on a high-to-low transition on the corresponding pin. If cleared, an interrupt occurs on a low-to-high transition on the corresponding pin.

Interrupt Flags. The corresponding bit in this read-write register is set automatically when an interrupt is generated. This register can be written to, and will generate an interrupt when a high level is written. When an interrupt occurs, this flag needs to be cleared before the `reti` instruction, or the same interrupt will call the ISR a second time.

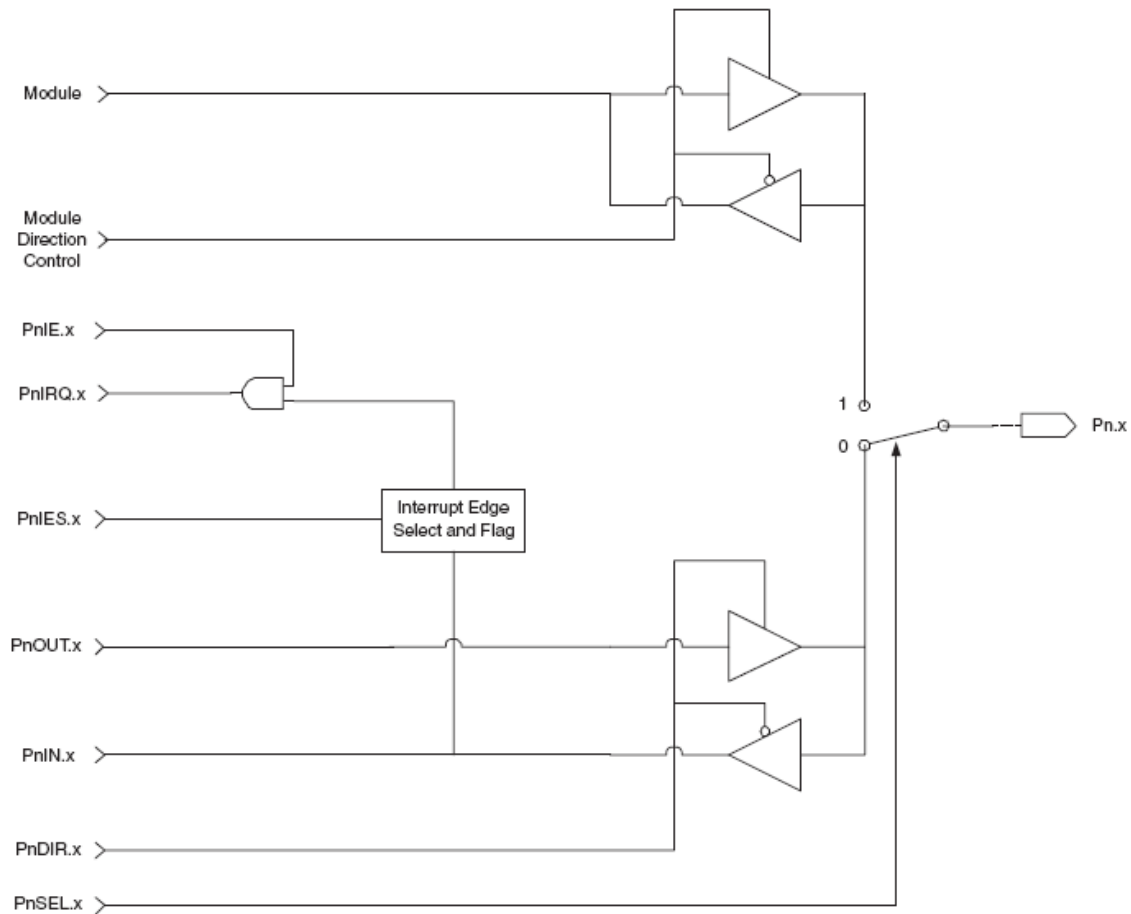


Figure: Interruptible I/O Pin Logic

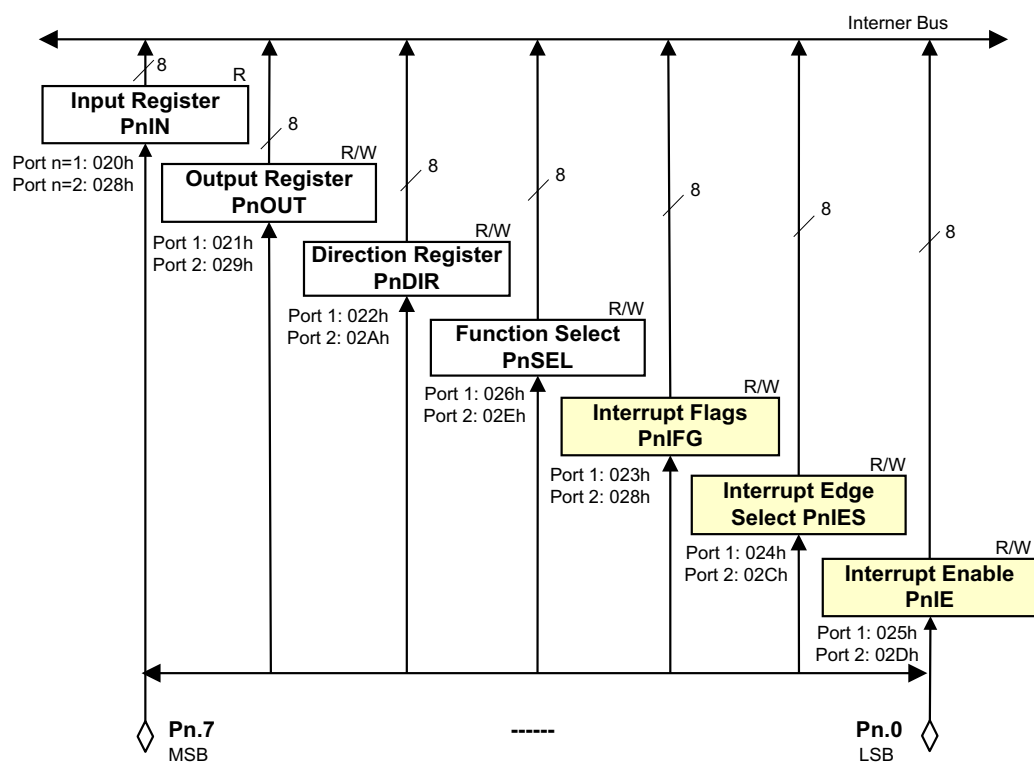


Bild: Digital I/O-Ports mit Interruptfähigkeit (Ports 1 u. 2)

8.5.3. Digital I/O-Registers

Register	Port 0 (‘3xx only)	Port 1	Port 2	Port 3	Port 4	Port 5	Port 6
Input Register	P0IN 010h	P1IN 020h	P2IN 028h	P3IN 018h	P4IN 01Ch	P5IN 030h	P6IN 034h
Output Register	P0OUT 011h	P1OUT 021h	P2OUT 029h	P3OUT 019h	P4OUT 01Dh	P5OUT 031h	P6OUT 035h
Direction Register	P0DIR 012h	P1DIR 022h	P2DIR 02Ah	P3DIR 01Ah	P4DIR 01Eh	P5DIR 032h	P6DIR 036h
Interrupt Flags Register	P0IFG 013h	P1IFG 023h	P2IFG 02Bh	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Interrupt Edge Select Register	P0IES 014h	P1IES 024h	P2IES 02Ch	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Interrupt Enable Register	P0IE 015h	P1IE 025h	P2IE 02Dh	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Function Select Register	Not Implemented	P1SEL 026h	P2SEL 02Eh	P3SEL 01Bh	P4SEL 01Fh	P5SEL 033h	P6SEL 037h

Bild: Digitale I/O-Register

Input Registers P1IN, P2IN

Both Input registers are read-only registers that reflect the signals at the I/O pins.

Note: Writing to Read-Only Registers P1IN, P2IN: Writing to these read-only registers results in increased current consumption while the write attempt is active.

Output Registers P1OUT, P2OUT

Each output register shows the information of the output buffer. The output buffer can be modified by all instructions that write to a destination. If read, the contents of the output buffer are independent of pin direction. A direction change does not modify the output buffer contents.

Direction Registers P1DIR, P2DIR

The direction registers contain eight independent bits that define the direction of the I/O pin. All bits are reset by the PUC signal. When:

- Bit = 0: The port pin is switched to input direction (3-state)
- Bit = 1: The port pin is switched to output direction

Interrupt Flags P1IFG, P2IFG

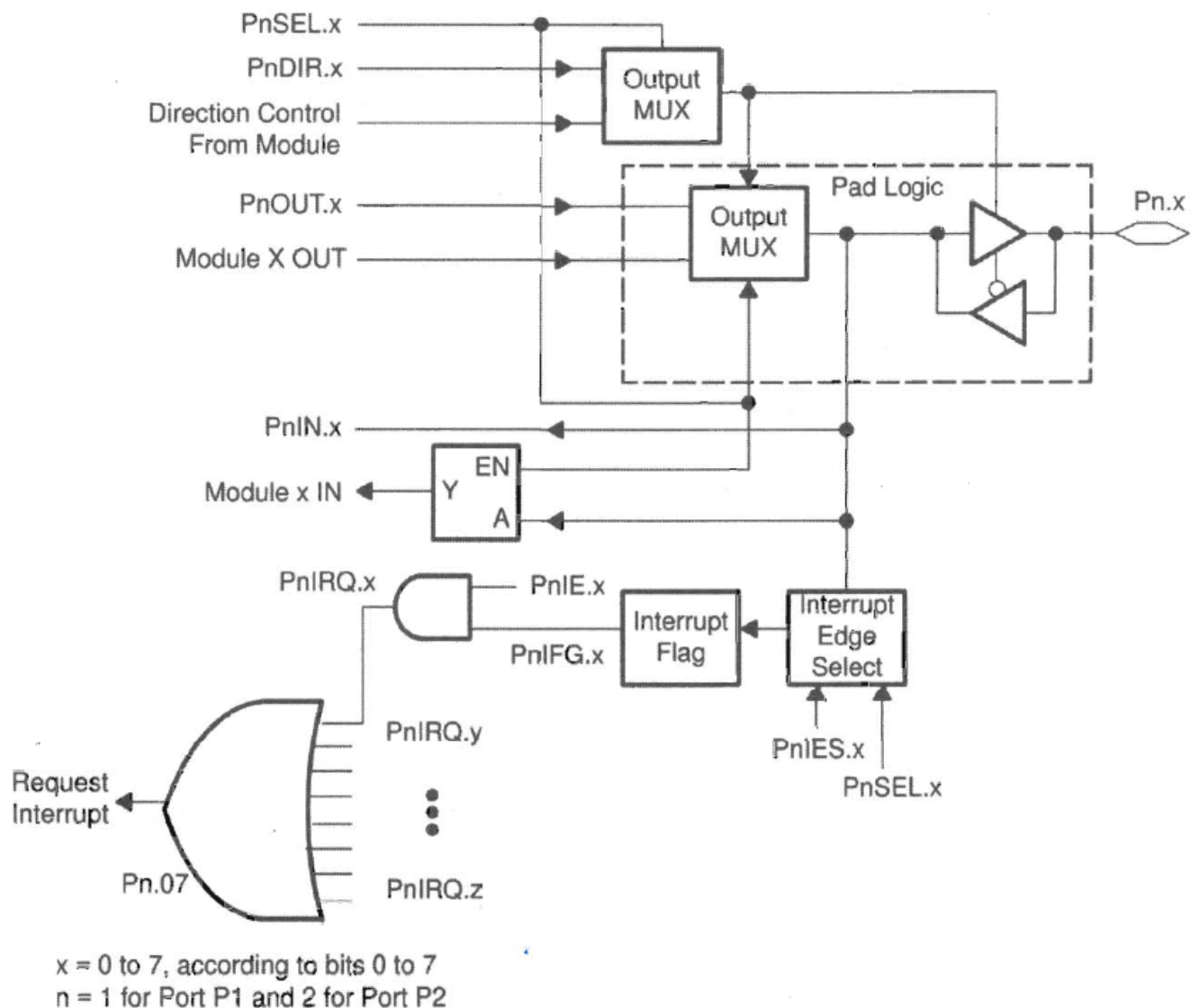


Bild: Sammel-Interrupt bei den digitalen I/O-Ports

Each interrupt flag register contains eight flags that reflect whether or not an interrupt is pending for the corresponding I/O pin, if the I/O is interrupt-enabled. When:

- Bit = 0: No interrupt is pending
- Bit = 1: An interrupt is pending due to a transition at the I/O pin or from software setting the bit.

Note:

Manipulating P1OUT and P1DIR, as well as P2OUT and P2DIR, can result in setting the P1IFG or P2IFG bits.

Writing a zero to an interrupt flag resets it; writing a one to an interrupt flag sets it and generates an interrupt.

Each group of interrupt flags P1FLG.0 to P1FLG.7 and P2FLG.0 to P2FLG.7 sources its own interrupt vector. Interrupt flags P1IFG.0 to P1IFG.7 and P2IFG.0 to P2IFG.7 are not reset

automatically when an interrupt from these events is serviced. The software should determine the origin of the interrupt and reset the appropriate flag(s).

Note: Any external interrupt event should be at least 1.5 times MCLK or longer, to ensure that it is accepted and the corresponding interrupt flag is set.

Interrupt Vectors der TI MSP430F449 Controllerfamilie

(Auszug aus msp430x44x.h)

```

/*****
 * Interrupt Vectors (offset from 0xFFE0)
 *****/

#define BASICTIMER_VECTOR    (0 * 2) /* 0xFFE0 Basic Timer */
#define PORT2_VECTOR         (1 * 2) /* 0xFFE2 Port 2 */
#define UART1TX_VECTOR       (2 * 2) /* 0xFFE4 UART 1 Transmit */
#define UART1RX_VECTOR       (3 * 2) /* 0xFFE6 UART 1 Receive */
#define PORT1_VECTOR         (4 * 2) /* 0xFFE8 Port 1 */
#define TIMERA1_VECTOR       (5 * 2) /* 0xFFEA Timer A CC1-2, TA */
#define TIMERA0_VECTOR       (6 * 2) /* 0xFFEC Timer A CC0 */
#define ADC_VECTOR           (7 * 2) /* 0xFFEE ADC */
#define UART0TX_VECTOR       (8 * 2) /* 0xFFF0 UART 0 Transmit */
#define UART0RX_VECTOR       (9 * 2) /* 0xFFF2 UART 0 Receive */
#define WDT_VECTOR           (10 * 2) /* 0xFFF4 Watchdog Timer */
#define COMPARATORA_VECTOR   (11 * 2) /* 0xFFF6 Comparator A */
#define TIMERB1_VECTOR       (12 * 2) /* 0xFFF8 Timer B 1-7 */
#define TIMERB0_VECTOR       (13 * 2) /* 0xFFFA Timer B 0 */
#define NMI_VECTOR           (14 * 2) /* 0xFFFC Non-maskable */
#define RESET_VECTOR         (15 * 2) /* 0xFFFE Reset [Highest Priority] */

```

Interrupt Edge Select P1IES, P2IES

Each interrupt edge select register contains a bit for each corresponding I/O pin to select what type of transition triggers the interrupt flag. When:

- Bit = 0: The interrupt flag is set with a low-to-high transition
- Bit = 1: The interrupt flag is set with a high-to-low transition

Note: Changing the P1IES and P2IES bits can result in setting the associated interrupt flags.

PnIES.x	PnIN.x	PnIFG.x
0 → 1	0	Unchanged
0 → 1	1	May be set

1 → 0	0	May be set
1 → 0	1	Unchanged

Interrupt Enable P1IE, P2IE

Each interrupt enable register contains bits to enable the interrupt flag for each I/O pin in the port. Each of the sixteen bits corresponding to pins P1.0 to P1.7 and P2.0 to P2.7 is located in the P1IE and P2IE registers. When:

- Bit = 0: The interrupt request is disabled
- Bit = 1: The interrupt request is enabled

Note: Port P1, Port P2 Interrupt Sensitivity. Only transitions, not static levels, cause interrupts.

If an interrupt flag is still set when the RETI instruction is executed (for example, a transition occurs during the interrupt service routine), an interrupt occurs again after RETI is completed. This ensures that each transition is acknowledged by the software.

The interrupt vectors and the power-up starting address are located in the address range 0FFFFh – 0FFE0h. The vector contains the 16-bit address of the appropriate interrupt-handler instruction sequence.

INTERRUPT SOURCE	INTERRUPT FLAG	SYSTEM INTERRUPT	WORD ADDRESS	PRIORITY
Power-up External Reset Watchdog Flash memory	WDTIFG KEYV (see Note 1)	Reset	0FFFEh	15, highest
NMI Oscillator Fault Flash memory access violation	NMIIFG (see Notes 1 & 4) OFIFG (see Notes 1 & 4) ACCVIFG (see Notes 1 & 4)	(Non)maskable (Non)maskable (Non)maskable	0FFFCh	14
Timer_B7 (see Note 5)	TBCCR0 CCIFG (see Note 2)	Maskable	0FFFAh	13
Timer_B7 (see Note 5)	TBCCR1 to 6 CCIFGs, TBIFG (see Notes 1 & 2)	Maskable	0FFF8h	12
Comparator_A	CAIFG	Maskable	0FFF6h	11
Watchdog timer	WDTIFG	Maskable	0FFF4h	10
USART0 receive	URXIFG0	Maskable	0FFF2h	9
USART0 transmit	UTXIFG0	Maskable	0FFF0h	8
ADC12	ADC12IFG (see Notes 1 & 2)	Maskable	0FFEEh	7
Timer_A3	TACCR0 CCIFG (see Note 2)	Maskable	0FFECCh	6
Timer_A3	TACCR1 CCIFG, TACCR2 CCIFG, TAIFG (see Notes 1 & 2)	Maskable	0FFEAh	5
I/O port P1 (eight flags)	P1IFG.0 (see Notes 1 & 2) To P1IFG.7 (see Notes 1 & 2)	Maskable	0FFE8h	4
USART1 receive	URXIFG1	Maskable	0FFE6h	3
USART1 transmit	UTXIFG1		0FFE4h	2
I/O port P2 (eight flags)	P2IFG.0 (see Notes 1 & 2) To P2IFG.7 (see Notes 1 & 2)	Maskable	0FFE2h	1
			0FFE0h	0, lowest

Bild: Interrupt vevtor addresses

Function Select Registers P1SEL, P2SEL

P1 and P2 port pins are often multiplexed with other peripheral modules to reduce overall pin count on MSP430 devices (see the specific device data sheet to determine which other peripherals also use the device pins). Control registers P1SEL and P2SEL are used to select

the desired pin function—I/O port or other peripheral module. Each register contains eight bits corresponding to each pin, and each pin's function is individually selectable. All bits in these registers are reset by the PUC signal. The bit definitions are:

- Bit = 0: Port P1 or P2 function is selected for the pin
- Bit = 1: Other peripheral module function is selected for the pin

Note: Function Select With P1SEL, P2SEL. The interrupt-edge-select circuitry is disabled if control bit PnSEL.x is set. Therefore, the input signal can no longer generate an interrupt.

When a port pin is selected to be used as an input to a peripheral module other than the I/O port (PnSEL.x = 1), the actual input signal to the peripheral module is a latched representation of the signal at the device pin.

The latch uses the PnSEL.x bit as its enable, so while PnSEL.x=1, the internal input signal simply follows the signal at the pin. However, if the PnSEL.x bit is reset, then the output of the latch (and therefore the input to the other peripheral module) represents the value of the signal at the device pin just prior to the bit being reset.

Port P1, P2 Interrupt Control Functions

Ports P1 and P2 use eight bits for interrupt flags, eight bits to enable interrupts, eight bits to select the effective edge of an interrupt event, one interrupt vector address for port P1, and one interrupt vector address for port P2.

Each signal uses three bits for configuration and interrupt:

- Interrupt flag, P1IFG.0 to P1IFG.7 and P2IFG.0 to P2IFG.7
- Interrupt enable bit, P1IE.0 to P1IE.7 and P2IE.0 to P2IE.7
- Interrupt edge select bit, P1IES.0 to P1IES.7 and P2IES.0 to P2IES.7

The interrupt flags P1IFG.0 to P1IFG.7 source one interrupt and P2IFG.0 to P2IFG.7 source one interrupt.

Any interrupt event on one or more pins of P1.0 to P1.7 or P2.0 to P2.7 requests an interrupt when two conditions are met: the appropriate individual bit PnIE.x is set, and the GIE bit is set. Interrupt flags P1IFG.0 to P1IFG.7 or P2IFG.0 to P2IFG.7 are not automatically reset. The software of the interrupt service routine should handle the detection of the source, and reset the appropriate flag when it is serviced.

Using I/O

- All port registers can be changed in software. This allows software to turn interrupts on and off, and to use a single pin for both input and output.
- When port pins are configured as inputs, they also function as high-impedance inputs. When being used as a high-impedance input, no pulling resistor is necessary, as long as any externally applied voltage is at the ground or supply rail. When configured this way, the leakage current is typically on the order of tens of nanoamperes.
- Interrupts on this device are edge triggered, and very susceptible to noise. A bit of filtering on interrupt lines will go a long way in noisy designs.

8.5.4. Electrical characteristics of Inputs/Outputs

Inputs

electrical characteristics over recommended operating free-air temperature (unless otherwise noted) (continued)

SCHMITT-trigger inputs – ports P1, P2, P3, P4, P5, and P6

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
V_{IT+}	Positive-going input threshold voltage	$V_{CC} = 2.2\text{ V}$	1.1		1.5	V
		$V_{CC} = 3\text{ V}$	1.5		1.9	
V_{IT-}	Negative-going input threshold voltage	$V_{CC} = 2.2\text{ V}$	0.4		0.9	V
		$V_{CC} = 3\text{ V}$	0.9		1.3	
V_{hys}	Input voltage hysteresis ($V_{IT+} - V_{IT-}$)	$V_{CC} = 2.2\text{ V}$	0.3		1.1	V
		$V_{CC} = 3\text{ V}$	0.5		1	

standard inputs – $\overline{RST}/NMI$; JTAG: TCK, TMS, TDI/TCLK

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
V_{IL}	Low-level input voltage	$V_{CC} = 2.2\text{ V} / 3\text{ V}$	V_{SS}		$V_{SS}+0.6$	V
V_{IH}	High-level input voltage		$0.8 \times V_{CC}$		V_{CC}	V

standard inputs – $\overline{RST}/NMI$; JTAG: TCK, TMS, TDI/TCLK

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
V_{IL}	Low-level input voltage	$V_{CC} = 2.2\text{ V} / 3\text{ V}$	V_{SS}		$V_{SS}+0.6$	V
V_{IH}	High-level input voltage		$0.8 \times V_{CC}$		V_{CC}	V

inputs Px.x, TA_x, TB_x

PARAMETER		TEST CONDITIONS	V_{CC}	MIN	TYP	MAX	UNIT
$t_{(int)}$	External interrupt timing	Port P1, P2: P1.x to P2.x, external trigger signal for the interrupt flag, (see Note 1)	2.2 V/3 V	1.5			cycle
			2.2 V	62			ns
			3 V	50			
$t_{(cap)}$	Timer_A, Timer_B capture timing	TA0, TA1, TA2 TB0, TB1, TB2, TB3, TB4, TB5, TB6 (see Note 2)	2.2 V	62			ns
			3 V	50			
$f_{(TAext)}$	Timer_A, Timer_B clock frequency externally applied to pin	TACLK, TBCLK, INCLK: $t_{(H)} = t_{(L)}$	2.2 V			8	MHz
$f_{(TBext)}$			3 V			10	
$f_{(TAint)}$	Timer_A, Timer_B clock frequency	SMCLK or ACLK signal selected	2.2 V			8	MHz
$f_{(TBint)}$			3 V			10	

- NOTES: 1. The external signal sets the interrupt flag every time the minimum $t_{(int)}$ cycle and time parameters are met. It may be set even with trigger signals shorter than $t_{(int)}$. Both the cycle and timing specifications must be met to ensure the flag is set. $t_{(int)}$ is measured in MCLK cycles.
2. Seven capture/compare registers in 'x44x and three capture/compare registers in 'x43x.

leakage current (see Notes 1 and 2)

PARAMETER			TEST CONDITIONS		MIN	TYP	MAX	UNIT
$I_{lkg}(P1.x)$	Leakage current	Port P1	Port 1: $V(P1.x)$	$V_{CC} = 2.2\text{ V} / 3\text{ V}$			± 50	nA
$I_{lkg}(P6.x)$		Port P6	Port 6: $V(P6.x)$				± 50	

- NOTES: 1. The leakage current is measured with V_{SS} or V_{CC} applied to the corresponding pin(s), unless otherwise noted.
2. The port pin must be selected as input and there must be no optional pullup or pulldown resistor.

Outputs

electrical characteristics over recommended operating free-air temperature (unless otherwise noted) (continued)

outputs – ports P1, P2, P3, P4, P5, and P6

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
V_{OH} High-level output voltage	$I_{OH(max)} = -1.5 \text{ mA}$, $V_{CC} = 2.2 \text{ V}$, See Note 1	$V_{CC}-0.25$		V_{CC}	V
	$I_{OH(max)} = -6 \text{ mA}$, $V_{CC} = 2.2 \text{ V}$, See Note 2	$V_{CC}-0.6$		V_{CC}	
	$I_{OH(max)} = -1.5 \text{ mA}$, $V_{CC} = 3 \text{ V}$, See Note 1	$V_{CC}-0.25$		V_{CC}	
	$I_{OH(max)} = -6 \text{ mA}$, $V_{CC} = 3 \text{ V}$, See Note 2	$V_{CC}-0.6$		V_{CC}	
V_{OL} Low-level output voltage	$I_{OL(max)} = 1.5 \text{ mA}$, $V_{CC} = 2.2 \text{ V}$, See Note 1	V_{SS}		$V_{SS}+0.25$	V
	$I_{OL(max)} = 6 \text{ mA}$, $V_{CC} = 2.2 \text{ V}$, See Note 2	V_{SS}		$V_{SS}+0.6$	
	$I_{OL(max)} = 1.5 \text{ mA}$, $V_{CC} = 3 \text{ V}$, See Note 1	V_{SS}		$V_{SS}+0.25$	
	$I_{OL(max)} = 6 \text{ mA}$, $V_{CC} = 3 \text{ V}$, See Note 2	V_{SS}		$V_{SS}+0.6$	

NOTES: 1. The maximum total current, $I_{OH(max)}$ and $I_{OL(max)}$, for all outputs combined, should not exceed $\pm 12 \text{ mA}$ to satisfy the maximum specified voltage drop.
2. The maximum total current, $I_{OH(max)}$ and $I_{OL(max)}$, for all outputs combined, should not exceed $\pm 48 \text{ mA}$ to satisfy the maximum specified voltage drop.

output frequency

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
$f_{(P_{x,y})}$ ($1 \leq x \leq 6$, $0 \leq y \leq 7$)	$C_L = 20 \text{ pF}$, $I_L = \pm 1.5 \text{ mA}$ $V_{CC} = 2.2 \text{ V}$ $V_{CC} = 3 \text{ V}$	DC		5 7.5	MHz
$f_{(ACLK)}$ $f_{(MCLK)}$ $f_{(SMCLK)}$	P1.1/TA0/MCLK, P1.5/TACLK/ ACLK P1.4/TBCLK/SMCLK $C_L = 20 \text{ pF}$			$f_{(System)}$	
$t_{(Xdc)}$ Duty cycle of output frequency	P1.5/TACLK/ACLK, $C_L = 20 \text{ pF}$, $V_{CC} = 2.2 \text{ V} / 3 \text{ V}$	$f_{(ACLK)} = f_{(LFXT1)} = f_{(XT1)}$	40%	60%	
		$f_{(ACLK)} = f_{(LFXT1)} = f_{(LF)}$	30%	70%	
		$f_{(ACLK)} = f_{(LFXT1)}$	50%		
	P1.1/TA0/MCLK, $C_L = 20 \text{ pF}$, $V_{CC} = 2.2 \text{ V} / 3 \text{ V}$	$f_{(MCLK)} = f_{(XT1)}$	40%	60%	
		$f_{(MCLK)} = f_{(DCOCLK)}$	50%– 15 ns	50% 50%+ 15 ns	
	P1.4/TBCLK/SMCLK, $C_L = 20 \text{ pF}$, $V_{CC} = 2.2 \text{ V} / 3 \text{ V}$	$f_{(SMCLK)} = f_{(XT2)}$	40%	60%	
		$f_{(SMCLK)} = f_{(DCOCLK)}$	50%– 15 ns	50% 50%+ 15 ns	

8.5.5. Programmbeispiel 1

```
// LED_Blinker.c
// Funktion: LED blinkt auf dem MSP-FET430-449 Board

#include <msp430x44x.h>    // hardwarespezifisches Header-File laden

// Funktionsdefinition
void wait_loop()
{
    unsigned int i;    // Laufvariable fuer Warteschleife
    for(i = 0; i < 60000; i++);
}

int main()
{
    WDTCTL = WDTPW + WDTHOLD;    // Watchdog deaktivieren

    P1SEL=0x00;    // Port1 als General Purpose Input Output (GPIO) verwenden

    P1DIR = 0xFF;    // Setze Port1 als Ausgang
    P1IE = 0x00;    // Interrupts disablen

    while(1)        // infinite blink loop
    {
        P1OUT = 0xFF; // LED ON
        wait_loop();

        P1OUT = 0x00; // LED OFF
        wait_loop();
    }
}
```

8.5.6. Programmbeispiel 2

```
/******
// MSP-FET430P440 Demo - Software Toggle P5.1
// Description; Toggle P5.1 by xor'ing P5.1 inside of a software loop.
// MCLK = DCO ~ 800k
//          MSP430F449
//          -----
//          /|\|          XIN|-
//          | |          |
//          --|RST        XOUT|-
//          |          |
//          |          P5.1|-->LED
//
//*****
#include <msp430x44x.h>
```

```

void main(void)
{
    WDTCTL = WDTPW + WDTNORM;           // Stop watchdog timer
    P5DIR |= 0x02;                       // Set P5.1 to output direction
    P5DIR |= 0x82;                       // Set P5.1 to output direction

    while (1)
    {
        volatile unsigned int i;        // if not volatile, the delay loop will
                                          // be optimized away by the compiler

        P5OUT = P5OUT ^ 0x82;           // Toggle P5.1 using exclusive-OR

        i = 50000;                       // Delay
        do while (i-- != 0);
    }
}

```

8.5.7. Programmbeispiel 3

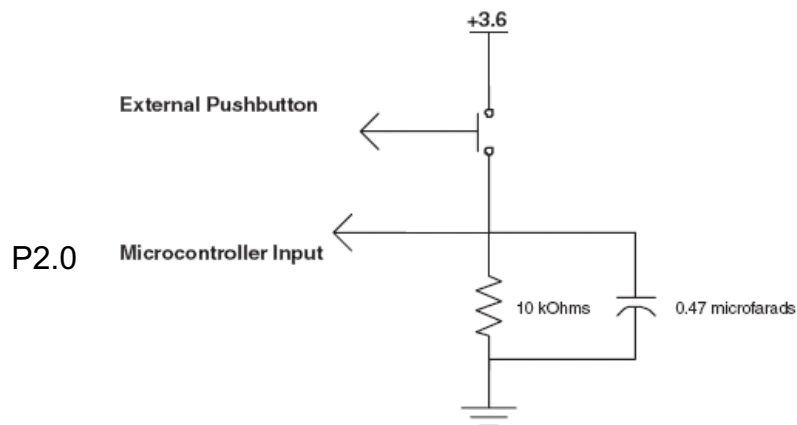


Bild: Pushbutton Interface

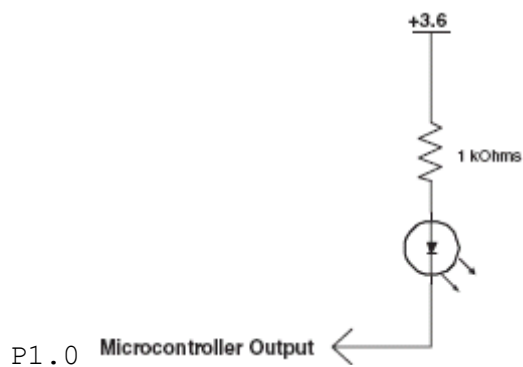


Bild: LED Interface

```

#include <msp430x44x.h>
//*****
// Tastatur-Interrupt über P2.0, dann LED togglen auf P1.0
//*****

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;          // Stop watchdog timer

    P1DIR = P1DIR | 0x01;              // Set P1.0 to output direction

    P2IE = P2IE | 0x01;                // interrupt for P2.0 enabled
    P2IES = P2IES | 0x01;              // interrupt select Hi/lo edge for P2.0
    P2IFG = P2IFG & ~0x01              // Interrupt Flag cleared for P2.0, set to 0
                                     // similar to: P2IFG &= ~0x01;

    _BIS_SR(LPM4_bits + GIE);          // Enter LPM4 w & Global interrupt enable
}

// Port 2 interrupt service routine
#pragma vector=PORT2_VECTOR
__interrupt void Port_2_keypressed(void)
{
    P1OUT = P1OUT ^ 0x01;               // P1.0 = toggle LED
    P2IFG = P2IFG & ~0x01;             // P2.0 IFG clear interrupt flag
}

```

8.5.8. Übung 1

1) Gegeben sind nachstehend die Timer A Register und deren Definition.

Die in der Tabelle Timer A genannten Register sind 16-Bit Register und müssen mit 16 Bit Befehlen bearbeitet werden.

Register	Short Form	Register Type	Address	Initial State
Timer A Control	TACTL	Read/write	160h	POR reset
Timer A Register	TAR	Read/write	170h	POR reset
Capture/Compare Control 0	CCTL0	Read/write	163h	POR reset
Capture/Compare 0	CCR0	Read/write	172h	POR reset
Capture/Compare Control 1	CCTL1	Read/write	164h	POR reset
Capture/Compare 1	CCR1	Read/write	174h	POR reset
Capture/Compare Control 2	CCTL2	Read/write	166h	POR reset
Capture/Compare 2	CCR2	Read/write	176h	POR reset
Interrupt Vector	TAIV	Read	12Eh	(POR reset)

Tabelle: Timer A Registers

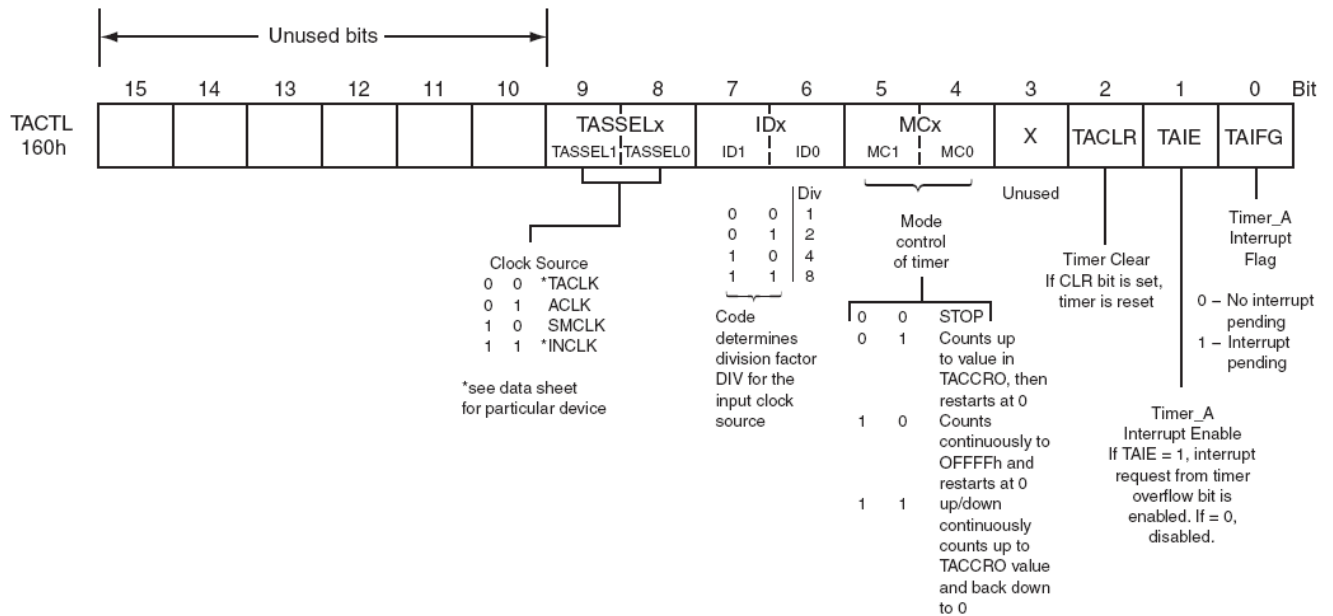


Bild: Timer A Control Register (TACTL)

```
// Timer A
TACTL = TASSEL_2 + MC_2 + ID_3 + TAIE;

TACCTL0 = CCI + CCIS0 + OUTMOD0 + CM_3 + CAP + CCIE;
```

- 1a) Aufgabe: Erläutern Sie die nachstehenden Konfigurationszeilen. In welchem Modus arbeitet der Timer A und die Capture Compare Unit 0?
- 1b) Welches Konfigurationsdatum wird dem Register TACTL zugewiesen?
- 1c) Geben Sie eine alternative Anweisungszeile für TACTL an.

- 2) Aufgabe: Erläutern Sie die nachstehenden Konfigurationszeilen. In welchem Modus arbeitet der Timer A und die Capture Compare Unit 0?

```
// Versuch 1-2: Initialisierung des Timer A als PWM mit CCR 0
void init_timer_A_PWM(void)
{
    TACCTL0 = 0x0050;

    TACCRO = 5000;
    TACTL = 0x0122;
}
```

8.5.9. Übung 2

Erstellen Sie ein Programm zur Steuerung einer Straßenampel mit Fußgängerübergang. Verwenden Sie hierzu den Port 2 und die Pins 1 (rot), 2 (gelb) und 3 (grün) für die Straßenampel, sowie die Pins 4 (rot) und 5 (grün) für die Steuerung der Fußgängerampel.

Fertigen Sie zunächst eine Skizze an und legen Sie dann die Reihenfolge der Signalübergänge fest. Erstellen Sie nun den entsprechenden Quellcode.

8.5.10. Übung 3

Erstellen Sie ein Programm zur Flankendetektion von Signalen. Das Signal liegt am Pin 2.1 an.

Erstellen Sie zunächst das Programm ohne Nutzung der Interruptfunktionalität des Ports 2 und später unter Nutzung des Portinterrupts.

8.5.11. Übung 4

Erstellen Sie ein C-Programm für den MSP430F449, dass die nachstehenden drei Blinkcodes zur Fehleranzeige eines Photovoltaikwechselrichters realisiert. Gehen Sie dabei von folgender Portpin-Signalkodierung aus:

Port 1.0 => Signal-LED grün

Port 1.1 => Signal-LED rot

Port 1.2 => Signal-LED **gelb**

Signalisierung A: Netzstörung

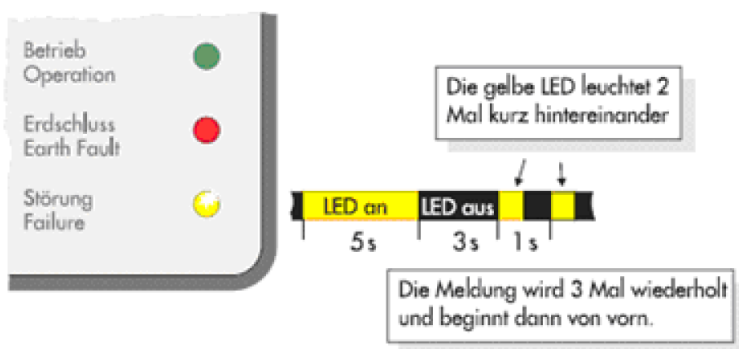


Bild: Signalisierung A: Netzstörung

Die gelbe Störungs-LED leuchtet beim Auftreten der Störung für 5 Sekunden und beginnt dann den Blinkcode auszugeben, indem sie 3 Sekunden lang dunkel bleibt und dann zweimal kurz blinkt. Der Code wird dreimal hintereinander gesendet. Liegt die Störung dann immer noch vor, beginnt die Meldung von vorn.

Signalisierung B: DC-Eingangsspannung des PV-Generators zu hoch

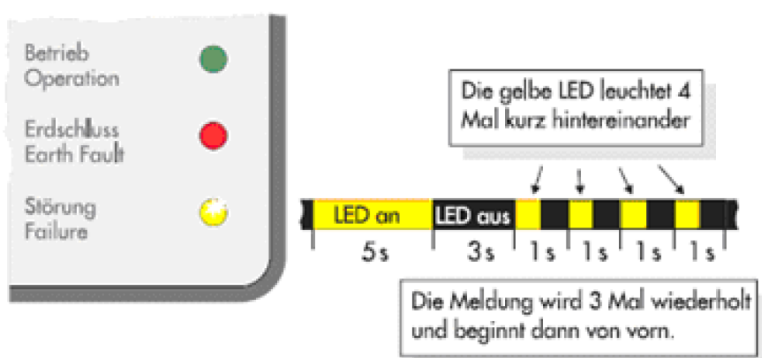


Bild: Signalisierung B: Eingangsspannung (PV-Generator) zu hoch

Die gelbe Störungs-LED leuchtet beim Auftreten der Störung für 5 Sekunden und beginnt dann den Blinkcode auszugeben, indem sie 3 Sekunden lang dunkel bleibt und dann viermal kurz blinkt. Der Code wird dreimal hintereinander gesendet. Liegt die Störung dann immer noch vor, beginnt die Meldung von vorn.

Signalisierung C: Netzimpedanz zu hoch

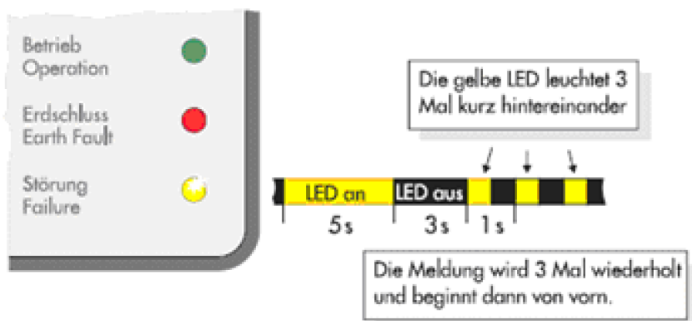


Bild: Signalisierung C: Netzimpedanz zu hoch

Die gelbe Störungs-LED leuchtet beim Auftreten der Störung für 5 Sekunden und beginnt dann den Blinkcode auszugeben, indem sie 3 Sekunden lang dunkel bleibt und dann dreimal kurz blinkt. Der Code wird dreimal hintereinander gesendet. Liegt die Störung dann immer noch vor, beginnt die Meldung von vorn.

9. INTERRUPTS UND INTERRUPT SERVICE ROUTINEN (ISR)

Begriffsdefinitionen:

Shared Interrupt (Wired OR auf Basis von Open Collector Schaltungen)

NMI (maskable, non maskable)

Message-Signaled Interrupts

Unterschiedliche Arten von Interrupts

Hardware-,

Softwareinterrupts (DOS 21h, BIOS 10h)

BIOS-Interrupts

Exceptions (Fault, Trap, Abort)

Interrupt-Controller

Kaskadierte Interruptcontroller

Bsp.: 8259

Vermittlung an den PC

Wired OR-Shared Interrupt

Einsatzgebiete von Interrupts

Anwendungsbsp.: Gegenüberstellung Polling - Interruptbasierende Verarbeitung

Wake on LAN

Diagnosebearbeitung und Power Save

Funktionsweise

IRQ -> Interruptvektortabelle -> ISR

C -> ASM -> HexCode bei der Implementierung von Interrupts am MSP430xx

Interruptvektoren „umbiegen“

IRQs am Systembus

Bsp: Interrupt-Ressourcen im PC

Bsp.: Interrupts beim MSP430, Auflösung der Makroanweisung #pragma

Literatur und Weblinks

<http://www.bjoern-koester.de/iogrundlagen/index.html>

<http://www.atwillys.de/imp/prj/index.htm>

9.1. Arten von Interrupts

Das englische Wort "Interrupt" bedeutet einfach Unterbrechung. Diese Bezeichnung verwendet man allgemein für Anweisungen an den Hauptprozessor, den gerade laufenden Prozess zu unterbrechen und vorübergehend einen anderen Befehl abzuarbeiten.

Es werden begrifflich verschiedene Arten von Interrupts unterschieden, auf die nachstehend eingegangen werden soll:

- **Prozessorsynchrone Interrupts.** Sie werden durch eine Folge konkreter Befehlsausführungen vom Prozessor selbst ausgelöst, man nennt sie auch Traps. Sie erfolgen selbstverständlich synchron zum Prozessortakt. Bsp.: Division durch Null.
- **Prozessorasynchrone Interrupts.** Sie werden von der Peripherie ausgelöst. Wenn man allgemein von Interrupts spricht, dann sind damit die prozessorasynchronen Interrupts gemeint. Mit dieser Sorte von Interrupts wollen wir uns im folgenden beschäftigen. Bsp.: Controller-Interrupt.

Der Prozessor wird beim Auftreten eines Interrupts dazu veranlasst, sein aktuell laufendes Programm (*Normalprogramm*) zu unterbrechen. Gäbe es nur ein Programm, welches auf jeden Interruptaufruf reagieren soll, so hätten wir gar kein Problem, dieses zu ermitteln. In der Praxis gibt es aber eine Vielzahl solcher Programme, deshalb muss nach Eingang des IRQs getestet werden, welches Programm anstelle des Normalprogramms auszuführen ist. Gehen wir zunächst davon aus, dass es nur einen einzigen IRQ gibt. Dazu muss die Ablaufsteuerung des *Fetching (Holephase des Prozessors)* um eine Interrupt-Erkennung erweitert werden. Ein IRQ kann, wie wir oben schon festgehalten haben, asynchron, d.h. zu beliebiger Zeit während eines Prozessorzyklus eintreffen. Damit der Prozessor nicht voll und ganz aus der Rolle gebracht wird, wird die Interruptanforderung zunächst zwischen gespeichert - man sagt, sie ist *hängend (pending)*. Das Testen des peripheren Signals wird bei neueren Prozessoren hardwareseitig vom Mikroprogramm eines jeden Befehls übernommen. Hierbei wird ein Statusbit abgefragt, das *Interrupt-Pending-Bit*. Zur Implementierung ist also eine Interruptleitung (*INT*) nötig, die das Peripheriegerät über einen Interrupt-Controller mit dem Prozessor verbindet. Hinzu kommt noch eine Quittungsleitung (*INTA, Interrupt Acknowledge*), die den Interrupt bestätigt.

Die Alternative zu dem Hardware-Test ist ein Software-Test, der über PIO den Zustand der entsprechenden Peripherieregister abfragt (Polling). Die speziellen Steuerleitungen zum Prozessor sind dann zwar nicht nötig, aber ein solcher *Interrupt-Handler* ist wegen den in Abschnitt <http://www.bjoern-koester.de/iogrundlagen/io.html> - *polling* bereits genannten Performance-Gründen nicht angebracht.

9.1.1. Interne Unterbrechungsursachen

Software-Interrupt, Supervisor-Call

- SWI n (INT n, SVC n) (n - Parameter zur Auswahl der Behandlungs-Routine; Bsp. für PC INT\$21 Rückkehr zu DOS)
- ähnlich einem Prozedur-Aufruf nur, dass die Aufrufadresse durch das System und nicht durch das Programm bereitgestellt wird; genutzt zum Ansprechen peripherer Komponenten und Betriebssystemdienste; Ziel ist die Schaffung eindeutiger Software-schnittstellen (vgl. Softwarelayer BIOS, DOS, COMMAND/WINDOWS)
- Prozessoren mit mehreren Privilegebenen Übergang vom Nutzer- in den Betriebssystem-Modus

Ausnahmesituationen bei der Befehlsabarbeitung (instruction exceptions, traps)

a) Einfügen einer Prozedur

- Einzelschritt-Modus, Einzelschritt-(single step) Flag gesetzt: Einfügen einer Prozedur nach jedem abgeschlossenen Befehl; genutzt für Inbetriebnahme, Test und Fehlersuche (Debugging)

b) Beseitigung der Ursache und Befehlswiederholung

- Seitenfehler (page segment fault): adressiertes Befehlswort, adressierte Daten nicht im Speicher;
Behebung: Laden der Seite; Problem: rückgängig machen nachfolgender, bereits gestarteter Befehle

c) Abbruch der aktuellen Aufgabe und Fortsetzung ab einer definierten Adresse

- ungültiger Befehlscode
- privilegierte Operation im Nutzermodus
- Verletzung der Zugriffsrechte auf den Speicher, Überschreitung der Speichergrenzen
- Division durch Null, Überlauf, Überschreitung von Feldgrenzen

9.1.2. Externe Unterbrechungsursache

Unterbrechungsursachen

- Reset: Initialisieren aller Register, Laden einer Anfangsadresse
- NMI, nicht unterbrechbarer Interrupt: wird zur Fehlerbehandlung genutzt, z. B. zum Anschluss eines Watchdogs

9.1.3. Hardware Interrupt Request-Leitungen (IRQs)

Über Interrupt Request Leitungen (Abkürzung IRQ, englisch für „Unterbrechungsanforderung“) wird in Mikrocomputern eine Unterbrechung eines Prozessors ausgelöst, in der Regel ausgehend von Peripheriegeräten im System.

Der Prozessor unterbricht dann seine derzeitige Aufgabe und führt die Unterbrechungsroutine (Interrupt Service Routine = ISR) aus. Danach wird die zuvor unterbrochene Aufgabe fortgesetzt.

Beispiele:

Der serielle Schnittstellenbaustein UART generiert einen Interrupt Request (IRQ), wenn ein Zeichen über das Kommunikationsmodul empfangen hat. Die dem IRQ zugeordnete Unterbrechungsroutine (ISR) kann nun das jeweilige Zeichen von dem UART-Baustein lesen

und z.B. in den Hauptspeicher (Arbeitsspeicher) übertragen.

Weitere Beispiele, in denen Geräte einen Interrupt Request generieren:

- Netzwerkkarten generieren einen Interrupt Request, wenn Daten empfangen wurden und im Puffer bereitliegen.
- Festplattencontroller generieren einen Interrupt Request, wenn die vorher angeforderten Daten von der Festplatte gelesen wurden und nun im Cache des Festplattencontrollers zur Abholung durch die CPU oder DMA abholbereit stehen.

Interrupt Requests (IRQ) und Interrupt Reaktionskette

Als IRQ bezeichnet man ein elektrisches Signal, das eine Hardwarekomponente (Peripheriemodul einer MCU) auslösen kann, um die CPU zur Abarbeitung von Befehlen aufzufordern. So meldet z. B. ein SCSI- oder IDE-Hostadapter dem Hauptprozessor, dass angeforderte Daten bereitstehen und jetzt verarbeitet werden müssen. Auch eine Soundkarte teilt über einen IRQ der CPU mit, wenn neue Daten benötigt werden, die von der CPU berechnet und zeitäquidistant aus dem Speicher zur Ausgabe gebracht werden sollen.

Die CPU besitzt nur einen Eingang für Interruptanforderungen. Um zu entscheiden, welche Hardwarekomponente die Aufmerksamkeit des Prozessor braucht, gibt es eine festgelegte Vorgehensweise: Ein spezieller Controller im Chipsatz, der Programmable Interrupt Controller (PIC), stellt mehrere Eingänge für IRQ-Anforderungen von Hardwarekomponenten zur Verfügung.

Löst nun eine Hardwarekomponente einen IRQ aus, so schickt der PIC seinerseits eine Interruptanforderung an die CPU und übermittelt dieser gleichzeitig einen Interrupt-Vektor, an der die jetzt auszuführenden Anweisungen stehen. Diese arbeitet die CPU dann ab.

Der PIC hat sich historisch aus der PC Architektur entwickelt. Damals bot er nur acht Eingänge (IRQ 0-7). Mit der AT-Architektur kam ein zweiter PIC hinzu, der an den IRQ2 des ersten PIC angeschlossen ist (PIC Kaskadierung).

Die Nummerierung der IRQs gibt die Priorität dieser IRQs an; diese ist aber nicht absolut zu nehmen. Die meisten der 15 verschiedenen IRQs sind fest belegt, andere lassen sich je nach Konfiguration des PIC unterschiedlich nutzen: Der IRQ0 besitzt jedoch in beiden Fällen stets die höchste und IRQ15 ebenfalls die niedrigste Priorität.

IRQ	typische Bedeutung	Alternativen
00	Systemtaktgeber	
01	Tastatur-Controller	
02	programmierbarer Interrupt-Controller	alte Modems, EGA, COM3, COM4
03	serielle Schnittstelle COM2 (E/A-Bereich 02F8)	COM4, Modems, SK, NWK
04	serielle Schnittstelle COM1 (E/A-Bereich 03F8)	COM3, Modems, SK, NWK
05	frei, oft Soundkarte (Soundblaster-Emulation) oder LPT2	LPT2, LPT3, COM3/4, Modems, NWK
06	Floppy-Controller	
07	parallele (Drucker-)Schnittstelle LPT1 (E/A-Bereich 0378)	LPT2, COM3/4, Modems, SK, NWK

08	Echtzeituhr	
09	frei	SK, NWK, SCSI, PCI
10	frei	SK, NWK, SCSI, 2.+4. IDE, PCI
11	frei	SK, NWK, SCSI, VGA, 3.+4. IDE, PCI
12	PS/2-Mausanschluss	SK, NWK, SCSI, VGA, 3. IDE, PCI
13	Math.-Coprozessor (ob separat oder in CPU integriert)	
14	1. IDE-Channel (primärer IDE-Kanal)	SCSI
15	2. IDE-Channeln (sekundärer IDE-Kanal)	NWK, SCSI

Bild: IRQ-Belegung in einem Pentium-Rechner

In der Regel stehen nur ungeteilte IRQ-Leitung zur Verfügung, eine Ausnahme stellen die seriellen COM-Ports dar. Stehen mehr als zwei serielle Schnittstellen zur Verfügung, so teilen sich COM1 und COM3 die IRQ4-Leitung und COM2 und COM4 die IRQ3-Leitung (Shared IRQ). Die E/A-Adressbereiche (RxD Input-Register, TxD Transfer Register) von COM1 und COM3 bzw. COM2 und COM4 sind jedoch verschieden.

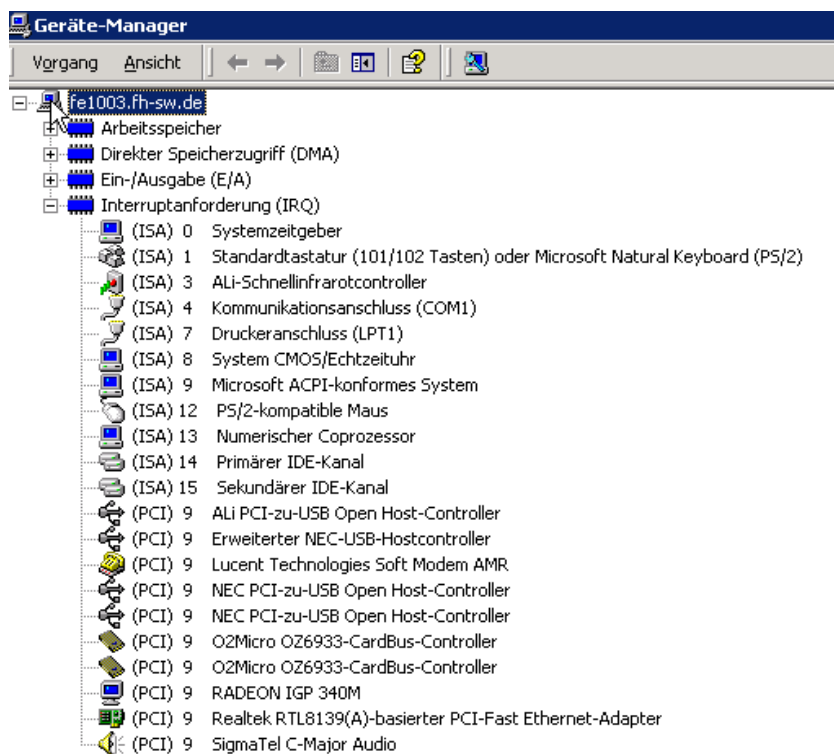


Bild: Interrupt-Liste eines Pentium-Rechners

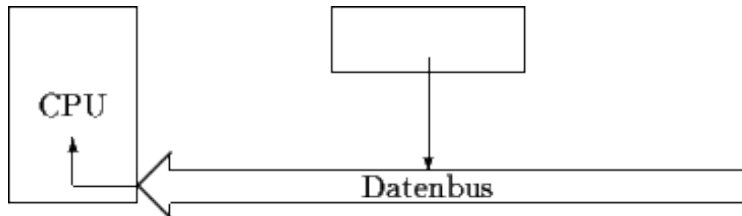
Die Interrupt Reaktionskette

9.2. Übertragung des Interruptvektors von der Peripherie an die CPU

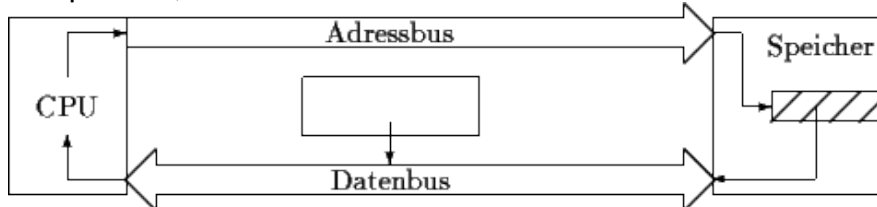
Übertragung des Interruptvektors von der Peripherie an die CPU: Vektorisierung

Das Verzweigen zur Interrupt-Service-Routine wird allgemein mit Vektorisierung von Interrupts bezeichnet. Bei der Vektorisierung gibt es mehrere Möglichkeiten, von denen hier die wichtigsten vorgestellt werden:

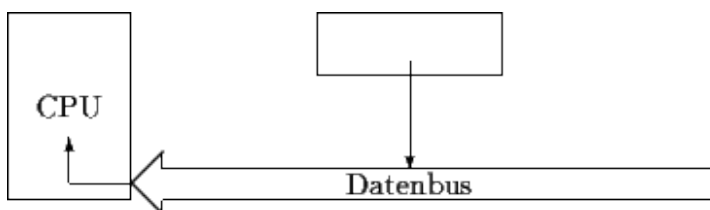
1. Die Interruptquelle legt den CALL-Befehl auf den Datenbus. Dieser wird vom Prozessor gelesen und wird daraufhin decodiert und ausgeführt.



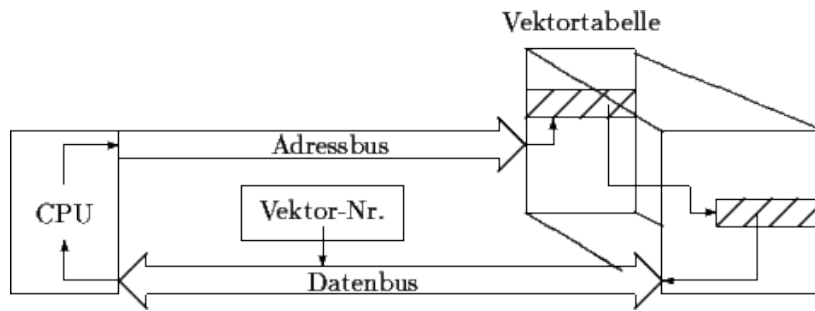
2. Die Interruptquelle legt nicht den CALL-Befehl selbst, sondern seine (Speicher-) Adresse auf den Datenbus. Der Prozessor liest mit dieser Adresse den CALL-Befehl aus dem Speicher, decodiert den Befehl und führt diesen aus.



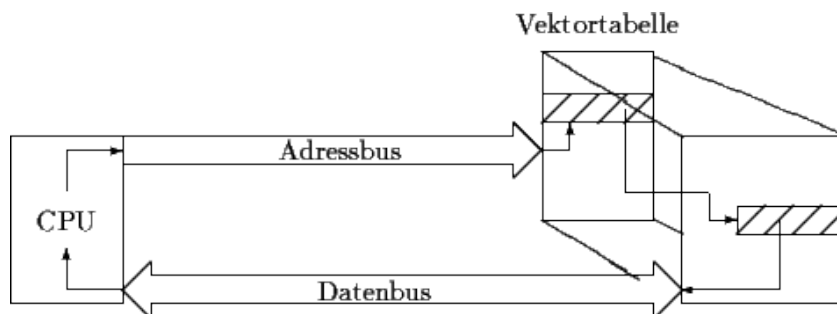
3. Anstelle des ganzen Befehls legt die Interruptquelle nur die im benötigten Befehl befindliche Adresse auf den Datenbus. Dies ist schließlich gleich der Adresse der Service-Routine. Der Prozessor liest die Service-Routine und verzweigt direkt zum Mikrocodeprogramm des CALL-Befehls (oder zu einem bestimmten Mikrocode - programm für den Interrupt). Die Decodierung entfällt.



4. Statt der Adresse der Service-Routine wird nur die Vektornummer auf den Datenbus gelegt. Durch den Interrupt wird die Vektornummer bestimmt. In der im Speicher befindlichen Interrupt-Vektortabelle werden die Startadressen der jeweiligen Service-Routinen festgehalten. Die Adresse der Service-Routine wird vom Prozessor aus dem Speicher, genauer, der Interrupt-Vektortabelle gelesen und verzweigt direkt zu einem bestimmten Mikroprogramm für den Interrupt (Interrupt Service Funktion - ISR). Solche Interrupts nennt man Vektor-Interrupts.



5. Anders als in Variante 4 legt die Interruptquelle keine Angabe für die Service-Routine auf den Datenbus. Die Vektornummer ist für den jeweiligen Interrupt prozessorintern gespeichert, so dass sie nicht erst (prozessorextern, z.B. über den Speicher) gelesen werden muss. Der Prozessor benutzt die Vektornummer unmittelbar zur Indizierung der Vektortabelle. Man nennt diese Interrupts Autovektor-Interrupts.



Bewertung der Methoden:

In der Praxis wird die Vektormethode 4. und 5. am häufigsten eingesetzt. Durch sie entsteht ein großer Hardwareaufwand, da für jede Interrupt-Anforderung ein Eingang am Prozessor vorgesehen sein muss. Die Sammlung und Priorisierung der Interruptanforderungen übernimmt CPU intern oft eine festverdrahtete Logik (Daisy-Chaining).

Flexibler sind die ersten drei Methoden, weil mehrere mögliche ISR auf einen einzigen Interrupt angewendet werden können. Die Effizienz ist allerdings geringer.

9.2.1. Maskierbare und nicht maskierbare Interrupts

Damit im unmittelbaren Befehlszyklus nach Ausführung der Service-Routine nicht wieder derselbe Interrupt wiederholt ausgelöst wird, auch wenn die Peripherie diesen nicht erwünscht, muss ein zusätzliches Bit untergebracht werden, welches das Interruptsignal unwirksam macht. Ein solches Bit heißt Maskenbit, da das Interruptsignal „maskiert“ wird.

Für den Prozessor ist das Interruptsignal, solange es maskiert ist, praktisch unsichtbar.

Das Entfernen, d. h., „Demaskieren“ des Interrupts sollte erst nach Abarbeitung der Service-Routine erfolgen, da damit zeitgleich auch das Interrupt-Pending-Bit gelöscht wird.

Nicht maskierbare Interrupts (non-maskable Interrupt, NMI)

Neben den maskierbaren Interrupts gibt es auch solche, die sich nicht maskieren lassen. Es handelt sich um nicht-maskierbare Interrupts (NMI).

NMIs können nicht ausmaskiert werden. Auch die Software besitzt keine Möglichkeit, diese Interrupts zu unterbinden. Die NMIs sind in der Lage, bereits laufende maskierbare Interrupts zu unterbrechen. NMIs haben stets die höchste Priorität und sind nur für die schlimmsten Ereignisse (BOR etc.) konzipiert worden. Ausgelöst werden sie z.B. bei unmittelbar bevorstehendem Stromverlust oder bei Speicher(zugriffs)fehlern (Page Faults).

Die NMIs muss die CPU unmittelbar verarbeiten, da sie bei zeitkritischen Unterbrechungsanforderungen zum Einsatz kommen.

9.2.2. Software-Interrupts (BIOS-Interrupt)

Über die so genannten BIOS-Interrupts stellt das BIOS bestimmte Funktionen bereit. Diese nutzt das Betriebssystem oder irgend ein Programm durch Aufruf des BIOS-Interrupts, ähnlich wie ein Unterprogramm. Bekannt ist etwa der INT 13h, über den man große Festplatten ansprechen kann oder INT 10h zum Ansprechen des Monitors.

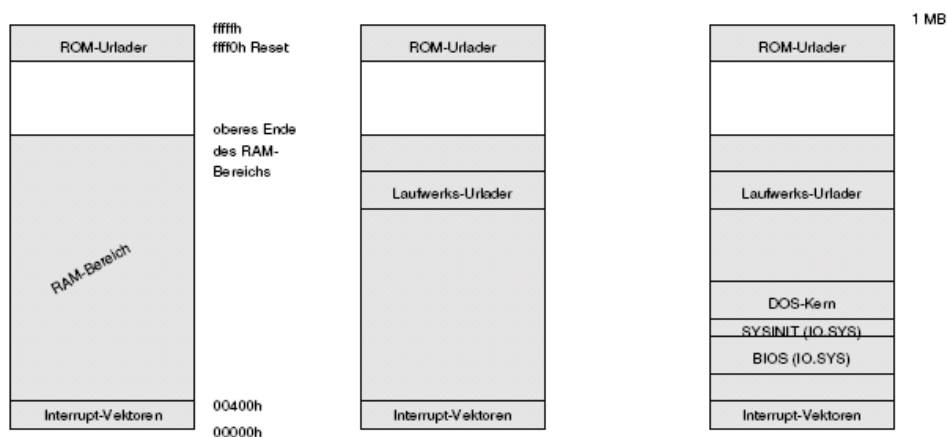


Bild: Schichtenmodell zum MS-DOS Betriebssystem

Software-Interrupts beim PC, z.B.

Interrupt 10h - BIOS (Bildschirm)

Interrupt 16h - BIOS (Tastatur)

Interrupt 21h - DOS (Betriebssystemfunktionen)

BIOS-Interrupt zur Tastaturabfrage: Interrupt 16h

Funktion 00h - Zeichen auslesen

Funktion 01h - Prüfen, ob Zeichen vorhanden ist

BIOS-Interrupt zur Bildschirmansteuerung: Interrupt 10h

Funktion 00h - Setzen des Videomodus
Funktion 02h - Positionierung des Cursors
Funktion 03h - Auslesen der Cursorposition
Funktion 09h - Schreiben eines Zeichens / Farbe
Funktion 0Eh - Schreiben eines Zeichens

9.2.3. Interrupt Service Routinen (ISR)

Unabhängig von der Art des Tests (Soft- oder Hardware) springt der Prozessor zu einem Programm, das der Interruptquelle zugeordnet ist. Diese Ausnahmeroutine bezeichnet man als die (Interrupt-)Service-Routine, ISR. Eine ISR ist ein Unterprogrammroutine im Programmspeicher, welche durch ein äußeres Ereignis unmittelbar aufgerufen wird.

Durch das IRQ-Signal (entspricht dem CALL) wird zu einer bestimmten ISR verzweigt und die Rücksprungadresse zum bisherigen Prozess auf dem Stack abgelegt.

Der Assembler-Befehl RETI (RETURN) bewirkt die Rückkehr. Damit es zu keinen Konflikten mit den aktuellen Statusregistern des Prozessors kommt, muss ein Unterprogramm in einer ISR reentrant (wiedereintrittsfest) sein. Konkret heißt das, dass am Anfang der ISR die Statusregister und der Programmzähler auf dem Stack abgelegt werden. Vor der Rückkehr werden diese wieder vom Stack zurückgelesen. Es gibt einen speziellen RETURN-Befehl für Interrupts, den EOI-Befehl (End Of Interrupt).

Nachdem die Service-Routine abgearbeitet worden ist, kehrt der Prozessor wieder an die Stelle im Programm zurück, von dem er unterbrochen. Nun wird der frühere CPU-Programmskontext wieder hergestellt. Der Aufruf von Interrupts ist daher schematisch gleichbedeutend mit der Quasi-Parallelverarbeitung von Prozessen.

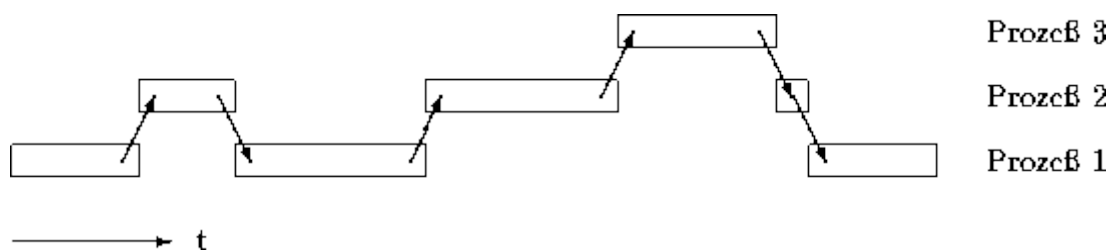


Bild: Quasi-Parallelität von Prozessen.

Werden die Prozesse wie in Bild gezeigt „verschachtelt“ und jeweils „verzahnt“ ausgeführt, so spricht man von „Quasi-Parallelität“. Voraussetzung dafür ist die „Unterbrechbarkeit“ des Prozessors - und damit der einzelnen Prozesse. Eine derart asynchrone Unterbrechung wird als (Prozessor-)Interrupt bezeichnet. Asynchron daher, weil der Interrupt nicht synchron, d.h. zu vorhersehbarer Zeit, sondern zu ganz bestimmten externen Ereignissen, also asynchron, eintrifft.

9.3. Der Programmable Interrupt-Controller 8259 (PIC)

Im Folgenden wird der Programmable Interrupt-Controller 8259 PIC von INTEL vorgestellt, der in der Praxis eine große Rolle spielt.

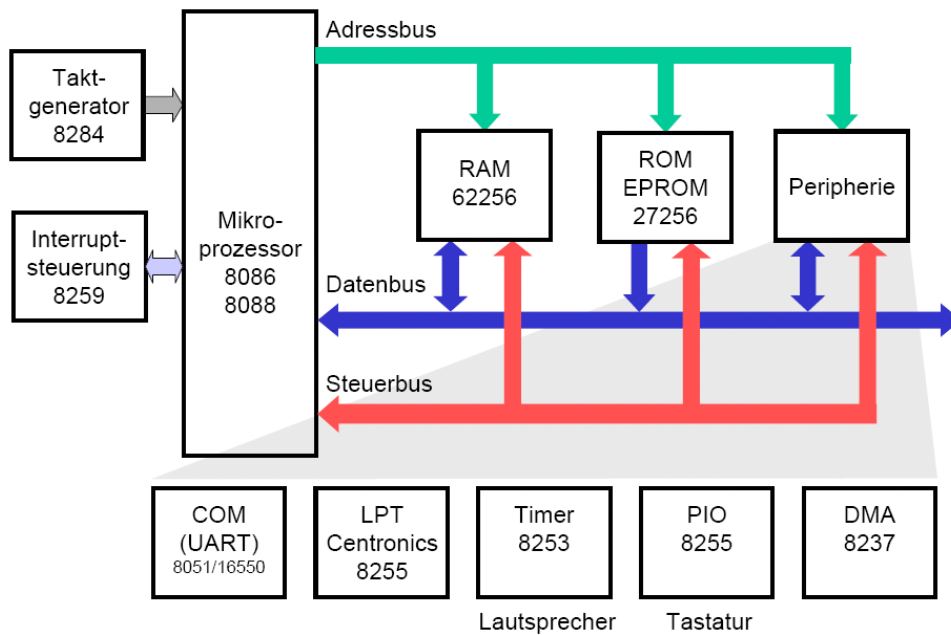


Bild: Ankopplung des Interruptcontrollers 8259 an die CPU

Peripheriebausteine:

- Timer-Baustein (8253)
- Paralleles Interface (8255)
- Serielles Interface (8251)
- DMA-Controller (8237)
- Programmierbare Interruptsteuerung (8259)

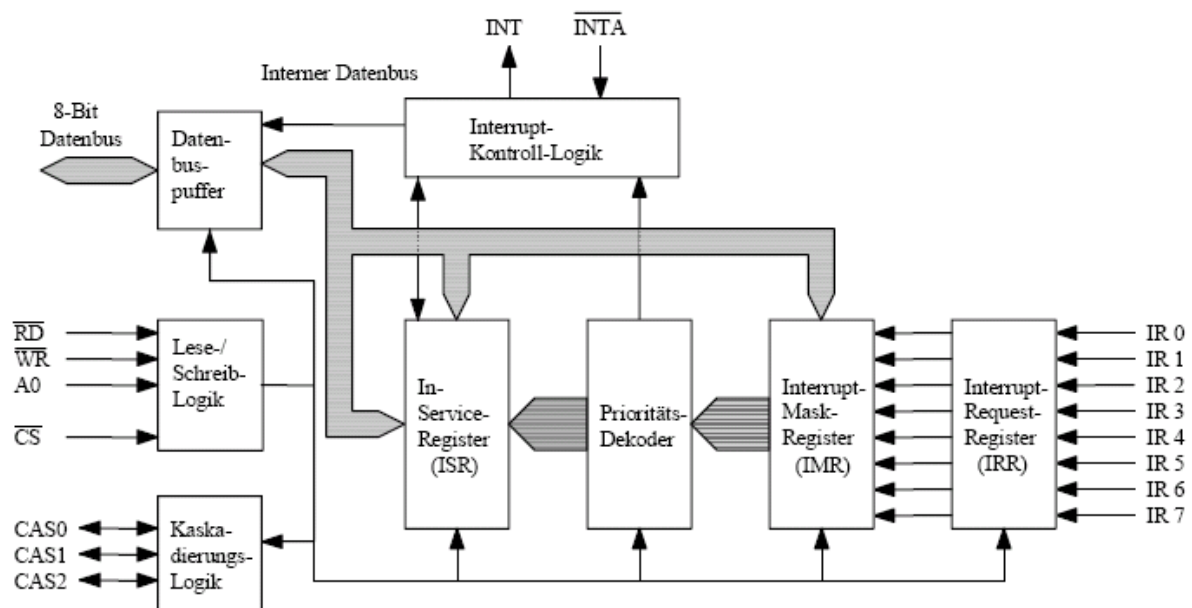


Bild: Blocksaltbild des PIC 8259

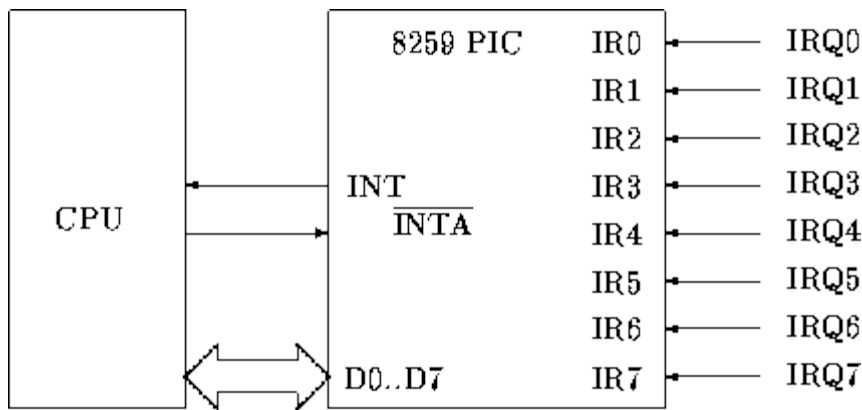


Bild: Stark vereinfachter Aufbau des 8259 PIC Interrupt-Controller von INTEL und dessen Anbindung an die CPU

Der 8259 PIC-Baustein kann einzeln bis zu 8 externe Interruptquellen verwalten, kann diese priorisieren, maskieren und erkennt, welche Interrupts gerade anliegend sind (pending) und welcher gerade bedient wird.

8259 PIC intern

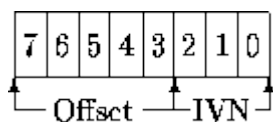


Bild: codierte Interrupt-Vektornummer

Die codierte Interrupt-Vektornummer ist in den niederwertigen 3 Bits angegeben. Die 5 übrigen Bits dienen als Offset des ersten IRQ (IRQ0) in der Vektortabelle, der bei Initialisierung des Interrupt-Controllers geändert werden kann („Umbiegen von Interrupts“). Die Vektornummer berechnet sich durch Offset + IRQ-Nummer.

Der Prozessor holt die Interrupt-Vektornummer vom Datenbus und ruft die entsprechende Interrupt-Service-Routine auf. Nach Beenden der Service-Routine wird ein EOI-Befehl an den Controller übergeben. Das ISR-Bit wird dann gelöscht.

In einer anderen Betriebsart, die den Normalfall darstellt, wird direkt beim Ausklingen des zweiten /INTA-Signals das ISR-Bit automatisch gelöscht. Befindet sich noch ein gesetztes Bit im PR, so beginnt dieser Vorgang von vorn. Eine Ausnahme besteht dann, wenn der PR erkennt, dass während der Abarbeitung einer Interrupt-Service-Routine ein weiterer Interrupt höherer Priorität angefordert wird. In diesem Fall wird die Interrupt-Service-Routine - entsprechend dem Abbruch eines Normalprogramms - abgebrochen und die neue Routine aufgerufen.

Kaskadierung von Interrupt-Controllern

Der ursprüngliche 8-Bit IBM-PC und der PC/XT besaßen nur einen einzigen IC, daher konnten nur 8 Hardware-Interrupts unterstützt werden. Wenig später, mit dem IBM PC/AT 16-Bit ISA-Bus, wurde aus Gründen der Abwärtskompatibilität ein zweiter PIC mit dem bereits vorhandenen zusammengeschaltet („kaskadiert“).

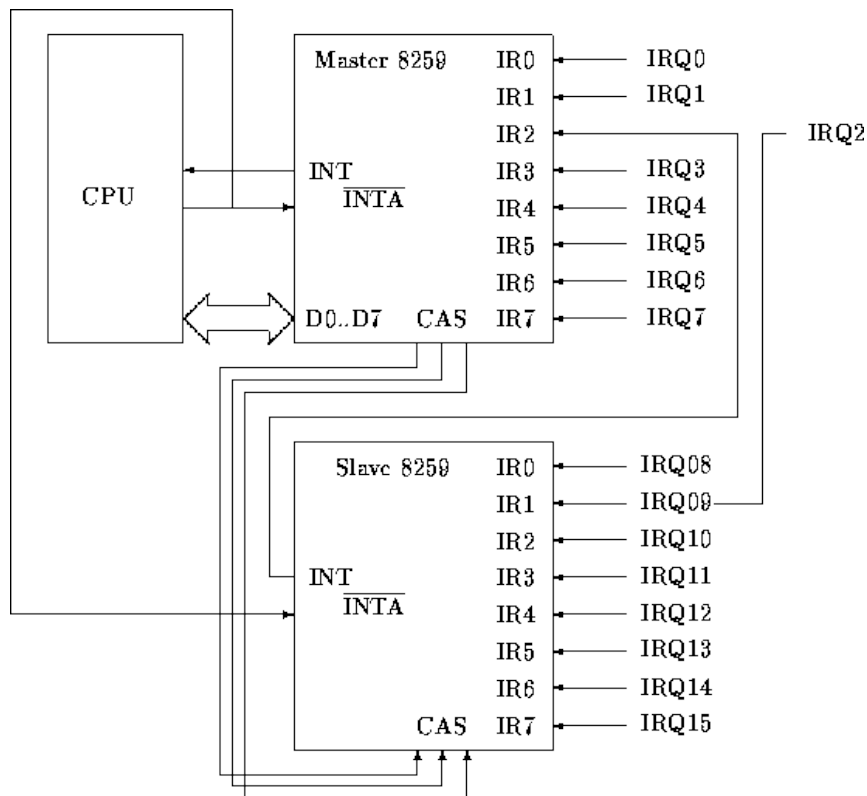


Bild: Zwei kaskadierte 8259 PIC Interrupt-Controller

Da die INTEL 80x86-Prozessorserie aber nur über eine einzige INT-Leitung verfügt, wurde der zweite PIC (PIC2) an den IRQ2-Anschluß des ersten PIC (PIC1) angeschlossen. Folglich sind insgesamt 15 Hardware-Interrupts implementiert.

Sind in einem System zwei PIC Interrupt-Controller kaskadiert, so ist einer Master und der andere Slave. Erreichen den Master Interruptanforderungen auf den Leitungen IR_i , ändert sich nichts im Vergleich zu einem Single-Betrieb. Wenn eine Anforderung auf die Leitungen des Slave eintrifft, dann wird INT signalisiert. INT ist nicht - wie bisher - mit dem Prozessor, sondern mit dem Master-PIC verbunden. Im IRR-Register wird das 2. Bit gesetzt und gegen die ISR- und IMR-Register verglichen. Anschließend, wenn nichts dagegen spricht, wird der Slave durch die CASi-Leitungen „aktiviert“. Wenn im folgenden der Prozessor sein /INTA-Signal setzt, um den Interrupt zu bestätigen, fühlt sich nur der Slave davon angesprochen. Der Master „ruht“ währenddessen. Der Rest geschieht analog zum Single-Betrieb.

Es ist denkbar, bis zu 8 Slaves an einem Master zu betreiben. In diesem Fall können bis zu 64 Interrupts verwaltet werden.

9.3.1. Zusammenspiel zwischen CPU und 8259-Controller

Der Interrupt-Controller bietet die Möglichkeit der Programmunterbrechung (Interrupts) durch externe Signale. Dazu besitzt der 80486 zwei Interrupteingänge. Der eine Eingang (NMI) ist für besonders wichtige Unterbrechungen vorgesehen. Der andere Eingang (INTR) ist für alle sonstigen Unterbrechungsquellen gedacht.

Da es nur einen Eingang gibt, muss dafür gesorgt werden, dass der Prozessor die einzelnen Quellen unterscheiden kann. Eine Möglichkeit wäre, die Quellen vom Behandlungsprogramm der Reihe nach abfragen zu lassen. Dieses Verfahren kostet aber Zeit und macht daher den eigentlichen Vorteil von Unterbrechungen, die schnelle Reaktion auf Ereignisse, wieder zunichte.

8 Bit XT Bus – top view

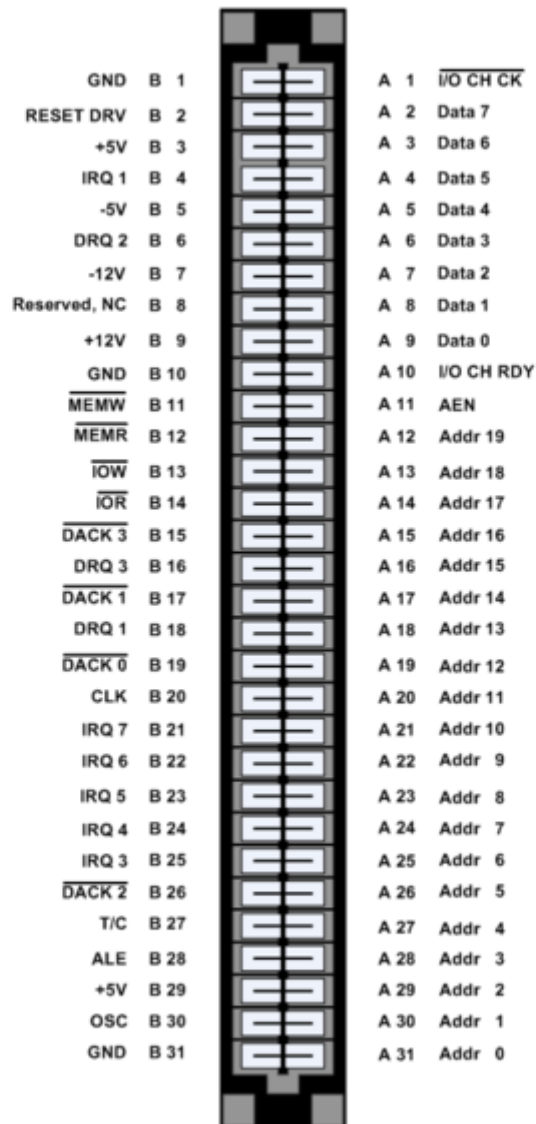


Bild: Der Industry Standard Architecture (ISA)-Bus

Eine andere, schnellere Möglichkeit wird bei Prozessoren der 80x86 Familie verwendet. Während der Interruptbestätigung erwartet der Prozessor die Nummer des Interrupts, der bearbeitet werden soll. Mit dieser Nummer, auch Vektor genannt, wird die Startadresse des Behandlungsprogramms ermittelt. Damit ist die Unterscheidung der verschiedenen Quellen geregelt.

Es gibt aber noch ein weiteres Problem, die Dringlichkeit eines Interrupts. So wollen schnelle Peripheriegeräte, wie z.B. Festplatten, möglichst sofort bedient werden, während die Datenübertragung an einen Drucker sicher nicht so dringend ist. Es wird also eine logische Schaltung benötigt, die dafür sorgt, dass wichtige Interrupts bevorzugt behandelt werden, den verschiedenen Quellen also unterschiedliche Prioritäten zuordnet.

Damit wären die Aufgaben des Interrupt-Controllers bekannt:

- Erzeugen des Interruptvektors, entsprechend der anfordernden Quelle
- Zuordnen von Prioritäten zu den verschiedenen Quellen..

Der 8259-A Interrupt-Controller erledigt die genannten Aufgaben. Dazu besitzt er acht Interrupteingänge mit unterschiedlicher Priorität. Wenn die acht Eingänge nicht ausreichen, können mehrere Controller verwendet werden. Dabei wird einer als Master-Controller definiert, welcher dem Prozessor Interrupts signalisiert. An den Master-Controller können dann bis zu acht Slave-Controller angeschlossen werden. Dazu werden die Eingänge des Masters mit den Interruptausgängen der Slave-Controller verbunden. Man spricht hier auch von einer Kaskadierung der Interruptcontroller. Zu diesem Thema folgen später noch einige Erläuterungen. Jetzt soll erst einmal der Aufbau und die Funktionsweise des Controllers allgemein beschrieben werden. Das folgende Bild zeigt den schematischen Aufbau des Controllers.

Der Baustein besitzt, wie in Bild zu sehen, über einen 8 Bit breiten Datenbus zur Verbindung mit dem Prozessor. Die Auswahl des Bausteins erfolgt durch einen LOW-Pegel am CS\ - Eingang. Der Baustein belegt zwei Adressen, die über den Pegel am Eingang A0 unterschieden werden. Zum Schreiben von Daten wird der Eingang WR\ auf Low-Pegel gelegt. Eine steigende Flanke überträgt das an den Datenleitungen anliegende Byte in das ausgewählte Register. Soll ein Byte gelesen werden, wird der RD\ - Eingang auf Low-Pegel gelegt, ungefähr 200ns später aktiviert der Baustein seinen Bustreiber und legt das angeforderte Byte auf den Datenbus. Der INT-Ausgang wird mit dem INTR-Eingang des Prozessors verbunden. Ein High-Pegel zeigt eine Interruptanforderung an. Die Bestätigung des Prozessors wird dem Controller durch einen Low-Pegel am INTA-Eingang angezeigt. Die mit IR0 bis IR7 bezeichneten Leitungen sind die Interrupteingänge. Die Leitungen CAS0 bis CAS2 sind nur dann von Bedeutung, wenn die Controller kaskadiert werden. Beim Master sind sie Ausgänge, beim Slave Eingänge. Alle Slaves sind über diese Leitungen mit dem Master verbunden.

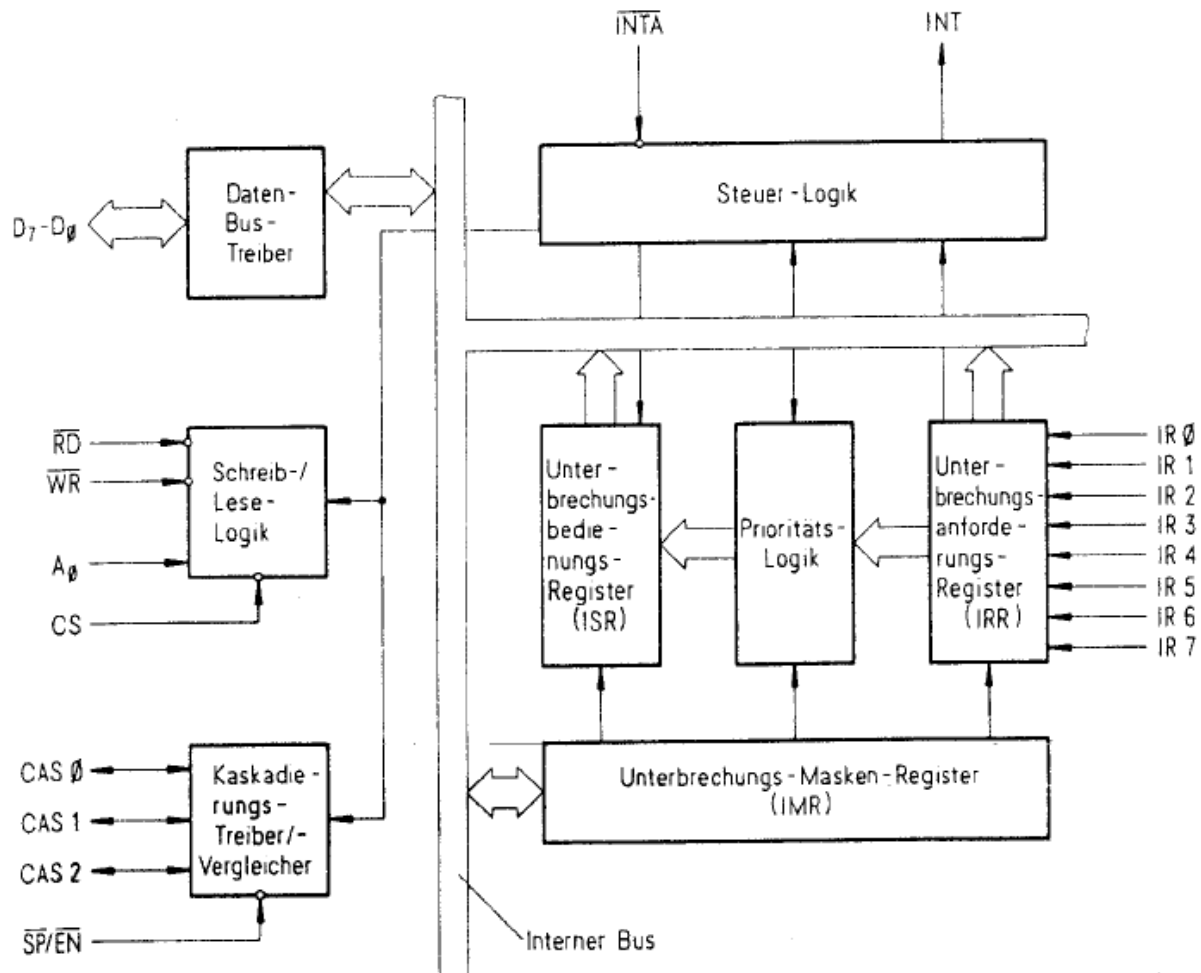


Bild: Blockschaltbild des PIC 8259

Nun zur Funktionsweise des Controllers. Eine Unterbrechung wird an den acht Eingängen durch den Übergang von Low- zu High-Pegel (steigende Flanke) angefordert. Die Anforderungen werden im Interrupt-Request-Register (IRR) gespeichert. Der Inhalt des Interrupt-Mask-Register (IMR) entscheidet nun, ob die Anforderungen beachtet werden oder nicht. In diesem Register ist für jeden Eingang ein Freigabebit vorhanden, dabei entspricht die Bitnummer der Nummer des Eingangs (Bit 7 für IR 7 usw.). Ein Eingang ist gesperrt, wenn das dazugehörige Bit gesetzt ist. Wenn die entsprechenden Eingänge freigegeben sind, wird, sofern mehrere Anforderungen vorliegen (maximal acht möglich) vom Prioritätsdekoder die Anforderung mit der höchsten Priorität ermittelt und weitergeleitet. Dann wird geprüft, ob zur Zeit ein Interrupt höherer oder gleicher Priorität (gleicher Eingang) behandelt wird. Dazu wird für jeden Eingang ein Bit im Interrupt-In-Service-Register (ISR) geführt, welches gesetzt ist, wenn ein Interrupt von diesem Eingang zur Zeit bearbeitet wird. Ist kein Interrupt gleicher oder höherer Priorität in Arbeit, so signalisiert der Controller dem Prozessor die Anforderung über die INT-Leitung. Diese Anforderung bleibt solange bestehen, bis der Prozessor durch einen ersten Low-Impuls am INTA-Eingang den Interrupt bestätigt. Durch diesen Impuls wird die Anforderung auf der INT-Leitung zurückgenommen, der Zustand des Interrupt-Request-Registers eingefroren, das heißt keine jetzt folgende Anforderung gespeichert, und das zugehörige Bit im In-Service-Register gesetzt. Durch einen zweiten Low-Impuls auf der INTA-Leitung fordert der Prozessor danach den Interruptvektor vom Controller an. Nach der Übertragung des Vektors wird die Anforderung im IRR gelöscht und das Register freigegeben.

Sobald die Behandlung des Interrupts beendet ist, muss das dem Controller mitgeteilt werden, damit dieser das entsprechende Bit im ISR zurücksetzt. Dazu wird dem Controller ein spezielles Steuerwort mit Namen EOI (End of Interrupt) übertragen. Dieser löscht daraufhin im ISR das Bit mit der höchsten Priorität und damit der letzten erfolgreichen Anforderung.

In einem System mit mehreren, kaskadierten Interruptcontrollern laufen prinzipiell die gleichen Vorgänge ab. Die Slave-Controller durchlaufen alle genannten Phasen bis zur Aktivierung der INT-Ausgänge. Diese fordern daraufhin Interrupts beim Mastercontroller an, welcher die Anforderung mit der höchsten Priorität an den Prozessor weiterleitet. Wird diese bestätigt (INTA-Impuls), so wählt der Master über die Leitungen CAS0 bis CAS2 den dazugehörenden Slave-Controller aus, der dann mit Erscheinen des zweiten INTA-Impulses die entsprechende Vektornummer ausgibt. Damit die ISR-Bits zurückgesetzt werden, muss der EOI-Befehl sowohl an den Master- als auch an den Slave-Controller geschickt werden. Man beachte außerdem, dass sämtliche Anforderungen eines Slaves ignoriert werden, wenn der von diesem verwendete Eingang des Masters nicht freigegeben ist.

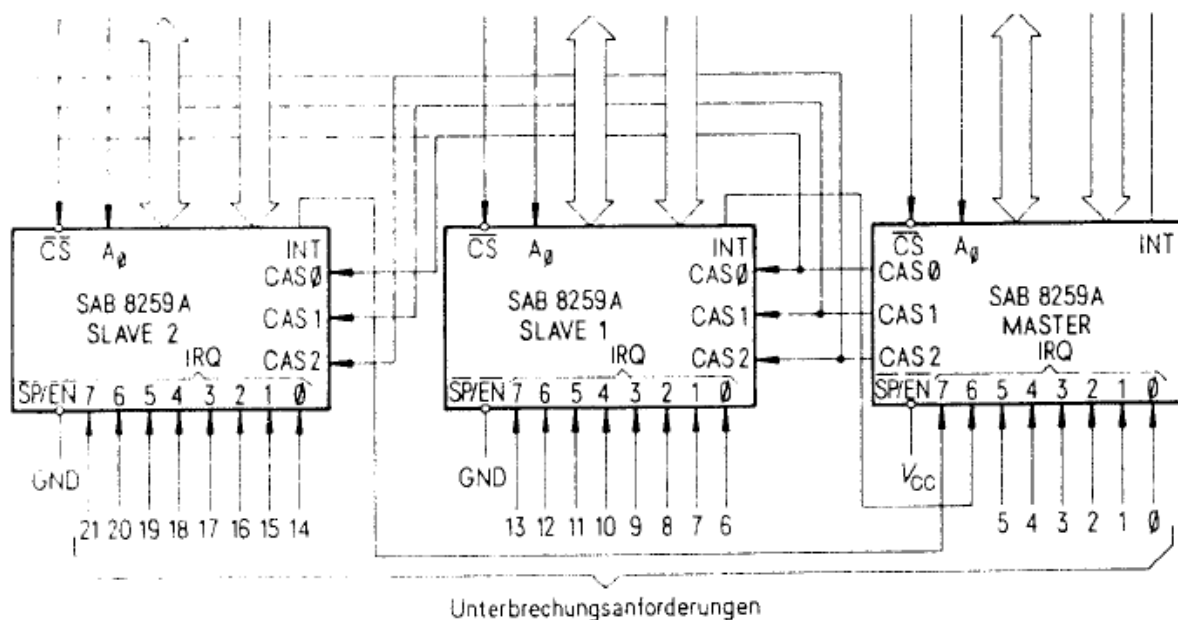


Bild: System mit mehreren, kaskadierten 8259A Interruptcontrollern

Die bisherige Beschreibung lässt schon erahnen, dass die Programmierung des 8259A recht kompliziert ist. Dies liegt zum Einen an den verschiedenen Betriebsarten, (einzelner Controller oder kaskadiert, dann entweder Master oder Slave), außerdem ist der Controller nicht nur für die Prozessoren der 80x86 Familie, sondern auch für die älteren Prozessoren 8080 und 8085 ausgelegt. Zum Anderen belegt er nur zwei Adressen. Bei der Programmierung müssen die einzelnen Steuer- und Datenbytes daher in einer bestimmten Reihenfolge übertragen werden. Bei AT-kompatiblen Rechnern unter MS-DOS wird einem die Programmierung allerdings vom Betriebssystem abgenommen. Dadurch sind die verwendeten Interruptvektoren und die Prioritäten der Eingänge festgelegt.

In einem AT-kompatiblen Rechner sind zwei Controller des Typs 8259A vorhanden. Die Priorität nimmt mit steigender Nummer des Eingangs ab. Der Slave-Controller fordert Interrupts am IR2-Eingang des Masters an, daher fügen sich seine acht Eingänge mit ihrer Priorität zwischen den Eingängen IR1 und IR3 ein.

Die folgende Tabelle gibt alle Interrupteingänge in absteigender Priorität an. Außerdem wird die Zuordnung der jeweiligen Controllereingänge zu den Interruptleitungen des AT-Rechnersystems und eventuell deren Verwendung angegeben.

Master-Controller	Slave-Controller	Int.-Nummer	Verwendung
IR 0		IRQ 0	Interner Timer-Interrupt
IR 1		IRQ 1	Tastatur
-	IR 0	IRQ 8	Echtzeit Uhr
-	IR 1	IRQ 9	Grafik-/ Netzwerkkarte
-	IR 2	IRQ 10	frei
-	IR 3	IRQ 11	frei
-	IR 4	IRQ 12	frei
-	IR 5	IRQ 13	num. Coprozessor
-	IR 6	IRQ 14	Festplatten Controller
-	IR 7	IRQ 15	reserviert BIOS
IR 3		IRQ 3	2. serielle Schnittstelle
IR 4		IRQ 4	1. serielle Schnittstelle
IR 5		IRQ 5	2. parallele Schnittstelle
IR 6		IRQ 6	Disketten Controller
IR 7		IRQ 7	1. parallele Schnittstelle

Bild: Zuordnung der Interruptleitungen zu den Controllereingängen

Vektortabelle

Die Vektortabelle enthält die Startadressen der Interrupt-Service-Routinen. Die Indizes werden *Vektornummern* (VN) genannt. Die (*Interrupt*-) *Vektortabelle* ist meistens im unteren Adreßbereich des Hauptspeichers abgelegt und besitzt die Offset-Adressen 0000 bis 1023. Es ergeben sich 256 verschiedene Vektoreinträge, wenn jeder Eintrag 4 Byte groß ist. Besitzt man bereits die Vektornummer, so kann man daraus die Offsetstartadresse der ISR berechnen:

Offset-Adresse der ISR = Vektornummer (VN) · 4

VN	Adresse	Belegung
00h	000h - 003h	(CPU) Division durch Null
08h	020h - 023h	(IRQ0) Timer
09h	024h - 027h	(IRQ1) Tastatur
0Ah	028h - 02Bh	(IRQ2) [Anschluß von PIC2]
0Bh	02Ch - 02Fh	(IRQ3) COM2
0Ch	030h - 033h	(IRQ4) COM1

0Dh	034h - 037h	(IRQ5) Festplatte
0Eh	038h - 03Bh	(IRQ6) Floppy
0Fh	03Ch - 03Fh	(IRQ7) Drucker
16h	058h - 05Bh	(BIOS) Tastaturabfrage
21h	084h - 087h	(DOS) DOS-Funktion aufrufen
70h	1C0h - 1C3h	(IRQ8) [Echtzeituhr]
71h	1C4h - 1C7h	(IRQ9)
72h	1C8h - 1CBh	(IRQ10)
73h	1CCh - 1CFh	(IRQ11)
74h	1D0h - 1D3h	(IRQ12)
75h	1D4h - 1D7h	(IRQ13) [80287 NMI]
76h	1D8h - 1DBh	(IRQ14) [Festplatte]
77h	1DCh - 1DFh	(IRQ15)

Bild: Vektortabelle beim IBM-PC/AT

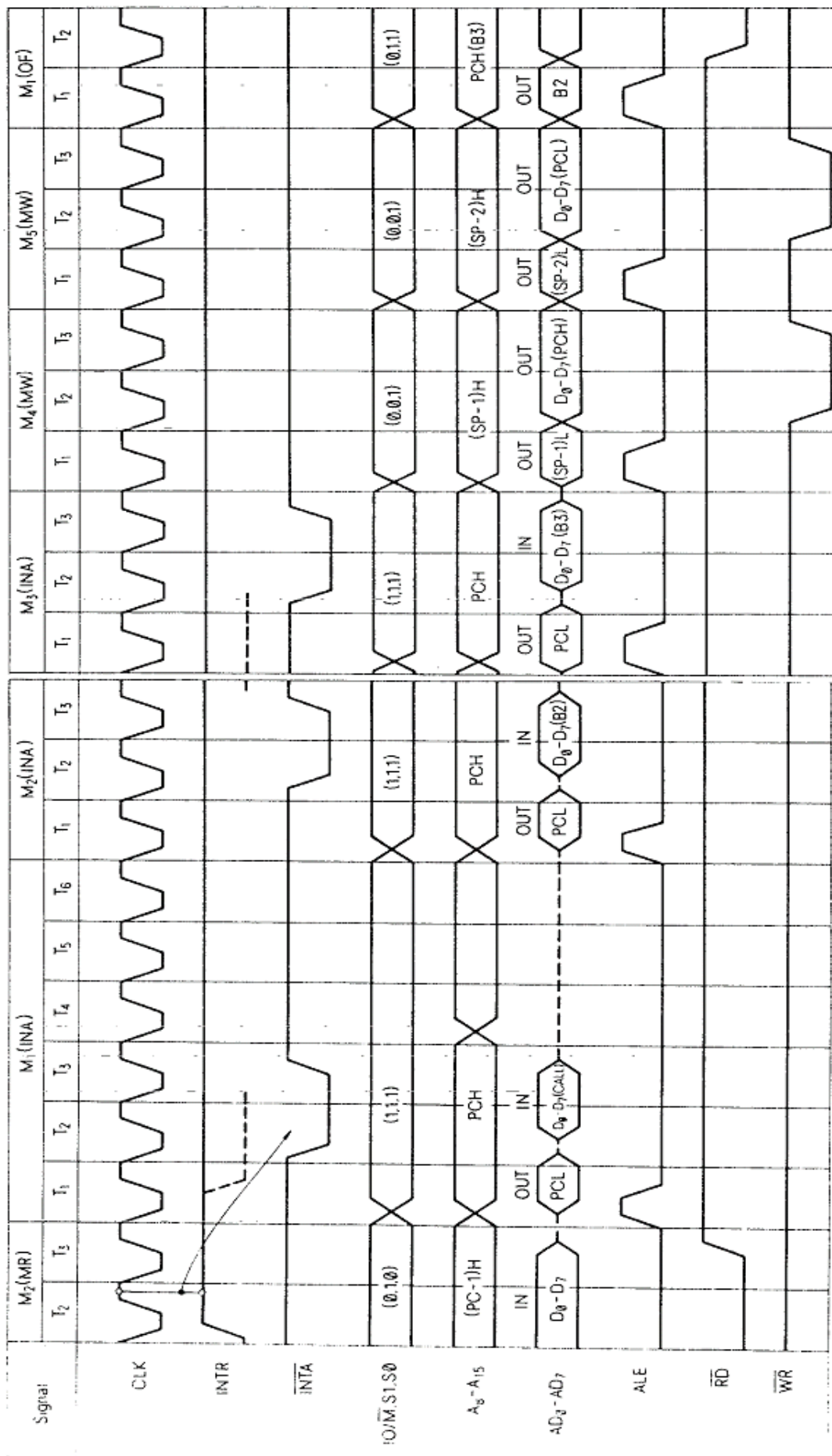


Bild: /INTA-Impulse zur Übertragung der Interrupt-Vektoradresse

9.4. CPU interne Bearbeitung von Interrupts und ISRs

9.4.1. Der normale Programmfluss

Für alle Prozessoren gilt folgender Programmablauf

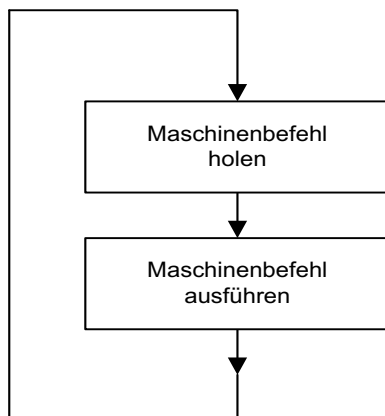


Bild: Allgemeiner Programmablauf eines Prozessors

Befehlszähler

- zeigt auf den nächsten Befehl im Speicher
- Befehlszählereinstellung ist Kern der Programmsteuerung
- Befehlszähler folgt normalerweise den hintereinanderliegenden Befehlen im Speicher
- Befehlszähler kann auch durch Maschinenbefehle eingestellt werden, z. B. durch programmsteuernde Sprung- und Verzweigungsbefehle (JMP etc.)
- Befehlszähler kann auch durch Eingriffe von außen verändert werden, z. B. durch Anforderung an die Programmsteuerung von außen (Interrupt); aber: Programmunterbrechung erfordert Zwischenspeicherung des Befehlszählerstandes

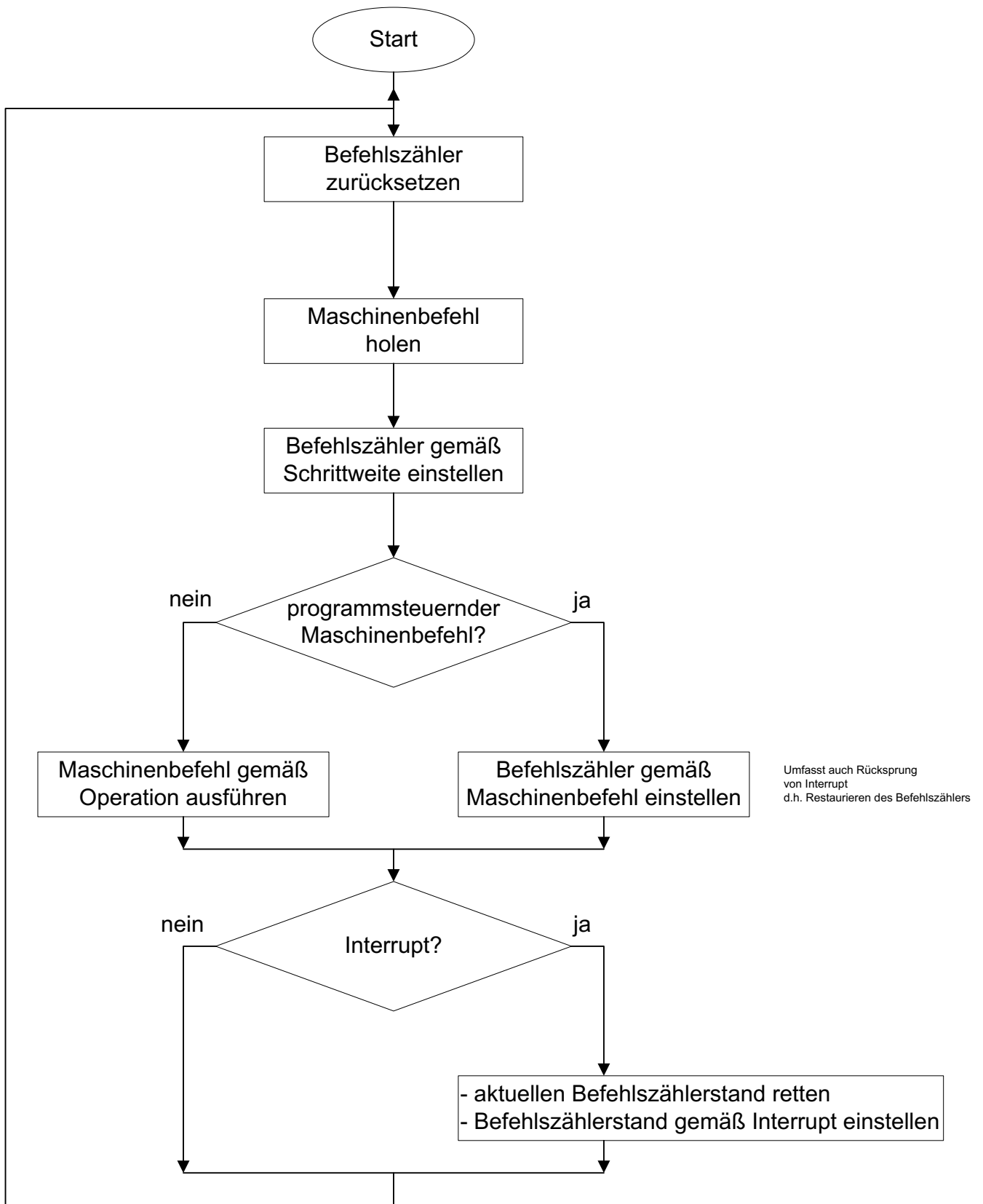


Bild: Programmablauf eines Prozessors im Detail

9.4.2. Priorisierung bei mehreren Interrupt-Quellen

- Überprüfen des/der Interrupt-Flags
- wenn Annahme erlaubt, signalisieren der Interrupt-Akzeptanz über den Steuerbus an alle Komponenten
- Komponente mit Forderung legt einen Vektor (typ. 8 bit) auf den Datenbus
- der Prozessor liest den Vektor und wählt entsprechend die Startadresse

Problem: INT-zeitgleich mehrere Anforderungen

- Prozessor kann nur eine Ausnahmesituation zu einem Zeitpunkt annehmen und bearbeiten
- Prioritätslogik: zentral/dezentral

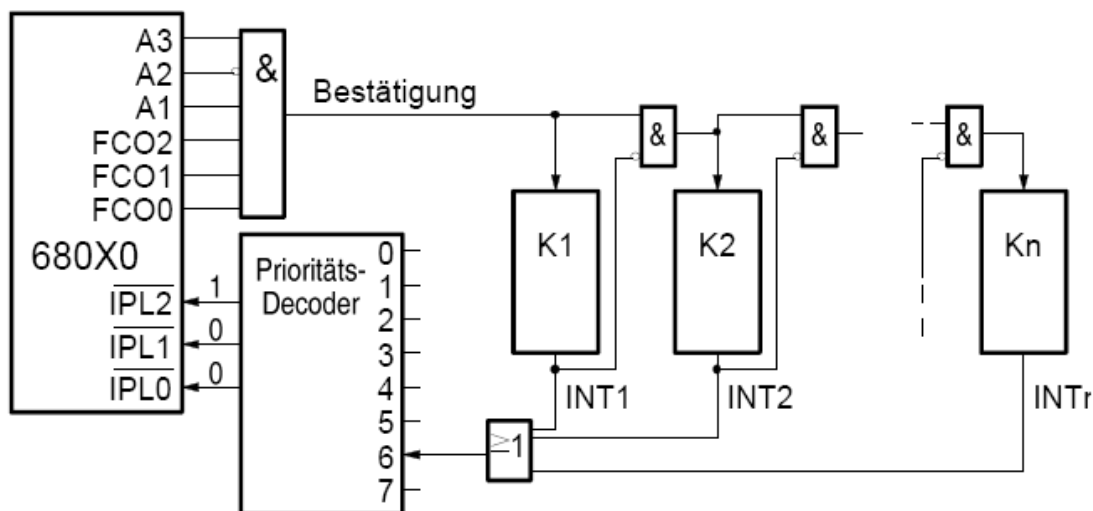


Bild: Daisy-Chain und Prioritäts-Decoder beim Motorola 68000

- Polling: gesteuert durch Software fragt der Prozessor nacheinander alle potentiellen Interrupt-Quellen ab
- Abfrage: Geräte besitzen Interrupt-Flag-Register, die vom Prozessor adressiert und gelesen werden können, ein Bit eines Registers ist entweder gesetzt oder zurückgesetzt, wenn Interrupt angefordert wird
- Nachteil: relativ hohe Anzahl der Verarbeitungsschritte für die Interrupt-Akzeptanz

9.5. Das MSP430 Interruptsystem

Die Interrupt-Vektortabelle steht meist am Ende der Speicherorganisation eines Embedded Systems und beinhaltet die Zeiger auf die Interrupt Service Routinen (ISR), d. h. in dieser Tabelle befinden sich die Anfangsadressen der Unterbrechungsroutinen.

					('43x and '44x only)
0xFFFF2	9	Timer A	USART Rx	Timer A	USART0 Rx ('43x and '44x only)
0xFFFF4	10	Watchdog Timer	Watchdog Timer	Watchdog Timer	Watchdog Timer
0xFFFF6	11	Comparator	Comparator	Comparator	Comparator
0xFFFF8	12	Unused	Timer B	Dedicated I/O	Timer B ('43x and '44x only)
0xFFFFA	13	Unused	Timer B	Dedicated I/O	Timer_B ('43x and '44x only)
0xFFFFC	14	Oscillator / Flash / NMI	Oscillator / Flash / NMI	Oscillator / Flash / NMI	Oscillator / Flash / NMI
0xFFFFE	15 Highest	Hard Re-set/Watchdog	Hard Re-set/Watchdog	Hard Re-set/Watchdog	Hard Re-set/Watchdog

Bild: Interrupt-Vektoren in der MSP430 Mikrocontrollerfamilie

Die ersten 14 Interrupts können maskiert, d. h. individuell ab- oder zugeschaltet werden durch das enable/disable Flag in den Peripherieregistern der Hardwaremodule.

Die letzten beiden Interrupts sind nichtmaskierbar: 0xFFFFC ist der NMI (non maskable interrupt) Vektor und 0xFFFFE ist der Resetvektor.

Zu diesen gehören:

- Der RST/NMI Pin kann dazu konfiguriert werden, dass er einen NMI statt auslöst, anstatt an dem Prozessor das Reset-Signal auszulösen und einen Neustart zu provozieren, wenn die NMI-Leitung auf Low gezogen wird.
- Flash Zugriffsverletzung (access violation)
- Ein Fehler im Oszillator tritt auf. Die neueren Modelle des MSP430 benutzen einen auf den Chip integrierten Taktgenerator namens FLL (frequency locked loop). Dieser kann programmiert werden um einen weiten Bereich an Kerntaktfrequenzen zu generieren, die an den externen Quarz (in der Regel ein 32kHz Quarz, wie er auch in Uhren zu finden ist) phasengebunden gekoppelt ist. Wenn die Frequenzabstimmung die äußersten Limits tangiert, und die FLL-Schaltung nicht mehr adaptieren kann, wird ein Fehler im Oszillator bekannt gegeben.

Andere MSP 430 Modelle benutzen ein anderes Oszillator Modul. Hier wird das Fehlerflag gesetzt, wenn der Oszillator überhaupt nicht mehr schwingt. Die CPU kann dann versuchen auf einen 2. Oszillator umzustellen, sollte dieser Fehlerfall eintreten.

Quelle	Flag	System	Adresse	Priorität
- Power-Up - ext. Reset - Watchdog	WDTIFG	Reset	0FFFEh	highest 15
NMI Oszillator- fehler	NMIIFG OFIFG	Non-maskable (non)maskable Mehrfachquelle	0FFFC h	14
P0.0	P0IFG.0	maskable	0FFFAh	13

P0.1	P0IFG.1	maskable	0FFF8h	12
		maskable	0FFF6h	11
Watchdog timer	WDTIF G	maskable	0FFF4h	10
		maskable	0FFF2h	9
		maskable	0FFF0h	8
		maskable	0FFEEh	7
		maskable	0FFEC h	6
ADC	ADCIFG	maskable	0FFEAh	5
Timer/Port		maskable	0FFE8h	4
		maskable	0FFE6h	3
		maskable	0FFE4h	2
Basic Timer	BTIFG	maskable	0FFE2h	1
Port 0 Bits 2..7	P0IFG.2 7	maskable Mehrfachquelle	0FFE0h	0, low

Bild: Maskable and non-maskable Interrupts

9.5.1. Behandlung eines Interrupts bei der TI MSP430-Familie

Wenn ein Interrupt auftritt, wird zunächst der Befehlszähler (PC, program counter) von der nächsten Instruction und das Prozessor Status Register (SR) auf den Stack geschoben.

Der Ablauf des Interrupts ist wie folgt:

- Push PC auf den Stack.
- Push SR auf den Stack.
- Lösche das Statusregister (SCG1, C, N, V und Z, mit Ausnahme von SCG0), Stelle die Interrupts (GIE) und das Energiemanagement (CPUOff, OSCOOFF) ab.
- Wenn nur eine Interruptquelle vorliegt, wird das entsprechende Interrupt Request Bit wieder zurückgesetzt. (Gibt es mehrere Interruptauslöser, wird er beibehalten um von der Software abgefragt werden zu können. Wenn NMI vorliegt, werden die enable Bits für die 3 NMI Quellen gelöscht.)
- Der Interrupt Vektor mit der höchsten Priorität aller aktivierten Interrupts wird in den Programm Counter der CPU geladen.
- Führe die ISR (Interrupt Handler) aus
- RETI (return from interrupt) restauriert den alten Zustand des Statusregisters und der Programm Counters.

Man kann Interruptbehandlungsroutinen verschachteln, indem man innerhalb einer Interrupt Service Routine die gegenwärtige Interruptquelle deaktiviert und das GIE Bit wieder auf 1 setzt.

Beachte: Es gibt keine Ausnahmebehandlungsmöglichkeiten für interne Prozessorfehler wie „divide by zero“ oder einem Adressierungsfehler. Jedoch kann ein Reset ausgelöst, wenn man in nicht beschreibbare Peripherieregister schreibt.

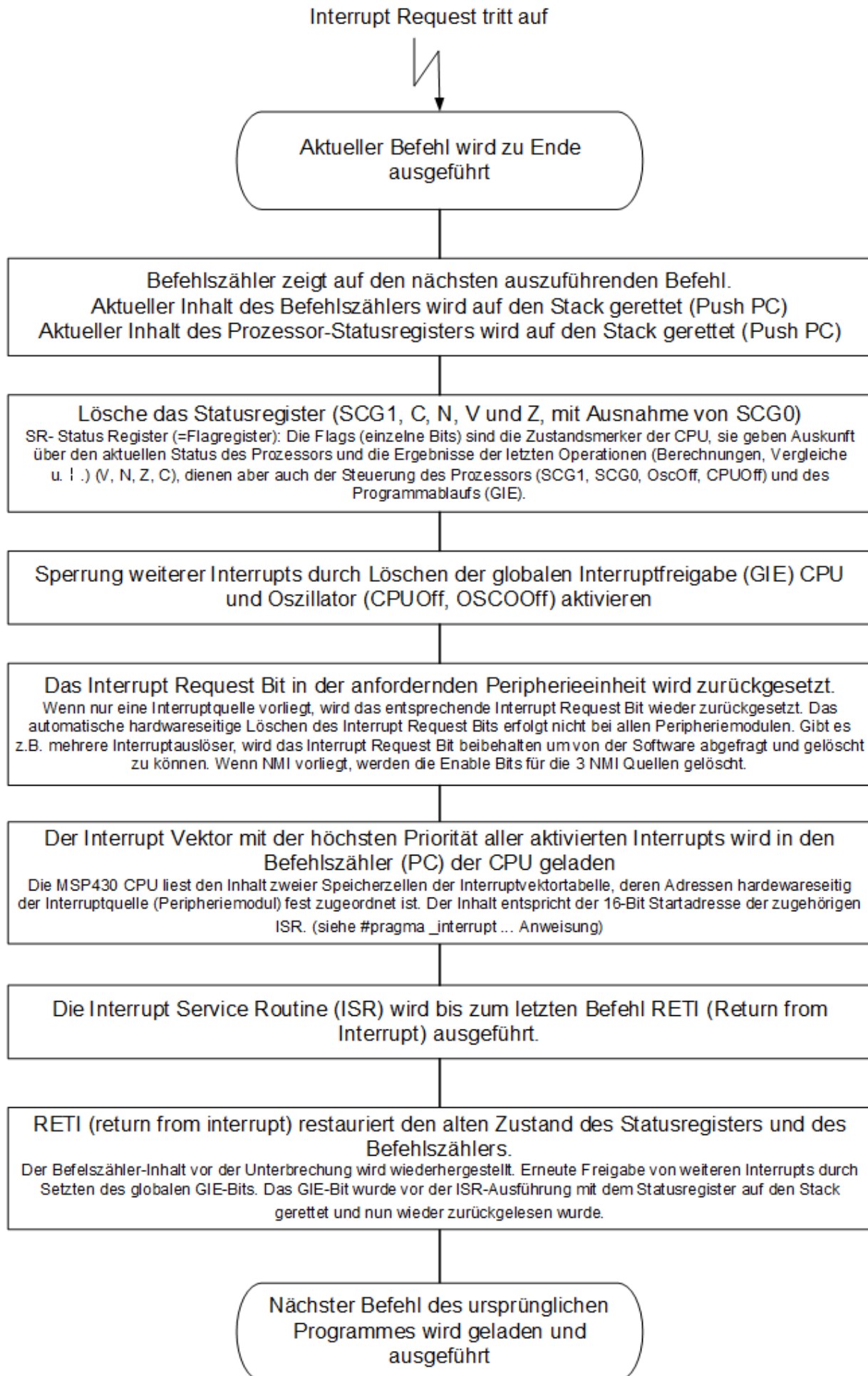


Bild: Behandlung eines Interrupts bei der TI MSP430-Familie

Veränderung des Stackpointers beim Interruptaufruf

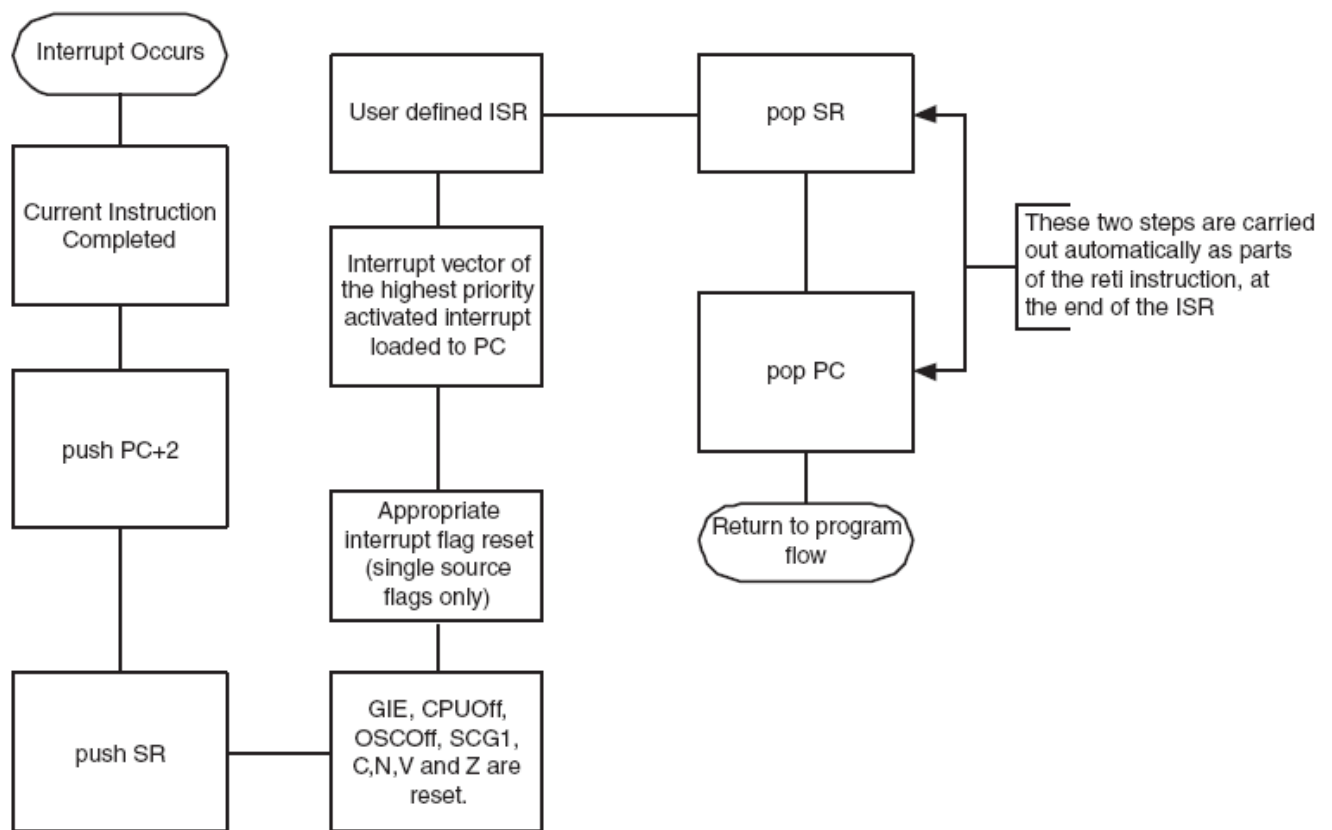


Bild: Interrupt Processing

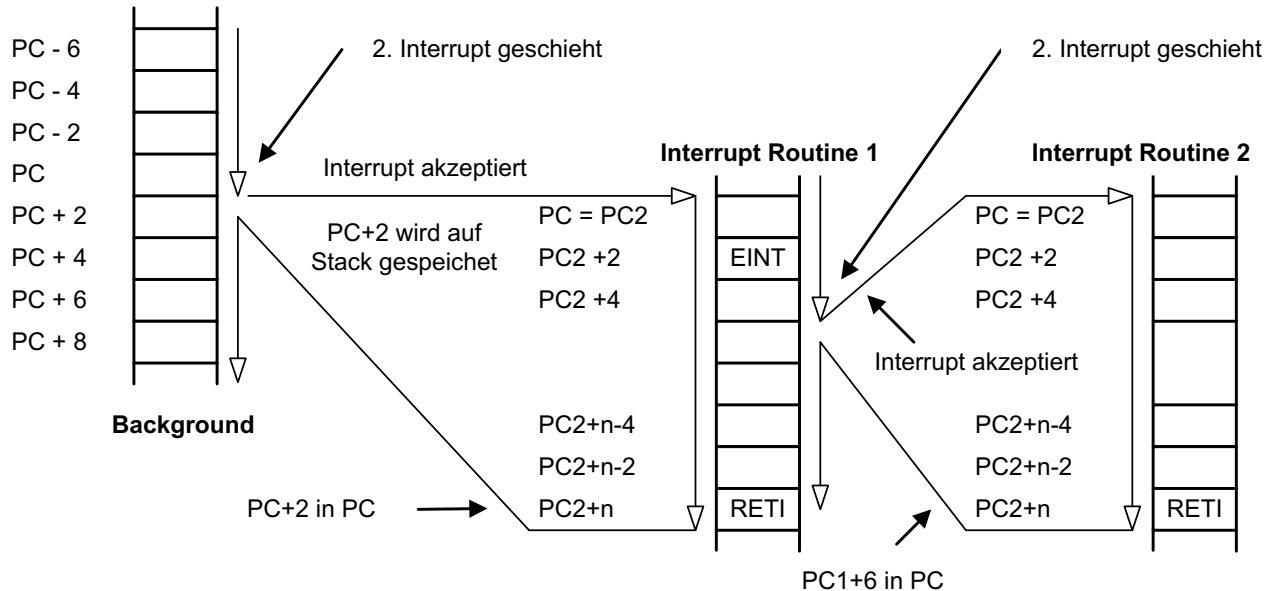
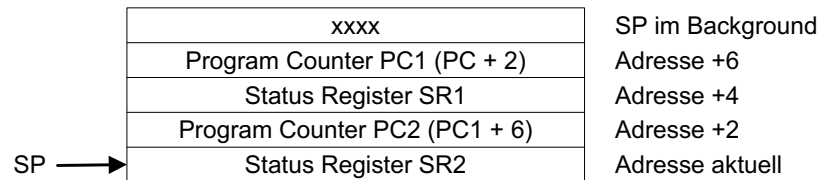


Bild: Interrupt-Verschachtelung (Interrupt Nesting)

9.5.2. Binden eines Interrupts an eine Interrupt Service Routine

Binden eines Interrupts an eine Interrupt Service Routine beim MSP430

The MSP430 microcontroller supports many interrupt sources. For each interrupt source, an interrupt routine can be written. Each interrupt routine is associated with a vector number which is specified in the MSP430 microcontroller documentation from the chip manufacturer.

The **iochip.h** header file, which corresponds to the selected derivative, contains predefined names for the existing exception vectors.

To define an interrupt function, the **__interrupt** keyword and the **#pragma vector** directive can be used, for example:

```
#pragma vector = PORT1_VECTOR __interrupt void my_interrupt_routine()
{
    // Do something
}
```

Note: An interrupt function must have the return type void, and it cannot specify any parameters.

When an interrupt function is defined with a vector, the processor interrupt vector table is

populated. It is also possible to define an interrupt function without a vector. This is useful if an application is capable of populating or changing the interrupt vector table at runtime. See the chip manufacturer's MSP430 microcontroller documentation for more information about the interrupt vector table.

The interrupt vector is the offset into the interrupt vector table.

For the MSP430 microcontroller, the interrupt vector table always starts at the address 0xFFE0.

9.5.3. Übersicht der Interrupt Vektoren (aus msp430x44x.h)

```
// entnommen aus msp430x44x.h
/*****
 * Interrupt Vectors (offset from 0xFFE0)
 *****/

#define BASICTIMER_VECTOR      (0 * 2u) /* 0xFFE0 Basic Timer */
#define PORT2_VECTOR           (1 * 2u) /* 0xFFE2 Port 2 */
#define USART1TX_VECTOR        (2 * 2u) /* 0xFFE4 USART 1 Transmit */
#define USART1RX_VECTOR        (3 * 2u) /* 0xFFE6 USART 1 Receive */
#define PORT1_VECTOR           (4 * 2u) /* 0xFFE8 Port 1 */
#define TIMERA1_VECTOR          (5 * 2u) /* 0xFFEA Timer A CC1-2, TA */
#define TIMERA0_VECTOR          (6 * 2u) /* 0xFFEC Timer A CC0 */
#define ADC_VECTOR              (7 * 2u) /* 0xFFEE ADC */
#define USART0TX_VECTOR         (8 * 2u) /* 0xFFFF0 USART 0 Transmit */
#define USART0RX_VECTOR         (9 * 2u) /* 0xFFFF2 USART 0 Receive */
#define WDT_VECTOR              (10 * 2u) /* 0xFFFF4 Watchdog Timer */
#define COMPARATORA_VECTOR      (11 * 2u) /* 0xFFFF6 Comparator A */
#define TIMERB1_VECTOR          (12 * 2u) /* 0xFFFF8 Timer B CC1-6, TB */
#define TIMERB0_VECTOR          (13 * 2u) /* 0xFFFFA Timer B CC0 */
#define NMI_VECTOR              (14 * 2u) /* 0xFFFFC Non-maskable */
#define RESET_VECTOR            (15 * 2u) /* 0xFFFFE Reset [Highest Priority] */
#define UART1TX_VECTOR          USART1TX_VECTOR
#define UART1RX_VECTOR          USART1RX_VECTOR
#define UART0TX_VECTOR          USART0TX_VECTOR
#define UART0RX_VECTOR          USART0RX_VECTOR
/*****
```

9.6. Beispiele und Übungen zum Interruptsystem MSP430I

9.6.1. Anwendungsbeispiel I

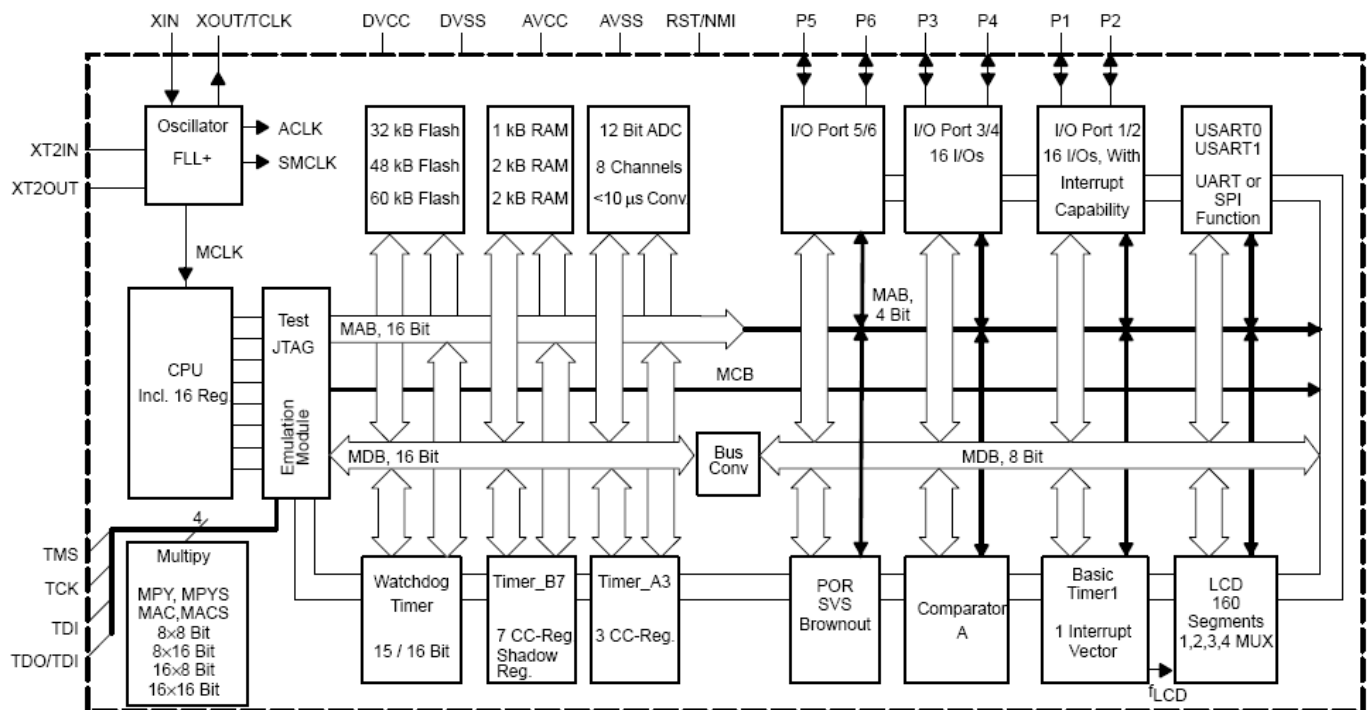


Bild: MSP430x44x functional block diagrams

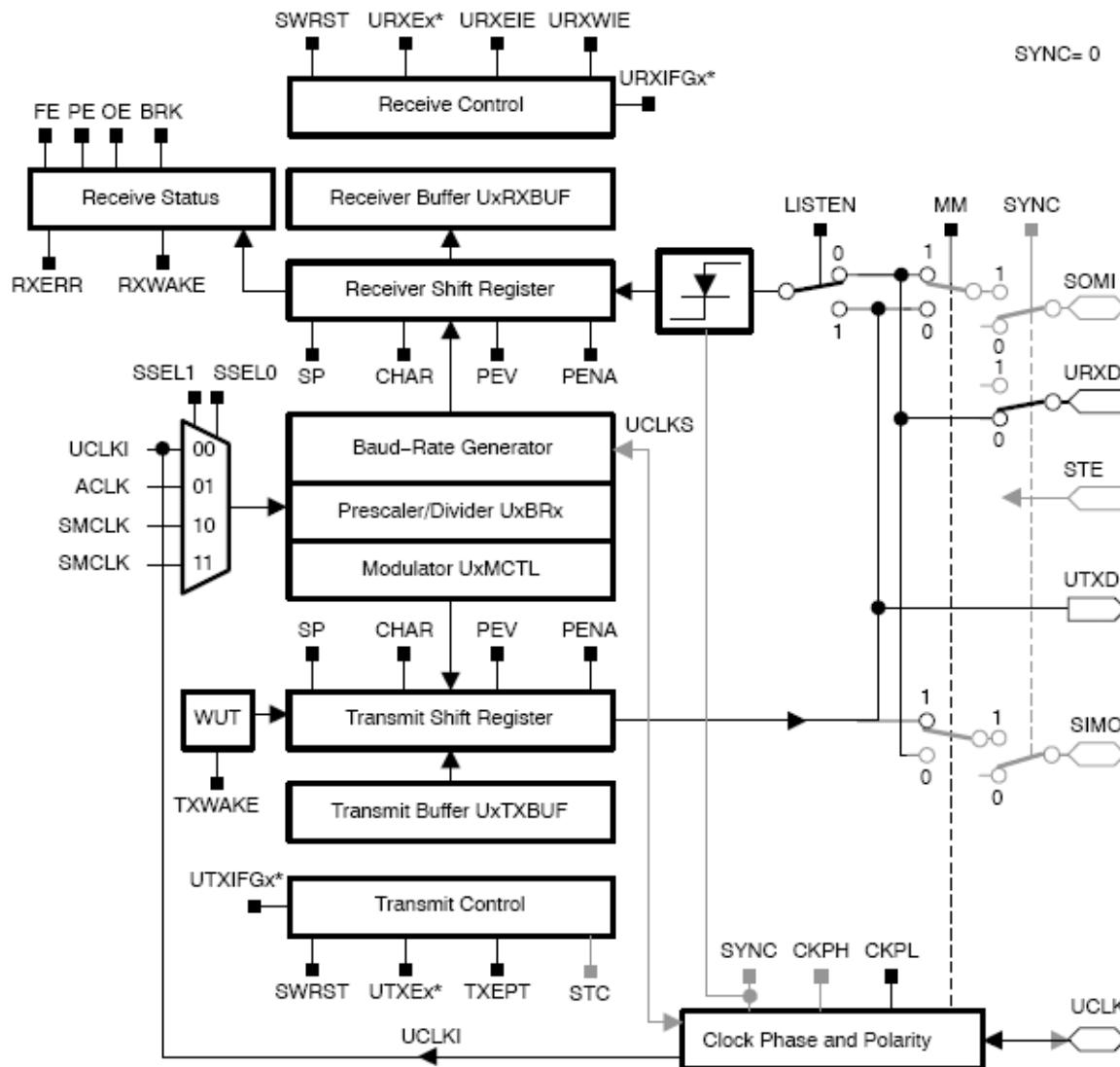


Bild: USARTx functional block diagram

Table 16–3. USART0 Control and Status Registers

Register	Short Form	Register Type	Address	Initial State
USART control register	U0CTL	Read/write	070h	001h with PUC
Transmit control register	U0TCTL	Read/write	071h	001h with PUC
Receive control register	U0RCTL	Read/write	072h	000h with PUC
Modulation control register	U0MCTL	Read/write	073h	Unchanged
Baud rate control register 0	U0BR0	Read/write	074h	Unchanged
Baud rate control register 1	U0BR1	Read/write	075h	Unchanged
Receive buffer register	U0RXBUF	Read	076h	Unchanged
Transmit buffer register	U0TXBUF	Read/write	077h	Unchanged
SFR module enable register 1	ME1	Read/write	004h	000h with PUC
SFR interrupt enable register 1	IE1	Read/write	000h	000h with PUC
SFR interrupt flag register 1	IFG1	Read/write	002h	082h with PUC

IFG2, Interrupt Flag Register 2

7	6	5	4	3	2	1	0
		UTXIFG1	URXIFG1				
		rw-1	rw-0				

Bits 7-6: These bits may be used by other modules. See device-specific datasheet.

UTXIFG1 Bit 5: USART1 transmit interrupt flag. UTXIFG1 is set when U1TXBUF empty.
 0 No interrupt pending
 1 Interrupt pending

URXIFG1 Bit 4: USART1 receive interrupt flag. URXIFG1 is set when U1RXBUF has received a complete character.
 0 No interrupt pending
 1 Interrupt pending

Bits 3-0: These bits may be used by other modules. See device-specific datasheet.

```
// Das Programm sendet das über die V24-Schnittstelle empfangene Zeichen wieder zurück
// (Echo Betrieb).
```

```
#include <msp430x44x.h>
```

```
void init_uart(void)
```

```
{
```

```
    // Einstellung der Baudrate
```

```
    // Modulationsregister U1MCTL
```

```
    U1BR0= 0x03;
```

```
    U1BR1= 0x00;
```

```
    U1MCTL= 0x4A;
```

```
    U1TCTL=0x10;          // SSEL0 defined in msp430x44x.hs
```

```
    U1CTL=0x10;
```

```
    // Module Enabel Register
```

```
    ME2 |= 0x30;
```

```
    P4SEL |= 0x03;          // Bit 0 und 1 == 1 -> Peripheral module function
```

```
    P4DIR |= 0x01;          // Bit 0 Output (Transmit)
```

```
}
```

```
void main(void)
```

```
{
```

```
    char zeichen;           // Speicher für empfangenes Zeichen
```

```
    WDTCTL = WDTPW + WDTHOLD; // Stop watchdog timer
```

```
    init_uart();           // Serielle Schnittstellenparameter setzen
```

```

while(1)
{
    while ( (IFG2 & URXIFG1) ==0 ); // Warten, bis Zeichen eintrifft; Interrupt Flag Register 2 lesen
    zeichen = RXBUF1;                // empfangenes Zeichen im Speicher lesen

    while ( (IFG2 & UTXIFG1) ==0 ); // Ist Transmitter frei für ein neues Datum? ; IFG2 lesen
    TXBUF1 = zeichen;                // Zeichen in Transmit Buffer schreiben
}

} // end main

```

```

// Interruptroutine serieller Empfang
#pragma vector = UART1RX_VECTOR
__interrupt void usart1_rx (void)
{
    IE2 = IE2 ^ URXIE1;              // Interrupt Flag für seriellen Empfang deaktivieren
    ( (char*) buffer ) [buffer_counter++] = RXBUF1; // Zeichen in Puffer lesen und
                                           // Pufferzähler erhöhen

    If ( RXBUF1 == 0x0d )              // Abfrage ob Return gesendet wurde
    {
        flag=1;                      // Flag zum Auslesen des Puffers setzen
        ( (char*) buffer) [buffer_counter] = 0; // Nullcharakter ans Ende vom String anfügen
    }
    else // kein Return gesendet
    {
        if ((buffer_counter+1)%10 == 0) // Ist Puffer voll?
            // können weitere 10 Byte an buffer anhängt werden?
            If ( (buffer_tmp = realloc (buffer, buffer_counter+1+10) ) == 0)
                flag=1; // wenn nicht möglich, dann Speicher auslesen aktivieren
    }

    If (flag==0)
        IE2 = IE2 | URXIE1; // Interrupt für Empfang freigeben,
                               // wenn nicht der Puffer ausgelesen werden soll
}

```

9.6.2. Anwendungsbeispiel II

```
//*****
// MSP-FET430P440 Demo - Timer A Toggle P5.1, CCR0 upmode ISR, 32kHz ACLK
//
// Description; Toggle P5.1 using software and the TA_0 ISR. Timer A is
// configured in an upmode, thus the the timer will overflow when TAR counts
// to CCR0. In this example, CCR0 is loaded with 1000.
// Toggle rate = 32768/(2*1000) = 16.384
// ACLK = TACLK = LFXTL = 32768, MCLK = SMCLK = DCO = 32xACLK = 1.048576MHz
// /*An external watch crystal on XIN XOUT is required for ACLK*/
//
//          MSP430F449
//          -----
//          /|\|          XIN|-
//          | |          | 32kHz
//          --|RST      XOUT|-
//          |          |
//          |          P5.1|-->LED
//*****
#include <msp430x44x.h>

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop WDT
    FLL_CTL0 = FLL_CTL0 | XCAP14PF;     // Configure load caps
    P5DIR = P5DIR | 0x02;               // Set P5.1 to output direction

    TACTL = TASSEL0 + TACLK;            // ACLK, clear TAR
    CCTLO = CCIE;                       // CCR0 interrupt enabled

    CCR0 = 1000;
    TACTL = TACTL | MC0;                 // Start Timer A in upmode

    _EINT();                             // Enable interrupts

    for (;;)
    {
        _BIS_SR(LPM3_bits);            // Enter LPM3
    }
}

// Timer A0 interrupt service routine
#pragma vector=TIMER_A0_VECTOR  __interrupt  void Timer A (void)
{
    P5OUT = P5OUT ^ 0x02;               // Toggle P5.1 using exclusive-OR
}
```

9.6.3. Übung I

1. Wie erfolgt die programmseitige „Bindung“ der Interrupt Service Routine an die Interruptleitung eines Hardwaremoduls?
2. Worin liegt der Unterschied zwischen einem maskierbaren und einem nichtmaskierbaren Interrupt?

3. Wie funktioniert prinzipiell die Interruptverarbeitung eines Mikrocomputersystems?
4. Wie kann die Interruptverarbeitung sinnvoll zur Steuerung eines Peripheriemoduls, z.B. eines USARTs eingesetzt werden?
5. Welche Werte stehen in der Interrupt-Vektortabelle?
6. Skizzieren Sie das Signal-Zeit-Diagramm eines Datenframes der RS232-Schnittstelle und erläutern Sie den Aufbau.
7. Auf welche Weise nimmt die CPU Kenntnis davon, dass ein Byte über den USART-Baustein eingetroffen ist.

10. WEITERE FRAGEN:

- a) Durch welche Leitung fordert der Programmable Interrupt Controller PIC 8259 die Unterbrechung bei der CPU an?
- b) Durch welche Leitung zeigt die CPU an, wann der Interrupt erfolgen kann?
- c) Welche Voraussetzungen müssen in der CPU vorliegen, damit die Unterbrechung erfolgen und abgearbeitet werden kann?

- d) Was versteht man unter „nested interrupts“? Wo liegen die Gefahren von „nested interrupts“ und wie können diese vermieden werden?
- e) Beschreiben Sie die Kommunikation des Interruptsteuerbausteins PIC 8259 A mit der CPU im 8086-Modus.
- f) Erläutern Sie den Unterschied zwischen Software- und Hardware-Interrupts. Wo werden diese insbesondere verwendet?
- g) Beschreiben Sie den Zweck und die Funktion der Vektortabelle? Wie berechnet sich die Einsprungadresse für die IRQ-Adresse eines externen Interrupts?
- h) Beschreiben Sie den Begriff „shared IRQ“.

10.1.1. Übung II

Gegeben ist der nachstehende Programmausschnitt mit dem Quellcode der Interruptservice-routine void usart1_rx(void) sowie zwei Auszüge (Hex-Dumps mit Disassemblierung) aus dem Speicherabbild.

Bitte geben Sie an, wie sich der Stackpointer bei Eintreffen des Interrupts UART1RX_VECTOR verändert und welche Werte auf dem Stack liegen. Wählen Sie selbst eine beliebige Stack-Basisadresse.

```
//Interruptroutine Timerüberlauf
#pragma vector=TIMER_A1_VECTOR
__interrupt void Timer_A1(void)
{
    P5OUT &= 0x7F;           // P5.7 aus
    TACTL ^= 0x0001;         // lösche Timerüberlauf Flag
}

// Interruptroutine Compareregister 0
#pragma vector=TIMER_A0_VECTOR
__interrupt void Timer_A (void)
{
    P5OUT |= 0x80;           // P5.7 ein
}

// Interruptroutine Serieller Empfang
#pragma vector=UART1RX_VECTOR
__interrupt void usart1_rx(void)
{
    IE2 ^= URXIE1;          // Interrupt für Seriellen empfang abschalten

    ((char*)buffer)[buffer_counter++]=RXBUF1; // Zeichen in Puffer lesen und Pufferzähler erhöhen
    if(RXBUF1==0x0d)         // Abfrage ob Return gesendet wurde
    {
        flag=1;             // Flag zum auslesen des Puffers setzen
        ((char*)buffer)[buffer_counter]=0; // Nullcharakter ans ende vom String anfügen
    }
    else
    {
        if((buffer_counter+1)%10==0) // Ist Puffer voll?
        {
            if((buffer_tmp=realloc(buffer,buffer_counter+1+10))==0)
                // weitere 10 Byte am Anschluß von Buffer reservieren
            {
                flag=1;      // Wenn kein Speicherplatz mehr frei war, Speicher auslesen aktivieren
            }
        }
    }
    if(flag==0) IE2 |= URXIE1; // Interrupt für Empfang freigeben, wenn nicht der Puffer ausgelesen w
}
```

```

Timer_A1:
001264 F2C080003100 bic.b #0x80,&P5OUT
00126A 92E36001 xor.w #0x1,&TACTL
00126E 0013 reti PC
Timer_A:
001270 F2D080003100 bis.b #0x80,&P5OUT
001276 0013 reti PC
usart1_rx:
001278 0D12 push.w R13
00127A 0C12 push.w R12
00127C 0F12 push.w R15
00127E 0E12 push.w R14
001280 F2E010000100 xor.b #0x10,&IE2
001286 1F420202 mov.w &buffer,R15
00128A 1F520602 add.w &buffer_counter,R15
00128E DF427E000000 mov.b &U1RXBUF,0x0(R15)
001294 92530602 inc.w &buffer_counter
001298 F2900D007E00 cmp.b #0xD,&U1RXBUF
00129E 0920 jne 0x12B2
0012A0 D2430802 mov.b #0x1,&flag
0012A4 1F420202 mov.w &buffer,R15
0012A8 1F520602 add.w &buffer_counter,R15
0012AC CF430000 clr.b 0x0(R15)
0012B0 173C jmp 0x12E0
0012B2 1C420602 mov.w &buffer_counter,R12
0012B6 1C53 inc.w R12
0012B8 3E400A00 mov.w #0xA,R14
0012BC B0125C13 call #?DivMod16u
0012C0 0E93 tst.w R14
0012C2 0E20 jne 0x12E0
0012C4 1E420602 mov.w &buffer_counter,R14
0012C8 3E500B00 add.w #0xB,R14
0012CC 1C420202 mov.w &buffer,R12
0012D0 B012201A call #realloc
0012D4 824C0402 mov.w R12,&buffer_tmp
0012D8 0C93 tst.w R12
0012DA 0220 jne 0x12E0
0012DC D2430802 mov.b #0x1,&flag
0012E0 C2930802 tst.b &flag
0012E4 0320 jne 0x12EC
0012E6 F2D010000100 bis.b #0x10,&IE2
0012EC 3E41 pop.w R14
0012EE 3F41 pop.w R15
0012F0 3C41 pop.w R12
0012F2 3D41 pop.w R13
0012F4 0013 reti PC

```

Bild: Disassembler Programmausschnitt

```

00FFC6 FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFCA FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFCE FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFD2 FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFD6 FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFDA FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFDE FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFE2 FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFE6 7812 push.b @R8+
00FFE8 FF DC8 0xFF
00FFE9 FF DC8 0xFF
00FFEA 6412 push.b @R4
00FFEC 7012FFFF push.b #0xFF
00FFF0 FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFF4 FFFFFFFF and.b @R15+,0xFFFF(R15)
00FFF8 FFFFFFFF and.b @R15+,0xFFFF(R15)

```

Bild: Interrupt-Vektor-Tabelle MSP430

10.1.2. Einsatz von Low Power Modes bei der MSP430F4xx-Familie

SCG1	SCG0	OSCOFF	CPUOFF	Mode	CPU and Clocks Status
0	0	0	0	Active	CPU is active, all enabled clocks are active
0	0	0	1	LPM0	CPU, MCLK are disabled (41x/42x peripheral MCLK remains on) SMCLK, ACLK are active
0	1	0	1	LPM1	CPU, MCLK, DCO oscillator are disabled (41x/42x peripheral MCLK remains on) DC generator is disabled if the DCO is not used for MCLK or SMCLK in active mode SMCLK, ACLK are active
1	0	0	1	LPM2	CPU, MCLK, SMCLK, DCO oscillator are disabled DC generator remains enabled ACLK is active
1	1	0	1	LPM3	CPU, MCLK, SMCLK, DCO oscillator are disabled DC generator disabled ACLK is active
1	1	1	1	LPM4	CPU and all clocks disabled

Bild: **MSP430F4xx Low Power Modes**

Auszug aus dem Headerfile „msp430x44x.h“

```

/* STATUS REGISTER BITS
#define C                (0x0001)
#define Z                (0x0002)
#define N                (0x0004)
#define V                (0x0100)
#define GIE              (0x0008)
#define CPUOFF           (0x0010)
#define OSCOFF           (0x0020)
#define SCG0              (0x0040)
#define SCG1              (0x0080)

/* Low Power Modes coded with Bits 4–7 in SR */

#define LPM0_bits         (CPUOFF)
#define LPM1_bits         (SCG0+CPUOFF)
#define LPM2_bits         (SCG1+CPUOFF)
#define LPM3_bits         (SCG1+SCG0+CPUOFF)
#define LPM4_bits         (SCG1+SCG0+OSCOFF+CPUOFF)

```

```

#define LPM0      _BIS_SR(LPM0_bits)      /* Enter Low Power Mode 0 */
#define LPM0_EXIT _BIC_SR_IRQ(LPM0_bits) /* Exit Low Power Mode 0 */
#define LPM1      _BIS_SR(LPM1_bits)      /* Enter Low Power Mode 1 */
#define LPM1_EXIT _BIC_SR_IRQ(LPM1_bits) /* Exit Low Power Mode 1 */
#define LPM2      _BIS_SR(LPM2_bits)      /* Enter Low Power Mode 2 */
#define LPM2_EXIT _BIC_SR_IRQ(LPM2_bits) /* Exit Low Power Mode 2 */
#define LPM3      _BIS_SR(LPM3_bits)      /* Enter Low Power Mode 3 */
#define LPM3_EXIT _BIC_SR_IRQ(LPM3_bits) /* Exit Low Power Mode 3 */
#define LPM4      _BIS_SR(LPM4_bits)      /* Enter Low Power Mode 4 */
#define LPM4_EXIT _BIC_SR_IRQ(LPM4_bits) /* Exit Low Power Mode 4 */

```

Beispielprogramm zum Einsatz von Low Power Modes

```

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;      // Stop WDT
    FLL_CTL0 = FLL_CTL0 | XCAP14PF; // Configure load caps
    P2DIR = ....BIT1;.....        // Set P2.1 to output direction
    P1IES = ....BIT0;.....        // H-L transition
    P1IE = ....BIT0;.....         // Enable interrupt
    _EINT();                       // Enable interrupts
    _BIT_SR (LPM3_bits);          // Enter Low Power Mode 3
}

#pragma vector=PORT1_VECTOR      // P1 interrupt service routine
__interrupt void P1ISR (void)
{
    ...P2OUT = P2OUT ^ BIT1;..... // toggle LED Pin
    P1IFG = ...P1IFG & ~BIT0;..... // Clear P1IFG
}

```

Einsatz von Low Power Modes und Servicebehandlung im Hauptprogramm

```

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;      // Stop WDT
    FLL_CTL0 = FLL_CTL0 | XCAP14PF; // Configure load caps
    P2DIR = ....BIT1;.....        // Set P2.1 to output direction

```

```

P1IES = ....BIT0;.....           // H-L transition
P1IE = ....BIT0;.....           // Enable interrupt
_EINT();                          // Enable interrupts
while (1)
{
    _BIT_SR (LPM3_bits);         // Enter Low Power Mode 3
    P2OUT = P2OUT ^ BIT1;        // toggle LED pin
    // do other operations

    P1IFG = ..P1IFG & ~BIT0;.....
}

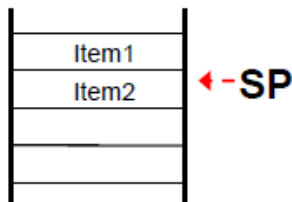
```

```

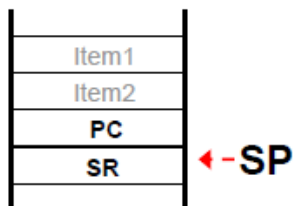
#pragma vector=PORT1_VECTOR       // P1 interrupt service routine
__interrupt void P1ISR (void)
{
    _BIC_SR_IRQ (LPM3_bits)      // Exit Low Power Mode 3, manipulate Stack
}

```

13) Bitmanipulation und Interrupts

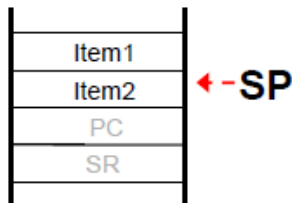


Prior to ISR



ISR hardware - automatically

- PC pushed
- SR pushed
- Interrupt vector moved to PC
- **GIE, CPUOFF, OSCOFF and SCG1** cleared
- IFG flag cleared on single source flags



reti - automatically

- SR popped - *original*
- PC popped

Bild: Interrupt Processing

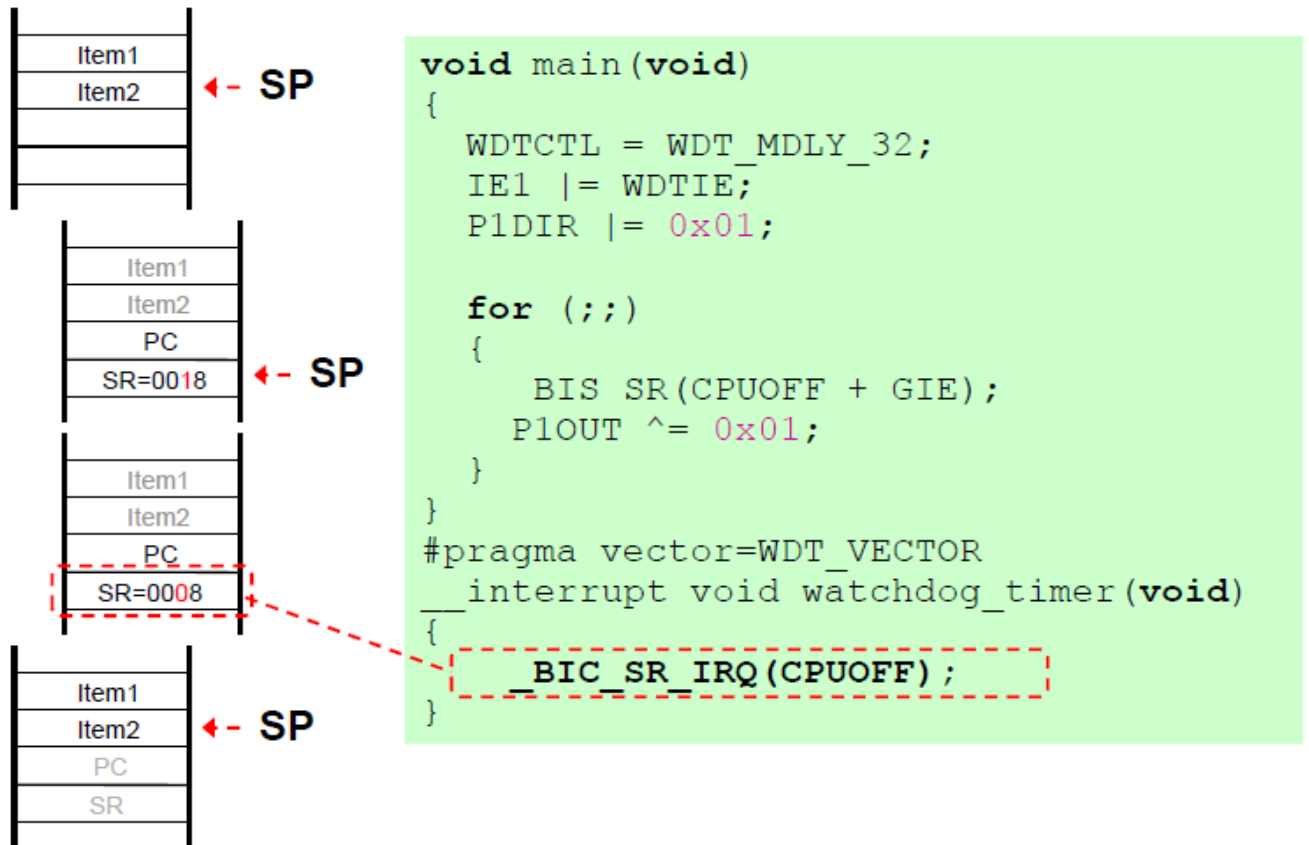


Bild: Low Power Modes In C